

Documentation Bundle

- **Root:**
C:\Users\dacoo\OneDrive\configure\edition4\OS\version2\TCP\services\Astronaut\Operate\Operate
- **Generated:** 2026-05-15T14:22:03
- **Included files:** 12
- **Max text file bytes:** 5000000
- **Ignored dirs:** .git, .hg, .idea, .svn, .venv, .vscode, __pycache__, build, dist, doc, node_modules, vendor, venv

Filesystem Tree (included paths)

```
Operate
├── Computer Engineering Codex
│   └── manuscript-cec.md
├── operator-alphabet.js
├── operator-build.js
├── operator-dropdown.js
├── operator-extensions.js
├── operator-framework.js
├── operator-language.js
├── operator-map.js
├── operator-persistence.js
├── operator.css
├── operator.html
└── operator.js
```

Table of Contents

1. Computer Engineering Codex/manuscript-cec.md
2. operator-alphabet.js
3. operator-build.js
4. operator-dropdown.js
5. operator-extensions.js
6. operator-framework.js
7. operator-language.js
8. operator-map.js
9. operator-persistence.js
10. operator.css
11. operator.html
12. operator.js

File Contents

[1] Computer Engineering Codex/manuscript-cec.md

- Bytes: 111587
- Type: text

Introduction: Theoretical Foundations of Electrical and Electrochemical Systems

1. Document Scope and Purpose

This technical document serves as a comprehensive ontological mapping and reference schema that delineates the theoretical infrastructure of modern electrical, electronic, and electrochemical systems. By organizing engineering disciplines into discrete component-oriented matrices—ranging from basic interconnects like wires and cables to active electromechanical switches, passives, and advanced electrochemical energy storage devices—this document bridges the gap between pure physics and applied engineering science. It explicitly categorizes the exact physical laws, mathematical models, and operational theories required to analyze, design, and validate robust technological solutions.

2. Use Cases for Theoretical Studies

For researchers, academics, and advanced engineering theorists, this document functions as a structured taxonomy to guide foundational inquiry and interdisciplinary research:

* **Cross-Disciplinary Paradigm Integration:** The text explicitly synthesizes fields that are traditionally siloed, such as linking chemical physics and thermodynamics (e.g., Redox, Nernst, Butler-Volmer, and Ion Transport theories) with classical network analytics (e.g., Randles circuit and Equivalent Circuit models). This supports deep theoretical studies into boundary-layer electrochemistry and energy storage mechanics.

* **Electromagnetic & Mathematical Modeling:** Researchers can utilize the text as a reference roadmap to trace how macro-level engineering challenges stem from fundamental field physics. For instance, it maps the progression from Maxwellian electromagnetism and classical electrodynamics down to localized phenomena like skin effect, proximity effect, and distributed element transmission line theory.

* **Ontological and Knowledge Graph Development:** In the domain of computer science and systems engineering theory, this document provides a clean structural framework for building semantic networks or AI-driven engineering ontologies. It cleanly maps structural, thermal, environmental, and electrical failure mechanics to their respective parent component architectures.

3. Engineering Science Applications for Solutions Engineering

In the practice of Solutions Engineering—where complex, multi-vendor, and cross-domain technical architectures must be synthesized into reliable commercial systems—this document provides immediate utility across several applied engineering vectors:

A. Holistic System Integration & Signal/Power Integrity

Solutions engineers tasked with deploying high-speed telemetry, data distribution networks, or heavy industrial power distribution can leverage the integrated frameworks for ****Wires, Cables, and Connectors****. It highlights the multi-variable nature of interconnect physics, prompting engineers to evaluate systems through the simultaneous lenses of:

- * Transmission line impedance matching and reflection coefficient mitigation.
- * Electromagnetic Compatibility (EMC) and Interference (EMI) shielding, grounding, and bonding architectures to suppress common-mode and differential-mode crosstalk.

B. Power Management and Energy Storage Design

When architecting power backup, microgrids, or electric mobility systems, the document serves as a strict analytical guide for ****Battery Cells, Supercapacitors, and Passive Networks****. Solution engineers can apply these theoretical frameworks to:

- * Optimize energy and power density trade-offs via Ragone plot analysis and Peukert's Law.
- * Incorporate Equivalent Series Resistance (ESR), Equivalent Series Inductance (ESL), and State of Health (SoH) tracking into automated Battery Management Systems (BMS) to handle mass transport gradients and SEI layer passivation.

C. Predictive Reliability, Degradation, and Root-Cause Analysis

A critical mandate of solutions engineering is ensuring system longevity and diagnosing failures under real-world stress. This document highlights a comprehensive array of engineering science degradation models:

- * ****Mechanical & Material Fatigue:**** Outlines tribology, fretting, corrosion, and moisture ingress theories essential for analyzing connector interface degradation and solder joint reliability under thermal expansion or vibration profiles.
- * ****Thermal Dynamics:**** Details Joule heating, thermal conduction, and dissipation physics to predict ampacity limits and manage severe system risks like thermal runaway.

D. Protective Architecture & Failsafe Engineering

For mission-critical infrastructure, the text provides the absolute blueprint for safety coordination and arc containment. Solutions engineers can utilize the theories embedded in the ****Switch, Relay, and Circuit Breaker**** sections to:

- * Coordinate complex protection schemes balancing inverse time, instantaneous trip, and thermal-magnetic clearing characteristics.
- * Mitigate destructive inductive kickback, corona discharge, and fault currents through the targeted application of snubber circuits, isolation boundaries, creepage/clearance constraints, and arc suppression mechanisms.

By grounding practical architectural decisions in these rigid engineering science principles, solutions engineers can confidently validate their technical designs, minimize deployment risks, and deliver highly optimized, resilient systems.

3699564407583367783087534503985189953682329669269635040697097924013451757233117171
5583116945581777750451695008052471225199384947172027734670115256720252609683812479
1410522130488465355566980938742714924868010172033412287358808763105223522026013695
6338207523057578520911219892511861392977750637361798880739217814197180851498989956
0683488175021654930881863684381952454197671458261841414828448620818170979122583331
0681338897372060738549006681582866296133734937437214871033911692578345962288913790
9593908210554410699013537159585081953795700393650139579787182301813515402683264793
9754368601059863549378430130466382011609684917299252452475212420058262974403335532
6647737459120385057797086901616795516709173730547933387859130075537876467856365375
0389918716711288529476127168439912944953718898044858883278180120217044526713632491
4629274441376743733289393944529276807682810222028083450436969870279645610418887974
9448859823582452119118754604524116520936625520020756566053995235660657286451247921
3101628837601775320018842145165100223712455317806894572860189367345371995023952630
5074415059081193893691603764718169130348782594424273511467570030597642517008286825
8302698506285242698540349654591284548348055882306682513351752542897621903553323143
6376213359632466074315737654993432479515323710030058024170699311655457423657285603
2247122082477320801941337512393464646182903170822648420214471211755658265188104932
1551823878126549323562013018275607036440613888606491429174513370462826267528336760
2433088195034811268180541640276519564416581204478791393660830537398168040601716185
5908857431197220570724914100565748497816063719251890222259692424569177061864273915
0622340318421199424029183579557693473391815237577641635528334821351501426085306961
8602615376654856679089907940134964462610745503208325507779296678744931233065973806
2231983276257272694157934807388047685816208414200043526057218003600890226225747853
6170554202709184704010801593800799379440589887141042790305325602725214475611840645
5249341173569177842284290407378099035589810909815749440698764933665951369618971763
2777053478856203823260792925524761629349764832127629290270428682825918950950582635
2260284916812478840898348510897736992236389168580849488699124262769198896907863613
1988092398594339813782143085467494218641492747020760418907195534100749806409389139
3901994962960631300397330103336566301913961539437944792882521826803219051714380790
5736792879104568898698768202887470484427671706517366492368266956111990553215581591
5175618580083690549620172412575211721049772671571710678426546158849942037999762451
2442731159421967645609519633392793609213625871866062288678146179395694783441717879
4263736213344014267404944191675568152309413959936592766980764019614901969704256186
1748234014856370291547851378019512739883667861544684465294850030365185777003569689
1415311563057723312401050324442231716853144817170434387305244057905044758591057327
8133229935006675869342499830702426481907744778571488298241198501675437653934214517
7448001998719028456057002211361521605362044700160852124539450381047187589880228987
5765178116208194324396426752748766055607250700832910616616807654474889712676901425
1923539954974304098543726628524753385272213496917415496183925316640256015086872532
6287201844456305593170765822755207324130259970261526428307926813002203419661743047
9117071191105425175602430340638562149448270648782856433284371971273218053101368438
5101321906925999742060830819855126951644834690357295987054604531129097183612060128
8610657218910535269206406746250398689627675920748789633395729972101215724561817158
6808371359607075093675844479379872950119925174246150678991502096616105785059449060
9645556154440524360469516664868996410602302039475000948822696409370326848138244216
9367866768706412907897682974898589734932780469675518803961619222726209359448789789
4625418515767084626349697265919084810769665535500011034354268965868046247638868180
2915666696369760112519897040432482673764096188690946383478053892951568093945532040
9075367339436058488271729033275703742926370893621790661162930015675025953222923699
1051238190840112065987802678340316181763362827984996335233531339291643082298112871
5655083833060625678436111633126987638843720636754295510622468110295824214335203024
6573529754074399474738301768416058802548417283995685171712287020456829850063136726
5508443081979455433860456702152059231214355203137517729208768953984635629130326456
3071879287495204053661351314774894332291753635636650100657845335476817494431143551

7611310189273709270755917970409806898085923567638422234660864549269009219996307981
5233024880293287745673572284886021120107928077488646491938638625147137256425523011
6828093996488003751308626389318157603642644748651674394081801936760015326892023302
1722755753049183175612314327787114379737406649904645438046006123665274739682278251
3199383287902096703649736951732073049731438935723360835746087018417714701081491950
3070553721633351162916655731587807735217502717228724662207358856304003173108919089
0690900015636557902588392565727291593721263985863117256423458466649732042852113926
6532367740658771722859774849276590222292545070589619437221867072771090580104452690
4598992358968847494035113506084276735506303908282853479390096008453771760661757273
3331300273387073408752527391447084012950062777296629274400590352784305759023229537
4432461767382616687571621043110003771908178880163393001423531238193646952992263271
9204905342714281041441010555137328044796168629506810333342114373538244089587870642
9222298304494904861968212312431004211376391423915461668530895910727570053824827751
5008265979829162496131349450619337109556512411331973020413757888507867386274419072
6751116339829079661742076701776079001048517322933507344538359592170774352767226754
4957497865387071045539896391353004546250228797639732213933264117735948896685200583
4504961945246955863745296438995690030621217679109398804717344434331963203543239476
7854269830084831975477228244647633606410883986032580589851664146439617647725504692
1613334338831423834162365731232967319991558927567295783320751705055690857378335794
5275861302275150591860346972838455376599278227921481219849830482871095138550858493
2111548366903317583850134823980270110159844105676358226077889747067604252170154630
3446850737634849558311744752102755927147692772646554712199347146446555930681832469
0364928769908204148888421037597531791653630529118353412542706499720441292370752461
9975302659293106957825399691639904002460072352608050458682719689323971256981447458
3778164744674997537731916945110887396333621783375695643694995927087063969505304656
1311614162594903931097208849926949824048802875228053512071109523509415745753787143
4740769815295062154113433646474887254039418191227128693892421529052081402265596776
7481875228190861305879314171202226475712665368375115622699445504470862873681612023
3648939122260977318591852905397735652142128920143664268207615364300062533772905362
5056835709104023346954568852125042003296549394711414704626524210570644563350054217
6986897815922442557012590352953539348642890797208297614971648271989839496763150814
1440451344594305783092365190258006610754044923676348146061858812682461725918001817
7460373825596736569715952255996821607976269979012666811604910882281863909210141629
8619163981175531787331866873161213931821710479484838952419371388998834263904973761
0088434315631715887683165329248964851659454761132739233613252886395700576435933285
5637934390216381793214060324152519531855751798175009181593907875922218033064069795
9406107247886503673175168050460664315915885877211473364359795152086254881909751513
1416136722873285408498735437889468451965632237685451969185879304354772290156405986
8488635683557695131127338854773406813504772289028093584906935889377904703588499285
8557861295460095931617147742093162707180826579774727982055318981480578808466488886
729615137948755760890300094714712166478579233908092199161415941896532733638363146
3952750548153152574728375298086596507318120959123612966982061641400708817903134375
1913583114060668055164834583145145878151738705883990288753561122809719906118544552
4182334471404837800913706055036626211634893306856144454677568640715409857710174515
8260005776344639443046333550026806078586841141271874068222078545976224832870829992
2672285315384470032821548463786713073057232979284539510817658513615854045219669659
1834770245771372908692794106513558077331297318339703437726930245902214586250475325
4766795753223301075616372490478275187858631062600777563326797724567527512035723608
3255324771184320993708087536451360708948552654372089421037261535588715821804089864
3081799299882141861684213217059820591069381892596712218129830240936561664745043826
7490631109384181341835432836008337248973432596516881904195247182699362057133270204
0934721470129437389021398657295783899288138088227123727079271690718431599068593196
4221133042770158325164001927730575871491277361053066508492906534071102018856148683
7045159231692513722329652377829395446642342230036660385950117941356203314172272084

0637003839880661722794014401066439907353032934601678086229943608391286745467875342
7492061945680994752614915982714775325354264071967046372214015464579574193756550663
6025739377673224163299249260882681873667228695589879202613725535081599353243665348
3704461680462578497381275075548113062165258219420108688656618549478487705262151420
0611369182814348463263763651744400002385155041124758632343984781432070311449689811
3702909599949209272604349586714658425753551449738804863638520905291146717491217192
7125239853993296271692646034080077158334301002677896193483380521069390732233346529
9794374378605979381047115936032233955198408387508392308175051839630778652937621405
3024385963455708727626076434672820515726065589239606433725335350954182652252990550
7851232011469688553611039284804419234927982835348243756788949735494793020747816664
3371011973339681355187561650893135928567897582733084120722955322445278723735503294
5071244922711190235617936569074986035556852772710523959580640857913755136189846916
9901015560131339330349020855371180202587600290437922380439754341392613987578666268
3019710627955150871664432804149270376545071401589321959786690968325746767280484156
6920789206653221776683899441748188446873973798272657756967441899190016890467449470
7770011493563392291484621912967983060202707656957556315112812961121698940709165083
5192458666482114428554924217657611243093817076510142815707164578715622837418437024
59653690623170180476598118503082812449029964534185593765671787681

{ wire }

Circuit theory . Network theory . Transmission line theory . Electromagnetic field theory . Maxwellian electromagnetism . Classical electrodynamics . Ohm's law theory . Kirchhoff circuit theory . Lumped element theory . Distributed element theory . Conductivity theory . Resistivity theory

Electron drift theory . Charge transport theory . Current density theory . Skin effect theory . Proximity effect theory . Signal integrity theory . Power integrity theory . Grounding theory . Bonding theory . Shielding theory . Electromagnetic compatibility theory . Electromagnetic interference theory

Antenna theory . Wave propagation theory . Impedance matching theory . Reflection coefficient theory . Standing wave theory . Crosstalk theory . Capacitance theory . Inductance theory . Mutual inductance theory . Parasitic element theory . Noise theory . Thermal noise theory

Johnson-Nyquist noise theory . Thermal conduction theory . Joule heating theory . Heat dissipation theory . Materials science theory . Solid state theory . Band theory of solids . Metallic bonding theory . Crystallographic defect theory . Contact resistance theory . Solder joint reliability theory . Mechanical strain theory

Fatigue theory . Vibration theory . Reliability theory . Failure mode theory . Insulation theory . Dielectric breakdown theory . Arc formation theory . Corona discharge theory . Electrostatic discharge theory . Safety grounding theory . Protective earthing theory . Fault current theory

Short circuit theory . Open circuit theory . Load flow theory . Power distribution theory . Return path theory . Loop area theory . Common mode theory . Differential mode theory . Common impedance coupling theory . Galvanic coupling theory . Capacitive coupling theory . Inductive coupling theory

Radiative coupling theory . Twisted pair theory . Coaxial transmission theory . Microstrip theory . Stripline theory . PCB interconnect theory . Wire harness

theory . Cable theory . Connector theory . Termination theory . Topology theory .
Graph theory

Control systems theory . Information theory . Communication theory . Sampling
theory . Digital logic theory . Analog signal theory . Power electronics theory .
Measurement theory . Instrumentation theory . Metrology theory

4924468474271341027951915311999788664118998326654754976731105762852340029558887097
8156711187634082281673883298147640437048474156832729645682881470734231797497253374
0293162397630458906043949344300984795357249095813110495957295460760055353900510012
6536291107790300757104046416807026736080754601335286820619924053318856939832851245
1960985342480365233871541554853242886941145427564955434274606167220127073372621935
6421176666512732743708893189468574644270866920479195129302614180102377238059693951
6504995955841169448467600095042664126640760729801515829572961450474227823838456603
5534949693699460695178497887363933657411694769243040710562048673187243442798006593
1539182825726211727976402884027641738338610290019884984821425648341438085639239500
2043944846167312130981795961427720319537898376588924010851653751133977288583281070
4732936397417083690586974672071569982856208103584682594338268518654742665496977306
1987365637600815519193741704340461306562231237883149558752527265609841543389705565
1391084091507510587180922908472051072213413879962265861480334821892120250528840316
2464685385682265322754749886920878456002585866570751525676988988475612568026617460
3035073754771437010572792794196979482239129415208027104074682758548527015063027819
2353257845667071218666475501107903262355022924833722761413067502134357043838487955
2419580285269531927404696221898527568592892673271927913598304453665399637863327152
9601683996506181741986220269155316972571231125581974101929689723310832194307091856
4439540487690788721489823022857686724093381188653800631870688383969907779884204036
4912707796534766844146189047765063545959647196388576015613377104918756385599911398
2992570204378068196362138150238605875708887771962604747136979125401487408234166865
1931424100824053205889867200994894410778065454226561818008239671707173666664620096
7360250902660056115909958562626960490980671154877142225047756725627302826076537215
3074542336044171877308426414051288783289188921632499811345938231883908055593891731
7392538326884803145360619817800058351526318803121168188539099363152005378076041842
0669248143475216869953637508571750545464699701770188022437307032745968721675949408
0768622558903859139407466746287077663020745217668682667194939016400624582943427372
4083631979954274323249415373675549762597301841583395405920412345950001812273697171
4532805044838369565589540648646969915197457910959222713391314564986087342649361386
9724597627128066809518707637239913745671705274182909644189832306123003293175953779
8716225172109935071216168610181527626976318550434949751333950888841491224743465107
4488300737397512968735303182896365690551420895398185148462840673023450142753773043
6692473666672543076962503044009218992762813565925332166043238663235946342624151089
6924772700490984058984393647223418992578656478599513010693740300582581031662142843
0851917179234259094276734168742644723915793308603909545736523815579370115017538593
5240593290559692321715811436756985752174763622210735365632564716656779880154589671
0069535054630684439187033338616782484054062825137190656332525617585658645898389678
7714656233715641005806825884096017744627548773618153545749744014982971919731381305
6993378248845876847092456038890666431439802774996780652782316204744019894819749854
9965682940718143489220612319528154325283360708492844623869659133962717493722924907
5198963047315580166576438552466192801673640859106403798473787967334489999617568239
2399759799821573872929564790388986981503660930250197585507963177332449419396176213
6176106990009191186718567546250357502404832157115950325608897778185114754533731013
8727008673236863742867798897528057690299830714777571385468560526637074378213061393
7817849639355193400834720915335787982281419081082532981566830369662472977787858643
4379055070722312109279975539146388600816633150802947701246561725401023267488595316
2880390994586540413424843583769889867752884562681936075689207984892536464154297432

3101229325791752891300132343040858874686334404384482606147808836586291172785115636
5877082381105685665039828848443907982146988695471180150154369615753009596903927048
5898829605478276275038769196545692256537050227738464045284349068935238647328150992
3419094645205381883183390932311371233814828604822245718388812460513436294078976749
0447626021962890125013266860609145296607820596205304018801388090683674607019064896
1305045489016263494749231940561138830292846711387451520257146492099417561120696455
4570044983685533502553351961415468322133339470973025608315335173974165228743176679
6133760282058990277810621972857167282768022748155542747366605508066445198777726915
9241530066595830455068801046183936939466016721647958047465401357168775669985176295
8486477398223536990582213095889891530509082564918704347328945892068711563058466910
1126784466156830116403112978424954368312145482171328379377670389487684025433016521
3590434769741004387755610283239718368055851104922140828657588062767143462741495433
2621644943848184371104611041907524628980947420550413873646629158946345410134505781
3224081293822993895943211055693686443278700991238990245623335443078537376043587387
5580599298969486637796031450442562945293528444339165591937614202399439193911346107
4072198525042800987957540516499058107151769242663264191004575974600753937525248410
7571969669855676473744547515362837867374475782258017707944312346862459180309278496
0492287146920890273978642643054046073331066142721752520006034980959402627213162751
2450189221500368267104405500757525835285582510382088747340997153842802609192550883
5756873137515879799768604385262496047006181030948647874121999396129264726100441506
0939784511824400728137620878457126033639385521295439563998768276202397882745913571
2963343029092945210967814450560855743269918563814192071313867112874682023031629672
5888564199906492299481489055779768670548425889099722638144142324512143471651709583
3996978070198416226562697166942232950925862867011137626098652842397609026664107457
4164490571890636097344261052439655587157849297841428051030880500385294105385040679
0113749580477549114202277193986857756783435709979810093689246369260880494177143568
0246202216554767832660983916061459825790328627659950673761659465026121846804749935
2325166784603918762461496928425262993431715684642016213584249704869722035813885186
7779407382742764248679305450155249532803622534495426757056320264587390259332451689
1330663222397791523808628327062279711350838192390274327013975375363281804787585296
3898172487706309298550522454827720861636092792037114663152098391733601833493922634
7431677833944704924015214806896238496242776226675685276730771948649631180682138990
9205530544748012274028471235031998207698446729916918783853404076182107863178617762
8795200646693546443331323887074622052750054619636541207379392971658454668922570768
6067366847147115158266023948711875028673322634718612100732235707222589755050826560
3957918912235646561064333196759789918174679990591074200430799710846464572576381075
9129950957266101336505298486911978311024102868789174289938153937411552231846951550
5316391885938129167425941355602703511915231486370097606951103080676758905754734624
3845615630311424382499766244778941409025278570479019557590859210582787438431887898
2669477544543037991658130971646799899988730289602347894409012175023954381483652089
7024290389922364266641143443106764084934032003450240528439822996628043557633767691
9316421201722531072705929016735123489530446240835328408185031705554828632796911467
3442851762347741141639244902667222638965753161295661256444978554989504077688511591
0184002813409979443860171762075414534900383932577714541569331597248520042667038213
5344120784479253680003865417851728639261706092116428333950259208312320795602963141
1210412215881865935353554404939964417777336776176466241600926157025083710226420255
0045600124235625928380506892671468587816226678816643419849559456204232528616764075
3812457432269241151313083529237482498366940305562971542787083293178976655298230086
4936748217442653530913213400013400907963594731844163481336069594382091456468676538
74367063721686343314557729

{ cable }

theory . Signal integrity theory . Power transmission theory . Skin effect theory . Proximity effect theory . Impedance matching theory . Distributed parameter theory . Wave propagation theory . Maxwellian electromagnetics

Dielectric theory . Insulation coordination theory . Capacitance theory . Inductance theory . Resistance theory . Conductance theory . Thermal conduction theory . Joule heating theory . Ampacity theory . Cable derating theory . Grounding theory . Earthing theory

Shielding theory . Electromagnetic compatibility theory . Electromagnetic interference theory . Crosstalk theory . Common mode theory . Differential mode theory . Noise coupling theory . Reflections theory . Standing wave theory . Characteristic impedance theory . Telegrapher equations theory . Coaxial cable theory

Twisted pair theory . Balanced line theory . Unbalanced line theory . Return path theory . Power loss theory . Attenuation theory . Dispersion theory . Phase velocity theory . Group velocity theory . Cable resonance theory . Cable fault theory . Partial discharge theory

High voltage cable theory . Low voltage cable theory . Power cable theory . Data cable theory . Fibre optic cable theory . Optical waveguide theory . Modal dispersion theory . Single mode propagation theory . Multimode propagation theory . Connector theory . Contact resistance theory . Termination theory

Cable harness theory . Wiring topology theory . Graph theory . Reliability theory . Failure mode theory . Aging theory . Corrosion theory . Moisture ingress theory . Polymer degradation theory . Mechanical strain theory . Flex fatigue theory . Vibration theory

3560501960047763156609515730929997946320655433864112434690945144315027583072788622
3299781527885202757646055999984184783358160537235193120520700114513814955912362755
3842460674556963824382650337747967267472953810333156714081033982974061955878395879
9213865348175812697345677590608262959291883220387405335668055072667443566055487520
3097836698751618275125917045427155533496196945545774955690264094761752922442222318
8659207814213473301594388556899174949794167981967507309522751657361259153181023926
2127111567075526381783912439883480299941289725992825167272510333302024336981100526
5508194885093448227679724779874622533796513014832790976002517186186229872819091540
6211178343369758381894255047346742325439434323532708557602156206833325195450022742
8420903919532310252023761246350957331298403393706234153522362858972537619633202183
9783062567838547058858267852104131145424159262520187099561911773621153328922176736
4885099358637597406097664501076576762208087021096218988102493393126421333633311289
9138947154360774140010400063318186887838351823334787722428200092159271326199415332
4437611267063787854554558870806293799652233081579982057886535610760144929147674323
7326849744775622278469924061317845416669665631382714105269674109677015706652335028
9838784198304177751252057901029388743154238988707524876523286536772301029107739466
3647654767072927998884719022126050439136195024320193679466794186038048282555922596
7576548500177083395640300238235799275954979463710655327802039029854168327636005601
6775392819950592745531407429750867826146874484971469231492069315272323313697768126
4459394299568660510689353137491820215461353985322911184054473864075603737286494964
5868961312386469440182560370909795360322102835471340660600622394367313639784520176
3103721771189629454831865971688817564005476596927387106791379653147827449546401861
5639515431966768321682170362342356728377412283077886777150109064564920944075819729
9236040582713285110681591785565016915000040579796956886923566603440399770882907666

5538794818335881508450834170159283150186409199313464064972023963553338148549072996
7284269462577790148016161664785610799044869880463737232302022482341214946539236061
2659455420198415099247729325485768263166349134406079683356074516278809971449404491
2277950666476794671608623815882217847560901091819145419290557485246985836255493781
9231953865224563853874055950839015591889986490678830254130499002664721823981989106
7845149526902099652044665817735660984643325398992625342647985433361189234268440058
8900724342954660576803859838313406019403861509707485763452356132053779258414078504
4688999520496550209896476610590220329072499819001671042297571157550515198300166043
2940415815947299959937410996618866956061756957798689263248914389133942770007440895
1729940419139592709815047268254898258616092799672657722590955789536211392672433969
9546754154100156276495352201834312990745532878450483137701645911816297892511947653
8927618418074432427023141552755568887300525376562486190361578127210687903292247760
7930419501707056541786701674681666607927705180739234301437875275952934699835133082
7299068816388845387396142409242451942870149050795054067331930827348859733044922529
440418210713981520216302939134061997407813217711998223590314305973709974368489772
6193754958974124128056703562563795250651565405734873300328676956456093992095892662
9149327709499867939890011885429894808562997094149252772602809954102517720091554106
0895662623849503548856693410112796764901918969942580997054695455010698584040157574
1361655120930365356109562279813804589975252459137567764318941906932077451658530445
435288503898026040317529385760255222489756478480912249241430139776829389615573409
3542719864770663507897718748782800299752881368040034009963267254213040969030894588
2714587267529013089070784148881002994724858032305350889860623014554646889315324035
8459929117641333188840981953432578818245715244625986665220165661621865554856972679
6235563184038751834758463712947642962134907904149609425344590018204257752603773878
7268550511727849323825834129693860566613022988577718389647837783724016474478708338
4147422414149867390995990015493180952560279496546115000241471324372375544307126098
3209407265856269972675774688883630980279624379241312166840370004437360654889920420
1190300706302784430688431056647163233824122511141496096851153651751674550203179390
9986162394163060518555746570681952287908484364045894200864967682584646020700457614
2539905218874233931878463088657468969656391431672770043418069122606865184353460019
2167850418650443439273775362639562000711245789581865977601427250284851729762022938
5006503467999628293502320381907798844387636447522978361172428867877804853449373274
7101879951921363125374918544782950872508040106087341600590576327140713074101908205
1414420027765719482792690688367059726526422021163997527831729052370540764088006291
4652669703992823931955757463355700281183298891040547699432711672875932043258901031
8931966963726588297093150787996860811297050392746755986025015808066017211865593613
8105803041467362800373734416526471387052694726023258507951910853513911548429205299
0941027309293874397408125765118479444084192714037127408204987622327325726723520018
1700280057779311414617942604040689165593746861176456007767810059966254102057848330
5314549206863951195021058492973794811429230177307361112741756246713859247161066359
6797182223933970719213064140217937703267024337574614414250663043311434430048549134
5349458971321572325941293258121890874799165341006134560300139200716192323151198066
6483831386334471061290139862097348114577947475761400139629006298317716048963210502
3214392325090309978487003978671363651756702149521061327518759654568206034140009792
5955324392632610143485335344378415729932146495685226321007596633648602704238944494
6188179165058775867979833771505719514119471595870204507128457601727436791458343548
6969014906250273592713029939694588790625389603752972465361682919904261708655188604
6501555475406227737946915196166922570341248211533044050477384004986391775083557163
7924775314016746406166954536200253176514994825144306041407194125145237239938733306
5310833143069646688119303009510901931107689084801726546227342134960753674240657102
152003645242524584847635447174018908703202905452236299246577057526901665

{ connector }

Contact resistance theory . Interface theory . Conductivity theory .
Electromagnetic theory . Transmission line theory . Signal integrity theory .
Impedance matching theory . Dielectric theory . Insulation theory . Capacitance
theory . Inductance theory . Skin effect theory

Corrosion theory . Galvanic theory . Tribology theory . Fretting theory . Thermal
expansion theory . Heat transfer theory . Joule heating theory . Reliability
theory . Failure analysis theory . Fatigue theory . Fracture mechanics theory .
Mechanical stress theory

Solder joint theory . Crimp connection theory . Compression contact theory .
Spring contact theory . Surface roughness theory . Oxidation theory .
Electrochemical theory . Materials science theory . Plating theory . Metallurgy
theory . Connector mating theory . Wear theory

Arc discharge theory . Breakdown voltage theory . Creepage theory . Clearance
theory . Grounding theory . Shielding theory . Crosstalk theory . Noise coupling
theory . Electromagnetic compatibility theory . Power distribution theory .
Current density theory . Thermal runaway theory

Vibration theory . Shock theory . Strain theory . Tolerance theory . Insertion
force theory . Retention force theory . Contact mechanics theory . Microcontact
theory . Percolation theory . Network theory . Circuit theory . Systems theory

4694334330861387823171892202071586219574883999099971095543421343898902836178999981
5759843132020646325509656015429462780840444158769101694003057235709142360496069032
5404941261671359442256101719909558763271591425755104305875136634012523478028621498
4581570732092199040026340173287283758449180778364240281020376266186057680507748365
0097287298829071226028537774602020112717022971660121124782669442033396272264539945
6598701936777796772170251553644706709942318027474293060874662273633957920284751498
6961316823247939148893818360890808678279417267076665704672901155610571265854909856
7199106292247733938224506357111051525568160380410007060790593629672904730973615998
9893931201498192943640215640868979839207137223507612535011282796514496798351201106
6355957848688786282115496019315098368124832592282624217867163900313930861088743984
7929686536207251104615485973738308703415042159288178637757305902087209446662053148
7962238910927343694064920306160646051686088589781988800934220896799772771560685140
9024280514340904434172558997375770914790718949412892618882583706316125562543733647
9459685527484688020253380006799668031106673504133898243268609313578050809811366498
2492667056165379609277585165184572250771153215230002475840695966956616675206060040
4670470653360396045908605196113305923462105661131576269599000312133238154448126083
6272602519941950924351070270569133471669286586519870727583779220908423060323058222
1930350681188043024025161260410339155091429883827452652292715339941876842344454528
7223129650252905312981425943849703889681309249720870566205497581364496832715537661
4987063531990270930160487706750604596829196279958649309152263415393801193991138384
2954254406224835146820292157752980477005259504184779000791683212228965900763402554
0610770259809322081604667581110411634434004423970102137413521142504699646184872901
1249691478778576869910395872088001645778551444274350671450034180244150522311636337
6126635166087825045697804926783935076911222945219344299552331893915933028939751285
2779125094089084935069070031050270419517052369518759275538240143196080317735849740
1275809847249463736750330789540105066660415811553446520154596593408127175389002109
1676126658967612504741129479539448106742715204570836554007612831427911662929634197
6638142663488029174320293341710105206903265465692716820177101797220950036712421383
6736982678865078383996567371870391926271095783208243595642635300239530429516424076
7100361552424401360054612258412262468020282621822379250379882655211457150793434091

9758243522946739376341182971155216233369261596441849212127515625138467412913685689
5901279429901305654287474279905154937132067614104879535092639694227968050260210818
7449723525083950181574880011801784937838430361192712037494565172655138619316561776
3136379222436428650824747466833472135315597001006800607899346789015655377875255333
1513361075550731956351562094082826718336066840700716702716010136293557242954205940
5187722810986453465369004902674030693730105590299015798466484007474124898267504068
0542090165906020540827775149978798539205142923191643367305370043427740253053573561
5345185959287915210483269757517505888492769977929069716551568930066570880190659405
1572075549276845789988512860205472885242828953764644916570867773430985797065708585
1714558086490162033378139304227669345750145802722207139071947322927153262222168441
9564189296949346083011681069038678412431497807902611208107866030061393272663649439
9607620645739404823277882794295144499573732373499425015745917246482364972474724025
6506416201008808291457619731125071690600575874997173683231970800350343471765919424
8329394526594253621612192627741906462396039918772991084158818290404685870336265482
1187805654149062522971489647266414579657483292068356642090185394023744044069848704
5608676232600181104418641167603574493807994633823000337515203280277971375945903958
3781766949811120388443299613692619286588793209770833026544138999921973282028456067
1812928633391532928535158064900881212910042571639868021989201831654110816666714123
6775412233259489419709789002707989590623943090299030889078926563900122687590643349
3489857744821722593224685788943222038357499161354414448771224594914806752952068777
5498711972014430422025493538905547264380489692852580023492849286637790034200644630
9084587200216043084941410553881975546741781009099445381587627244240850166455074860
3012256926714335427610887261597282350832464821406013207726587910441207870818339869
1802127911712771813312288900298345640813248307152010247394996494889665079424603387
0741722596884462768974557396061486302426183781741766513644501811162617077199451678
0643515051533907171882620618830571904979220417567096783947833499490293619263905003
4466231532021572090617395415445693980000090081359411980189349496886823246276349024
8517758557095289031715198769377364946839000091493731333996167765922255864122785761
2118855390727164131460035455065148793929018600663537660461170886931908519663644436
7492514373373282088341773759964486576042567972207872970333294497943896732294906740
6139129070523840416977561804147275778671362217742800792903904813515618823999683415
9663006800654044336544210673806649131255161520597668116784221225751471064018877383
3757417925948222046175382362029951059861538281383499183311974229847768398182654746
7174879509511559389560982512545716824853564161796357287566535318281203029419798104
9640608388941260386104378792630603243099126199681104230079599208803096851478303210
7819342579147486186150993634321639307579961762754290645550768192175447116946814864
1

{ battery cell }

Electrochemical theory . Cell potential theory . Redox theory . Ion transport theory . Electrode kinetics theory . Double layer theory . Nernst theory . Butler Volmer theory . Faraday law theory . Charge transfer theory . Diffusion theory . Migration theory

Ohmic resistance theory . Internal resistance theory . Polarization theory . Overpotential theory . Mass transport theory . Concentration gradient theory . Electrolyte conductivity theory . Solid electrolyte theory . Intercalation theory . Passivation theory . SEI formation theory . Corrosion theory

Battery degradation theory . Cycle life theory . Calendar ageing theory . Thermal runaway theory . Heat generation theory . Joule heating theory . Energy density theory . Power density theory . State of charge theory . State of health theory . Capacity fade theory . Coulombic efficiency theory

Equivalent circuit theory . Thevenin battery model theory . Randles circuit theory . Impedance spectroscopy theory . Open circuit voltage theory . Load matching theory . Discharge curve theory . Peukert law theory . Charge acceptance theory . Battery management theory . Safety protection theory . Failure mode theory

8175981048535746503803402972882679311821530940874709652799800629296791495408458194
9858307060708982700933964221652512406483039130151425701944607760541646277901755424
6001377350223030386772957065667523312101081694121143420618276380446751431417042895
2817826137729162362471490233320865583727335151102156876263353613106396007703704703
8515990550349504539755204673130632659228947065598235131822595663754308268052412968
3236098770334989228251968435267146498495453058058724626397231483786254371106097924
7144239218285499046699908568165624838337544116276891357453491453899796851653518216
7420985204681243246765690734912826956532534616779630816298275852534493214822570139
8437862164219015105231033424993480915429599387185641366170818299275103735907779875
9834301171182799055373791202113797030438500598444598072086004474560212282727421460
9502137945459083868799772797900127991104042197306547336237799325116472104278395037
8159836883478528358394740623215084376654569846961065597795535514430606350951271690
2609844936171966628505964412670786770455747483912284105251330719770805543408736566
4777097836627310851083911312406578241521784565455522052561802010107763626833229017
4250815614938936689563885696600606152202682765474306995729464734554957270343886304
7670123789618018663073290297219738314374035314907737732018376345955862762723626751
3481696428952998247242241990979766734151656297541030287639945300032813727850950879
2793375403206902562776421845689556974821319604267833230657962063503207875220380943
2949451107770981221281440076622042405119669815467425921190849496516816964553072813
2224959996869555140230088523245931367813258543664702866468695586272441037573467384
2030393145453744787599368770924436904327246046304178779648579779643727679035014918
2462655931305996435915139158309782065061551748170204992788756710565049960643004494
5584389185849297816411818008479377653707599153777496816275370139408178154602876303
8582701833011572584204418409419161748716613856050239308385019913492834400675672493
8431726526803543695045975987531154144436815574124233905543431759510733672213208399
8299001278466235246550874007629997348796261256774280418007497372685274381260060765
6065064963286619785402059371444671176922385994524083117142626581729473038616614620
8682994419925348732821296421684399793153930933098446583575056494280452764423961243
7338675704374420915783964283253447347466084992632409141974408221077839170754651725
2393374267864183328094602922748755197252033320944647466178263311942493939182275521
1003989210022017123426852603790725503338146385111610108964981640031638573085944820
1804988680836767696779972921154414191003762423134639569985751524447557250523318499
7810083243678860586277290262186313513203230789423368545668173123862785210042814309
7134106628103125968063965692173337730989311302584766887870355233386437834467717687
7945148376382792707081391353523594850661797402200155175774887559215830046988584362
5755655363395175899104592243224364098954661231406937072637137190679867053224058974
0603469207782347292870908349774384466527695723267231262542350300481021828145038091
5216502880702393498340821863828132074939644913298625845582960697624655126134642892
3681031275997053211795679573607291007058301171396486753029045723744221558020179623
2065485039169184314994967511100820015622049502576114789125979093821207439636815347
2216077971918014337124872525181055759749004238026361678778857667914691318122665934
3437400469259740695332992711747744443875423291009223942233231851763727039688566667
933842048809257848824365399686685523752477322544830155754024630009705643930102579
1434854852586383241833538940377419522871596211672861334248362080714010581975070103
3357467220497869504627712566328317605277513486422746518165851299459052860538626111
1396788061076630570650105934268612435110468219958666831483272630828453298140194872
6798828817055982063893812829846797276706134989245517874796413045700020884522965331
6976482759471807872618392879810455402485651899387470531190354529881398429033928065

7747763250427346064261979122433410793525507879602031384146990919870170383296927722
1659747165347146540408283744429403668204430175646767896204015174387231647331908122
5682499603314490227614037879773354504203685148856890066551877294764091028337331468
2057798372294886020767947731826099251957602837059728156512266819228602916879337251
8018894575161420484519256457444726963253161104412591487548474761531128834001400230
2367710013564144547457715811245025070830435266498056746871689258301719156858998150
9524213093333320009726795250848512888524787550031925854822949665536483749850273109
6640370594525023471051646075759753669297996214925800444166528533880096144687006453
4605519335476661247626618503145923120793523500820022616158973284311196767985392893
4913177684892726363256114017421849175770181394565323466240423531273896307315030407
3705071753088649345971910581176094232636946318018700652199182447873179435134838252
8180287692530234605415465666312996068190239199229864430520039834526792209640202341
0832814054341555650172784593588876570810568811451347964878369353105520308610841011
4506828380130957192666680920394951369026104319357817091157508080976466940689064175
7458349753644283361136024978691818833823639709222725089401635138467528341290206120
5670456351576576982207554899463890995312545997048837220584966269812327409597290733
1059087917935977454939381749742520158829267275873260818568095549921936229051059732
7815427296837741696129280803217657397971547734637933140145344576362308867509271303
7040106884782413309294858438783315561038514065520677204691971774575309828696920668
9704291867720355129149440295322626646855398587049732081809639781337901169870420891
1435704461924581818780031948743842309115100368628152178503161437853247999610545640
1675772759854797131958878722179563269069241636894258074658573735624753855600977669
93672411297728032988333359189765394317348859175198348511843655363489

{ supercapacitor }

Electric double layer theory . Pseudocapacitance theory . Electrochemical capacitance theory . Charge storage theory . Ion adsorption theory . Surface area theory . Porous electrode theory . Electrolyte conductivity theory . Ion transport theory . Diffusion theory . Migration theory . Electrode polarization theory

Capacitance theory . Equivalent series resistance theory . Equivalent parallel resistance theory . Leakage current theory . Self discharge theory . Energy density theory . Power density theory . Charge transfer theory . Faraday reaction theory . Redox capacitance theory . Double layer capacitance theory . Helmholtz layer theory

Gouy Chapman theory . Stern layer theory . Nernst theory . Butler Volmer theory . Impedance spectroscopy theory . Frequency response theory . Relaxation time theory . RC time constant theory . Equivalent circuit theory . Thermal stability theory . Joule heating theory . Ageing theory

Cycle life theory . Voltage balancing theory . Cell balancing theory . Overvoltage protection theory . Electrochemical stability theory . Dielectric breakdown theory . Activated carbon theory . Graphene capacitance theory . Metal oxide capacitance theory . Conductive polymer theory . Ragone plot theory . Failure mode theory

2435409830918790316953113645224309966273844644604878523373500988417621881330633612
4464042544563824823253099542890011894994244473666523659959130177994417903978053095
0321243504808413137301046871203629322219071750592709211059605875017916806648715780
8503442506829319758774381769762942846117471269232521013508195558928299295026278859
0571471729238714096324365391535352169933495926611714985064059652332269348439682907
3036007978285939949222282131290538775959628745530761553777850692745520753483605532
9000356810027748543987119649621453157555086154424493240338659740352358683422838643

4565826873809906741833236375469859574635603505676730561523842697833061949090254396
2094566319070674573967274121680391039221364903592034102829860424647404421519497768
2223754995983706363149196869134350528339692284883478807688828339640549196983293886
2428093332068609317598326988901277184876552640746854989635802261906410024076359124
5338164852633543756628416393868737252601029972568895467734906299961983326881987754
0245319878703539088280153131270990756996618971417591335088736526999318430336900555
1138753063198575956648378598283712487363066266220277099395690106595139559314302246
7391725859776873816488073401807346335512424559101483185255522071410215858062638795
5675389476994407947877794947232022395430280213313488394963168885278311352499651235
0866326801843033763712043730770773188764476098733812398674191296558899000829451764
7990861389251115234126374229390383841949216488444374450592511094931995486680390158
5385987008586355027784337886259003947355971054047261641654040445662420211458204539
9829525464597198966922444377597887461848207807360875457370802467695586229135469722
1499319158196931384311970010136232264370155782948226994712817769474621419915869211
0916750740907934844342122749691221144511037007786534029157825453053848752687078731
9903241117521267034077279217657647374745758393934833419342708401520876879651517890
0743015151946133901205982242006039367498902551798270212575283358983162667093629372
9801153781537846811428901881116794536435079551131784905741548181459688116347453849
9917343844185149280218955460156172422996857522509471135772937866965270642371268144
8975335842311155816353742416064194780254027881411584406094577020093562504416552151
1558846451203682411795631132707915805628441392973212249640914653571093243387931279
8570742337912940812892325895861230507591519049149131062351571398247472831932439903
1932213594175718072854552333561200356315953922982128725521794894180774379556675023
4841927221852724301420936330145630577197395331291188306600950772989214178667016891
0149110776608755290890373143610341062335868540625454870487008200406904010989997471
4364832050538016858589437968578284409506708617209493044884412200031965868396786124
4566017298729326052925831602869215755824836622661873677845327269522451336725235615
5893148973749384971917743197096288993484706747088238782585747207364206839528347914
2635046131908348339013435368950952560533111175247437505840376469608835853413772719
0054446188385877249320804299538211286759438288938817927989137566471165070362787032
6156140354617457062904887049227920958097961052896581058421646223532370805615049733
9290643959351460294993783360710252225400216188290496378754230684767152247943386972
1140313611055630956051857133754247377432937397404163700348232189491601055766528001
9565151230180331160036780634476892801424849098579455564321660252408521522771889271
4637943573694733656186702819671340918072432107052165723499725917649293408687523116
9065540899277857064218378695402421509637390045830945526799312255576287763732506513
4858731335622482420003847040391576538700066771676166501649668569061070348452038784
9830972699726358285568996794942424713662397234029957800157133285479694754669199165
0867883165516438871500871788878376973765243495609263265093106029463023601508697431
1210155779351478661709574211081520393451029385288495060812077237147641333136970622
3895098318355314893471832721180456255594805476556558217042296958499083184522560621
0191866739616716763411219623412999636452177864177439047030547241829334800421813656
4583761645637023797327064227431813919165636485830107035899705731079186259659982678
7556969646662111671914528722212046670310352503290352963728421445447379763622016426
1636117901681853890039835502115657510162632786275262450362979333464356099423522353
6455476245903038316688418650078749950454607559590874736510025799236153818212309883
0237451440555747458726017746900372397567950911942987527740831354242645990866885991
6202508139053619239726632149831053229423223895495956753801944636138047103167921293
5154903432373272082764906282808915778093545611335120805590651380261266687642106126
8552760003947203549008691635233542661896288172388015185846766306924483781742304875
0040105063284130212371604663982335175355555420818708675041942546691195629189130559
8013476265383150768506166228941233238766345337505994432606526877237365445317880629
2489439408766943365216393126249408126228752521021020073358288670696240219847831997
5474203945846666239806914383437255397383048131472194350290789942132583881443576270

2676051849092080475424411927859077503444896811721151380677111572560789966630767067
1793937361989971829817763927207942981669558279941719567715354779823490663687779477
8365165935214921765237731578655786092889774286323048594463629999760763543789659018
3395029787669871260693281806863430062204231010716809902598585581620969315949138446
5741868747855149826887681754041764408215591941035818655630836107381236576667162796
382045096402331817225121

{ switch }

Switching theory . Contact theory . Circuit interruption theory . Boolean
switching theory . Digital logic theory . State transition theory . Relay theory .
Mechanical contact theory . Contact resistance theory . Contact bounce theory .
Debounce theory . Arc suppression theory

Transient response theory . Load switching theory . Power switching theory .
Signal switching theory . Semiconductor switching theory . MOSFET switching theory
. BJT switching theory . Thyristor switching theory . Gate drive theory .
Threshold voltage theory . Hysteresis theory . Latching theory

Open circuit theory . Closed circuit theory . Short circuit theory . Isolation
theory . Pull up theory . Pull down theory . Multiplexing theory . Demultiplexing
theory . Crosspoint switching theory . Matrix switching theory . Switching network
theory . Fault tolerance theory

Electromagnetic compatibility theory . Inductive kickback theory . Snubber circuit
theory . Contact wear theory . Dielectric breakdown theory . Insulation
coordination theory . Safety interlock theory . Control systems theory . Finite
state machine theory . Human machine interface theory . Reliability theory .
Failure mode theory

4462271240371935960306567827162556558207039030784077114260606210507819071526177116
4842499207449418305408512484170744606730939648465954525761981071777347905218690419
5885043684585021900210363353134835439269598948744257703577320435695364958023710651
6847249228300034135639757282699787719849528883768198829158183063390622825696943214
8133889877507403001716568006719818935552931751289901278839020494615047833903518787
9741511700629947098936797446268433717634943413928030171664179639955875676378294117
3562670001302509652136417533971295958920683989112011688785521291514606616616333540
6163998347613105778390488172783571304782999494465253430639759349027354187277403755
1962960162682273227430181091497933770183570328713650832392819163895002481769354948
7422314891296391793246922495087475147516078696360274950271678197131795239396690136
1418560009300021064614734004722715745913903082549243019139664856343833086323452750
3845823644821731039095741535285031797002998943586506537286957766397569753892223505
6978242475313302204801465644542846220054082038742660704338960614024925104756740870
8217497003534338009768346263438206468174303316412161755679423853851170524174295560
4324571646479164995812050159423503977409115520371464462243325199186307717620257574
0614061113189852130099244138460961175189938584417754478458244846648365274421432436
3676338827580997566587734017576897788670524083495493742706519321627409199999258955
2653828931518628494542439272720674707071251792733533633606467656613508125873149936
2195171618948369033587624686244273330105205444882103780701577185459502953658580625
4579682275329226259548051095314950419173817110385637482269516112987665228715704116
8536745191550482068635772335634299195863508719423502415092933302910862036660515846
5492437790448492584515154965883761435587550350062388560545601418127984407575987283
7768953742527668119128585010536689850416886578179518135102696615006246232121281362
1923390399237605730787095757626702909267026557688369967504021738806600071326317383

5053746718655206762877765060120580935670937535235841584158085513142989232012605791
6076958365466936542592626889630565708944985280114699956310829070466836182801643842
5613236671608435099554409270359844006215398540416694673087331443190562638695227581
2326141176722979299109907162918700340838755796747190977083455959671727506174149328
9094317635245160970503500971649321530025956902665271635033904645207617904241454276
8846327117111768330355525747877400887908267655497673993686380315404962301619008969
1905047667002391885826730392746245167552300995977569798793503994921304348390678665
0699717481625685757147563869217899489192289537728723050344960645407502618848052109
9729204788809856180175235997375118742129574685924717025488935188471194658486634147
7188969025441643613916528428589889445743301729177566168040294619769205615895551669
5840977775653252446773720293186934237901617640727799630829573797576348480359030060
5229091649858139368182755141140398320262189981486728527201626080956463804278245834
9044837891103394226038171243281978532454264702406396431574900537132367150912831616
5592313931240359587884483959808029593511007991416530738148679668193098138372352286
3075076879594106289598816785624311191257898167944768687717667673091668940181608597
6691094179342850030027583288699614929386267796073072721021350504941932947977127698
9028963426598805886369683369298704347727583728649129451912738863693485997330388506
5457004949871407021198973094435088658103045422971064116200423520125588899608243716
0857148691063960788739727253089026216829153748664727482703861544677301861989215516
0786048041585255243930114791431586503569202807587223982033041598565070966650778169
8527115560056408669456009595063392926384619338856494977827838688645635903472492763
3093747190483809631123626014124238070254913440763883517024078537866038609925537899
2495754060019080788889072437988813569284284926037550053378445278871600621188473089
3284354746317389326593232688905790354893913845209184992260359707830956059237265589
4053167778218585492866374387860861977281056546902361245062900567796069111506092082
791681219387906760167736912696582819996596293519456348540844628235723335948338604
8774691668129802334628862580603646121798767864506694110012667659908714416134768232
5731762422476581562196446231212623856915348367069869248655462308664975951864796804
2896627413033730203838635297906898611124852399031012743527454107571808547807869656
6973253655887801267166433365415515123300685343328707938211292389013989564530058247
6581361338226477715377610823468653299885789330223504690525486725498958028958749982
5977844596467189050110029661027879756704787592715384316993900097095127886108727746
4976987598131428753333582538131383403810807044118528374992705986526177854247815577
8689763894030853755665734898043122660843060376302292609442230055039302196127307479
2768646034635908117007624325714066722517265621575880919060476163419924728363273092
6921772248037197357596046364003480108395075204795151771805194965569175297797279331
7467207963144357856680615437090918913378445223572305336395670403840311014871506468
4825661685199300177620558185216778809916431320457685972286531726766892514371180686
3095219466972161367730300112085864045858528668583280170007447331066167060368747669
1873691612142544916301119332589786331671157880831885899693201300419425633883524999
2075447397629732273590869017260919583274164113029337210017014267831240283651151125
1597578612043407097636743815665198663285866471177371770805645884829365545778974712
2345609086518942992445281076005603714304868173365924351651768620135718491201246039
40580349386993263726751960501117901231898048311454191010924222660842888692243038
0018314109782911991420388736031348303769887777

{ relay }

Electromagnetic relay theory . Electromechanical switching theory . Contact theory
. Coil excitation theory . Magnetic field theory . Armature motion theory .
Contact resistance theory . Contact bounce theory . Debounce theory . Switching
theory . Isolation theory . Galvanic isolation theory

Relay logic theory . Boolean switching theory . Latching relay theory . Reed relay

theory . Solid state relay theory . Optocoupler isolation theory . Load switching theory . Signal switching theory . Power switching theory . Control circuit theory . Driver circuit theory . Flyback suppression theory

Inductive kickback theory . Snubber circuit theory . Arc suppression theory . Contact wear theory . Contact welding theory . Coil inductance theory . Pull in voltage theory . Drop out voltage theory . Hysteresis theory . Response time theory . Transient response theory . Electromagnetic compatibility theory

Normally open contact theory . Normally closed contact theory . Changeover contact theory . Pole and throw theory . Contact rating theory . Insulation coordination theory . Dielectric strength theory . Safety interlock theory . Fault isolation theory . Reliability theory . Failure mode theory . Protection circuit theory

1601773462191809915887003701380795497143850690976292685885041117712415908472796403
5425682063803722983028376809095736578817921781397801326241799224071028634975226410
4640067871421722561237621226831708203727624119802548466770461283093003525404777452
6889847184778475870314470536099474827902944760027912823567338764505426732393922702
6989919073617899308913721887441923936886531481518799001162396160666383005975464999
3039056631638817043213337957919424060325682723856591372952623312891766854753939077
3838367051252793201286306900050419760806096843878842817983446295210138996811758818
8905019145257127643601226813439402268805296184916683772450377812471053169889663859
1666062981423485124265896900743383553013662178642014295995524714613279279541424475
5636301588019841853142115578616407565627531596649151638942308494702657339602636498
6509162951707802384244046057509086658000945295740314899071984313836642790248842109
8352947101833005564273194385186849540158358843472002900983071366774102254699552442
4543926850568061188278723288758319631529713235703598733169064329090671027507514007
4394262256084123695494067923556842267938189685406631505435639662695167454699840021
2862133082530966301399016807080608498898011481043718425067960821496151362650213784
3752280119276499652991158273606092952693337489959389469647923741637609051234438098
1641469230385674736854390152005089318187916057796129717069995136510260361754123282
2344970649675591193989993887692614984901714287547405001463597114055849736031525535
7447156953869338483764668507492922814741419974598993258275818409629166041649662139
7587172874589061109675464333038451032559995443065185461527646570674604537372830266
1974328033891572825517125930806811121147629381425276548757884742679667052135617963
5173404113447736254241756787058446091623305640136000560981444149439911603980569176
1966408651579112384397981759299357191432386557348969612727862588651911110452354452
7704880810691407664899026236492440867216541882015654494797202269706173718836141991
5953994792536958370745729511304212048923151500388506747647944166935604712292350208
3495654376716822018099222776779814448424607296082206849190864875938764337149375861
9633335649990958929267426717988479081679546842725147828711608443085526413236072950
8067946804495157766098316685327735998151293418965075797758903342308948571219157008
5866502316969354155673197832520370309402387924470352288533035095726079426617263578
8124678077326438020857342986671608745337763197305571507258081897082506829988240835
0893689726002853567296045224976434541726499869516914846241121363812724518657727429
6116543643714039795863676376325219735853834050502895454179196636515978084829879144
6311082991042608854445954216904137374210608034849750075618312867170074498111685288
3393872165300061268442404515307765120868738588892744585577666543538887137739068091
5530138875852695807342454364552774173025859306567496355504778168592847185310946505
6030569033559809558235702074697930960839732392359363877012890060845006307952123093
8808240759373985260658242848383317825947784956608073390594358837991246033124162289
8525484020156754128527063110637701795879146437280724689628397027846166075151255146
5151825438223975677311516464877266120931227480178303467297295563172749947862356197
1984467940551862805228390861036148672104463651874232436037025813166936234894456366

0022164266986544255155901361949411032703281877108715918696239227988251396148933357
1104940154564701979982891891379187614958428935077309180060321339856681884025731853
0105176242737723601526782680295375483509088190180461102445695703440333244401052252
9084362256413604564752662168573776923088130360583595997854838802793366067758637923
1319943564973295445000217141584260314036623209174597886654496404980707013787583949
0723277182875672403005779746616651839323820880577095022820206667613451603520172241
1010461098614933742880183222398498432993442555445057672227380931386524905385131727
5578218241475570279636500577697995333452335798293509366625591113786862765166945527
2978754161257232785669858111071363474829951301526234778366508202820548279482046741
968300613216370451302331982934902132564222088821873628142131082740240634015483197
3151708223927757004735751257707206871429371991994553258744034485136283372292900297
8204647708351853667130777284529407040790821872360241756511610255556476212150809430
024618902309868021310323338226196929003029376876883235722393703140373467198058012
4053888013575778035644370993795366473904333959994949104378135275600682149586726191
7743735732906170342738083586947225147447327313926518555121952726031429576361490246
7403546284641020903726621121908182602300476704651139829356486983118264628864759232
9297406863587745957201054144933514477762233507246892930909730546035825986396622077
3432304800527707449303267955228356310603925143340774588822120747295908689158791622
3741493614713132646081920903500314871195996628031483592098839903145139119144548064
6999574924079315849330572523357777006958561344043145683451519924701606233111505100
2870718222818558993117230669672730721063015371486678977658178493374248742642551470
5261144316765946429520791266777654792287639813515487648464924483937030964505161260
3627532593408929

{ resistor }

Resistance theory . Ohm law theory . Circuit theory . Network theory . Linear
circuit theory . Passive component theory . Energy dissipation theory . Joule
heating theory . Power rating theory . Voltage drop theory . Current limiting
theory . Load theory

Conductance theory . Resistivity theory . Conductivity theory . Material
resistance theory . Temperature coefficient theory . Thermal noise theory .
Johnson Nyquist noise theory . Shot noise theory . Tolerance theory . Precision
resistance theory . Drift theory . Ageing theory

Series resistance theory . Parallel resistance theory . Voltage divider theory .
Current divider theory . Pull up theory . Pull down theory . Biasing theory .
Impedance matching theory . Termination theory . Attenuation theory . Feedback
theory . Gain setting theory

Equivalent resistance theory . Thevenin equivalent theory . Norton equivalent
theory . Wheatstone bridge theory . Kirchhoff voltage law theory . Kirchhoff
current law theory . Maximum power transfer theory . RC time constant theory .
Filtering theory . Damping theory . Snubber theory . Failure mode theory

1223070752382914734506624999538428815020387407276272673061691118846421258857799104
8908579901862357393482391231276567614918586889219043918616735987405433996267580949
0420962152322189129684932533900215660892144034333492081445337195760830592175141981
8427919732662075944558801638939048578113032653561866395460578436626371482759695404
6105801413901347443779758286102868850797288004057613166444051236008971243523370718
6698317117920938052236838195472331162119805498753061989120690367556610573291552869
9917321507775953766286439809738270294043896954076163296592654024894196159362145542
0865778156894549955932398719987558870130443894967376482413587060639595868447554823

1087923389327179482454500183505655313711436150423336175474400073766917635496614204
3583526701685498460763367329128659483440946623786695522256960464513718017528896289
3708439706338146445998414007245520811400887549182422948780855512095865223214859497
1306853970164577741489813900529396308427622317028474775308372802282979252918482275
5306758435483436059188241657969826940959000343537023710634190414660370565417721469
3361636847119746750168702378415779923009558332107482438749388023380129030615240525
3674249113733290378453458019184057947519149008066305370397519878908200099835364959
1537483211471983801770076069521838505925065103173937460116485267002541809755299136
3728848750634651627301322065504638861577528854306426806721396794102370505340798098
664659310732360922671513176545388258736257503001857688966267935613495252526557114
9767956639046608003602577919568540785920263602042042940852165922063435071869409229
6947659520393850798335633271213352557866924171813169100533592553827788747060728247
6945168528118681501364835106852928130018362774958602741166833467503689705522003461
2495225577103014994068825492062579643518920226647017667613796192907159053905214444
9388512378416532678228633086114811146493575500848969201088723518224774101961060504
9828180205444482135129653652703331781932725754866760649926042308992857651488477450
5260490198465019863847901163454062885625720685347767480874267469778636771056742933
1087945666753421299004849279926211513537098202866720770511806688660549008735319684
0328443132279648934124446141552066259906952586399970304209169856873771964912762868
3963177550403756721669937623010859515379866829762290176087389305002704826443564765
4226467630537600028157867984282874661686699148504498991963077262204463661020734216
5860437565583089131983850278911512441843251860878531915537598866363322761640950525
2447671894545475603074790619452127321638500915080464381600542407076227306256240502
0937337351965648758401027005654670381287082443977538532317335334050279101554019610
3293057981186997797260613781736759319742270811961528502636479263274883573941335841
0450283208302391468431931623547657233754433962989446332228299267517307963258329153
7869149610323941806160561099291473172213641551995954922852629662745247313434446941
4783120516793270759100025279042781699907622150279241494966885299413331477021777045
0277530298548999674355226099399581655392379116175496772311820485275394517590171424
8940420458515252232784825664322708208866939111559090948384657644528193021600101005
9625405781682611735824292859856371479935871079225224690268718821329377268675805735
0568807356762445932641094570715656926070172019389202557390455533296619685333461338
9846669820548488533302348790879904054945907534508292553786492734162374162984672096
1454589215380998619181864276366511829745440490029735266295239431654123603972538929
8796051474187540806155052090713041123399580968738396771110334641859361971539425951
1367392332213903683174212465496438618404499676603730233452083593841460793655606444
4737928084514443628180367250992629455489249923271313322125210397754502039075106722
6353874813171726040736646484327247461740264906454561340931984236190388715747344387
7892334668491322150144185752534755768516523845317404412327637853011676645396214917
2077120238209482760451301041383868379803552366934597356076283741231598554561690134
0545239394206202447987581334000730556038879157768465577617881720740861964703917922
7058220690726967955117069473222969697361369212648339628356435135600759001936360803
0465167799046935237794644966740941262289625576753893781789673727590503272260768842
6352424712264070408454605913102707873422823904903763534961713284591061272335845196
7493810443284984701505081496224765423609998909786478655397740028642015048818517746
8824780570688921030193110767593382417059612206360758324867382009792131446993598573
0429348590869871042196002104667254372441160627339521150822323106726970847320232905
7705661590098661894533458584567152905895361520019021951198496644399366124061203213
2453253763510951166360699381367122830049945344207038377914522274116646739216504634
6120142181273320182052472103223956285787280155090794586128801670014797208116887501
7675072741905800803395271709083496705630586996454616473323014054835921480104845541
7891739985126866111270790944201292469116027314409460216450283588908321873439566087
7896043370468117412710628220955001438330044419756376962082047379683593701316654024
8481483770363929763102656617923731136082432680748655688749497979360633609442033563

7900212146646009114534280225277862752647276581534731876733858944680440775418519329
5516692736161098641200517849000265051952950316948544340145861403622610963570766490
1143692581899061093283299178203121858779123070670116574277209115697569197623568066
6987361937560763468638762138426420134076372969365462792193496353790847108366593181
5434934894537730562546326158692863789173255097511874740675721374971978147323618522
6198785495357869764936394559950529742681205318072006139181587447407227341253921213
2885103743219260225714668165855871262301787884605523127515198098436431384085989842
7043281194497718190745058820032023398272252596988118030770734656865038529234078251
1387231228182320391412518268948295826781664181668419227164130247547366193868074992
4385390680865159911881011023306574732353330257741274182702430204146534629079178375
7326109901225967216003723583510144591470859201076651934047700689951951159473739710
1170971969525811034862009779751022731642839772015286889496922513002334774673771250
3160902021781670739899633159207486296134694930367609765444686148811521757789414119
8833437765926944050752797550884124869516031378855587347661514070563023015653333626
6250537407887644832213822487698152965943033125644152265876105427418866135314019181
9477947895171971156643786126985328974950772971877773573457148777647408472563006529
5850878189964562539220968045786659723883074357615832102330736909285028788596843171
3397086577053499784846418899074529023169975078222616902419479524557431087744445514
6112001739524275880867682866302024965250641126927489994735975015964146461170437299
4523915127688591593299263830886926059202940145124957365677429006599580001081459932
4241725117400071102574016315437494802987102691282569239635603795561828065465355971
4392120840119023206217505

{ capacitor }

Capacitance theory . Electrostatic field theory . Dielectric theory . Charge storage theory . Gauss law . Coulomb law . Electric potential theory . Energy storage theory . Polarization theory . Permittivity theory . Boundary condition theory . Displacement current theory

RC time constant theory . Transient response theory . Exponential charging theory . Exponential discharging theory . Impedance theory . Reactance theory . Phasor theory . AC circuit theory . Frequency response theory . Filter theory . Transfer function theory . Network theory

Laplace transform theory . Complex frequency theory . Resonance theory . LC circuit theory . RLC circuit theory . Quality factor theory . Damping theory . Signal coupling theory . Decoupling theory . Bypass theory . Noise suppression theory . Ripple smoothing theory

Power supply filtering theory . Capacitive divider theory . Miller effect theory . Parasitic capacitance theory . Equivalent series resistance theory . Equivalent series inductance theory . Leakage current theory . Breakdown voltage theory . Dielectric absorption theory . Self resonance theory . Thermal stability theory . Aging theory

Electrolytic capacitor theory . Ceramic capacitor theory . Film capacitor theory . Supercapacitor theory . Variable capacitance theory . Varactor theory . Integrated circuit capacitance theory . Transmission line capacitance theory . PCB capacitance theory . Stray capacitance theory . Charge pump theory . Switched capacitor theory

3023227362659661090850084689634930423203095037631320319557833220311610586145450924
6235337673516358934010260966503664090254930901083099857086155541616220960644822204

0161211000199661235881965662329201996401800812614697508421802206077328052510701195
4524299198036961921270459630955816468591896048214602628856486906288300086270001812
9477532197436869729599381255442261384616622353009222550049566147680671839865786834
9133612288328513655409393001762442455760361447068307080468235678698384358061843326
6146742895869417953940751935645952300602669041331827828807095016085003715739899285
1991318472154194254845107131851199918091597875552899822430717585506154278414500775
4375985239753006400529530075843712455786738414161831395795267890312425431092950670
6533354169583558028059675902072952441004391365555786374759833203419778712465083365
6724601073209546998216226709988144959399289427043475551930574185427818058960754182
3500934276357984676815236918654687437631525061914098324904951738991740940126821906
1972812942726115391660895257384419207423165751308653279851709183074898331410469621
0892957923272833566692834724006955645075857907083117079511202003752458766285346897
2759713459518316355378641907994683751801626440722351074389988922942527227275117654
7929006864221317313961312471754554308471459382406411974517269038211758796095571323
2521562747138096421392303839890455179716578202095726382897024895355974204228918382
3913480396747350185925340320404959147469246672645101011648317527342357286193358151
1442074258814376450099058343833099806190045580737050764271371659626604490194070090
5533882104676670503927302092088823803122971093613493892401559270008748385939678496
6301586801678133231020731185560970128259102153815816276963398655773933015253644961
3075542895444152891951412977763563718316930923063723008249048361289105211525866435
2776541425888107367215148719017210458697691569944025919271982915263990148528175660
365554628114955520043342002752553401400274791244241586200492581089743091668695020
5030361348730257831235872110421869599932834817312910316665720017056001427342750910
5527024098878251680764351004041274203261301437452435544840339580665916451661356101
5013041987469459110839385548127644981710902063011833335202029438282605324642276774
8609905906151329882483120282980415206512037333239719842103238277218719269743837767
0377292919273756266314302657072607303287821820282275788900359472759746428771908938
6736098550235836805263547578384705897034834440099231205993728072048366167959158190
0477202959976951243007153110664618909783376358753509966084841936969822077257766273
3941553016268917636503362121634749399052986758910307564704268465291793532200036955
4682254463315660518875873499479954799344962095039731098497104710210714981437390890
5988698964496979590687029568504626909634858264193051265430708906992673687483573819
7890630610733898981065072937601942213769975230168665298824829048047481523598530703
2881461323155371089980976938120724896437001644207551516385751580790958178836749761
7599250266475942943860473065786918874106649919833757852037912210108523948802409696
4942023920318660715022567029087267900022933964171522887809398797641233602070657388
8486558896100986026117995329443967594205688902423267915345794464059937751166803215
4724591779097012452030484570594202605111800926851173006565888691513721883031961494
9039408890202081650078992141154967098221152654507696331488508199472578265690076646
2244045562944862300874921861501073123941834489043181366750469155047767745808273816
9041640939837407021845481489722450944735983879148973560200355033120455418522257288
5685565848401748936250023199287983333995653356558589947261406907340090257908328708
7830949513197695328800833480739119344898259067683468988314776587822267498620110444
1714968186810711147003338130877780672189651543087606869654319586235936408888830625
5273882489076752928646664781116910764480679707098280292039195180810539385667157235
3638937597627817365514821673674729029998073607089250949982699949035369324740452274
6888524868175534867620894822817766125038860637057810080173386622171067131718646314
9569755440668282451405641356190543197023887983531289737561632485523190924663717727
137558275468641497049869358124927019152060663808590401765341859316533730686747831
8211401327179009633115140443419612336096702851555409380740196537497984241731556195
2831673382174359027001900538509013629784688586624070493191594660362682202267728086
0946951895638887468506762127865472114691760851120754723835803487426141287380013333
9484004990770999652170209252339269105063173879297496254759604754417259526116873788
9290114817308293385850267805034173760241892734217280350592641914102377537978580870

1222731111495558545822088311403051480128747139293563711442818871813121094123634673
791209699157988201038777132727955018018066258581466401462616957976602139674036353
2044270839758812644961064095712378530538071664590297405671415332137321290675116382
4975465687952477289213041171151463795927362632718021486119081629456577643975799729
3063501415687533509749516808313106505910176281958708887930872256828073283753099634
5852177358670527913194802338188866830435533310365668725189221948130567110126742567
0346108163428687943810148299133008770809377258158664900853324864749091509229009881
0578294036843509751183644868766780357826384972813994516769419852195216658600848145
8402543308694378469946115443928550789245276884274924268665508779308632383178633712
7139325593774078612078824623828910573093262395602297020040075832372624986564172272
3953445466712819230124431697951502172405991850275645440826655603795919891767575971
9659941721044493167209680152668547246501556111710875803887013484367035701404870427
9853518553838538167447536028008685729618680602531988537141861600803350636733435987
3239624920021939643360608311115181928801055568276242376917025862943771061012261169
6041798232323366503285905644191598312024280742304408509447671338802504670074209365
0533294855258970012752094182777366147932159495247175117720942866770391968080589210
4078587169559853667731423122922394751965219479115432374302743130722881757761378397
7358140925650239258521093510726881577458108219665228138772437341800111104453549648
2142888520592380642534080552235446940574687228477722978079712335168827017472823494
5029207154431855999133581329026108180121359408157193915936824076664970808642621240
6380721477675089975286081384222009349919353493800248522465986674674197171868124945
1865521932527786836187334536966564996751647301769071013038577257765872052944149320
6210301768051724628655591752153258401582634044880824055228830365961897304306011626
9763939586001098378674950998808475682333963147238012127433832358972697175867819593
52131241912907851685926562239553643538236468777

{ inductor }

Inductance theory . Electromagnetic field theory . Faraday law . Lenz law . Ampere law . Magnetic flux theory . Flux linkage theory . Magnetic energy storage theory . Self induction theory . Mutual induction theory . Magnetic permeability theory . Core magnetization theory

RL time constant theory . Transient response theory . Current ramp theory . Back electromotive force theory . Impedance theory . Reactance theory . Phasor theory . AC circuit theory . Frequency response theory . Low pass filter theory . High pass filter theory . Transfer function theory

Laplace transform theory . Complex frequency theory . Resonance theory . LC circuit theory . RLC circuit theory . Quality factor theory . Damping theory . Coupling theory . Transformer theory . Choke theory . Noise filtering theory . Ripple suppression theory

Power supply filtering theory . Buck converter theory . Boost converter theory . Flyback converter theory . Energy transfer theory . Saturation theory . Hysteresis theory . Eddy current theory . Skin effect theory . Proximity effect theory . Core loss theory . Copper loss theory

Parasitic capacitance theory . Equivalent series resistance theory . Self resonance theory . Magnetic shielding theory . Air core inductor theory . Ferrite core theory . Toroidal inductor theory . Solenoid theory . PCB trace inductance theory . Stray inductance theory . Inductive kickback theory . Electromagnetic compatibility theory

3814422719072160743758260453626093641957980782642900839056682608944975635196853056
7626915184027487450770085146716330177660746581309247803570756024064074624914144805
0579567299031102257636643526434430107555146260452684415977581382015874071732498862
0389285493213296565495991856132851256838837344186129867123770448733179855104809475
7206690507463672683555755735002510452533746916916653016131229888535221509001152654
4570977935370638069363927147295664860174129990017700935735902470664817325482014909
4054647499107414642979778975699785638588745286148540160898966575624181283137292251
6122735095245022914341825232232647859450941797433634422911921839197591176032254098
6676493694002835155080716232823107339388171200878377657031236590139301046809618701
8803565577662132119283585668582901659313480784865910235042564408468040576275020098
8370988349009862122364965761001433771331037387333983783965278628949251409427778393
0499003656027055552193266764519628463295309764606205938349946733334484578151837401
204280299706829254185142444425953078106819942786793506770088551134429417083672749
4442401717227338335204968286402833379119823534933368328297436157194455675448346160
7328399031078546264427732980817821801216990870941893104946640436555299175991244050
1330852964742993942391904338161523096202813180376960578380248911176002433013163392
0967328983612958299812635350138905790915721662233056928960151454556637585070536134
4488023721727093076004185561511872946908780492208796978377452001108139677812865993
4506560558070345051469223185662359288570197670189265385204182043363503383755323154
7646601581945779400704572475080737113656888729859640059421582515610811622084537535
6370835744771186040127347787826497034826622127168765821894485533619585761840730472
8474603868258248521303996793119098266956598336844066345983240594712825673551080709
7007187941190011608436279333426451097477292036381603930487391878324276035207978513
4488968374698189562697670990082308495303980553341390209299144709377343872232317571
6446279688394278763482396039960912797437317796270695303808682102827801568911786814
0236076346092419165676870675106304384672276993732678626718160431311365907571758720
1077841682074368894728120276883426778004451781157608617260437408659159042679416302
2796796949744181938635134971190844706574813556178637588777262051733389659360482040
6909110925341793912666587951479577693158326150303230648517164627052249721961545480
5025158369972052318589723941530030030975123782963394835886722602827322877543507395
2116949729066849097421783047509925131721097579473032970780002651246827859902901142
9447996122396491669702106397660438363962826131376046784364921442848236134235991915
0456395138152594744804009461533127480870624997767611914506039757780081433141824937
3973611179635102791352198879301017015221860188373053607015488454325800905664474489
4095941891363533606713114429608301374391418026478261337905065893759879961705795656
0625046837315064169579897343796901188295138257423619680050896826321759998906873738
6553024230014699916478474715970813940416292577208779408649375621626924044508723046
5779854429408504294415506542336741519602506331212804739776828305588822771272559842
7613126208337716290873342609899172890749710830309019785468826643237416882787020071
0022964251882110225498757187840574279681642425303204449166647205030562341633089370
343764426402277527219067203084874687953628672025182085807378939165270079931964923
4440344584854252723631646447794668916883238492869807929875511202648377393041915407
7561510361047986801925938462016287828515322648153068373431742096183840216595375650
1237276373206011172625018652670180155395829908826194962913670503770090003238365696
5644312452832944526076903715010102354334348220043780618776284042267029623999654988
8296283031552326857375285372004633900841891997006775477729983966000467769162096088
8708359803433856649869863051626331022536281192398400035334133929164486320448941364
7619524281582664015617556731070492724233230277645079775643400284842455314954285962
0451751647574074351061231136091485841177194872883488179866799016227858353713954482
6328263177082818211030317464770369779987717142115664779141541596087325355879681404
4981785652093437278590196276022457852022997849692433783731148477946341849050719472
9526863883080360804098234184719204285667523129327655197408128731696548054834774061
1482532671886505655751662869832743243707516058658790465055244474586555619403820574
6459664807436317669010280397067482945388514816097335563995632118226191328362389917

5824954609319022606404359055214824769631350390539233797881303323128321498770465838
9860573922308720107633711417264481314452474753685441994618551806613519690086747483
8362037125810719350533596649567600241438039651026515524162700670500366052220996064
3855358927030743982563520345449097075114198224519593000082844418212811019096333898
1476532619011837163309489986172707105719365297986429919731621585298691235542360947
9717707040240041864594754889006498009047884171323061791899529991451001380852075698
5707733734001003682330077613317168701645463729607230685868855162425823730208934575
4143142326883786652693241750958274273870582683048014912696296961576369868771181034
1028891610621393863555312414596129865410997896387494333758417097729453833192003565
2238542816981789527872926669452152029948334123311812633177798127643275066201808570
0151534525886925050142452633637143966102794928531615053905968151550443098365212223
5097420053953762967361545133972002344819253169705174420457473361213396940959659306
6896535569539301567466626415867732988520198060212352483440654802281408753739439348
1059993373890931534937930466859283496505344257047133137167675764235787297864954720
1779207432730933672086150300698132437940767051523662795133957786171951386260702447
9172513858350749236437422411180426614203448495375983295480105007106072793308879526
4804687962874199648785067665160228420501333820774236330599853575182589985579624691
0403211469983922244925753991541297039916999854816816302728443934684660317589498200
5226586010210700114861843803057338740412680519238086599104309121573081600688594971
3018745726993318966327464056219533914116215063326193162708620382851913839670185379
1188284187879297315059972879937480261259708189437503788063797462291311917755090504
7780125364677554721794710691002482076603754785836288417384936512046367364588880676
3016678401214110787389727958459894901184150454614333529339411858182537911508698781
7449929920217064020692053265923194767722887075281133956279359418128209295226022765
7877037438555109060958981905696827353624747323553524678038650830371185652434660720
8378819053382851122690849566291537679424939680124482103426574468612021716052593545
5645668444482736684849441396863085790004924418036629943842634156170676352041643339
3305840784142224649421782365884877093338233966726691211303637612193346183162127622
3195318436685329989898078067485202390972649450666048685622216379502199953938509457
5944875617350237881579884715008182805756318532671972327670145785945143181761207716
1334562239969193833310114127942105522867633242560908906534999796383411366249735645
4749122276207887344172566328416237636250525441620234276964293838429512996148838299
751384541276790095345060791658416782724513

{ transformer }

Transformer theory . Electromagnetic induction theory . Faraday law . Lenz law .
Ampere law . Magnetic flux theory . Flux linkage theory . Mutual induction theory
. Self induction theory . Magnetic coupling theory . Magnetic permeability theory
. Core magnetization theory

Turns ratio theory . Voltage transformation theory . Current transformation theory
. Impedance transformation theory . Power transfer theory . Energy conservation
theory . Ideal transformer theory . Non ideal transformer theory . Equivalent
circuit theory . Load regulation theory . Voltage regulation theory . Efficiency
theory

AC circuit theory . Phasor theory . Sinusoidal steady state theory . Frequency
response theory . Complex impedance theory . Reflected impedance theory .
Magnetizing current theory . Excitation current theory . Leakage inductance theory
. Winding resistance theory . Core loss theory . Copper loss theory

Hysteresis theory . Eddy current theory . Saturation theory . Flux density theory
. Magnetic reluctance theory . Magnetic circuit theory . Lamination theory . Skin

effect theory . Proximity effect theory . Thermal rise theory . Insulation theory . Dielectric breakdown theory

Step up transformer theory . Step down transformer theory . Isolation transformer theory . Autotransformer theory . Current transformer theory . Potential transformer theory . Flyback transformer theory . Pulse transformer theory . Toroidal transformer theory . Three phase transformer theory . Power distribution theory . Electromagnetic compatibility theory

1508124034642624971981345651683276655788807712480804838767839153998277501503683780
9105761006443027012213281161306353133840368751618166807895377509685051029986503103
5881324534220637923142580571775199918991979084359784675148167578572164272970486968
7761633326829186392491198914341480106634070076562063480234090355596255731198094615
5251822513869256768634055499266462466604122024381164674904498952517753107985027095
4156897293035267647462433665893689598325820293471065606431923825806326874687448401
6297646228899434989515165654718181137681327057044432093849118972837202035823922632
6372561935056512077530039096528847287309276602081942288677381946698578675757625712
1909039016073470959271913573995325629368298689241839637906225704171951516857106016
8237116793493678689139140492160480168743812496225979315647143799341095210427830853
277975565579014112239322593256138328464494780905983479561152178646895949310321535
8338862189340748726515324215976409797499248757819344560832125800167410118886393924
5441457149762647598026051825220982768432584699763200557303197515025899022182464510
9161256185834605611840918355967954763676577324598430612125267470787121844848222199
7270511643591848608424535649391170506386568268831606521211263219189932759450111066
3993901313572384032447533796092044238287637015949513357684649492582965413116711994
8797725989079616037108844337982012060321992328503488291209819322263301956572734224
8677104265993459149891702591492471762421717939402023818704019971545109140199133489
8870084054151801526615384017902845926316425205109958613781260181412066767461950315
1073569923970107135703976746926837520858474874915029448390304006515305078706270850
2999859539982988774011374228446349983110244377380317579692456138971718000322955986
8942001864395812565655348904460120961943380792651819197361011604950448990534717733
3071543317536348148421616717886286803052893699206104088312074604946080900732721306
1389072579623822343945065643294067693227683651624464908307354883407060779020918170
9507117670170897786670755388802655205284781456167440888587361904036928945947053658
5542627838336378563773574057837585776685233142370691241033915811427981110729003364
8749794667614670395670829089560620747299028545025667944302077840673598829337504645
3120791894512128598229214503286425958403625495432570951733967276432326848823770523
1775436802642965144786422372882770519289670327966333882016667598817905486021772387
3381937170760475563663151881606562107479315179881213867108858292702761021216139888
4426363455516010940263451112129027718027893996712126112944006723334693222654678231
2865012762472454919127403745399867316308131190553683232057020851751839948381476037
5391818639736937670204606794782817798624361132852929443331197072032753399131902707
5384628606222091644884226828070134725629705532566840347559488964187683996567623367
6879849946021771478554955974440154034250627278544156266181537400941795672843554181
1488404260672538387293225048401259695366054365281940756552865526989651563466999438
1067890342327910332573597791694949885405623283924594272660129132194443444768644085
4677505261182121476696830821722857586826338330068902917384876125519881510790008442
1151867074153270975904461584721153960393181943287680285759687104668916232554669018
0938801829575528107737888667596858969753415840013094394994672509319350537795335039
0330532402450703530815075921578342647974564300478906979289054634323823744785452772
7426753178441302952248634372882775627230759817440157320252845722897621721068102745
1108040435222234795203481143032354794578481302474297919891465480764973371540062443
6082546323714446311065727157422120868812095852503643842702213007845669825652187737
9116373368535849327374547724559973331143598099462755167501263875569610816405529554

2780387197847263109513890774725015816715297727679444299506007630565754907583500159
2370959692005143241412815592290547915338269285931418473689948482493717985122103117
9356407391695394052342613216047065686568091759248741174162126682223479187154998637
7228763855234030932102520588872710480383321171906559592163609290085546995856914493
1038165612101530572138803938934964495692469134404225941281127624363980093127879517
9179317169019092843728144112435702602110292013397987695599709572526827579407706639
8873395168950905049511739588969068159949466785790080781321693510690579434058119929
7121045549470963832660155794587808329388148916089774562630801908704617057241698412
9193308919912368619849995810278990076299632902880624183168469254739254800417460906
9320105133159612522367168847653954489669410617903434872673395818104672987009094253
9612902564202963177052164851876829970751482766860518977450569145711132170023860792
9167518909139363910997864694098352464661918487537405015865606721094633621911331020
4469046243434454440317614309017998116881434832319407334945023347241264048278824630
4198515853083119961400913716848921734795214946981695558715497359676734977299309555
6262946988843478934927899574981569253000529620085329752496410341498607612299918972
5633973222066434899528765922403906642954483297413596450484631884013297518961598857
2024918325561716460272883246109979397600946740037142887236027418262482491430836359
2138075109578954714813064440422410331007621977898536243706059631185237637132512881
5465119332022736888501570286692369646285946078532832059164057578484944937996483014
9314589226320635049778490727620060693875211699501518042103957159685340221815595017
5419905281264554884843327341188161022049489238711672417652989240148771993328753289
2195375493277793078669008174633939554307397589940154200632944882844409866526727690
0356890513740981442618222597587189454427242437111506384898643051806279537578993122
0695218819622155930214578967867473922451863696911839243510442181215056257982418954
0796769765370944637896244865715870636875846611913551938508190667088808717976464848
0213599456689020260208021982582805717604441525940062094510265262648628213047066969
5278623593304768962414061465252466067710156116946435493165995107567330932493333742
0154741066246989605316914471278998102162921680264581930231244088658578782778687539
4693865664319496956285020475657461536414209856174805556594475881962099227748087930
6741987342705201289959643907241012952002541351996156184868890170843363600988316481
1072144914868011983446528819750830496050405591702192118502973584765365572919996779
8303296336449081495701439434113272695767733163421346899240560729552760445611072612
2523943967329404402929745410763627931351323023572426035088086871462903341987168758
8868978267738209799323863726944064948503617586706925148313448949448339685652742055
1618856280958583032006837852304778182760228898275453748068317265766043162171677191
3728990818385751017826005965756420768975980538106547947813507920146531600391765428
10641671110491692854474112697905698511708189345287134507902752984353172713134918086
7564628559600865125616417093299911301151337283875206353330899664328642323621865627
0051004307044067616151397248279271194198641843637583917048949749698370549149475624
0085945430625103716829226686716505896959196436535363558999959938811817671557060992
33129602444284149567391600033

{ fuse }

Fuse theory . Overcurrent protection theory . Circuit protection theory . Joule heating theory . Thermal runaway theory . Melting point theory . I squared t theory . Current rating theory . Voltage rating theory . Interrupting rating theory . Breaking capacity theory . Fault current theory

Short circuit theory . Overload theory . Arc formation theory . Arc suppression theory . Arc extinction theory . Contact separation theory . Thermal response theory . Time current characteristic theory . Fast acting fuse theory . Slow blow fuse theory . Time delay fuse theory . Inrush current theory

Coordination theory . Selectivity theory . Discrimination theory . Protection grading theory . Fault isolation theory . Energy let through theory . Prospective fault current theory . Current limiting theory . Peak let through current theory . Thermal stress theory . Mechanical stress theory . Safety margin theory

Cartridge fuse theory . Blade fuse theory . Glass fuse theory . Ceramic fuse theory . Resettable fuse theory . Polymer PTC theory . Thermal fuse theory . Semiconductor fuse theory . High rupture capacity theory . Low voltage fuse theory . High voltage fuse theory . Automotive fuse theory

Fuse aging theory . Derating theory . Ambient temperature theory . Heat dissipation theory . Insulation breakdown theory . Dielectric withstand theory . Electrical safety theory . Fire prevention theory . Equipment protection theory . Power distribution protection theory . Electromagnetic compatibility theory . Reliability theory

8098300627201059839906489040030726454262564530365105051923216833405103403547717611
0728506623876272310871742132031240997678888537262434414511749700829376787140687341
6587479487014880277525851999910188979853943319837013017564083016045166784919144439
1603789964723741401887980483963981998795186564777432966288015005656500905136718103
0734914239341765168049387186488559676064642439474383567729659635212592619057947206
8494350213078829379176587854439591508862601994751776245553327965100179137748729038
1987702376502194753407063227091883494651940799399252981483600535523990418150111308
8288677283509073204562709851907626378511258058579784181943218468397808067763432726
2319967130125813640516651585909314164794008681677050956068339076485342594017117477
7714926730798697010196123876056830453107246323715726799358598453226394753259586912
0489286039417188099476986607468271131485605000658269282183260743412392138082833699
8644391734955492999853677773127796679995096690246204530370969922092700720786723949
28541048901464047019552592493609788818451307194036919479888641074374440574064723990
5722281568598172235443953394889978887527935957565329104633564287876547638318898541
8711603951383005674423644990366297211209611675032994393332063806868953126749907471
7302195667980799173282885999287079483216504687815689521674856222612796613963184552
5736983801292525810792978707745605430351850841224746771822903880949317736799390241
7246661518886767352951846924049242824573691824909156342330801227625345451211385122
0186941251836256899040208741788705927913912152143511288667776876023879646004014897
1399571339986293985591057560065101024487057545293727765813069992508136072179594916
0167049482198558528929944908570517114473182557444051795394827378831987569563149837
1596761405803746138496756975593166479453216114908647035181681544170108992769521321
2608059777928864056354356585212338222083273218928020329103952464360813481845162575
9119085786386612596649607079717956077940233812225849697924130011417199921084136961
0529399640624280906149928391497664683622386747597585005690807901437666768512004986
9693994143701202541670376304420304776661559047349242556266571563624611421048055698
4745978675258655585762663038332095782767885515002586857802483850088781638054643048
0012171104923466249831478040052640429404286499744937307329384218851981329412457706
4162182596979430914682015440420803664318677806231605385095890127954468987197826545
1072557923034223542620370086824090845931593838447916029527170972416354394142940388
3179377502439851795952302502885532947169024363652605427864148796989255372260353474
6867537599961642301274340290475120026140859764000709941970444306680208255698317016
7378080939838376566226115301122822392967594309573680433086737715553295695024526430
8378461468523278526082146562852442954049619773773758304121405050699499545912527207
4855068521629990670137652451848434595905467491603130660287260479381942896290159877
7988639791011638046368542654700147841322562292360619233945745609172355777757945102
6049768591867954507409652885958817513247104927756754712319608529136619646100322998
1008316708036100956552264861230421392911544678600949347446746031439320257108165692

0718637879801343631513425184196896443777064404211966149788829442063039970153341178
6521081693352139219056673768093754739100049110477744883407036775853252218274884437
6349353756405202796114857166648351540366153214082653123547442825145706545165115622
485994063748738593374608954536447258885839682937031229424162165732759021534717517
2651076639333581325603150937923006816615272001649305549150915172524256687806525899
8483881600124353035792422513049401629580629655170573640325066642578975287728706680
6789426701891620947639735428550684626285515606363526243611687868360865265791314072
6534598368560036877640499863530735791270422565020458990323246368557670889928792815
0052023130461339817955282713677256569504077074514449868790377885412381678858862659
3939301532635525292983843063159432698382753561040230797046144282262592254463098966
1744942752892148490873613095782173073105072002703677027964440948971650546357466900
0338980092436012333154369646891001378433380613567611750194724069066516207086137210
5692037270723205888278745564235964598309108859381815088106827243794085876599654648
2911577148416063461317759162190732909243525539139830333502874309200937516346967435
4637465737764111675941997261248523086965439684028831023856444916024809511887483460
9672699006709930922089553758526749199052492426313056755479991994730537410123042443
6973337585392965394566934652950274143795443714567873015763642965298224203350668730
9492760790834176770621191656408687593970405364688470338652946711846349517261089897
6629717833279417734436959699541848567477215655556404503474086247414010554680243082
2323760191462433711875457678916126668514995063045163849940081259733823454894234190
0446130433165469048219314180341053382862814732066783737047226317235433859231800786
9941305540834284036374814887251376099368631865690218582336552563103790535261611316
5023665874473608124358180514752197392263361881739918429955003366400061522982559829
7936490026607763787875808988294419715205570360243994194741082459030407472500404477
6588537411410153643274051661231315233663350949980021561584646942104396396339164239
8355824148603604673287742125685964727804548436109834027207468911315491216786356296
8561113939392838644650110156651510385563372331632196726231467284931824522243741392
5785814667095514001717197384934229381272353768089596966581476262155974503394937311
3377954507415776681550915866982083869973693043163766088277561599589300158450829292
1907228170353569595709725762328481201025694818521791467081971158036205479920059445
3353793061115470202681182252761753389684897181060566135031569822014933820783304080
1848653634192609589112826848378150479338606223878973558514183095986195132723978477
9070145354521028316766561241780967695323999231573559390173020975952847551406120177
0386715065941001427653178181088177413594010203049059177053576370042246997985645577
3622743204266991599952688702659000102804628379758756384089260406880824386008481716
7820670636899453804186903567788649227325889942689501041954082793760920840290336096
0393795106535869084648405691099132014396284866922737234285357699127688749199314290
4851613316749176650708556812377177524858170290837303993200583072256347765980305015
9544338558399021019529866056976302317651782511977048447858346666364173985072709595
9826423479873173766216853644571868092078172085268327569956296811895031954911802337
7255041840997890492347856008331588479301624996840718822458270526421955403234248554
5016294384956323715676821496837296454571190835292793737387522126978560378787422741
4261541386221250844454530838486141329856708111358594074749487420326952128917048977
1076754759587031442863045316736656938298453413410792938873355466031538326882395275
8234412246937660468711090068104576586307525482333946926441677175407223103532729050
1494541590825267089709915617375867998922735415030718246312324803712716108295430612
3075661736982851407007910280848795363961904592579272877318075091360449049804052505
7700788419388919706477591688053561798441686204008115288408173446749639689234719545
6964933251294819008236023239670279990243872108705812175779324417592624572615917543
7459750507683776789467535527005329874665061921392908512679950121838019900559585961
8680281893387605885247514012739497490139458009503856131844208764502695225379549618
5284127298302078842084941226180035536749304792127582761056615274534341409004313411
2994813190186370173122201176823110909642402078156074185212881975885212884256944913
586880466773796053878357507761835425


```
{ circuit breaker }
```

Circuit breaker theory . Overcurrent protection theory . Fault protection theory . Short circuit theory . Overload theory . Arc formation theory . Arc suppression theory . Arc extinction theory . Contact separation theory . Switching theory . Trip mechanism theory . Protection coordination theory

Thermal trip theory . Magnetic trip theory . Thermal magnetic theory . Electronic trip theory . Residual current theory . Ground fault theory . Earth leakage theory . Differential current theory . Fault current theory . Prospective fault current theory . Interrupting rating theory . Breaking capacity theory

Time current characteristic theory . Inverse time theory . Instantaneous trip theory . Selectivity theory . Discrimination theory . Protection grading theory . Current limiting theory . Energy let through theory . I^2t theory . Peak let through current theory . Contact resistance theory . Contact wear theory

Air circuit breaker theory . Miniature circuit breaker theory . Molded case circuit breaker theory . Residual current breaker theory . Ground fault interrupter theory . Vacuum circuit breaker theory . Oil circuit breaker theory . SF6 circuit breaker theory . High voltage switching theory . Low voltage protection theory . Distribution protection theory . Motor protection theory

Inrush current theory . Nuisance tripping theory . Derating theory . Thermal stress theory . Mechanical stress theory . Dielectric withstand theory . Insulation coordination theory . Electromagnetic compatibility theory . Electrical safety theory . Fire prevention theory . Equipment protection theory . Reliability theory

[2] `operator-alphabet.js`

- **Bytes:** 42865
- **Type:** text

```
/**
 * operator-alphabet.js
 *
 * This module implements the Alphabet System (Tile Color Viewer) for the Operator
framework,
 * providing grid visualization and manipulation capabilities.
 * It integrates with the operator-dropdown.js system.
 *
 * @module OperatorAlphabet
 * @version 1.0.0
 * @depends OperatorFramework.Dropdown
 */

// Access the shared namespace for Operator framework

/**
```

```

* Alphabet System for grid visualization and tile color operations
* @namespace OperatorAlphabet
*/
OperatorFramework.Alphabet = (function() {
    'use strict';

    // Private module variables
    let _initialized = false;
    let _section = null;
    let _grids = []; // Array to store color sequences of each grid
    let _currentGridIndex = 0; // Index of the currently displayed grid
    let _defaultOptions = {
        containerId: 'main', // ID of the container to add the alphabet section to
        sectionId: 'alphabet-system-section', // ID for the created section
        defaultTileColor: '#ff0000', // Default tile color
        defaultTileSize: 100 // Default tile size in pixels
    };
    let _options = {};

    /**
     * Initialize the Alphabet System
     * @param {Object} options - Configuration options
     * @param {string} [options.containerId='main'] - ID of the container to add
the alphabet section to
     * @param {string} [options.sectionId='alphabet-system-section'] - ID for the
created section element
     * @param {string} [options.defaultTileColor='#ff0000'] - Default tile color
     * @param {number} [options.defaultTileSize=100] - Default tile size in pixels
     * @returns {boolean} Success status
     */
    function initialize(options = {}) {
        if (_initialized) {
            console.warn('OperatorAlphabet is already initialized');
            return false;
        }

        // Merge options with defaults
        _options = { ..._defaultOptions, ...options };

        console.log('OperatorAlphabet initialized with default options');
        _initialized = true;

        // Register with dropdown system if available
        if (OperatorFramework.Dropdown) {
            OperatorFramework.Dropdown.addSystem({
                id: 'alphabet',
                name: 'Tile Color Viewer',
                description: 'Create and manipulate colorful grid visualizations',
                loadFunction: loadAlphabetSystem
            });
            console.log('Alphabet System registered with
OperatorFramework.Dropdown');
        } else {

```

```

        console.warn('OperatorFramework.Dropdown not available, Alphabet
System not registered');
    }

    return true;
}

/**
 * Load the Alphabet System UI and functionality
 * @returns {boolean} Success status
 */
function loadAlphabetSystem() {
    if (!_initialized) {
        console.error('OperatorAlphabet not initialized, call initialize()
first');
        return false;
    }

    // Check if already loaded
    if (_section && document.getElementById(_options.sectionId)) {
        showAlphabetSection();
        return true;
    }

    // Get container
    //const container = document.getElementById(_options.containerId);
    // Use:
    const container = document.querySelector('main');
    if (!container) {
        console.error(`Container with ID "${_options.containerId}" not
found`);
        return false;
    }

    // Create Alphabet System section
    _section = document.createElement('section');
    _section.id = _options.sectionId;
    _section.className = 'operation-section';

    // Populate with Alphabet System content
    _section.innerHTML = _getAlphabetSystemHTML();

    // Add to container
    container.appendChild(_section);

    // Add styles
    _addAlphabetSystemStyles();

    // Initialize event listeners
    _initAlphabetSystemEventListeners();

    // Show the section
    showAlphabetSection();

```

```

// Initialize the grid with default settings
updateGrid();

console.log('Alphabet System loaded successfully');
return true;
}

/**
 * Show the Alphabet System section and hide other sections
 */
function showAlphabetSection() {
    if (!_section) return;

    // Hide all sections
    const allSections = document.querySelectorAll('.operation-section');
    allSections.forEach(section => {
        section.classList.remove('active');
    });

    // Show Alphabet section
    _section.classList.add('active');
}

/**
 * Generate the Alphabet System HTML content
 * @private
 * @returns {string} HTML content
 */
function _getAlphabetSystemHTML() {
    return `
        <h2>Tile Color Viewer</h2>
        <div class="operation-container">
            <div class="operation-controls">
                <!-- Hero Section -->
                <div class="alphabet-hero">
                    <p>Create and customize colorful grids with ease.</p>
                </div>

                <!-- Controls Section -->
                <div id="controls" class="controls-container">
                    <label for="tile-color">Tile Color: </label>
                    <input type="color" id="tile-color"
value="${_options.defaultTileColor}">

                    <label for="tile-size">Tile Size (px): </label>
                    <input type="number" id="tile-size"
value="${_options.defaultTileSize}" placeholder="Tile size in pixels">

                    <button id="update-grid">Update Grid</button>
                </div>

                <!-- Compounded ID Input Container -->
                <div id="compounded-id-input-container" class="controls-
container">

```

```

        <h3>Generate Grids by Compounded IDs</h3>
        <div id="compounded-id-inputs">
            <div class="compounded-id-input-row">
                <input type="text" class="compounded-id-input"
placeholder="Enter Compounded Grid ID">
            </div>
        </div>
        <button id="add-compounded-id">+</button>
        <button id="generate-grid-by-id">Generate Grids</button>
        <br>
        <!-- File Upload Input -->
        <p>Upload a file containing compounded grid IDs (e.g.,
sequences.txt).</p>
        <input type="file" id="file-input" accept=".txt">

        <!-- Execute Button -->
        <button id="execute-button" aria-label="Process uploaded
file">Execute</button>
    </div>

    <!-- Color List Container -->
    <div id="color-list-container" class="controls-container">
        <h3>Sequence List</h3>
        <ul id="color-list"></ul>
        <button id="download-colors">Download Tile ID</button>
    </div>
</div>

<div class="operation-result">
    <h3>Grid Display</h3>

    <!-- Grid Display Section -->
    <canvas id="grid-canvas" style="border: 1px solid #ccc;">
</canvas>

    <!-- Navigation Controls -->
    <div id="navigation-controls">
        <button id="prev-grid" aria-label="Go to previous
grid">Previous Grid</button>
        <button id="next-grid" aria-label="Go to next grid">Next
Grid</button>
        <button id="download-grid-image" aria-label="Download
current grid as image">Download Grid Image</button>
    </div>

    <!-- Current Grid Information -->
    <div id="current-grid-id"></div>
    <div id="current-grid-position"></div>
</div>
</div>
`;
}

/**

```



```

* Add CSS styles for the Alphabet System
* @private
*/
function _addAlphabetSystemStyles() {
    const styleId = 'operator-alphabet-styles';

    // Don't add styles if they already exist
    if (document.getElementById(styleId)) return;

    const styleElement = document.createElement('style');
    styleElement.id = styleId;
    styleElement.textContent = `
        /* CSS for Alphabet System */
        #alphabet-system-section {
            --primary-color: #4B5320; /* Army Green */
            --secondary-color: #FFFFFF; /* White */
            --accent-color: #BDB76B; /* Dark Khaki */
            --accent-hover-color: #D2B48C; /* Tan */
            --background-color: #F5F5DC; /* Beige */
            --text-color: #333333; /* Dark Gray */
        }

        /* General Reset - scoped to alphabet section */
        #alphabet-system-section * {
            box-sizing: border-box;
        }

        .alphabet-hero {
            width: 100%;
            text-align: center;
            padding: 20px;
            background-color: var(--primary-color);
            color: var(--secondary-color);
            margin-bottom: 20px;
        }

        .alphabet-hero p {
            font-size: 1.2rem;
        }

        /* Controls Container */
        .controls-container {
            background: var(--secondary-color);
            padding: 20px;
            border: 2px solid var(--accent-color);
            box-shadow: 0 2px 4px rgba(0, 0, 0, 0.4);
            width: 100%;
            margin-bottom: 20px;
        }

        #controls {
            display: flex;
            flex-wrap: wrap;
            justify-content: space-between;

```

```

        align-items: center;
        gap: 10px;
    }

    #controls label {
        flex: 1 1 200px;
        font-size: 1rem;
        color: var(--primary-color);
        font-weight: 500;
    }

    #controls input {
        flex: 1 1 200px;
        padding: 8px 12px;
        border: 2px solid var(--accent-color);
        font-size: 1rem;
        color: var(--text-color);
        background-color: #F5F5DC; /* Beige */
        font-family: "Fira Code", "Consolas", monospace;
    }

    #controls button {
        flex: 1 1 100%;
        padding: 12px 25px;
        font-size: 1rem;
        color: var(--secondary-color);
        background-color: var(--primary-color);
        border: 2px solid var(--accent-color);
        cursor: pointer;
        transition: background-color 0.3s ease, transform 0.2s ease, color
0.3s ease;

        margin-top: 10px;
        font-family: "Fira Code", "Consolas", monospace;
    }

    #controls button:hover {
        background-color: var(--accent-color);
        color: var(--text-color);
        transform: translateY(-2px);
    }

    /* Grid Canvas */
    #grid-canvas {
        border: 2px solid var(--accent-color);
        margin-bottom: 20px;
        width: 100%;
        max-width: 400px;
        height: auto;
    }

    /* Navigation Controls */
    #navigation-controls {
        display: flex;
        justify-content: center;

```

```
        align-items: center;
        gap: 10px;
        width: 100%;
        margin-bottom: 20px;
        flex-wrap: wrap;
    }

    #navigation-controls button {
        padding: 12px 25px;
        font-size: 1rem;
        color: var(--secondary-color);
        background-color: var(--primary-color);
        border: 2px solid var(--accent-color);
        cursor: pointer;
        font-weight: bold;
        transition: background-color 0.3s ease, transform 0.2s ease, color
0.3s ease;
        font-family: "Fira Code", "Consolas", monospace;
    }

    #navigation-controls button:hover {
        background-color: var(--accent-color);
        color: var(--text-color);
        transform: translateY(-2px);
    }

    #navigation-controls button:disabled {
        background-color: #ccc;
        border: 2px solid #999;
        color: #666;
        cursor: not-allowed;
        transform: none;
    }

    /* Current Grid Information */
    #current-grid-id,
    #current-grid-position {
        font-size: 1rem;
        color: var(--primary-color);
        margin-bottom: 10px;
        text-align: center;
        font-weight: 500;
        font-family: "Fira Code", "Consolas", monospace;
    }

    /* Color List Container */
    #color-list-container h3 {
        font-size: 1.2rem;
        margin-bottom: 10px;
        color: var(--primary-color);
        font-weight: 600;
        text-transform: uppercase;
        letter-spacing: 1px;
    }
```

```
#color-list {
  list-style-type: none;
  padding: 0;
  margin: 0;
}

#color-list li {
  background: #F5F5DC; /* Beige */
  padding: 10px;
  margin-bottom: 5px;
  border: 2px solid var(--accent-color);
  font-size: 1rem;
  color: var(--text-color);
  font-family: "Fira Code", "Consolas", monospace;
}

/* Download Colors Button */
#download-colors {
  margin-top: 10px;
  padding: 12px 25px;
  font-size: 1rem;
  color: var(--secondary-color);
  background-color: var(--primary-color);
  border: 2px solid var(--accent-color);
  cursor: pointer;
  transition: background-color 0.3s ease, transform 0.2s ease, color
0.3s ease;
  font-family: "Fira Code", "Consolas", monospace;
}

#download-colors:hover {
  background-color: var(--accent-color);
  color: var(--text-color);
  transform: translateY(-2px);
}

/* Compounded ID Input Container */
#compounded-id-inputs {
  display: flex;
  flex-direction: column;
  gap: 10px;
  margin-bottom: 10px;
}

.compounded-id-input-row {
  display: flex;
  align-items: center;
}

.compounded-id-input {
  flex: 1;
  padding: 8px 12px;
  border: 2px solid var(--accent-color);
```

```

        font-size: 1rem;
        color: var(--text-color);
        background-color: #F5F5DC; /* Beige */
        font-family: "Fira Code", "Consolas", monospace;
    }

    /* Add Compounded ID Button */
    #add-compounded-id {
        padding: 10px;
        font-size: 1.5rem;
        color: var(--secondary-color);
        background-color: var(--primary-color);
        border: 2px solid var(--accent-color);
        cursor: pointer;
        transition: background-color 0.3s ease, transform 0.2s ease, color
0.3s ease;

        margin-bottom: 10px;
        width: 40px;
        height: 40px;
        line-height: 20px;
        font-family: "Fira Code", "Consolas", monospace;
    }

    #add-compounded-id:hover {
        background-color: var(--accent-color);
        color: var(--text-color);
        transform: translateY(-2px);
    }

    /* Generate Grids Button */
    #generate-grid-by-id, #execute-button {
        padding: 12px 25px;
        font-size: 1rem;
        color: var(--secondary-color);
        background-color: var(--primary-color);
        border: 2px solid var(--accent-color);
        cursor: pointer;
        transition: background-color 0.3s ease, transform 0.2s ease, color
0.3s ease;

        font-family: "Fira Code", "Consolas", monospace;
        margin-top: 10px;
    }

    #generate-grid-by-id:hover, #execute-button:hover {
        background-color: var(--accent-color);
        color: var(--text-color);
        transform: translateY(-2px);
    }

    /* Error messages */
    .error-message {
        color: #e74c3c;
        background-color: rgba(231, 76, 60, 0.1);
        padding: 10px;
    }

```

```

        border-left: 4px solid #e74c3c;
        margin-top: 10px;
        font-size: 0.9rem;
    }

    /* Success messages */
    .success-message {
        color: #2ecc71;
        background-color: rgba(46, 204, 113, 0.1);
        padding: 10px;
        border-left: 4px solid #2ecc71;
        margin-top: 10px;
        font-size: 0.9rem;
    }
    `;

    document.head.appendChild(styleElement);
}

/**
 * Initialize event listeners for the Alphabet System
 * @private
 */
function _initAlphabetSystemEventListeners() {
    // Main buttons
    document.getElementById('update-grid').addEventListener('click',
updateGrid);
    document.getElementById('add-compounded-id').addEventListener('click',
addNewCompoundedIDInput);
    document.getElementById('generate-grid-by-id').addEventListener('click',
generateGridsFromInputs);
    document.getElementById('execute-button').addEventListener('click',
processUploadedFile);
    document.getElementById('download-colors').addEventListener('click',
downloadTileID);

    // Navigation buttons
    document.getElementById('prev-grid').addEventListener('click', () => {
        if (_currentGridIndex > 0) {
            _currentGridIndex--;
            displayCurrentGrid();
        }
    });

    document.getElementById('next-grid').addEventListener('click', () => {
        if (_currentGridIndex < _grids.length - 1) {
            _currentGridIndex++;
            displayCurrentGrid();
        }
    });

    document.getElementById('download-grid-image').addEventListener('click',
downloadGridImage);

```



```

        // Initialize with default value
        const defaultCompoundedID = '1443664';
        document.querySelector('.compounded-id-input').value =
defaultCompoundedID;
    }

    /**
     * Convert hex color code to a unique numeric ID (BigInt)
     * @param {string} hex - Hex color code
     * @returns {BigInt} Unique numeric ID
     */
    function hexToID(hex) {
        return BigInt('0x' + hex.replace('#', '')) + BigInt(1);
    }

    /**
     * Convert numeric ID (BigInt) back to hex color code
     * @param {BigInt|string|number} id - Numeric ID
     * @returns {string} Hex color code
     */
    function idToHex(id) {
        id = BigInt(id);
        if (id <= 0n) {
            return '#000000'; // Default to black
        }
        let hex = (id - BigInt(1)).toString(16).padStart(6, '0');
        return `#${hex}`;
    }

    /**
     * Calculate the compounded grid ID from a color sequence
     * @param {Array<string>} colorSequence - Array of hex color codes
     * @returns {string} Compounded grid ID
     */
    function calculateCompoundedGridID(colorSequence) {
        return colorSequence.map(color => hexToID(color).toString()).join('');
    }

    /**
     * Calculate grid dimensions based on the number of tiles
     * @param {number} numTiles - Number of tiles
     * @returns {Object} Grid dimensions {rows, cols}
     */
    function calculateGridDimensions(numTiles) {
        const sideLength = Math.ceil(Math.sqrt(numTiles));
        return { rows: sideLength, cols: sideLength };
    }

    /**
     * Validate hex color code
     * @param {string} hex - Hex color code
     * @returns {boolean} Whether the hex color code is valid
     */
    function isValidHexColor(hex) {

```

```

        return /^[0-9A-F]{6}$/i.test(hex);
    }

    /**
     * Update the grid based on user inputs
     */
    function updateGrid() {
        try {
            const tileColor = document.getElementById('tile-color').value;
            const tileSize = parseInt(document.getElementById('tile-size').value);

            if (isNaN(tileSize) || tileSize <= 0) {
                showError("Please enter a valid positive number for tile size.");
                return;
            }

            const numTiles = 1; // Start with a single tile
            const colorSequence = Array(numTiles).fill(tileColor);
            _grids = [colorSequence]; // Reset grids array with the new grid
            _currentGridIndex = 0; // Reset to the first grid

            displayCurrentGrid();

            showSuccess("Grid updated successfully");
        } catch (error) {
            showError(`Error updating grid: ${error.message}`);
            console.error("Error updating grid:", error);
        }
    }

    /**
     * Display the current grid based on _currentGridIndex
     */
    function displayCurrentGrid() {
        try {
            const canvas = document.getElementById('grid-canvas');
            const context = canvas.getContext('2d');

            if (_grids.length === 0) {
                context.clearRect(0, 0, canvas.width, canvas.height);
                return;
            }

            const tileSize = parseInt(document.getElementById('tile-size').value)
            || _options.defaultTileSize;
            const colorSequence = _grids[_currentGridIndex];
            const numTiles = colorSequence.length;
            const { rows, cols } = calculateGridDimensions(numTiles);

            // Set canvas dimensions
            canvas.width = cols * tileSize;
            canvas.height = rows * tileSize;

            // Clear the canvas

```

```

context.clearRect(0, 0, canvas.width, canvas.height);

// Draw the grid
colorSequence.forEach((color, i) => {
  const col = i % cols;
  const row = Math.floor(i / cols);
  const x = col * tileSize;
  const y = row * tileSize;

  context.fillStyle = color;
  context.fillRect(x, y, tileSize, tileSize);

  // Optional: Add a border to each tile
  context.strokeStyle = '#ccc';
  context.strokeRect(x, y, tileSize, tileSize);
});

// Remove existing event listeners to prevent stacking
canvas.onclick = null;

// Add a single event listener to the canvas
canvas.addEventListener('click', function(event) {
  const rect = canvas.getBoundingClientRect();
  const clickX = event.clientX - rect.left;
  const clickY = event.clientY - rect.top;

  const clickedCol = Math.floor(clickX / tileSize);
  const clickedRow = Math.floor(clickY / tileSize);
  const clickedIndex = clickedRow * cols + clickedCol;

  if (clickedIndex >= 0 && clickedIndex < colorSequence.length) {
    const newColor = prompt("Enter new hex color (e.g.,
#00ff00):", colorSequence[clickedIndex]);
    if (newColor) {
      if (isValidHexColor(newColor)) {
        colorSequence[clickedIndex] = newColor;
        displayCurrentGrid(); // Redraw the grid
        showSuccess("Tile color updated");
      } else {
        showError('Please enter a valid hex color code (e.g.,
#00ff00).');
      }
    }
  }
});

updateColorList();
updateNavigationButtons();
} catch (error) {
  showError(`Error displaying grid: ${error.message}`);
  console.error("Error displaying grid:", error);
}
}

```

```

/**
 * Update the color list and display the compounded grid ID
 */
function updateColorList() {
  try {
    const colorList = document.getElementById('color-list');
    colorList.innerHTML = '';
    const colorSequence = _grids[_currentGridIndex];

    colorSequence.forEach((color, index) => {
      const listItem = document.createElement('li');
      listItem.textContent = `ID ${hexToID(color)}: ${color}`;
      colorList.appendChild(listItem);
    });

    const compoundedGridID = calculateCompoundedGridID(colorSequence);
    const gridIdElement = document.getElementById('current-grid-id');
    gridIdElement.textContent = `Compounded Grid ID: ${compoundedGridID}`;

    const gridPositionElement = document.getElementById('current-grid-
position');
    gridPositionElement.textContent = `Grid ${_currentGridIndex + 1} of
${_grids.length}`;
  } catch (error) {
    showError(`Error updating color list: ${error.message}`);
    console.error("Error updating color list:", error);
  }
}

/**
 * Update the navigation buttons' disabled state
 */
function updateNavigationButtons() {
  try {
    const prevButton = document.getElementById('prev-grid');
    const nextButton = document.getElementById('next-grid');

    prevButton.disabled = _currentGridIndex === 0;
    nextButton.disabled = _currentGridIndex === _grids.length - 1;
  } catch (error) {
    console.error("Error updating navigation buttons:", error);
  }
}

/**
 * Split a compounded ID into tile IDs
 * @param {string} compoundedID - Compounded grid ID
 * @param {number} tileIDLength - Length of each tile ID
 * @returns {Array<string>} Array of tile IDs
 */
function splitCompoundedID(compoundedID, tileIDLength) {
  const tileIDs = [];
  let index = 0;
  while (index < compoundedID.length) {

```

```

        const tileID = compoundedID.substr(index, tileIDLength);
        tileIDs.push(tileID);
        index += tileIDLength;
    }
    return tileIDs;
}

/**
 * Regenerate grids from an array of compounded grid IDs
 * @param {Array<string>} compoundedIDs - Array of compounded grid IDs
 */
function regenerateGrids(compoundedIDs) {
    try {
        if (!compoundedIDs || !Array.isArray(compoundedIDs) ||
compoundedIDs.length === 0) {
            showError("No valid compounded IDs provided");
            return;
        }

        _grids = []; // Reset the grids array

        compoundedIDs.forEach((idString, index) => {
            const tileIDLength = 7; // Standard tile ID length
            let tileIDs = [];

            idString = idString.trim();

            if (/^\d+$/.test(idString)) {
                // It's a single large number; split it into tile IDs
                tileIDs = splitCompoundedID(idString, tileIDLength);
            } else {
                showError(`Invalid Compounded Grid ID format at index
${index}. Please enter a single large integer.`);
                throw new Error('Invalid Compounded Grid ID format');
            }

            const colorSequenceFromID = tileIDs.map(id => {
                try {
                    const hexColor = idToHex(id);
                    if (isValidHexColor(hexColor)) {
                        return hexColor;
                    } else {
                        showError(`Invalid tile ID: ${id}`);
                        return '#000000'; // Default to black
                    }
                } catch (error) {
                    console.error(`Error processing tile ID ${id}:`, error);
                    return '#000000'; // Default to black
                }
            });

            _grids.push(colorSequenceFromID);
        });
    }
}

```

```

        _currentGridIndex = 0; // Start from the first grid
        displayCurrentGrid();

        showSuccess(`Successfully regenerated ${_grids.length} ${_grids.length
=== 1 ? 'grid' : 'grids'}`);
    } catch (error) {
        showError(`Error regenerating grids: ${error.message}`);
        console.error("Error regenerating grids:", error);
    }
}

/**
 * Add a new compounded ID input field
 */
function addNewCompoundedIDInput() {
    try {
        const inputsContainer = document.getElementById('compounded-id-
inputs');

        const newInputRow = document.createElement('div');
        newInputRow.className = 'compounded-id-input-row';

        const newInput = document.createElement('input');
        newInput.type = 'text';
        newInput.className = 'compounded-id-input';
        newInput.placeholder = 'Enter Compounded Grid ID';

        newInputRow.appendChild(newInput);
        inputsContainer.appendChild(newInputRow);
    } catch (error) {
        showError(`Error adding input field: ${error.message}`);
        console.error("Error adding input field:", error);
    }
}

/**
 * Generate grids from input fields
 */
function generateGridsFromInputs() {
    try {
        const inputFields = document.querySelectorAll('.compounded-id-input');
        const compoundedIDStrings = [];

        inputFields.forEach(input => {
            const idString = input.value.trim();
            if (idString !== '') {
                compoundedIDStrings.push(idString);
            }
        });

        if (compoundedIDStrings.length === 0) {
            showError('Please enter at least one Compounded Grid ID.');
```



```

        regenerateGrids(compoundedIDStrings);
    } catch (error) {
        showError(`Error generating grids: ${error.message}`);
        console.error("Error generating grids:", error);
    }
}

/**
 * Process an uploaded file containing compounded grid IDs
 */
function processUploadedFile() {
    try {
        const fileInput = document.getElementById('file-input');
        const file = fileInput.files[0];

        if (!file) {
            showError("Please select a sequences.txt file before executing.");
            return;
        }

        const reader = new FileReader();

        reader.onload = function(event) {
            try {
                const content = event.target.result;
                const lines = content.split('\n').map(line => line.trim());

                const validIDs = lines.filter(line => /^d+$/ .test(line)); //
Only numeric lines
                if (validIDs.length > 0) {
                    regenerateGrids(validIDs); // Process the valid compound
IDs
                } else {
                    showError("No valid compound IDs found in the file.");
                }
            } catch (error) {
                showError(`Error processing file content: ${error.message}`);
                console.error("Error processing file content:", error);
            }
        };

        reader.onerror = function() {
            showError("Failed to read the file.");
        };

        reader.readAsText(file);
    } catch (error) {
        showError(`Error processing file: ${error.message}`);
        console.error("Error processing file:", error);
    }
}

/**
 * Download the tile ID information

```

```

*/
function downloadTileID() {
  try {
    if (_grids.length === 0 || _currentGridIndex >= _grids.length) {
      showError("No grid available to download");
      return;
    }

    const colorSequence = _grids[_currentGridIndex];
    const compoundedGridID = calculateCompoundedGridID(colorSequence);
    const fileContent = colorSequence.map((color, index) => `ID
${hexToID(color)}: ${color}`).join('\n');
    const fullContent = `Compounded Grid ID:
${compoundedGridID}\n\n${fileContent}`;

    // Create and download the file
    const blob = new Blob([fullContent], { type: 'text/plain' });
    const url = URL.createObjectURL(blob);

    const a = document.createElement('a');
    a.href = url;
    a.download = 'color_sequence.txt';
    a.click();

    URL.revokeObjectURL(url);

    showSuccess("Tile ID information downloaded");
  } catch (error) {
    showError(`Error downloading tile ID: ${error.message}`);
    console.error("Error downloading tile ID:", error);
  }
}

/**
 * Download the current grid as an image
 */
function downloadGridImage() {
  try {
    const canvas = document.getElementById('grid-canvas');
    const image = canvas.toDataURL('image/png');

    // Create and download the image
    const a = document.createElement('a');
    a.href = image;
    a.download = `grid_${_currentGridIndex + 1}.png`;
    a.click();

    showSuccess("Grid image downloaded");
  } catch (error) {
    showError(`Error downloading grid image: ${error.message}`);
    console.error("Error downloading grid image:", error);
  }
}

```

```

/**
 * Show an error message
 * @param {string} message - Error message
 */
function showError(message) {
    // Remove any existing error messages
    const existingError = document.querySelector('.error-message');
    if (existingError) {
        existingError.remove();
    }

    // Remove any existing success messages
    const existingSuccess = document.querySelector('.success-message');
    if (existingSuccess) {
        existingSuccess.remove();
    }

    // Create and show the error message
    const container = document.getElementById('compounded-id-input-
container');
    const errorElement = document.createElement('div');
    errorElement.className = 'error-message';
    errorElement.textContent = message;

    container.appendChild(errorElement);

    // Auto-remove after 5 seconds
    setTimeout(() => {
        if (document.body.contains(errorElement)) {
            errorElement.remove();
        }
    }, 5000);
}

/**
 * Show a success message
 * @param {string} message - Success message
 */
function showSuccess(message) {
    // Remove any existing error messages
    const existingError = document.querySelector('.error-message');
    if (existingError) {
        existingError.remove();
    }

    // Remove any existing success messages
    const existingSuccess = document.querySelector('.success-message');
    if (existingSuccess) {
        existingSuccess.remove();
    }

    // Create and show the success message
    const container = document.getElementById('compounded-id-input-
container');

```

```

    const successElement = document.createElement('div');
    successElement.className = 'success-message';
    successElement.textContent = message;

    container.appendChild(successElement);

    // Auto-remove after 5 seconds
    setTimeout(() => {
        if (document.body.contains(successElement)) {
            successElement.remove();
        }
    }, 5000);
}

/**
 * Get the current grids
 * @returns {Array<Array<string>>} The current grids
 */
function getGrids() {
    return [..._grids];
}

/**
 * Get the current grid
 * @returns {Array<string>|null} The current grid or null if no grid is
available
 */
function getCurrentGrid() {
    if (_grids.length === 0 || _currentGridIndex >= _grids.length) {
        return null;
    }
    return [..._grids[_currentGridIndex]];
}

/**
 * Set the current grid
 * @param {Array<string>} colorSequence - The color sequence for the grid
 * @param {boolean} [replace=false] - Whether to replace the current grid or
add a new one
 * @returns {boolean} Success status
 */
function setCurrentGrid(colorSequence, replace = false) {
    if (!colorSequence || !Array.isArray(colorSequence)) {
        return false;
    }

    // Validate all colors
    for (const color of colorSequence) {
        if (!isValidHexColor(color)) {
            return false;
        }
    }

    if (replace && _grids.length > 0) {

```

```

        _grids[_currentGridIndex] = [...colorSequence];
    } else {
        _grids.push([...colorSequence]);
        _currentGridIndex = _grids.length - 1;
    }

    displayCurrentGrid();
    return true;
}

/**
 * Set the current grid index
 * @param {number} index - The index to set
 * @returns {boolean} Success status
 */
function setCurrentGridIndex(index) {
    if (index < 0 || index >= _grids.length) {
        return false;
    }

    _currentGridIndex = index;
    displayCurrentGrid();
    return true;
}

/**
 * Convert a grid to a data URL
 * @param {number} [index] - Index of the grid to convert, defaults to current
grid
 * @returns {string|null} Data URL of the grid or null if no grid is available
 */
function gridToDataURL(index = null) {
    const gridIndex = index !== null ? index : _currentGridIndex;

    if (_grids.length === 0 || gridIndex < 0 || gridIndex >= _grids.length) {
        return null;
    }

    const colorSequence = _grids[gridIndex];
    const numTiles = colorSequence.length;
    const { rows, cols } = calculateGridDimensions(numTiles);
    const tileSize = parseInt(document.getElementById('tile-size').value) ||
_options.defaultTileSize;

    // Create a temporary canvas
    const canvas = document.createElement('canvas');
    canvas.width = cols * tileSize;
    canvas.height = rows * tileSize;
    const context = canvas.getContext('2d');

    // Draw the grid
    colorSequence.forEach((color, i) => {
        const col = i % cols;
        const row = Math.floor(i / cols);

```

```

        const x = col * tileSize;
        const y = row * tileSize;

        context.fillStyle = color;
        context.fillRect(x, y, tileSize, tileSize);

        // Add a border to each tile
        context.strokeStyle = '#ccc';
        context.strokeRect(x, y, tileSize, tileSize);
    });

    return canvas.toDataURL('image/png');
}

// Public API
return {
    initialize,
    loadAlphabetSystem,
    showAlphabetSection,
    updateGrid,
    displayCurrentGrid,
    regenerateGrids,
    downloadTileID,
    downloadGridImage,
    getGrids,
    getCurrentGrid,
    setCurrentGrid,
    setCurrentGridIndex,
    gridToDataURL,
    hexToID,
    idToHex,
    calculateCompoundedGridID,
    isValidHexColor
};
})();

// Automatically initialize when the DOM is ready
document.addEventListener('DOMContentLoaded', function() {
    // Look for the main container
    const mainElement = document.querySelector('main');
    if (mainElement) {
        OperatorFramework.Alphabet.initialize({
            containerId: mainElement.id || 'main'
        });
        console.log('Alphabet System initialized');
    } else {
        console.warn('Main element not found, Alphabet System not automatically initialized');
    }
});

```


- **Bytes:** 128872
- **Type:** text

```

/**
 * operator-build.js
 *
 * This module implements the Build System (Memory Slot Manager & Logic Framework)
 * for the Operator framework, providing memory management and logical operations.
 * It integrates with the operator-dropdown.js system.
 *
 * @module OperatorBuild
 * @version 1.0.0
 * @depends OperatorFramework.Dropdown
 */

// Access the shared namespace for Operator framework

/**
 * Build System for memory management and logical operations
 * @namespace OperatorBuild
 */
OperatorFramework.Build = (function() {
    'use strict';

    // Private module variables
    let _initialized = false;
    let _section = null;
    let _memoryManager = null;
    let _logicEngine = null;
    let _iframe = null;
    let _defaultOptions = {
        containerId: 'main', // ID of the container to add the build section to
        sectionId: 'build-system-section', // ID for the created section element
        iframeId: 'build-system-iframe', // ID for the iframe element
        iframeHeight: '800px' // Default height for the iframe
    };
    let _options = {};
    let _loadCallbacks = [];

    /**
     * Initialize the Build System
     * @param {Object} options - Configuration options
     * @param {string} [options.containerId='main'] - ID of the container to add
the build section to
     * @param {string} [options.sectionId='build-system-section'] - ID for the
created section element
     * @param {string} [options.iframeId='build-system-iframe'] - ID for the
iframe element
     * @param {string} [options.iframeHeight='800px'] - Height for the iframe
     * @returns {boolean} Success status
     */
    function initialize(options = {}) {

```

```

    if (!_initialized) {
        console.warn('OperatorBuild is already initialized');
        return false;
    }

    // Merge options with defaults
    _options = { ..._defaultOptions, ...options };

    console.log('OperatorBuild initialized with default options');
    _initialized = true;

    // Register with dropdown system if available
    if (OperatorFramework.Dropdown) {
        OperatorFramework.Dropdown.addSystem({
            id: 'build',
            name: 'Memory Slot Manager & Logic Framework',
            description: 'Manage memory slots and perform logical operations',
            loadFunction: loadBuildSystem
        });
        console.log('Build System registered with
OperatorFramework.Dropdown');
    } else {
        console.warn('OperatorFramework.Dropdown not available, Build System
not registered');
    }

    return true;
}

/**
 * Load the Build System UI and functionality
 * @returns {boolean} Success status
 */
function loadBuildSystem() {
    if (!_initialized) {
        console.error('OperatorBuild not initialized, call initialize()
first');
        return false;
    }

    // Check if already loaded
    if (_section && document.getElementById(_options.sectionId)) {
        showBuildSection();
        return true;
    }

    // Get container
    //const container = document.getElementById(_options.containerId);
    // Use:
    const container = document.querySelector('main');
    if (!container) {
        console.error(`Container with ID "${_options.containerId}" not
found`);
        return false;
    }

```

```

}

// Create Build System section
_section = document.createElement('section');
_section.id = _options.sectionId;
_section.className = 'operation-section';

// Create header
const header = document.createElement('h2');
header.textContent = 'Memory Slot Manager & Logic Framework';
_section.appendChild(header);

// Create container div
const operationContainer = document.createElement('div');
operationContainer.className = 'operation-container';
_section.appendChild(operationContainer);

// Create loading indicator
const loadingIndicator = document.createElement('div');
loadingIndicator.id = 'build-loading-indicator';
loadingIndicator.style.textAlign = 'center';
loadingIndicator.style.padding = '20px';
loadingIndicator.innerHTML = `
    <div style="margin-bottom: 15px;">Loading Memory Slot Manager & Logic
Framework...</div>
    <div class="spinner" style="
        border: 4px solid rgba(0, 0, 0, 0.1);
        border-left-color: #4B5320;
        border-radius: 50%;
        width: 30px;
        height: 30px;
        animation: spin 1s linear infinite;
        margin: 0 auto;
    "></div>
    <style>
        @keyframes spin {
            0% { transform: rotate(0deg); }
            100% { transform: rotate(360deg); }
        }
    </style>
`;
operationContainer.appendChild(loadingIndicator);

// Create iframe (initially hidden)
_iframe = document.createElement('iframe');
_iframe.id = _options.iframeId;
_iframe.style.width = '100%';
_iframe.style.height = _options.iframeHeight;
_iframe.style.border = 'none';
_iframe.style.display = 'none';
operationContainer.appendChild(_iframe);

// Add to container
container.appendChild(_section);

```

```

    // Show the section
    showBuildSection();

    // Create the iframe content
    createBuildSystemIframe();

    console.log('Build System loaded successfully');
    return true;
}

/**
 * Show the Build System section and hide other sections
 */
function showBuildSection() {
    if (!_section) return;

    // Hide all sections
    const allSections = document.querySelectorAll('.operation-section');
    allSections.forEach(section => {
        section.classList.remove('active');
    });

    // Show Build section
    _section.classList.add('active');
}

/**
 * Create and populate the Build System iframe
 */
function createBuildSystemIframe() {
    if (!_iframe) return;

    // Generate iframe content
    const iframeContent = generateIframeHTML();

    // Set up iframe load event
    _iframe.onload = function() {
        // Hide loading indicator
        const loadingIndicator = document.getElementById('build-loading-indicator');
        if (loadingIndicator) {
            loadingIndicator.style.display = 'none';
        }

        // Show iframe
        _iframe.style.display = 'block';

        // Access iframe contents
        const iframeWindow = _iframe.contentWindow;
        const iframeDocument = _iframe.contentDocument ||
iframeWindow.document;

        // Expose parent functions to iframe

```

```

        iframeWindow.parentAPI = createParentAPI();

        // Add event listeners for iframe-to-parent communication
        setupIframeCommunication(iframeWindow);

        // Store references to memory manager and logic engine
        if (iframeWindow.memoryManager) {
            _memoryManager = iframeWindow.memoryManager;
        }

        if (iframeWindow.logicEngine) {
            _logicEngine = iframeWindow.logicEngine;
        }

        // Execute any queued load callbacks
        for (const callback of _loadCallbacks) {
            try {
                callback(_memoryManager, _logicEngine, iframeWindow);
            } catch (error) {
                console.error('Error executing load callback:', error);
            }
        }

        // Clear load callbacks
        _loadCallbacks = [];

        console.log('Build System iframe loaded and initialized');
    };

    // Load content into iframe
    _iframe.srcdoc = iframeContent;
}

/**
 * Create API object for parent-to-iframe communication
 * @returns {Object} API object
 */
function createParentAPI() {
    return {
        executeCommand: function(command) {
            return executeCommand(command);
        },
        notifyParent: function(message, type) {
            console.log(`Message from iframe: ${message}`, type);
        },
        getParentData: function() {
            return {
                timestamp: new Date().toISOString(),
                origin: window.location.origin,
                framework: 'OperatorFramework',
                version: '1.0.0'
            };
        }
    };
};

```

```

}

/**
 * Set up communication between parent and iframe
 * @param {Window} iframeWindow - The iframe window object
 */
function setupIframeCommunication(iframeWindow) {
    // Listen for messages from iframe
    window.addEventListener('message', function(event) {
        // Verify origin
        if (event.source !== iframeWindow) return;

        const { action, data } = event.data || {};

        if (!action) return;

        switch (action) {
            case 'execute-command':
                if (data && data.command) {
                    const result = executeCommand(data.command);
                    // Send result back to iframe
                    iframeWindow.postMessage({
                        action: 'command-result',
                        data: { result }
                    }, '*');
                }
                break;

            case 'notify-parent':
                if (data && data.message) {
                    console.log(`Notification from iframe: ${data.message}`,
data.type);
                }
                break;

            case 'request-data':
                // Send data to iframe
                iframeWindow.postMessage({
                    action: 'parent-data',
                    data: {
                        timestamp: new Date().toISOString(),
                        origin: window.location.origin,
                        framework: 'OperatorFramework',
                        version: '1.0.0'
                    }
                }, '*');
                break;
        }
    });
}

/**
 * Execute a command in the iframe
 * @param {string} command - Command to execute

```

```

    * @returns {Promise<any>} Command result
    */
    function executeCommand(command) {
        return new Promise((resolve, reject) => {
            if (!_iframe || !_iframe.contentWindow) {
                reject(new Error('Iframe not available'));
                return;
            }

            try {
                // Process the command
                const iframeWindow = _iframe.contentWindow;

                if (iframeWindow.submitCommand) {
                    // For terminal commands
                    iframeWindow.submitCommand(command);
                    resolve(true);
                } else if (iframeWindow.eval) {
                    // Direct evaluation (less secure but more flexible)
                    const result = iframeWindow.eval(command);
                    resolve(result);
                } else {
                    reject(new Error('No command execution method available in
iframe'));
                }
            } catch (error) {
                reject(error);
            }
        });
    }

    /**
     * Register a callback to be executed when the iframe is loaded
     * @param {Function} callback - Callback function(memoryManager, logicEngine,
iframeWindow)
     */
    function onLoad(callback) {
        if (typeof callback !== 'function') {
            console.error('onLoad requires a function parameter');
            return;
        }

        if (_memoryManager && _logicEngine && _iframe && _iframe.contentWindow) {
            // If already loaded, execute immediately
            try {
                callback(_memoryManager, _logicEngine, _iframe.contentWindow);
            } catch (error) {
                console.error('Error executing load callback:', error);
            }
        } else {
            // Otherwise, queue for later execution
            _loadCallbacks.push(callback);
        }
    }
}

```



```

/**
 * Generate the HTML content for the iframe
 * @returns {string} HTML content
 */
function generateIframeHTML() {
    return `<!DOCTYPE html>
    <html lang="en">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Memory Slot Manager & Logic Framework</title>
        <style>
            :root {
                --primary-color: #0078d7;
                --secondary-color: #4a4a4a;
                --background-color: #f9f9f9;
                --terminal-bg: #1e1e1e;
                --terminal-text: #f0f0f0;
                --border-color: #dbdbdb;
                --highlight-color: #e6f2ff;
            }

            * {
                box-sizing: border-box;
                margin: 0;
                padding: 0;
                font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            }

            body {
                background-color: var(--background-color);
                line-height: 1.6;
                color: var(--secondary-color);
                padding: 20px;
                max-width: 1200px;
                margin: 0 auto;
            }

            header {
                text-align: center;
                margin-bottom: 30px;
                padding: 20px;
                background-color: white;
                border-radius: 5px;
                box-shadow: 0 2px 5px rgba(0,0,0,0.1);
            }

            h1 {
                color: var(--primary-color);
                margin-bottom: 10px;
            }

            .system-info {

```

```
        font-size: 0.9em;
        color: var(--secondary-color);
    }

    .main-container {
        display: flex;
        flex-direction: column;
        gap: 20px;
    }

    @media (min-width: 992px) {
        .main-container {
            flex-direction: row;
        }

        .terminal-section {
            flex: 1;
        }

        .results-section {
            flex: 1;
        }
    }

    .section {
        background-color: white;
        border-radius: 5px;
        padding: 20px;
        box-shadow: 0 2px 5px rgba(0,0,0,0.1);
    }

    .section-title {
        font-size: 1.2em;
        margin-bottom: 15px;
        padding-bottom: 10px;
        border-bottom: 1px solid var(--border-color);
        color: var(--primary-color);
        display: flex;
        justify-content: space-between;
        align-items: center;
    }

    .section-title-actions {
        display: flex;
        gap: 10px;
    }

    .action-button {
        padding: 3px 8px;
        font-size: 0.8em;
        background-color: var(--primary-color);
        color: white;
        border: none;
        border-radius: 3px;
    }
```

```
        cursor: pointer;
    }

    .action-button:hover {
        background-color: #005a9e;
    }

    .terminal {
        background-color: var(--terminal-bg);
        color: var(--terminal-text);
        padding: 15px;
        border-radius: 5px;
        font-family: 'Consolas', 'Courier New', monospace;
        height: 350px;
        overflow-y: auto;
        margin-bottom: 15px;
    }

    .terminal-input {
        display: flex;
        margin-bottom: 10px;
    }

    .terminal-input span {
        margin-right: 10px;
        color: #4caf50;
    }

    #commandInput {
        width: 100%;
        padding: 10px;
        font-family: 'Consolas', 'Courier New', monospace;
        margin-bottom: 10px;
        border: 1px solid var(--border-color);
        border-radius: 5px;
    }

    .button {
        background-color: var(--primary-color);
        color: white;
        border: none;
        padding: 10px 15px;
        border-radius: 5px;
        cursor: pointer;
        font-size: 0.9em;
        transition: background-color 0.3s;
    }

    .button:hover {
        background-color: #005a9e;
    }

    .button-secondary {
        background-color: #6c757d;
```

```
}

.button-secondary:hover {
    background-color: #5a6268;
}

#multilineInput {
    width: 100%;
    min-height: 100px;
    padding: 10px;
    font-family: 'Consolas', 'Courier New', monospace;
    margin-bottom: 10px;
    border: 1px solid var(--border-color);
    border-radius: 5px;
    display: none;
    white-space: pre;
    overflow-wrap: normal;
    overflow-x: auto;
}

.memory-display {
    background-color: white;
    border: 1px solid var(--border-color);
    border-radius: 5px;
    padding: 15px;
    height: 500px;
    overflow-y: auto;
}

.memory-file {
    margin-bottom: 20px;
}

.memory-file-title {
    font-weight: bold;
    margin-bottom: 5px;
    padding-bottom: 5px;
    border-bottom: 1px solid var(--border-color);
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.memory-file-actions {
    display: flex;
    gap: 5px;
}

.file-action {
    font-size: 0.7em;
    padding: 2px 5px;
    background-color: var(--primary-color);
    color: white;
    border: none;
```

```
        border-radius: 3px;
        cursor: pointer;
    }

    .file-action:hover {
        background-color: #005a9e;
    }

    .memory-slot {
        padding: 8px;
        border-bottom: 1px solid #f0f0f0;
    }

    .memory-slot:hover {
        background-color: var(--highlight-color);
    }

    .memory-slot-title {
        font-weight: bold;
        margin-bottom: 5px;
    }

    .memory-slot-value {
        white-space: pre-wrap;
        font-family: 'Consolas', 'Courier New', monospace;
        padding: 5px;
        background-color: #f8f8f8;
        border-radius: 3px;
        font-size: 0.9em;
    }

    .quick-commands {
        display: flex;
        flex-wrap: wrap;
        gap: 10px;
        margin-top: 15px;
    }

    .quick-command {
        background-color: #f0f0f0;
        padding: 5px 10px;
        border-radius: 3px;
        font-size: 0.8em;
        cursor: pointer;
        transition: background-color 0.3s;
    }

    .quick-command:hover {
        background-color: #e0e0e0;
    }

    .modal {
        display: none;
        position: fixed;
```

```

        z-index: 1;
        left: 0;
        top: 0;
        width: 100%;
        height: 100%;
        overflow: auto;
        background-color: rgba(0,0,0,0.5);
    }

    .modal-content {
        background-color: #fefefe;
        margin: 15% auto;
        padding: 20px;
        border: 1px solid #888;
        width: 80%;
        max-width: 600px;
        border-radius: 5px;
    }

    .close-modal {
        color: #aaa;
        float: right;
        font-size: 28px;
        font-weight: bold;
        cursor: pointer;
    }

    .close-modal:hover {
        color: black;
    }

    .progress-bar-container {
        width: 100%;
        background-color: #e0e0e0;
        border-radius: 3px;
        margin: 10px 0;
    }

    .progress-bar {
        height: 20px;
        background-color: var(--primary-color);
        border-radius: 3px;
        width: 0%;
        transition: width 0.3s;
    }

    #generationStatus {
        margin-top: 10px;
        font-size: 0.9em;
    }

    /* File system styles */
    .files-section {
        margin-top: 20px;
    }

```

```
}

.file-system-actions {
    display: flex;
    gap: 10px;
    margin-bottom: 10px;
}

/* Radio button styling */
.radio-group {
    margin: 15px 0;
}

.radio-option {
    margin-bottom: 10px;
    display: flex;
    align-items: flex-start;
}

.radio-option input[type="radio"] {
    margin-top: 3px;
    margin-right: 10px;
}

.radio-option .option-label {
    font-weight: bold;
}

.radio-option .option-description {
    font-size: 0.85em;
    color: #666;
    margin-top: 2px;
}

/* File input styling */
.file-input-container {
    margin: 15px 0;
}

.file-input-label {
    font-weight: bold;
    margin-bottom: 5px;
    display: block;
}

.file-input {
    width: 100%;
    padding: 8px;
    border: 1px solid var(--border-color);
    border-radius: 4px;
}

/* Logic engine styles */
.nav-tabs {
```



```
        display: flex;
        border-bottom: 1px solid var(--border-color);
        margin-bottom: 15px;
    }

    .nav-tab {
        padding: 8px 15px;
        cursor: pointer;
        border: 1px solid transparent;
        border-bottom: none;
        border-radius: 5px 5px 0 0;
        margin-right: 5px;
        background-color: #f8f8f8;
    }

    .nav-tab.active {
        background-color: white;
        border-color: var(--border-color);
        border-bottom: 1px solid white;
        margin-bottom: -1px;
        font-weight: bold;
    }

    .tab-content {
        display: none;
    }

    .tab-content.active {
        display: block;
    }

    #logicTerminal {
        background-color: var(--terminal-bg);
        color: var(--terminal-text);
        padding: 15px;
        border-radius: 5px;
        font-family: 'Consolas', 'Courier New', monospace;
        height: 300px;
        overflow-y: auto;
        margin-bottom: 15px;
    }

    @media (max-width: 768px) {
        body {
            padding: 10px;
        }

        .terminal {
            height: 250px;
        }

        .memory-display {
            height: 300px;
        }
    }
```

```

        .modal-content {
            width: 95%;
            margin: 10% auto;
        }

        .file-system-actions {
            flex-direction: column;
        }
    }
</style>
</head>
<body>
    <header>
        <h1>INTEGRATED MEMORY & LOGIC FRAMEWORK</h1>
        <div class="system-info">
            <div id="systemTime"></div>
            <div>MEMORY MANAGER & LOGIC ENGINE LOADED</div>
            <div>READY FOR INPUT</div>
        </div>
    </header>

    <div class="main-container">
        <div class="terminal-section">
            <div class="section">
                <div class="nav-tabs">
                    <div class="nav-tab active" data-tab="memory">Memory
Manager</div>
                    <div class="nav-tab" data-tab="logic">Logic
Engine</div>
                </div>

                <div id="memoryTab" class="tab-content active">
                    <div class="section-title">
                        Command Terminal
                    <div class="section-title-actions">
                        <button class="action-button"
onclick="clearTerminal()">Clear</button>
                    </div>
                </div>
                <div id="terminal" class="terminal"></div>
                <input type="text" id="commandInput"
placeholder="Enter command (type 'help' for list of commands)">
                <textarea id="multilineInput" placeholder="Enter
multi-line content (e.g. for assign or search)" wrap="off"></textarea>
                <div class="quick-commands">
                    <span class="quick-command"
onclick="insertCommand('help')">help</span>
                    <span class="quick-command"
onclick="insertCommand('display')">display</span>
                    <span class="quick-command"
onclick="insertCommand('assign')">assign</span>
                    <span class="quick-command"
onclick="insertCommand('read')">read</span>

```

```

        <span class="quick-command"
onclick="insertCommand('search')">search</span>
        <span class="quick-command"
onclick="insertCommand('generate')">generate</span>
        <span class="quick-command"
onclick="insertCommand('save_all')">save_all</span>
        <span class="quick-command"
onclick="insertCommand('load_all')">load_all</span>
    </div>
    <div style="display: flex; gap: 10px; margin-top:
10px;">
        <button id="submitCommand" class="button">Execute
Command</button>
        <button id="submitMultiline" class="button"
style="display: none;">Submit</button>
        <button id="cancelMultiline" class="button button-
secondary" style="display: none;">Cancel</button>
    </div>
</div>

    <div id="logicTab" class="tab-content">
        <div class="section-title">
            Proof by Contradiction Engine
            <div class="section-title-actions">
                <button class="action-button"
onclick="clearLogicTerminal()">Clear</button>
            </div>
        </div>
        <div id="logicTerminal" class="terminal"></div>
        <input type="text" id="logicCommandInput"
placeholder="Enter instruction (type 'help' for available instructions)">
        <div class="quick-commands">
            <span class="quick-command"
onclick="insertLogicCommand('help')">help</span>
            <span class="quick-command"
onclick="insertLogicCommand('ASSERT')">ASSERT</span>
            <span class="quick-command"
onclick="insertLogicCommand('NOT')">NOT</span>
            <span class="quick-command"
onclick="insertLogicCommand('AND')">AND</span>
            <span class="quick-command"
onclick="insertLogicCommand('OR')">OR</span>
            <span class="quick-command"
onclick="insertLogicCommand('IMPLIES')">IMPLIES</span>
            <span class="quick-command"
onclick="insertLogicCommand('CONTRADICTION')">CONTRADICTION</span>
        </div>
        <div style="display: flex; gap: 10px; margin-top:
10px;">
            <button id="submitLogicCommand"
class="button">Execute Instruction</button>
            <button id="checkContradictions"
class="button">Check Contradictions</button>
            <button id="clearStatements" class="button button-

```

```

secondary">Clear Statements</button>
                <button id="generateStatements"
class="button">Generate Statements</button>
            </div>
        </div>
    </div>

    <div class="section files-section">
        <div class="section-title">File System</div>
        <div class="file-system-actions">
            <button class="button"
onclick="saveAllMemorySlots()">Save All Files</button>
            <button class="button"
onclick="loadMemorySlotsFromFile()">Load Files</button>
            <button class="button button-secondary"
onclick="clearAllMemorySlots()">Clear All Files</button>
        </div>
    </div>
</div>

<div class="results-section">
    <div class="section">
        <div class="section-title">
            Memory Slots
            <div class="section-title-actions">
                <button class="action-button"
onclick="updateMemoryDisplay()">Refresh</button>
            </div>
        </div>
        <div id="memoryDisplay" class="memory-display">
            <div style="text-align: center; color: #888; margin-
top: 20px;">
                No memory slots available.
                <br><br>
                Use the terminal to add memory slots.
            </div>
        </div>
    </div>
</div>

<div id="generationModal" class="modal">
    <div class="modal-content">
        <span class="close-modal"
onclick="document.getElementById('generationModal').style.display='none'">&times;
    </span>

    <h2>Generating Combinations</h2>

    <div class="radio-group">
        <div class="radio-option">
            <input type="radio" id="generateSingleFile"
name="generateType" value="singleFile" checked>
            <div>
                <div class="option-label">Single File with

```

```

Multiple Slots</div>
        <div class="option-description">Store all
combinations in one file with each combination as a separate slot</div>
        </div>
    </div>
    <div class="radio-option">
        <input type="radio" id="generateMultiFile"
name="generateType" value="multiFile">
        <div>
            <div class="option-label">One File Per
Combination</div>
            <div class="option-description">Create a separate
file for each generated combination</div>
            </div>
        </div>
    </div>

    <div id="fileInputContainer" class="file-input-container">
        <div class="file-input-label">Enter Filename for Single
File Mode:</div>
        <input type="text" id="fileNameInput" class="file-input"
value="combinations" placeholder="Enter filename (e.g. 3, combinations, etc.)">
        </div>

        <div class="progress-bar-container">
            <div id="generationProgress" class="progress-bar"></div>
        </div>
        <div id="generationStatus">Ready to generate</div>
        <div style="margin-top: 20px; display: flex; gap: 10px; flex-
wrap: wrap;">
            <button id="startGenerationBtn" class="button">Start
Generation</button>
            <button id="downloadGeneratedBtn" class="button"
disabled>Download Results</button>
            <button id="importGeneratedBtn" class="button"
disabled>Import to Memory Slots</button>
            <button class="button button-secondary"
onclick="document.getElementById('generationModal').style.display='none'">Close</b
utton>
        </div>
    </div>
</div>
</div>

    <div id="statementsGenerationModal" class="modal">
        <div class="modal-content">
            <span class="close-modal"
onclick="document.getElementById('statementsGenerationModal').style.display='none'
">&times;</span>
            <h2>Generate Statements</h2>

            <div class="file-input-container">
                <div class="file-input-label">Number of statements to
generate:</div>
                <input type="number" id="numStatementsInput" class="file-

```

```

input" value="5" min="1" max="100">
    </div>

    <div style="margin-top: 20px; display: flex; gap: 10px; flex-
wrap: wrap;">
        <button id="startStatementsGenBtn" class="button">Generate
Statements</button>
        <button class="button button-secondary"
onclick="document.getElementById('statementsGenerationModal').style.display='none'
">Cancel</button>
    </div>
</div>
</div>

<div id="helpModal" class="modal">
    <div class="modal-content">
        <span class="close-modal"
onclick="document.getElementById('helpModal').style.display='none'">&times;</span>
        <h2>Available Commands</h2>
        <pre style="white-space: pre-wrap; max-height: 400px;
overflow-y: auto; margin-top: 10px;">
AVAILABLE COMMANDS:
-----
assign &lt;fileNumber&gt; &lt;slotNumber&gt;
    &lt;value&gt;                : Assign a multi-line value to a slot
                           (Enter your multi-line value in the text area)

read &lt;fileNumber&gt; &lt;slotNumber&gt;
    : Read the value from a slot

last_slot &lt;fileNumber&gt;
    : Get the last slot number in a file

search
    &lt;value&gt;                : Search for a value across all slots

call &lt;fileNumber&gt; &lt;startSlot&gt; &lt;endSlot&gt;
    : Call values from a range of slots

export &lt;fileNumber&gt; : Export a file's memory slots to a text file

generate &lt;n&gt;          : Generate all combinations with n cells
                        using the standard character set
                        Results are ready to use with this application

                        Generation Options:
                        - Single File: All combinations in one file with multiple
slots
                        - One File Per Combination: Each combination in its own
file

custom_generate &lt;n&gt; : Generate combinations with custom character set

display          : Show all memory slots

```

```
clear          : Clear the terminal
help           : Display this help message
```

```
save_json <fileNumber> : Save a specific file to JSON
save_all       : Save all memory slots to a ZIP archive
load_json      : Load memory slots from a JSON file
load_all       : Load memory slots from a ZIP archive
```

File System Operations:

The file system supports downloading all your data as a ZIP archive and uploading it again for continuous use. This provides an automatic way to save your work and continue where you left off.

Generated files are automatically structured to be used with the memory slot manager system without any additional processing.

```
</pre>
```

```
</div>
```

```
</div>
```

```
<div id="logicHelpModal" class="modal">
```

```
<div class="modal-content">
```

```
<span class="close-modal"
```

```
onclick="document.getElementById('logicHelpModal').style.display='none'>&times;
</span>
```

```
<h2>Proof by Contradiction Engine Help</h2>
```

```
<pre style="white-space: pre-wrap; max-height: 400px;
```

```
overflow-y: auto; margin-top: 10px;">
```

Proof by Contradiction Engine Help

1. Add Instruction:

- ASSERT <statement>
- NOT <statement>
- AND <statement1> <statement2>
- OR <statement1> <statement2>
- IMPLIES <statement1> <statement2>
- CONTRADICTION <statement1> <statement2>

2. Clear Statements: Clears all statements from the current framework.

3. Check Contradictions: Checks for contradictions within the current framework using defined rules.

4. Generate Statements: Automatically generates a framework of statements.

5. Help: Displays this help message.

Example:

```
ASSERT Statement1
```

```
NOT Statement1
```

```
CONTRADICTION Statement1 NOT Statement1
```

```
(This will detect a contradiction based on the statement_not_statement rule)
```

```
</pre>
```

```
</div>
```

```
</div>
```

```
<script>
```

```
class MemorySlotManager {
```

```

        constructor() {
            this.memorySlots = {};
        }

        assignSlot(fileNumber, slotNumber, value) {
            if (!fileNumber || !slotNumber || !value) {
                return "Error: Invalid input parameters.";
            }

            if (this.valueExists(value)) {
                return "Error: The value already exists in another
memory slot.";
            }

            if (!this.memorySlots[fileNumber]) {
                this.memorySlots[fileNumber] = {};
            }

            this.memorySlots[fileNumber][slotNumber] = value;
            return `Assigned value to slot ${slotNumber} in file
${fileNumber}.`;
        }

        readSlot(fileNumber, slotNumber) {
            if (this.memorySlots[fileNumber] &&
this.memorySlots[fileNumber][slotNumber] !== undefined) {
                return this.memorySlots[fileNumber][slotNumber];
            }
            return `Slot ${slotNumber} not found in file
${fileNumber}.`;
        }

        getLastSlotNumber(fileNumber) {
            if (this.memorySlots[fileNumber] &&
Object.keys(this.memorySlots[fileNumber]).length > 0) {
                try {
                    const slots = [];
                    for (const slot in this.memorySlots[fileNumber]) {
                        if (/^\d+$/.test(slot)) {
                            slots.push(parseInt(slot));
                        }
                    }

                    if (slots.length > 0) {
                        return Math.max(...slots);
                    }
                } catch (e) {
                    // Ignore exceptions
                }
            }
            return null;
        }

        searchValue(value) {

```



```

        const results = [];
        const searchValueLower = value.toLowerCase();

        for (const file in this.memorySlots) {
            for (const slot in this.memorySlots[file]) {
                const val = this.memorySlots[file][slot];
                if (val.toLowerCase().includes(searchValueLower))
{
                    results.push({
                        file,
                        slot,
                        value: val
                    });
                }
            }
        }

        return results;
    }

    valueExists(value) {
        const valueLower = value.toLowerCase();

        for (const file in this.memorySlots) {
            for (const slot in this.memorySlots[file]) {
                if (this.memorySlots[file][slot].toLowerCase() ===
valueLower) {
                    return true;
                }
            }
        }

        return false;
    }

    callSlotRange(fileNumber, startSlot, endSlot) {
        if (!this.memorySlots[fileNumber]) {
            return `File \${fileNumber} not found.`;
        }

        const results = [];
        for (let slot = startSlot; slot <= endSlot; slot++) {
            const slotStr = slot.toString();
            if (this.memorySlots[fileNumber][slotStr] !==
undefined) {
                results.push(`Slot
\${slotStr}: \n\${this.memorySlots[fileNumber][slotStr]}`);
            } else {
                results.push(`Slot \${slotStr} not found.`);
            }
        }

        return results.join('\n\n');
    }
}

```

```

exportToTextFile(fileNumber) {
  if (!this.memorySlots[fileNumber]) {
    return `Error: File \${fileNumber} not found.`;
  }

  try {
    let content = '';
    const slots = this.memorySlots[fileNumber];
    let slotKeys = [];

    try {
      const numericKeys = [];
      for (const key in slots) {
        if (/^\d+$/.test(key)) {
          numericKeys.push(parseInt(key));
        }
      }
      numericKeys.sort((a, b) => a - b);
      slotKeys = numericKeys.map(key => key.toString());
    } catch (e) {
      // If conversion fails, sort alphabetically
      slotKeys = Object.keys(slots).sort();
    }

    for (const slot of slotKeys) {
      const value = slots[slot];
      // Format: SLOT <number>
      content += `SLOT \${slot}\\n\${value}\\n-----`
    }

    content += `\\n`;

    // Create a download link
    const blob = new Blob([content], { type: 'text/plain' });

    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `file\${fileNumber}.txt`;
    a.click();
    URL.revokeObjectURL(url);

    return `Exported file \${fileNumber} to`
    `file\${fileNumber}.txt`;
  } catch (e) {
    return `Error exporting to text file:`
    `${e.message}`;
  }
}

saveToJSON(fileNumber) {
  try {
    if (fileNumber && !this.memorySlots[fileNumber]) {
      return `Error: File \${fileNumber} not found.`;
    }
  }
}

```

```

    }

    let jsonData;
    let filename;

    if (fileNumber) {
        jsonData = this.memorySlots[fileNumber];
        filename = `file\${fileNumber}.json`;
    } else {
        jsonData = this.memorySlots;
        filename = 'memory_slots.json';
    }

    const jsonString = JSON.stringify(jsonData, null, 2);
    const blob = new Blob([jsonString], { type:
'application/json' });

    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = filename;
    a.click();
    URL.revokeObjectURL(url);

    return `Memory slots saved to \${filename}`;
} catch (e) {
    return `Error saving to JSON: \${e.message}`;
}
}

loadJSON(jsonData) {
    try {
        const content = JSON.parse(jsonData);

        if (typeof content === 'object' && content !== null) {
            // Check if this is our expected format
            let isFullStructure = false;
            for (const file in content) {
                if (typeof content[file] === 'object' &&
content[file] !== null) {
                    isFullStructure = true;
                    break;
                }
            }

            if (isFullStructure) {
                // Merge with existing memory slots
                for (const file in content) {
                    if (!this.memorySlots[file]) {
                        this.memorySlots[file] = {};
                    }

                    for (const slot in content[file]) {
                        this.memorySlots[file][slot] =
content[file][slot];

```

```

        }
    }
    } else {
        // Assume it's a single file's slots
        const fileNumber = '1'; // Default file number
        if (!this.memorySlots[fileNumber]) {
            this.memorySlots[fileNumber] = {};
        }

        for (const slot in content) {
            this.memorySlots[fileNumber][slot] =
content[slot];
        }
    }

    return 'Successfully loaded JSON data.';
}

return 'Error: Invalid JSON structure.';
} catch (e) {
    return `Error loading JSON: ${e.message}`;
}
}

// Save all memory slots as a ZIP file
saveAllAsZip() {
    try {
        // Create a direct download of all slots as JSON for
simpler implementation
        const jsonData = JSON.stringify(this.memorySlots,
null, 2);

        const blob = new Blob([jsonData], { type:
'application/json' });

        const url = URL.createObjectURL(blob);
        const a = document.createElement('a');
        a.href = url;
        a.download = 'all_memory_slots.json';
        a.click();
        URL.revokeObjectURL(url);

        return 'All memory slots saved to
all_memory_slots.json';
    } catch (e) {
        return `Error saving memory slots: ${e.message}`;
    }
}

// Load memory slots from a JSON file (simplified without ZIP)
loadFromFile(file) {
    return new Promise((resolve, reject) => {
        const reader = new FileReader();

        reader.onload = (e) => {
            try {

```

```

        const result = this.loadJSON(e.target.result);
        resolve(result);
    } catch (error) {
        reject(`Error parsing file:
    \${error.message}`);
    }
};

reader.onerror = () => {
    reject('Error reading file');
};

reader.readAsText(file);
});
}

generateCharSet() {
    let chars =
        "abcdefghijklmnopqrstuvwxyz" +
        "ABCDEFGHIJKLMNOPQRSTUVWXYZ" +
        "0123456789" +
        "!@#$%^&*()-_+=[]{}|;:,<>/?" +
        "\\t\\n\\r" +
        "~\\`'\\\"\\\\\\";

    // Ensure exactly 100 characters
    if (chars.length > 100) {
        chars = chars.substring(0, 100);
    } else if (chars.length < 100) {
        // Pad with additional characters if needed
        while (chars.length < 100) {
            chars += '?';
        }
    }

    return chars;
}

async generateCombinations(chars, n, generateType, fileName,
progressCallback) {
    // The number of possible combinations is (k+1)^n
    const k = chars.length - 1;
    const nbrComb = Math.pow(k + 1, n);

    let fileContent = '';

    if (nbrComb > 1000000) {
        progressCallback(0, "Warning: Generating a large
number of combinations. This may take a while.");
    }

    // Clear any existing memory slots that would be affected
    if (generateType === 'singleFile') {
        // Clear just the target file

```

```

        this.memorySlots[fileName] = {};
    }

    // To avoid blocking the UI for too long, break up the
work    const batchSize = 10000; // Process in batches to keep UI
responsive    const totalBatches = Math.ceil(nbrComb / batchSize);

    for (let batch = 0; batch < totalBatches; batch++) {
        await new Promise(resolve => setTimeout(resolve, 0));
// Allow UI to update

        const startRow = batch * batchSize;
        const endRow = Math.min((batch + 1) * batchSize,
nbrComb);

        for (let row = startRow; row < endRow; row++) {
            const id = row + 1;
            let combination = '';

            for (let col = n - 1; col >= 0; col--) {
                const rdiv = Math.pow(k + 1, col);
                const cell = Math.floor(row / rdiv) % (k + 1);
                combination += chars[cell];
            }

            // Handle different generation types
            if (generateType === 'singleFile') {
                // Store all combinations in a single file
with the given name

                const slotNumber = id.toString();
                this.memorySlots[fileName][slotNumber] =
combination;

            }
            else if (generateType === 'multiFile') {
                // Each combination gets its own file
                const fileNumber = id.toString();
                if (!this.memorySlots[fileNumber]) {
                    this.memorySlots[fileNumber] = {};
                }
                // Always use slot 1 for each file
                this.memorySlots[fileNumber]['1'] =
combination;

            }

            // Add to file content for download
            if (generateType === 'singleFile') {
                fileContent += `SLOT
\\${id}\\n\\${combination}\\n-----\\n`;
            }
            else {
                fileContent += `FILE \\${id}\\nSLOT
1\\n\\${combination}\\n-----\\n`;
            }
        }
    }

```

```

    }

    // Update progress approximately every 5%
    if (batch % Math.max(1, Math.floor(totalBatches / 20))
=== 0 || batch === totalBatches - 1) {
        const progress = ((batch + 1) / totalBatches) *
100;

        progressCallback(progress, `\Generated
\${Math.min((batch + 1) * batchSize, nbrComb)} of \${nbrComb} combinations
(\${progress.toFixed(1)}%)\`);
    }
}

// Format the complete output in a way that's ready for
the application

let finalContent = `\GENERATED COMBINATIONS\\n\`;
finalContent += `\Generation Mode: \${generateType} ===
'singleFile' ? 'Single File' : 'One File Per Combination'\\n\`;
finalContent += `(k+1)^n = (\${k} + 1)^\${n} =
\${nbrComb}\\n\`;

finalContent +=
`\`=====\\n\`;
finalContent += fileContent;
finalContent += `\\n\\nEnd. Total combinations:
\${nbrComb}\`;

return {
    content: finalContent,
    count: nbrComb,
    type: generateType,
    fileName: fileName
};
}

// Import generated combinations from text based on generation
type
importCombinationsFromText(text, type, fileName) {
    try {
        if (type === 'singleFile') {
            // Parse single file format
            const pattern = /SLOT (\d+)\n([\s\S]*?)(?=\n-
{20}|\nZ)/g;

            let match;
            let count = 0;

            // Create or clear the target file
            if (!this.memorySlots[fileName]) {
                this.memorySlots[fileName] = {};
            } else {
                // Clear existing slots
                this.memorySlots[fileName] = {};
            }

            while ((match = pattern.exec(text)) !== null) {

```

```

        const slotNumber = match[1];
        let value = match[2].trim();

        // Store in memory slots
        this.memorySlots[fileName][slotNumber] =
value;

        count++;
    }

    return `Imported ${count} combinations into file
"${fileName}"`;
    }
    else if (type === 'multiFile') {
        // Parse multi-file format
        const filePattern = /FILE ((\d+)\nSLOT
((\d+)\n([\s\S]*?)(?=\n-{20}|\nZ))/g;
        let fileMatch;
        let count = 0;

        while ((fileMatch = filePattern.exec(text)) !==
null) {

            const fileNumber = fileMatch[1];
            const slotNumber = fileMatch[2];
            let value = fileMatch[3].trim();

            // Create file if it doesn't exist
            if (!this.memorySlots[fileNumber]) {
                this.memorySlots[fileNumber] = {};
            }

            // Store in memory slots
            this.memorySlots[fileNumber][slotNumber] =
value;

            count++;
        }

        // Fallback for standard format without FILE
headers

        if (count === 0) {
            const pattern = /SLOT ((\d+)\n([\s\S]*?)(?
=\n-{20}|\nZ))/g;

            let match;
            let fileNumber = 1;

            while ((match = pattern.exec(text)) !== null)
            {

                const slotNumber = match[1];
                let value = match[2].trim();

                // Create file if it doesn't exist
                if
(!this.memorySlots[fileNumber.toString()]) {
this.memorySlots[fileNumber.toString()] = {};

```



```

        }

        // Store in memory slots
        this.memorySlots[fileNumber.toString()]

        count++;
        fileNumber++;
    }
}

return `Imported ${count} combinations into
separate files`;

    }

    return "Unknown generation type for import";
} catch (e) {
    return `Error importing combinations:
    ${e.message}`;
}
}

// Clear all memory slots
clearAll() {
    this.memorySlots = {};
    return 'All memory slots cleared';
}
}

// Proof by Contradiction Engine integration
class ProofByContradictionEngine {
    constructor() {
        this.statementsFileId = "statements"; // File ID for
storing statements in memory manager
        this.char_set =
"0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ+-*/=<>!@#$$%^&()[]
{ }, ; : . ? ~ \ ` | " ;

        this.MAX_STATEMENT_LENGTH = 256;
        this.NUM_CHARACTERS = this.char_set.length;

        // Initialize the statements file
        if (!memoryManager.memorySlots[this.statementsFileId]) {
            memoryManager.memorySlots[this.statementsFileId] = {};
        }
    }

    clearStatements() {
        memoryManager.memorySlots[this.statementsFileId] = {};
        return "All statements cleared.";
    }

    addStatement(statement) {
        // Get the next slot number
        let slotNumber = 1;
        const existingSlots =

```

```

Object.keys(memoryManager.memorySlots[this.statementsFileId])
    .map(key => parseInt(key))
    .filter(num => !isNaN(num));

    if (existingSlots.length > 0) {
        slotNumber = Math.max(...existingSlots) + 1;
    }

    // Store the statement
    memoryManager.memorySlots[this.statementsFileId]
[slotNumber] = statement;
    return slotNumber;
}

getStatements() {
    const statements = [];
    const slots =
memoryManager.memorySlots[this.statementsFileId];

    const sortedKeys = Object.keys(slots)
        .map(key => parseInt(key))
        .filter(num => !isNaN(num))
        .sort((a, b) => a - b);

    for (const key of sortedKeys) {
        statements.push(slots[key]);
    }

    return statements;
}

generateStatement() {
    const length = Math.floor(Math.random() *
this.MAX_STATEMENT_LENGTH) + 1;
    let statement = '';

    for (let i = 0; i < length; i++) {
        statement += this.char_set[Math.floor(Math.random() *
this.NUM_CHARACTERS)];
    }

    return statement;
}

generateStatements(numStatements) {
    this.clearStatements();

    for (let i = 0; i < numStatements; i++) {
        const statement = this.generateStatement();
        this.addStatement(statement);
    }

    return ``\${numStatements} statements generated.`;
}

```

```

checkContradiction() {
    const statements = this.getStatements();

    // Implementation of rule_statement_not_statement
    for (let i = 0; i < statements.length; i++) {
        const statement = statements[i];
        const notStatement = `NOT \${statement}`;

        for (let j = 0; j < statements.length; j++) {
            if (i !== j && statements[j] === notStatement) {
                return {
                    found: true,
                    indices: [i, j],
                    statements: [statement, notStatement]
                };
            }
        }
    }

    // Implementation of rule_identical_statements
    for (let i = 0; i < statements.length; i++) {
        for (let j = i + 1; j < statements.length; j++) {
            if (statements[i] === statements[j]) {
                return {
                    found: true,
                    indices: [i, j],
                    statements: [statements[i],
statements[j]],

                    rule: "identical_statements"
                };
            }
        }
    }

    return { found: false };
}

processInstruction(instruction) {
    const parts = instruction.split(' ');
    const cmd = parts[0];

    switch (cmd) {
        case 'ASSERT':
            if (parts.length < 2) {
                return "Error: ASSERT requires a statement.";
            }
            const statement = parts.slice(1).join(' ');
            const slotNumber = this.addStatement(statement);
            return `Added statement "\${statement}" at
position \${slotNumber}.`;

        case 'NOT':
            if (parts.length < 2) {

```

```

        return "Error: NOT requires a statement.";
    }
    const notStatement = parts.slice(1).join(' ');
    const notSlotNumber = this.addStatement(`NOT
\${notStatement}`);
    return `Added statement "NOT \${notStatement}" at
position \${notSlotNumber}.`;

    case 'AND':
        if (parts.length < 3) {
            return "Error: AND requires two statements.";
        }
        const andStatement1 = parts[1];
        const andStatement2 = parts.slice(2).join(' ');
        const andSlotNumber =
this.addStatement(`(\${andStatement1}) AND (\${andStatement2})`);
        return `Added statement "(\${andStatement1}) AND
(\${andStatement2})" at position \${andSlotNumber}.`;

    case 'OR':
        if (parts.length < 3) {
            return "Error: OR requires two statements.";
        }
        const orStatement1 = parts[1];
        const orStatement2 = parts.slice(2).join(' ');
        const orSlotNumber =
this.addStatement(`\${orStatement1} OR \${orStatement2}`);
        return `Added statement "\${orStatement1} OR
\${orStatement2}" at position \${orSlotNumber}.`;

    case 'IMPLIES':
        if (parts.length < 3) {
            return "Error: IMPLIES requires two
statements.";
        }
        const impliesStatement1 = parts[1];
        const impliesStatement2 = parts.slice(2).join('
');
        const impliesSlotNumber =
this.addStatement(`\${impliesStatement1} IMPLIES \${impliesStatement2}`);
        return `Added statement "\${impliesStatement1}
IMPLIES \${impliesStatement2}" at position \${impliesSlotNumber}.`;

    case 'CONTRADICTION':
        if (parts.length < 3) {
            return "Error: CONTRADICTION requires two
statements.";
        }

        const contradictionStatement1 = parts[1];
        const contradictionStatement2 =
parts.slice(2).join(' ');

        // Check if the statements exist in our framework

```

```

        const statements = this.getStatements();
        let found1 = false, found2 = false;
        let index1 = -1, index2 = -1;

        for (let i = 0; i < statements.length; i++) {
            if (statements[i] === contradictionStatement1)
{
                found1 = true;
                index1 = i;
            }
            if (statements[i] === contradictionStatement2)
{
                found2 = true;
                index2 = i;
            }
        }

        if (found1 && found2) {
            return `Contradiction found between
statements \${index1 + 1} and \${index2 + 1}:\\n  \${contradictionStatement1}\\n
\${contradictionStatement2}\\`;
        } else {
            return "No contradiction found between the
specified statements.";
        }

        default:
            return `Unknown instruction: \${cmd}\\`;
        }
    }
}

// Initialize the applications
const memoryManager = new MemorySlotManager();
const logicEngine = new ProofByContradictionEngine();

// DOM Elements
const terminal = document.getElementById('terminal');
const commandInput = document.getElementById('commandInput');
const multilineInput = document.getElementById('multilineInput');
const submitCommandBtn = document.getElementById('submitCommand');
const submitMultilineBtn =
document.getElementById('submitMultiline');
const cancelMultilineBtn =
document.getElementById('cancelMultiline');
const memoryDisplay = document.getElementById('memoryDisplay');
const systemTimeEl = document.getElementById('systemTime');

const logicTerminal = document.getElementById('logicTerminal');
const logicCommandInput =
document.getElementById('logicCommandInput');
const submitLogicCommandBtn =
document.getElementById('submitLogicCommand');
const checkContradictionsBtn =

```

```

document.getElementById('checkContradictions');
    const clearStatementsBtn =
document.getElementById('clearStatements');
    const generateStatementsBtn =
document.getElementById('generateStatements');

    const generationModal =
document.getElementById('generationModal');
    const generationProgress =
document.getElementById('generationProgress');
    const generationStatus =
document.getElementById('generationStatus');
    const downloadGeneratedBtn =
document.getElementById('downloadGeneratedBtn');
    const importGeneratedBtn =
document.getElementById('importGeneratedBtn');
    const startGenerationBtn =
document.getElementById('startGenerationBtn');
    const fileNameInput = document.getElementById('fileNameInput');
    let generatedResults = null;

    // Set current time
    const now = new Date();
    systemTimeEl.textContent = `SYSTEM INITIALIZED:
\${now.toISOString().replace('T', ' ').substr(0, 19)}\`;

    // Add initial welcome message
    appendToTerminal('INTEGRATED MEMORY & LOGIC FRAMEWORK', 'system');
    appendToTerminal('Type "help" for a list of commands.', 'system');

    // Add initial welcome message to logic terminal
    appendToLogicTerminal('PROOF BY CONTRADICTION ENGINE INITIALIZED',
'system');
    appendToLogicTerminal('Type "help" for a list of instructions.',
'system');

    // Set up tab navigation
    document.querySelectorAll('.nav-tab').forEach(tab => {
        tab.addEventListener('click', function() {
            // Remove active class from all tabs
            document.querySelectorAll('.nav-tab').forEach(t => {
                t.classList.remove('active');
            });

            // Add active class to clicked tab
            this.classList.add('active');

            // Hide all tab content
            document.querySelectorAll('.tab-content').forEach(content
=> {
                content.classList.remove('active');
            });

            // Show the selected tab content

```

```

        const tabId = this.getAttribute('data-tab');

document.getElementById(`\`${tabId}Tab\`).classList.add('active');
    });
});

// Event handler for generate type change

document.querySelectorAll('input[name="generateType"]').forEach(radio => {
    radio.addEventListener('change', function() {
        const fileInputContainer =
document.getElementById('fileInputContainer');
        if (this.value === 'singleFile') {
            fileInputContainer.style.display = 'block';
        } else {
            fileInputContainer.style.display = 'none';
        }
    });
});

// Start generation button handler
startGenerationBtn.addEventListener('click', async () => {
    const generateType =
document.querySelector('input[name="generateType"]:checked').value;
    let fileName = fileNameInput.value.trim();

    // Default to "combinations" if empty
    if (generateType === 'singleFile' && !fileName) {
        fileName = 'combinations';
    }

    try {
        startGenerationBtn.disabled = true;
        downloadGeneratedBtn.disabled = true;
        importGeneratedBtn.disabled = true;
        generationProgress.style.width = '0%';
        generationStatus.textContent = 'Starting generation...';

        // Get n value from stored data
        const n = parseInt(generationModal.dataset.n);
        const chars = generationModal.dataset.customChars ||
memoryManager.generateCharSet();

        generatedResults = await
memoryManager.generateCombinations(
            chars,
            n,
            generateType,
            fileName,
            (progress, status) => {
                generationProgress.style.width =
`\`${progress}%\`;

                generationStatus.textContent = status;
            }

```

```

    );

    downloadGeneratedBtn.disabled = false;
    importGeneratedBtn.disabled = false;

    let modeDesc = generateType === 'singleFile' ?
        `single file "${file}"` :
        'separate files (one per combination)';

    generationStatus.textContent = `Generation complete!
    ${generatedResults.count} combinations generated in ${modeDesc}`;

    // If the count is manageable, automatically import
    if (generatedResults.count <= 1000) {
        const result =
memoryManager.importCombinationsFromText(
            generatedResults.content,
            generatedResults.type,
            generatedResults.fileName
        );
        appendToTerminal(result, 'system');
        updateMemoryDisplay();
    }
    } catch (e) {
        generationStatus.textContent = `Error during generation:
    ${e.message}`;
        appendToTerminal(`Error during generation:
    ${e.message}`, 'system');
    } finally {
        startGenerationBtn.disabled = false;
    }
    });

    // Generate statements button handler
    generateStatementsBtn.addEventListener('click', () => {

document.getElementById('statementsGenerationModal').style.display = 'block';
    });

    // Start statements generation button handler

document.getElementById('startStatementsGenBtn').addEventListener('click', () => {
    const numStatements =
parseInt(document.getElementById('numStatementsInput').value);
    if (isNaN(numStatements) || numStatements <= 0) {
        appendToLogicTerminal('Error: Please enter a valid
positive number.', 'system');
        return;
    }

    const result = logicEngine.generateStatements(numStatements);
    appendToLogicTerminal(result, 'system');

document.getElementById('statementsGenerationModal').style.display = 'none';

```



```

        // Display the current statements
        const statements = logicEngine.getStatements();
        let output = 'Current statements in framework: ';
        for (let i = 0; i < statements.length; i++) {
            output += `\\n\\${i + 1}. \\${statements[i]}\\`;
        }
        appendToLogicTerminal(output, 'system');
    });

    // Check contradictions button handler
    checkContradictionsBtn.addEventListener('click', () => {
        const statements = logicEngine.getStatements();

        if (statements.length === 0) {
            appendToLogicTerminal('No statements in framework.',
'system');
            return;
        }

        let output = 'Current statements in framework: ';
        for (let i = 0; i < statements.length; i++) {
            output += `\\n\\${i + 1}. \\${statements[i]}\\`;
        }
        appendToLogicTerminal(output, 'system');

        const result = logicEngine.checkContradiction();
        if (result.found) {
            let contradictionMsg = `Contradiction found between
statements \\${result.indices[0] + 1} and \\${result.indices[1] + 1}:\\n\\`;
            contradictionMsg += ` \\${result.statements[0]}\\n
\\${result.statements[1]}\\n\\`;

            if (result.rule) {
                contradictionMsg += `(Detected by \\${result.rule}
rule)\\`;
            }

            appendToLogicTerminal(contradictionMsg, 'system');
        } else {
            appendToLogicTerminal('No contradictions found in the
current framework.', 'system');
        }
    });

    // Clear statements button handler
    clearStatementsBtn.addEventListener('click', () => {
        const result = logicEngine.clearStatements();
        appendToLogicTerminal(result, 'system');
    });

    // Enable tab key in the multiline input
    multilineInput.addEventListener('keydown', function(e) {
        if (e.key === 'Tab') {

```

```

        e.preventDefault();

        // Insert a tab at the current cursor position
        const start = this.selectionStart;
        const end = this.selectionEnd;
        const value = this.value;

        // Insert the tab character
        this.value = value.substring(0, start) + '\\t' +
value.substring(end);

        // Move the cursor after the tab
        this.selectionStart = this.selectionEnd = start + 1;
    }
});

// Event listeners for memory manager
commandInput.addEventListener('keydown', (e) => {
    if (e.key === 'Enter') {
        const command = commandInput.value.trim();
        submitCommand(command);
    }
});

submitCommandBtn.addEventListener('click', () => {
    const command = commandInput.value.trim();
    submitCommand(command);
});

submitMultilineBtn.addEventListener('click', () => {
    const multilineValue = multilineInput.value;
    finishMultilineInput(multilineValue);
});

cancelMultilineBtn.addEventListener('click', () => {
    // Reset the UI
    resetToCommandMode();
    appendToTerminal('Multi-line input canceled.', 'system');
});

// Event listeners for logic engine
logicCommandInput.addEventListener('keydown', (e) => {
    if (e.key === 'Enter') {
        const command = logicCommandInput.value.trim();
        submitLogicCommand(command);
    }
});

submitLogicCommandBtn.addEventListener('click', () => {
    const command = logicCommandInput.value.trim();
    submitLogicCommand(command);
});

downloadGeneratedBtn.addEventListener('click', () => {

```

```

        if (generatedResults) {
            const blob = new Blob([generatedResults.content], { type:
'text/plain' });

            const url = URL.createObjectURL(blob);
            const a = document.createElement('a');
            a.href = url;
            a.download = 'GENERATED_COMBINATIONS.txt';
            a.click();
            URL.revokeObjectURL(url);
        }
    });

importGeneratedBtn.addEventListener('click', () => {
    if (generatedResults) {
        // Import the generated combinations into memory slots
        const result = memoryManager.importCombinationsFromText(
            generatedResults.content,
            generatedResults.type,
            generatedResults.fileName
        );
        appendToTerminal(result, 'system');
        updateMemoryDisplay();
        document.getElementById('generationModal').style.display =
'none';
    }
});

// Function to append text to the terminal
function appendToTerminal(text, type = 'command') {
    const entryDiv = document.createElement('div');

    if (type === 'input') {
        entryDiv.className = 'terminal-input';
        entryDiv.innerHTML = \`${escapeHtml(text)}\`;
    } else if (type === 'system') {
        entryDiv.className = 'terminal-system';
        entryDiv.textContent = text;
    } else {
        entryDiv.className = 'terminal-output';
        entryDiv.innerHTML = text.replace(/\n/g, '<br>');
    }

    terminal.appendChild(entryDiv);
    terminal.scrollTop = terminal.scrollHeight;
}

// Function to append text to the logic terminal
function appendToLogicTerminal(text, type = 'command') {
    const entryDiv = document.createElement('div');

    if (type === 'input') {
        entryDiv.className = 'terminal-input';
        entryDiv.innerHTML = \`${escapeHtml(text)}\`;
    }

```

```

\${escapeHtml(text)}\` ;
    } else if (type === 'system') {
        entryDiv.className = 'terminal-system';
        entryDiv.textContent = text;
    } else {
        entryDiv.className = 'terminal-output';
        entryDiv.innerHTML = text.replace(/\n/g, '<br>');
    }

    logicTerminal.appendChild(entryDiv);
    logicTerminal.scrollTop = logicTerminal.scrollHeight;
}

function escapeHtml(text) {
    const div = document.createElement('div');
    div.textContent = text;
    return div.innerHTML;
}

// Clear the terminal
function clearTerminal() {
    terminal.innerHTML = '';
    addToTerminal('Terminal cleared.', 'system');
}

// Clear the logic terminal
function clearLogicTerminal() {
    logicTerminal.innerHTML = '';
    addToLogicTerminal('Terminal cleared.', 'system');
}

// Process a command for memory manager
async function submitCommand(command) {
    if (!command) return;

    addToTerminal(command, 'input');
    commandInput.value = '';

    const commandParts = command.split(' ');
    const cmd = commandParts[0].toLowerCase();

    try {
        switch (cmd) {
            case 'help':
            case '?':
                document.getElementById('helpModal').style.display
= 'block';

                break;

            case 'assign':
                if (commandParts.length < 3) {
                    addToTerminal('Error: Missing arguments for
assign command.', 'system');

                    break;

```

```

    }

    const fileNumber = commandParts[1];
    const slotNumber = commandParts[2];

    // Switch to multi-line input mode
    switchToMultilineMode('assign', fileNumber,
slotNumber);

    break;

    case 'read':
        if (commandParts.length < 3) {
            appendToTerminal('Error: Missing arguments for
read command.', 'system');

            break;
        }

        const readResult =
memoryManager.readSlot(commandParts[1], commandParts[2]);
        appendToTerminal(readResult, 'system');
        break;

    case 'last_slot':
        if (commandParts.length < 2) {
            appendToTerminal('Error: Missing file number
for last_slot command.', 'system');

            break;
        }

        const lastSlot =
memoryManager.getLastSlotNumber(commandParts[1]);
        if (lastSlot !== null) {
            appendToTerminal(`Last slot number in file
\${commandParts[1]} is \${lastSlot}.`, 'system');
        } else {
            appendToTerminal(`No slots found in file
\${commandParts[1]}.`, 'system');
        }
        break;

    case 'search':
        // Switch to multi-line input mode
        switchToMultilineMode('search');
        break;

    case 'call':
        if (commandParts.length < 4) {
            appendToTerminal('Error: Missing arguments for
call command.', 'system');

            break;
        }

        try {
            const callFileNumber = commandParts[1];

```

```

        const startSlot = parseInt(commandParts[2]);
        const endSlot = parseInt(commandParts[3]);
        const callResult =
memoryManager.callSlotRange(callFileNumber, startSlot, endSlot);
        appendToTerminal(callResult, 'system');
    } catch (e) {
        appendToTerminal('Error: Slot numbers must be
integers.', 'system');
    }
    break;

    case 'export':
        if (commandParts.length < 2) {
            appendToTerminal('Error: Missing file number
for export command.', 'system');
            break;
        }

        const exportResult =
memoryManager.exportToTextFile(commandParts[1]);
        appendToTerminal(exportResult, 'system');
        break;

    case 'generate':
        if (commandParts.length < 2) {
            appendToTerminal('Error: Please specify the
number of cells (n).', 'system');
            break;
        }

        const n = parseInt(commandParts[1]);
        if (n <= 0) {
            appendToTerminal('Error: n must be a positive
integer.', 'system');
            break;
        }

        if (n > 4) {
            appendToTerminal('Warning: Large values of n
will generate very large outputs.', 'system');
            if (!confirm('Continue with generating a large
number of combinations? n=' + n)) {
                appendToTerminal('Generation canceled.',
'system');
                break;
            }
        }

        const charSet = memoryManager.generateCharSet();
        appendToTerminal(`Using standard character set
(${charSet.length} chars)` , 'system');

        // Show the generation modal with options
        generationModal.style.display = 'block';

```

```

        generationProgress.style.width = '0%';
        generationStatus.textContent = 'Select options and
click "Start Generation"';

        // Set up initial state for the modal

document.querySelector('#generateSingleFile').checked = true;

document.getElementById('fileInputContainer').style.display = 'block';
        fileNameInput.value = 'combinations';
        downloadGeneratedBtn.disabled = true;
        importGeneratedBtn.disabled = true;
        startGenerationBtn.disabled = false;

        // Store n value for later use
        generationModal.dataset.n = n;
        generationModal.dataset.customChars = '';

        break;

        case 'custom_generate':
            if (commandParts.length < 2) {
                appendToTerminal('Error: Please specify the
number of cells (n).', 'system');
                break;
            }

            const customN = parseInt(commandParts[1]);
            if (customN <= 0) {
                appendToTerminal('Error: n must be a positive
integer.', 'system');
                break;
            }

            // Switch to multi-line input mode for custom
            character set

            switchToMultilineMode('custom_generate', customN);
            break;

        case 'display':
        case 'show':
            updateMemoryDisplay();
            appendToTerminal('Memory display updated.',
'system');
            break;

        case 'clear':
            clearTerminal();
            break;

        case 'save_json':
            const saveFileNumber = commandParts.length > 1 ?
commandParts[1] : null;
            const saveResult =

```

```

memoryManager.saveToJSON(saveFileNumber);
        appendToTerminal(saveResult, 'system');
        break;

        case 'save_all':
            const saveAllResult =
memoryManager.saveAllAsZip();
            appendToTerminal(saveAllResult, 'system');
            break;

        case 'load_json':
            // Create a file input element
            const fileInput = document.createElement('input');
            fileInput.type = 'file';
            fileInput.accept = '.json';
            fileInput.style.display = 'none';
            document.body.appendChild(fileInput);

            fileInput.onchange = function(e) {
                const file = e.target.files[0];
                if (!file) {
                    appendToTerminal('No file selected.',
'system');

                    return;
                }

                memoryManager.loadFromFile(file)
                    .then(result => {
                        appendToTerminal(result, 'system');
                        updateMemoryDisplay();
                    })
                    .catch(error => {
                        appendToTerminal(`Error: \${error}`,
'system');

                    });
            });

            fileInput.click();
            document.body.removeChild(fileInput);
            break;

        case 'load_all':
            // Simplified version just using JSON
            const zipInput = document.createElement('input');
            zipInput.type = 'file';
            zipInput.accept = '.json';
            zipInput.style.display = 'none';
            document.body.appendChild(zipInput);

            zipInput.onchange = function(e) {
                const file = e.target.files[0];
                if (!file) {
                    appendToTerminal('No file selected.',
'system');

```



```

        return;
    }

    memoryManager.loadFromFile(file)
        .then(result => {
            appendToTerminal(result, 'system');
            updateMemoryDisplay();
        })
        .catch(error => {
            appendToTerminal(`Error: ${error}`,
'system');
        });

    });

    zipInput.click();
    document.body.removeChild(zipInput);
    break;

    case 'exit':
    case 'quit':
        appendToTerminal('This is a web application. To
exit, close the browser tab.', 'system');
        break;

    default:
        appendToTerminal(`Unknown command: ${cmd}`,
'system');
        appendToTerminal("Type 'help' for available
commands.", 'system');
    }
    } catch (error) {
        appendToTerminal(`Error executing command:
${error.message}`, 'system');
    }
}

// Process a command for logic engine
function submitLogicCommand(command) {
    if (!command) return;

    appendToLogicTerminal(command, 'input');
    logicCommandInput.value = '';

    if (command.toLowerCase() === 'help') {
        document.getElementById('logicHelpModal').style.display =
'block';
        return;
    }

    try {
        const result = logicEngine.processInstruction(command);
        appendToLogicTerminal(result, 'system');
    } catch (error) {
        appendToLogicTerminal(`Error: ${error.message}`,

```

```

'system');
    }
}

// Switch to multi-line input mode
function switchToMultilineMode(mode, ...args) {
    // Hide command input and button
    commandInput.style.display = 'none';
    submitCommandBtn.style.display = 'none';

    // Show multi-line input, submit and cancel buttons
    multilineInput.style.display = 'block';
    submitMultilineBtn.style.display = 'inline-block';
    cancelMultilineBtn.style.display = 'inline-block';

    // Clear the multi-line input
    multilineInput.value = '';

    // Focus the multi-line input
    multilineInput.focus();

    // Store the mode and arguments
    multilineInput.dataset.mode = mode;
    multilineInput.dataset.args = JSON.stringify(args);

    // Set appropriate placeholder text
    if (mode === 'assign') {
        multilineInput.placeholder = `Enter value for file
\${args[0]}, slot \${args[1]} (TAB key supported)`;
    } else if (mode === 'search') {
        multilineInput.placeholder = 'Enter search value (TAB key
supported)';
    } else if (mode === 'custom_generate') {
        multilineInput.placeholder = 'Enter custom character set
for combination generation';
    }

    appendToTerminal(`Enter multi-line value for "\${mode}"
(Submit when finished):`, 'system');
}

// Finish multi-line input
async function finishMultilineInput(value) {
    const mode = multilineInput.dataset.mode;
    const args = JSON.parse(multilineInput.dataset.args || '[]');

    resetToCommandMode();

    if (mode === 'assign') {
        const [fileNumber, slotNumber] = args;
        const result = memoryManager.assignSlot(fileNumber,
slotNumber, value);

        appendToTerminal(result, 'system');
        updateMemoryDisplay();
    }
}

```

```

        } else if (mode === 'search') {
            if (!value.trim()) {
                appendToTerminal('Error: No search value provided.',
'system');

                return;
            }

            const searchResults = memoryManager.searchValue(value);
            if (searchResults.length > 0) {
                let resultOutput = 'Search Results: ';
                for (const result of searchResults) {
                    resultOutput += `\\nFile: ${result.file}, Slot:
\\${result.slot}, Value:\\n\\${result.value}\\n\\`;
                }
                appendToTerminal(resultOutput, 'system');
            } else {
                appendToTerminal('No results found.', 'system');
            }
        } else if (mode === 'custom_generate') {
            const customN = args[0];
            const customCharSet = value.trim() ||
memoryManager.generateCharSet();

            appendToTerminal(`Using \\${value.trim()} ? 'custom' :
'default'} character set (\\${customCharSet.length} chars)\\`, 'system');

            // Show the generation modal with options
            generationModal.style.display = 'block';
            generationProgress.style.width = '0%';
            generationStatus.textContent = 'Select options and click
"Start Generation"';

            // Set up initial state for the modal
            document.querySelector('#generateSingleFile').checked =
true;

            document.getElementById('fileInputContainer').style.display = 'block';
            fileNameInput.value = 'combinations';
            downloadGeneratedBtn.disabled = true;
            importGeneratedBtn.disabled = true;
            startGenerationBtn.disabled = false;

            // Store data for later use
            generationModal.dataset.n = customN;
            generationModal.dataset.customChars = customCharSet;
        }
    }

    // Reset to command mode
    function resetToCommandMode() {
        multilineInput.style.display = 'none';
        submitMultilineBtn.style.display = 'none';
        cancelMultilineBtn.style.display = 'none';
    }

```

```

        commandInput.style.display = 'block';
        submitCommandBtn.style.display = 'inline-block';

        commandInput.focus();
    }

    // Update the memory display
    function updateMemoryDisplay() {
        const memorySlots = memoryManager.memorySlots;

        if (Object.keys(memorySlots).length === 0) {
            memoryDisplay.innerHTML = `
                <div style="text-align: center; color: #888; margin-
top: 20px;">
                    No memory slots available.
                    <br><br>
                    Use the terminal to add memory slots.
                </div>
            `;
            return;
        }

        let html = '';

        // Sort file numbers numerically if possible
        const sortedFiles = Object.keys(memorySlots).sort((a, b) => {
            if (/^\d+$/.test(a) && /^\d+$/.test(b)) {
                return parseInt(a) - parseInt(b);
            }
            return a.localeCompare(b);
        });

        for (const fileNumber of sortedFiles) {
            const slots = memorySlots[fileNumber];
            const slotCount = Object.keys(slots).length;

            // Skip displaying the statements file in memory display -
            // it's shown in the logic terminal
            if (fileNumber === logicEngine.statementsFileId) {
                continue;
            }

            html += `<div class="memory-file">`;
            html += `
                <div class="memory-file-title">
                    FILE \${fileNumber} (\${slotCount} slots)
                <div class="memory-file-actions">
                    <button class="file-action"
onclick="exportFile('\${fileNumber}')">Export</button>
                    <button class="file-action"
onclick="saveFileAsJson('\${fileNumber}')">Save JSON</button>
                </div>
                </div>
            `;
        }
    }

```

```

        // Sort slot numbers numerically if possible
        const sortedSlots = Object.keys(slots).sort((a, b) => {
            if (/^\d+$/.test(a) && /^\d+$/.test(b)) {
                return parseInt(a) - parseInt(b);
            }
            return a.localeCompare(b);
        });

        // Limit display to max 10 slots per file for readability
        const displaySlots = sortedSlots.slice(0, 10);
        const remainingSlots = sortedSlots.length - 10;

        for (const slotNumber of displaySlots) {
            const value = slots[slotNumber];

            // Truncate long values for display
            let displayValue = value;
            if (displayValue.length > 100) {
                displayValue = displayValue.substring(0, 97) +
'...';
            }

            html += `

\`;
            html += `

Slot
\${slotNumber}</div>\`;

            html += `

\${escapeHtml(displayValue)}</div>\`;
            html += `</div>\`;
        }

        if (remainingSlots > 0) {
            html += `

... and \${remainingSlots} more slots (not
displayed)

</div>\`;
        }

        html += `</div>\`;
    }

    if (html === '') {
        memoryDisplay.innerHTML = `
<div style="text-align: center; color: #888; margin-
top: 20px;">

            No memory slots available.
            <br><br>
            Use the terminal to add memory slots.
        </div>
    `;
    } else {
        memoryDisplay.innerHTML = html;
    }
}


```

```

    }

    // Export a file
    function exportFile(fileNumber) {
        const result = memoryManager.exportToTextFile(fileNumber);
        appendToTerminal(result, 'system');
    }

    // Save a file as JSON
    function saveFileAsJson(fileNumber) {
        const result = memoryManager.saveToJSON(fileNumber);
        appendToTerminal(result, 'system');
    }

    // Save all memory slots to a file
    function saveAllMemorySlots() {
        const result = memoryManager.saveAllAsZip();
        appendToTerminal(result, 'system');
    }

    // Load memory slots from a file
    async function loadMemorySlotsFromFile() {
        // Create a file input element
        const fileInput = document.createElement('input');
        fileInput.type = 'file';
        fileInput.accept = '.json';
        fileInput.style.display = 'none';
        document.body.appendChild(fileInput);

        fileInput.onchange = function(e) {
            const file = e.target.files[0];
            if (!file) {
                appendToTerminal('No file selected.', 'system');
                return;
            }

            memoryManager.loadFromFile(file)
                .then(result => {
                    appendToTerminal(result, 'system');
                    updateMemoryDisplay();
                })
                .catch(error => {
                    appendToTerminal(`Error: ${error}`, 'system');
                });
        };

        fileInput.click();
        document.body.removeChild(fileInput);
    }

    // Clear all memory slots
    function clearAllMemorySlots() {
        if (confirm('Are you sure you want to clear all memory slots?
This cannot be undone.')) {

```

```

        const result = memoryManager.clearAll();
        appendToTerminal(result, 'system');
        updateMemoryDisplay();
    }
}

// Insert command to input field
function insertCommand(command) {
    commandInput.value = command;
    commandInput.focus();
}

// Insert logic command to logic input field
function insertLogicCommand(command) {
    logicCommandInput.value = command;
    logicCommandInput.focus();
}

// Make utility functions global for access from HTML
window.exportFile = exportFile;
window.saveFileAsJson = saveFileAsJson;
window.saveAllMemorySlots = saveAllMemorySlots;
window.loadMemorySlotsFromFile = loadMemorySlotsFromFile;
window.clearAllMemorySlots = clearAllMemorySlots;
window.clearTerminal = clearTerminal;
window.clearLogicTerminal = clearLogicTerminal;
window.updateMemoryDisplay = updateMemoryDisplay;
window.insertCommand = insertCommand;
window.insertLogicCommand = insertLogicCommand;
window.submitCommand = submitCommand;

// Check for parent frame communication
if (window.parent !== window) {
    try {
        // Notify parent we're loaded
        if (window.parent.postMessage) {
            window.parent.postMessage({
                action: 'iframe-loaded',
                data: {
                    message: 'Build System iframe loaded'
                }
            }, '*');
        }

        // Check if parent provided an API
        if (window.parentAPI) {
            console.log('Parent API available');
        }
    } catch (e) {
        console.warn('Error communicating with parent frame:', e);
    }
}

</script>
</body>

```

```

    </html>`;
}

/**
 * Access the memory manager
 * @returns {Object|null} Memory manager object
 */
function getMemoryManager() {
    return _memoryManager;
}

/**
 * Access the logic engine
 * @returns {Object|null} Logic engine object
 */
function getLogicEngine() {
    return _logicEngine;
}

/**
 * Access the iframe window
 * @returns {Window|null} Iframe window object
 */
function getIframeWindow() {
    return _iframe ? _iframe.contentWindow : null;
}

/**
 * Execute a command in the memory manager
 * @param {string} command - Command to execute
 * @returns {Promise<any>} Command result
 */
function executeMemoryCommand(command) {
    return executeCommand(command);
}

/**
 * Execute a command in the logic engine
 * @param {string} command - Command to execute
 * @returns {Promise<any>} Command result
 */
function executeLogicCommand(command) {
    return new Promise((resolve, reject) => {
        if (!_iframe || !_iframe.contentWindow) {
            reject(new Error('Iframe not available'));
            return;
        }

        try {
            // Process the command
            const iframeWindow = _iframe.contentWindow;

            if (iframeWindow.submitLogicCommand) {
                // For logic terminal commands
            }
        }
    });
}

```



```

        iframeWindow.submitLogicCommand(command);
        resolve(true);
    } else {
        reject(new Error('Logic command execution not available'));
    }
} catch (error) {
    reject(error);
}
});
}

// Public API
return {
    initialize,
    loadBuildSystem,
    showBuildSection,
    getMemoryManager,
    getLogicEngine,
    getIframeWindow,
    executeMemoryCommand,
    executeLogicCommand,
    onLoad
};
})();

// Automatically initialize when the DOM is ready
document.addEventListener('DOMContentLoaded', function() {
    // Look for the main container
    const mainElement = document.querySelector('main');
    if (mainElement) {
        OperatorFramework.Build.initialize({
            containerId: mainElement.id || 'main'
        });
        console.log('Build System initialized');
    } else {
        console.warn('Main element not found, Build System not automatically
initialized');
    }
});

```

[4] operator-dropdown.js

- **Bytes:** 17992
- **Type:** text

```

/**
 * operator-dropdown.js
 *
 * This file provides the dropdown menu system for the Operator framework,
 * allowing seamless integration of different subsystems into the main interface.
 *

```

```

* @module OperatorDropdown
* @version 1.0.0
* @author Claude AI
*/

// Access the shared namespace for Operator framework

/**
 * Dropdown system for integrating external systems
 * @namespace OperatorDropdown
 */
OperatorFramework.Dropdown = (function() {
    'use strict';

    // Private variables
    let _initialized = false;
    let _availableSystems = [];
    let _activeSystem = null;
    let _navigationElement = null;

    /**
     * System configuration object format
     * @typedef {Object} SystemConfig
     * @property {string} id - Unique identifier for the system
     * @property {string} name - Display name for the system
     * @property {string} description - Brief description of the system
     * @property {Function} loadFunction - Function to call to load the system
     * @property {boolean} [isLoading=false] - Whether the system is currently
loaded
    */

    /**
     * Initialize the dropdown system
     * @param {string} navElementId - ID of the DOM element to contain the
dropdown
     * @returns {boolean} Success status
    */
    function initialize(navElementId) {
        if (_initialized) {
            console.warn('OperatorDropdown is already initialized');
            return false;
        }

        // Find the navigation element
        _navigationElement = document.getElementById(navElementId);
        if (!_navigationElement) {
            console.error(`Navigation element with ID "${navElementId}" not
found`);
            return false;
        }

        // Add styles for the dropdown
        _addDropdownStyles();
    }

```

```

    // Create the initial dropdown structure
    _createDropdownStructure();

    _initialized = true;
    console.log('OperatorDropdown initialized successfully');
    return true;
}

/**
 * Add a system to the dropdown
 * @param {SystemConfig} systemConfig - The system configuration
 * @returns {boolean} Success status
 */
function addSystem(systemConfig) {
    if (!_initialized) {
        console.error('OperatorDropdown not initialized, call initialize() first');
        return false;
    }

    // Validate the system config
    if (!_validateSystemConfig(systemConfig)) {
        return false;
    }

    // Check for duplicate IDs
    if (_availableSystems.some(sys => sys.id === systemConfig.id)) {
        console.error(`System with ID "${systemConfig.id}" already exists`);
        return false;
    }

    // Add default properties if not provided
    const system = {
        ...systemConfig,
        isLoading: false
    };

    // Add to available systems
    _availableSystems.push(system);

    // Add to the dropdown menu
    _addSystemToDropdown(system);

    console.log(`Added system "${system.name}" to dropdown`);
    return true;
}

/**
 * Load a system by ID
 * @param {string} systemId - ID of the system to load
 * @returns {boolean} Success status
 */
function loadSystem(systemId) {
    if (!_initialized) {

```

```

        console.error('OperatorDropdown not initialized, call initialize()
first');
        return false;
    }

    // Find the system
    const system = _availableSystems.find(sys => sys.id === systemId);
    if (!system) {
        console.error(`System with ID "${systemId}" not found`);
        return false;
    }

    // Don't reload if already active
    if (_activeSystem === systemId && system.isLoaded) {
        console.log(`System "${system.name}" is already active`);
        return true;
    }

    try {
        // Call the system's load function
        system.loadFunction();

        // Update system status
        system.isLoaded = true;
        _activeSystem = systemId;

        // Update UI to indicate active system
        _updateActiveSystemUI(systemId);

        console.log(`Successfully loaded system "${system.name}"`);
        return true;
    } catch (error) {
        console.error(`Error loading system "${system.name}":`, error);
        return false;
    }
}

/**
 * Unload the currently active system
 * @returns {boolean} Success status
 */
function unloadActiveSystem() {
    if (!_activeSystem) {
        console.log('No active system to unload');
        return false;
    }

    const system = _availableSystems.find(sys => sys.id === _activeSystem);
    if (!system) {
        console.error(`Active system with ID "${_activeSystem}" not found`);
        return false;
    }

    if (typeof system.unloadFunction === 'function') {

```

```

        try {
            system.unloadFunction();
        } catch (error) {
            console.error(`Error unloading system "${system.name}":`, error);
        }
    }

    system.isLoaded = false;
    _activeSystem = null;

    // Update UI
    _updateActiveSystemUI(null);

    console.log(`Unloaded system "${system.name}"`);
    return true;
}

/**
 * Get the currently active system
 * @returns {string|null} ID of the active system, or null if none
 */
function getActiveSystem() {
    return _activeSystem;
}

/**
 * Get a list of all available systems
 * @returns {Array<SystemConfig>} Array of system configurations
 */
function getAllSystems() {
    return [..._availableSystems];
}

/**
 * Check if a system is loaded
 * @param {string} systemId - ID of the system to check
 * @returns {boolean} Whether the system is loaded
 */
function isSystemLoaded(systemId) {
    const system = _availableSystems.find(sys => sys.id === systemId);
    return system ? system.isLoaded : false;
}

/**
 * Validate a system configuration object
 * @private
 * @param {SystemConfig} config - The system configuration to validate
 * @returns {boolean} Whether the configuration is valid
 */
function _validateSystemConfig(config) {
    if (!config) {
        console.error('System configuration is required');
        return false;
    }
}

```

```

    if (!config.id || typeof config.id !== 'string') {
        console.error('System must have a valid string ID');
        return false;
    }

    if (!config.name || typeof config.name !== 'string') {
        console.error('System must have a valid string name');
        return false;
    }

    if (!config.loadFunction || typeof config.loadFunction !== 'function') {
        console.error('System must have a valid load function');
        return false;
    }

    return true;
}

/**
 * Add CSS styles for the dropdown system
 * @private
 */
function _addDropdownStyles() {
    const styleElement = document.createElement('style');
    styleElement.textContent = `
        /* Operator Dropdown Styles */
        .op-dropdown {
            position: relative;
            display: inline-block;
            width: 100%;
            margin-bottom: 10px;
        }

        .op-dropdown-button {
            width: 100%;
            padding: 10px;
            text-align: center;
            cursor: pointer;
            background-color: var(--primary-color, #4B5320);
            color: white;
            border: 2px solid var(--secondary-color, #BDB76B);
            transition: background-color 0.3s ease;
            font-family: var(--font-main, "Fira Code", "Consolas", monospace);
        }

        .op-dropdown-button:hover {
            background-color: var(--secondary-color, #BDB76B);
            color: var(--text-color, #333333);
        }

        .op-dropdown-content {
            display: none;
            position: absolute;
    `;
}

```

```

        background-color: white;
        min-width: 160px;
        box-shadow: 0px 8px 16px 0px rgba(0,0,0,0.2);
        z-index: 1000;
        width: 100%;
    }

    .op-dropdown:hover .op-dropdown-content {
        display: block;
    }

    .op-system-button {
        width: 100%;
        text-align: left;
        padding: 10px;
        display: block;
        background-color: white;
        color: var(--text-color, #333333);
        border: none;
        border-bottom: 1px solid #e0e0e0;
        cursor: pointer;
        transition: background-color 0.2s ease;
        font-family: var(--font-main, "Fira Code", "Consolas", monospace);
    }

    .op-system-button:hover {
        background-color: var(--background-color, #F5F5DC);
    }

    .op-system-button.active {
        border-left: 5px solid var(--primary-color, #4B5320);
        background-color: var(--background-color, #F5F5DC);
        font-weight: bold;
    }

    .op-system-tooltip {
        position: absolute;
        background-color: #333;
        color: white;
        padding: 5px 10px;
        border-radius: 4px;
        font-size: 0.8rem;
        z-index: 1001;
        visibility: hidden;
        opacity: 0;
        transition: opacity 0.3s;
        max-width: 200px;
        /* Change these properties */
        right: 105%;    /* Position to the left instead of using left
property */
        left: auto;    /* Clear the left property */
        top: 50%;
        transform: translateY(-50%);
    }

```

```

        /* Add a small triangle/arrow pointing right */
        .op-system-tooltip::after {
            content: "";
            position: absolute;
            top: 50%;
            left: 100%; /* Position at the right edge */
            margin-top: -5px;
            border-width: 5px;
            border-style: solid;
            border-color: transparent transparent transparent #333; /* Make
triangle point right */
        }

        .op-system-button:hover .op-system-tooltip {
            visibility: visible;
            opacity: 1;
        }
    `;
    document.head.appendChild(styleElement);
}

/**
 * Create the initial dropdown structure
 * @private
 */
function _createDropdownStructure() {
    const navSection = document.createElement('div');
    navSection.className = 'nav-section';
    navSection.innerHTML = `
        <h3>External Systems</h3>
        <div class="op-dropdown">
            <button class="op-dropdown-button">Select System</button>
            <div class="op-dropdown-content" id="op-systems-list">
                <!-- Systems will be added here -->
                <div class="op-no-systems">No systems available</div>
            </div>
        </div>
    `;

    _navigationElement.appendChild(navSection);
}

/**
 * Add a system to the dropdown UI
 * @private
 * @param {SystemConfig} system - The system to add
 */
function _addSystemToDropdown(system) {
    const systemsList = document.getElementById('op-systems-list');

    // Remove the "no systems" message if it exists
    const noSystems = systemsList.querySelector('.op-no-systems');
    if (noSystems) {

```



```

        systemsList.removeChild(noSystems);
    }

    // Create button for the system
    const systemButton = document.createElement('button');
    systemButton.id = `op-system-${system.id}`;
    systemButton.className = 'op-system-button';
    systemButton.dataset.systemId = system.id;
    systemButton.innerHTML = `
        ${system.name}
        <span class="op-system-tooltip">${system.description || system.name}
</span>
    `;

    // Add click handler
    systemButton.addEventListener('click', () => {
        loadSystem(system.id);
    });

    // Add to the dropdown
    systemsList.appendChild(systemButton);
}

/**
 * Update the UI to indicate the active system
 * @private
 * @param {string|null} systemId - ID of the active system, or null if none
 */
function _updateActiveSystemUI(systemId) {
    // Remove active class from all buttons
    const allButtons = document.querySelectorAll('.op-system-button');
    allButtons.forEach(button => {
        button.classList.remove('active');
    });

    // Add active class to the active system
    if (systemId) {
        const activeButton = document.getElementById(`op-system-${systemId}`);
        if (activeButton) {
            activeButton.classList.add('active');
        }
    }

    // Update dropdown button text
    const dropdownButton = document.querySelector('.op-dropdown-button');
    if (dropdownButton) {
        if (systemId) {
            const system = _availableSystems.find(sys => sys.id === systemId);
            dropdownButton.textContent = system ? system.name : 'Select
System';
        } else {
            dropdownButton.textContent = 'Select System';
        }
    }
}

```

```

}

/**
 * Search for systems by name or description
 * @param {string} query - The search query
 * @returns {Array<SystemConfig>} Array of matching system configurations
 */
function searchSystems(query) {
  if (!query || typeof query !== 'string') {
    return [..._availableSystems];
  }

  const normalizedQuery = query.toLowerCase();
  return _availableSystems.filter(system => {
    return system.name.toLowerCase().includes(normalizedQuery) ||
      (system.description &&
system.description.toLowerCase().includes(normalizedQuery));
  });
}

/**
 * Filter and update the dropdown based on a search query
 * @param {string} query - The search query
 */
function filterDropdown(query) {
  const matchingSystems = searchSystems(query);
  const systemsList = document.getElementById('op-systems-list');

  // Remove all existing items
  systemsList.innerHTML = '';

  if (matchingSystems.length === 0) {
    const noResults = document.createElement('div');
    noResults.className = 'op-no-systems';
    noResults.textContent = 'No matching systems found';
    systemsList.appendChild(noResults);
  } else {
    // Add matching systems
    matchingSystems.forEach(system => {
      _addSystemToDropdown(system);
    });

    // Restore active state
    if (_activeSystem) {
      _updateActiveSystemUI(_activeSystem);
    }
  }
}

/**
 * Add a search box to filter systems
 * @param {string} [placeholder="Search systems..."] - Placeholder text for
the search box
 */

```

```

function addSearchBox(placeholder = "Search systems...") {
  const navSection = _navigationElement.querySelector('.nav-section:last-child');
  if (!navSection) return;

  const searchContainer = document.createElement('div');
  searchContainer.className = 'op-search-container';
  searchContainer.style.padding = '10px 0';

  const searchInput = document.createElement('input');
  searchInput.type = 'text';
  searchInput.placeholder = placeholder;
  searchInput.className = 'op-search-input';
  searchInput.style.width = '100%';
  searchInput.style.padding = '8px';
  searchInput.style.border = '1px solid var(--secondary-color, #BDB76B)';
  searchInput.style.fontFamily = 'var(--font-main, "Fira Code", "Consolas", monospace)';

  searchInput.addEventListener('input', (e) => {
    filterDropdown(e.target.value);
  });

  searchContainer.appendChild(searchInput);
  navSection.appendChild(searchContainer);
}

// Public API
return {
  initialize,
  addSystem,
  loadSystem,
  unloadActiveSystem,
  getActiveSystem,
  getAllSystems,
  isSystemLoaded,
  searchSystems,
  filterDropdown,
  addSearchBox
};
})();

// Automatically initialize when the DOM is ready
document.addEventListener('DOMContentLoaded', function() {
  // Look for the main navigation element
  const mainNav = document.getElementById('main-nav');
  if (mainNav) {
    OperatorFramework.Dropdown.initialize('main-nav');

    // Optional: Add search capability
    OperatorFramework.Dropdown.addSearchBox();
  } else {
    console.warn('Main navigation element #main-nav not found, OperatorDropdown not initialized');
  }
});

```

```
}  
});
```

[5] operator-extensions.js

- **Bytes:** 35813
- **Type:** text

```
/**  
 * Operator Extensions  
 * Additional features for the Operator System  
 *  
 * This file adds advanced features to the quadtree implementation:  
 * - Custom cell colors  
 * - Custom cell text  
 * - Image insertion into cells (file upload and clipboard paste)  
 * - Enhanced PNG export  
 */  
  
(function() {  
    'use strict';  
  
    // Track when the page is fully loaded  
    document.addEventListener('DOMContentLoaded', function() {  
        // Initialize extensions after a short delay to ensure main app is loaded  
        setTimeout(initExtensions, 300);  
    });  
  
    /**  
     * Initialize extension features  
     */  
    function initExtensions() {  
        // Find tree operations section  
        const treeSection = document.getElementById('tree-operations');  
        if (!treeSection) {  
            console.error('Tree operations section not found');  
            return;  
        }  
  
        // Enhance the quadtree UI  
        enhanceQuadtreeUI();  
  
        // Create and add context menu  
        createContextMenu();  
  
        // Set up global clipboard paste handler  
        setupClipboardPaste();  
  
        // Override the export PNG functionality  
        overrideExportPNG();  
    }  
})
```

```

        console.log('Operator Extensions loaded successfully');
    }

    /**
     * Enhance the quadtree UI with additional controls
     */
    function enhanceQuadtreeUI() {
        const quadtreeContainer = document.getElementById('quadtree-container');
        if (!quadtreeContainer) {
            console.error('Quadtree container not found');
            return;
        }

        // Add right-click event listener to quadtree container
        quadtreeContainer.addEventListener('contextmenu',
handleQuadtreeContextMenu);

        // Add custom styles
        addCustomStyles();

        // Add file input for images (hidden by default)
        const fileInput = document.createElement('input');
        fileInput.type = 'file';
        fileInput.id = 'quadtree-image-input';
        fileInput.accept = 'image/*';
        fileInput.style.display = 'none';
        document.body.appendChild(fileInput);

        fileInput.addEventListener('change', handleImageUpload);
    }

    /**
     * Add custom styles for the context menu
     */
    function addCustomStyles() {
        const styleElement = document.createElement('style');
        styleElement.textContent = `
            /* Context Menu Styles */
            .quadtree-context-menu {
                position: absolute;
                background-color: white;
                border: 1px solid var(--secondary-color);
                border-radius: 4px;
                box-shadow: 0 2px 10px rgba(0, 0, 0, 0.2);
                padding: 8px 0;
                min-width: 180px;
                z-index: 1000;
                display: none;
            }

            .quadtree-context-menu.active {
                display: block;
            }
        `;
        document.head.appendChild(styleElement);
    }

```

```
.context-menu-item {
    padding: 8px 16px;
    cursor: pointer;
    transition: background-color 0.2s ease;
    display: flex;
    align-items: center;
}

.context-menu-item:hover {
    background-color: var(--background-color);
}

.context-menu-item-icon {
    margin-right: 10px;
    width: 16px;
    text-align: center;
}

.context-menu-separator {
    height: 1px;
    background-color: var(--secondary-color);
    margin: 6px 0;
    opacity: 0.3;
}

/* Color Picker */
.color-picker-container {
    padding: 8px 16px;
}

/* Cell customizations */
.quadtree-cell-custom-text {
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
    max-width: 90%;
    font-family: var(--font-main);
    pointer-events: none;
}

.quadtree-cell-image {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    object-fit: cover;
    pointer-events: none;
}
```

```
/* Hide the original cell content when customized */
.quadtree-cell.customized .cell-content {
  display: none;
}

/* Cell with focus for pasting */
.quadtree-cell.paste-focus {
  outline: 2px dashed var(--primary-color);
  z-index: 10;
  position: relative;
}

/* Paste indicator */
.paste-indicator {
  position: fixed;
  bottom: 20px;
  right: 20px;
  background-color: var(--background-color);
  border: 1px solid var(--secondary-color);
  border-left: 4px solid var(--primary-color);
  padding: 10px 15px;
  font-size: 0.9rem;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
  border-radius: 4px;
  z-index: 1000;
  display: none;
}

.paste-indicator.active {
  display: block;
  animation: fadeIn 0.3s ease-in-out;
}

/* Export loading indicator */
.export-loading {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.5);
  display: flex;
  justify-content: center;
  align-items: center;
  z-index: 2000;
}

.export-loading-content {
  background-color: white;
  padding: 20px;
  border-radius: 4px;
  text-align: center;
}
```

```
.export-spinner {
  border: 4px solid #f3f3f3;
  border-top: 4px solid var(--primary-color);
  border-radius: 50%;
  width: 40px;
  height: 40px;
  animation: spin 2s linear infinite;
  margin: 0 auto 15px auto;
}

@keyframes spin {
  0% { transform: rotate(0deg); }
  100% { transform: rotate(360deg); }
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(10px); }
  to { opacity: 1; transform: translateY(0); }
}

/* Custom cell modal */
.quadtree-modal {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.5);
  display: flex;
  justify-content: center;
  align-items: center;
  z-index: 1100;
}

.quadtree-modal-content {
  background-color: white;
  padding: 20px;
  border-radius: 4px;
  max-width: 500px;
  width: 100%;
}

.quadtree-modal-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 20px;
}

.quadtree-modal-close {
  background: none;
  border: none;
  font-size: 1.5rem;
  cursor: pointer;
}
```



```

        padding: 0;
        color: var(--text-color);
    }

    .quadtree-modal-body {
        margin-bottom: 20px;
    }

    .quadtree-form-group {
        margin-bottom: 15px;
    }

    .quadtree-form-group label {
        display: block;
        margin-bottom: 5px;
        font-weight: bold;
    }

    .quadtree-form-group input,
    .quadtree-form-group textarea {
        width: 100%;
        padding: 8px;
        border: 1px solid var(--secondary-color);
        border-radius: 4px;
    }

    .quadtree-form-actions {
        display: flex;
        justify-content: flex-end;
        gap: 10px;
    }
`;
document.head.appendChild(styleElement);
}

/**
 * Create the context menu
 */
function createContextMenu() {
    // Check if it already exists
    if (document.getElementById('quadtree-context-menu')) {
        return;
    }

    // Create context menu
    const contextMenu = document.createElement('div');
    contextMenu.id = 'quadtree-context-menu';
    contextMenu.className = 'quadtree-context-menu';

    // Menu items
    contextMenu.innerHTML = `
        <div class="context-menu-item" data-action="change-color">
            <span class="context-menu-item-icon"> 🎨 </span>
            <span>Change Color...</span>
    `;

```

```

    </div>
    <div class="context-menu-item" data-action="add-text">
      <span class="context-menu-item-icon">T</span>
      <span>Add Text...</span>
    </div>
    <div class="context-menu-item" data-action="add-image">
      <span class="context-menu-item-icon">🖼️</span>
      <span>Add Image...</span>
    </div>
    <div class="context-menu-item" data-action="paste-image">
      <span class="context-menu-item-icon">📋</span>
      <span>Paste From Clipboard</span>
    </div>
    <div class="context-menu-separator"></div>
    <div class="context-menu-item" data-action="reset-cell">
      <span class="context-menu-item-icon">↺</span>
      <span>Reset Cell</span>
    </div>
  `;

  // Add context menu to document
  document.body.appendChild(contextMenu);

  // Add click listeners to menu items
  contextMenu.querySelectorAll('.context-menu-item').forEach(item => {
    item.addEventListener('click', handleContextMenuAction);
  });

  // Click outside to close
  document.addEventListener('click', function(e) {
    if (!contextMenu.contains(e.target)) {
      contextMenu.classList.remove('active');
    }
  });

  // Create paste indicator
  const pasteIndicator = document.createElement('div');
  pasteIndicator.id = 'paste-indicator';
  pasteIndicator.className = 'paste-indicator';
  pasteIndicator.textContent = 'Press Ctrl+V to paste image';
  document.body.appendChild(pasteIndicator);
}

/**
 * Handle right-click on quadtree cells
 */
function handleQuadtreeContextMenu(event) {
  // Prevent default context menu
  event.preventDefault();

  // Find if click was on a cell
  const cell = event.target.closest('.quadtree-cell');
  if (!cell) return;

```

```

// Store reference to target cell
window.activeQuadtreeCell = cell;

// Position and show the context menu
const contextMenu = document.getElementById('quadtree-context-menu');
contextMenu.style.left = `${event.clientX}px`;
contextMenu.style.top = `${event.clientY}px`;
contextMenu.classList.add('active');
}

/**
 * Handle context menu actions
 */
function handleContextMenuAction(event) {
    const action = event.currentTarget.getAttribute('data-action');
    const cell = window.activeQuadtreeCell;

    // Hide context menu
    document.getElementById('quadtree-context-menu').classList.remove('active');

    if (!cell) return;

    // Execute action
    switch (action) {
        case 'change-color':
            showColorPickerModal(cell);
            break;

        case 'add-text':
            showTextModal(cell);
            break;

        case 'add-image':
            const fileInput = document.getElementById('quadtree-image-input');
            fileInput.click();
            break;

        case 'paste-image':
            prepareCellForPaste(cell);
            break;

        case 'reset-cell':
            resetCell(cell);
            break;
    }
}

/**
 * Mark cell as customized to hide original content
 */
function markCellAsCustomized(cell) {
    cell.classList.add('customized');
}

```

```

/**
 * Show color picker modal
 */
function showColorPickerModal(cell) {
  // Create modal
  const modal = document.createElement('div');
  modal.className = 'quadtree-modal';

  // Modal content
  modal.innerHTML = `
    <div class="quadtree-modal-content">
      <div class="quadtree-modal-header">
        <h3>Change Cell Color</h3>
        <button class="quadtree-modal-close">&times;</button>
      </div>
      <div class="quadtree-modal-body">
        <div class="quadtree-form-group">
          <label for="cell-color">Select Color:</label>
          <input type="color" id="cell-color"
value="${getComputedStyle(cell).backgroundColor || '#f0f0f0'}">
        </div>
        <div class="quadtree-form-group">
          <label for="cell-opacity">Opacity:</label>
          <input type="range" id="cell-opacity" min="0" max="100"
value="100">
        </div>
      </div>
      <div class="quadtree-form-actions">
        <button class="button button-secondary modal-
cancel">Cancel</button>
        <button class="button modal-apply">Apply</button>
      </div>
    </div>
  `;

  // Add modal to document
  document.body.appendChild(modal);

  // Handle close button
  modal.querySelector('.quadtree-modal-close').addEventListener('click', ()
=> {
    document.body.removeChild(modal);
  });

  // Handle cancel button
  modal.querySelector('.modal-cancel').addEventListener('click', () => {
    document.body.removeChild(modal);
  });

  // Handle apply button
  modal.querySelector('.modal-apply').addEventListener('click', () => {
    const color = document.getElementById('cell-color').value;
    const opacity = document.getElementById('cell-opacity').value;

```

```

        // Apply color to cell
        cell.style.backgroundColor = color;
        cell.style.opacity = opacity / 100;

        // Mark cell as customized
        markCellAsCustomized(cell);

        // Close modal
        document.body.removeChild(modal);
    });
}

/**
 * Show text input modal
 */
function showTextModal(cell) {
    // Find existing text element
    let existingText = '';
    const textElement = cell.querySelector('.quadtree-cell-custom-text');
    if (textElement) {
        existingText = textElement.textContent;
    }

    // Create modal
    const modal = document.createElement('div');
    modal.className = 'quadtree-modal';

    // Modal content
    modal.innerHTML = `
        <div class="quadtree-modal-content">
            <div class="quadtree-modal-header">
                <h3>Add Text to Cell</h3>
                <button class="quadtree-modal-close">&times;</button>
            </div>
            <div class="quadtree-modal-body">
                <div class="quadtree-form-group">
                    <label for="cell-text">Text:</label>
                    <input type="text" id="cell-text" value="${existingText}">
                </div>
                <div class="quadtree-form-group">
                    <label for="text-color">Text Color:</label>
                    <input type="color" id="text-color" value="#000000">
                </div>
                <div class="quadtree-form-group">
                    <label for="text-size">Text Size:</label>
                    <input type="range" id="text-size" min="10" max="32"
value="14">
                    <span id="text-size-value">14px</span>
                </div>
            </div>
            <div class="quadtree-form-actions">
                <button class="button button-secondary modal-
cancel">Cancel</button>
    `;

```

```

        <button class="button modal-apply">Apply</button>
      </div>
    </div>
  `;

  // Add modal to document
  document.body.appendChild(modal);

  // Update size value display
  const sizeInput = document.getElementById('text-size');
  const sizeValue = document.getElementById('text-size-value');
  sizeInput.addEventListener('input', () => {
    sizeValue.textContent = `${sizeInput.value}px`;
  });

  // Handle close button
  modal.querySelector('.quadtree-modal-close').addEventListener('click', ()
=> {
    document.body.removeChild(modal);
  });

  // Handle cancel button
  modal.querySelector('.modal-cancel').addEventListener('click', () => {
    document.body.removeChild(modal);
  });

  // Handle apply button
  modal.querySelector('.modal-apply').addEventListener('click', () => {
    const text = document.getElementById('cell-text').value;
    const color = document.getElementById('text-color').value;
    const size = document.getElementById('text-size').value;

    // Remove any existing text element
    if (textElement) {
      cell.removeChild(textElement);
    }

    // If text is not empty, add it to the cell
    if (text.trim() !== '') {
      const newTextElement = document.createElement('div');
      newTextElement.className = 'quadtree-cell-custom-text';
      newTextElement.textContent = text;
      newTextElement.style.color = color;
      newTextElement.style.fontSize = `${size}px`;

      cell.appendChild(newTextElement);

      // Mark cell as customized
      markCellAsCustomized(cell);
    }

    // Close modal
    document.body.removeChild(modal);
  });

```

```

}

/**
 * Handle image upload
 */
function handleImageUpload(event) {
    const cell = window.activeQuadtreeCell;
    if (!cell) return;

    const file = event.target.files[0];
    if (!file) return;

    // Check if file is an image
    if (!file.type.startsWith('image/')) {
        alert('Please select an image file');
        return;
    }

    // Read file as data URL
    const reader = new FileReader();
    reader.onload = function(e) {
        addImageToCell(cell, e.target.result);
    };

    reader.readAsDataURL(file);

    // Reset file input
    event.target.value = '';
}

/**
 * Add an image to a cell
 */
function addImageToCell(cell, imageDataUrl) {
    // Remove any existing image
    const existingImage = cell.querySelector('.quadtree-cell-image');
    if (existingImage) {
        cell.removeChild(existingImage);
    }

    // Create image element
    const img = document.createElement('img');
    img.className = 'quadtree-cell-image';
    img.src = imageDataUrl;

    // Add image to cell
    cell.appendChild(img);

    // Mark cell as customized
    markCellAsCustomized(cell);
}

/**
 * Reset cell to default state

```

```

*/
function resetCell(cell) {
  // Remove custom text
  const textElement = cell.querySelector('.quadtree-cell-custom-text');
  if (textElement) {
    cell.removeChild(textElement);
  }

  // Remove image
  const imageElement = cell.querySelector('.quadtree-cell-image');
  if (imageElement) {
    cell.removeChild(imageElement);
  }

  // Reset styles
  cell.style.backgroundColor = '';
  cell.style.opacity = '';

  // Remove customized class to show original content
  cell.classList.remove('customized');

  // Remove paste focus if present
  cell.classList.remove('paste-focus');

  // Trigger a redraw of the cell
  setTimeout(() => {
    const event = new Event('cellreset');
    cell.dispatchEvent(event);
  }, 0);
}

/**
 * Prepare a cell for clipboard paste
 */
function prepareCellForPaste(cell) {
  // Remove focus from any previously focused cell
  const focusedCell = document.querySelector('.quadtree-cell.paste-focus');
  if (focusedCell) {
    focusedCell.classList.remove('paste-focus');
  }

  // Add focus to this cell
  cell.classList.add('paste-focus');

  // Show the paste indicator
  const pasteIndicator = document.getElementById('paste-indicator');
  pasteIndicator.classList.add('active');

  // Auto hide the paste indicator after 3 seconds
  setTimeout(() => {
    pasteIndicator.classList.remove('active');
  }, 3000);
}

```



```

/**
 * Setup clipboard paste functionality
 */
function setupClipboardPaste() {
  // Listen for paste events on the document
  document.addEventListener('paste', function(event) {
    // Find the focused cell for paste
    const cell = document.querySelector('.quadtree-cell.paste-focus');
    if (!cell) return;

    // Get clipboard items
    const items = (event.clipboardData ||
event.originalEvent.clipboardData).items;

    // Look for image content
    for (const item of items) {
      if (item.type.indexOf('image') === 0) {
        // Get image from clipboard
        const blob = item.getAsFile();
        const reader = new FileReader();

        reader.onload = function(e) {
          // Add the image to the cell
          addImageToCell(cell, e.target.result);

          // Remove the paste focus
          cell.classList.remove('paste-focus');

          // Hide the paste indicator
          document.getElementById('paste-
indicator').classList.remove('active');
        };

        reader.readAsDataURL(blob);

        // Prevent default paste behavior
        event.preventDefault();
        return;
      }
    }

    // If we get here, no image was found in clipboard
    console.log('No image found in clipboard');
  });
}

/**
 * Override the export PNG functionality to include custom elements
 */
function overrideExportPNG() {
  // Find the export button
  const exportButton = document.getElementById('export-quadtree');
  if (!exportButton) {
    console.error('Export button not found');
  }
}

```

```

        return;
    }

    // Store the original click handler
    const originalHandler = exportButton.onclick;

    // Override with our enhanced version
    exportButton.onclick = function(event) {
        // Prevent the default action
        event.preventDefault();

        // Call our enhanced export function
        exportQuadtreeAsPNG();
    };
}

/**
 * Export the quadtree as a PNG with all custom elements
 */
function exportQuadtreeAsPNG() {
    // Show loading indicator
    showLoadingIndicator('Generating PNG...');

    // Get the quadtree container
    const quadtreeContainer = document.getElementById('quadtree-container');
    if (!quadtreeContainer) {
        hideLoadingIndicator();
        alert('Quadtree container not found');
        return;
    }

    // Use html2canvas to capture the DOM elements (including custom elements)
    // If html2canvas is not available, we'll use a fallback method
    if (typeof html2canvas === 'function') {
        html2canvas(quadtreeContainer, {
            scale: 2, // Higher quality
            backgroundColor: null,
            logging: false,
            useCORS: true, // For images from different origins
            allowTaint: true // Allow cross-origin images
        }).then(canvas => {
            // Convert canvas to PNG and download
            finishExport(canvas);
        }).catch(error => {
            console.error('Error capturing quadtree:', error);
            hideLoadingIndicator();
            alert('Failed to export quadtree: ' + error.message);

            // Try fallback method
            exportQuadtreeWithDOM2Canvas();
        });
    } else {
        // Fallback method
        exportQuadtreeWithDOM2Canvas();
    }
}

```

```

    }
}

/**
 * Fallback method to export quadtree using DOM-to-Canvas drawing
 */
function exportQuadtreeWithDOM2Canvas() {
    // Get the quadtree container
    const quadtreeContainer = document.getElementById('quadtree-container');
    if (!quadtreeContainer) {
        hideLoadingIndicator();
        alert('Quadtree container not found');
        return;
    }

    // Create a canvas
    const canvas = document.createElement('canvas');
    const context = canvas.getContext('2d');

    // Set canvas size to match container
    const width = quadtreeContainer.offsetWidth;
    const height = quadtreeContainer.offsetHeight;
    canvas.width = width * 2; // Higher quality
    canvas.height = height * 2;
    context.scale(2, 2);

    // Fill background
    context.fillStyle = 'white';
    context.fillRect(0, 0, width, height);

    // Draw each cell
    const cells = quadtreeContainer.querySelectorAll('.quadtree-cell');

    // First, render the cells and their backgrounds
    cells.forEach(cell => {
        // Only draw visible cells
        if (cell.style.display !== 'none') {
            // Get cell position and dimensions
            const rect = cell.getBoundingClientRect();
            const containerRect = quadtreeContainer.getBoundingClientRect();

            // Calculate position relative to container
            const x = rect.left - containerRect.left;
            const y = rect.top - containerRect.top;
            const cellWidth = rect.width;
            const cellHeight = rect.height;

            // Draw cell background
            context.fillStyle = getComputedStyle(cell).backgroundColor ||
'#f0f0f0';

            context.globalAlpha = parseFloat(getComputedStyle(cell).opacity)
|| 1;

            context.fillRect(x, y, cellWidth, cellHeight);

```

```

        // Draw cell border
        context.strokeStyle = getComputedStyle(cell).borderColor ||
'#ccc';
        context.lineWidth = parseFloat(getComputedStyle(cell).borderWidth)
|| 1;
        context.globalAlpha = 1;
        context.strokeRect(x, y, cellWidth, cellHeight);
    }
});

// Then, render the content (for proper z-order)
cells.forEach(cell => {
    // Only draw visible cells
    if (cell.style.display !== 'none') {
        // Get cell position and dimensions
        const rect = cell.getBoundingClientRect();
        const containerRect = quadtreeContainer.getBoundingClientRect();

        // Calculate position relative to container
        const x = rect.left - containerRect.left;
        const y = rect.top - containerRect.top;
        const cellWidth = rect.width;
        const cellHeight = rect.height;

        // Draw original cell content if not customized
        if (!cell.classList.contains('customized')) {
            const contentElement = cell.querySelector('.cell-content');
            if (contentElement) {
                context.fillStyle = getComputedStyle(contentElement).color
|| '#333';
                context.font = getComputedStyle(contentElement).font ||
'12px sans-serif';
                context.textAlign = 'center';
                context.textBaseline = 'middle';
                context.globalAlpha = 1;
                context.fillText(
                    contentElement.textContent,
                    x + cellWidth / 2,
                    y + cellHeight / 2
                );
            }
        }

        // Draw custom text if present
        const textElement = cell.querySelector('.quadtree-cell-custom-
text');
        if (textElement) {
            context.fillStyle = getComputedStyle(textElement).color ||
'#000';
            context.font = getComputedStyle(textElement).font || '14px
sans-serif';
            context.textAlign = 'center';
            context.textBaseline = 'middle';
            context.globalAlpha = 1;

```

```

        context.fillText(
            textElement.textContent,
            x + cellWidth / 2,
            y + cellHeight / 2
        );
    }

    // Draw image if present (this requires handling async loading)
    const imageElement = cell.querySelector('.quadtree-cell-image');
    if (imageElement) {
        try {
            context.globalAlpha = 1;
            context.drawImage(imageElement, x, y, cellWidth,
cellHeight);

        } catch (error) {
            console.error('Error drawing image:', error);
        }
    }
}

});

// Convert canvas to PNG and download
finishExport(canvas);
}

/**
 * Finish the export process by downloading the canvas as PNG
 */
function finishExport(canvas) {
    try {
        // Convert canvas to data URL
        const dataUrl = canvas.toDataURL('image/png');

        // Create download link
        const link = document.createElement('a');
        link.download = 'quadtree.png';
        link.href = dataUrl;
        link.click();

        // Hide loading indicator
        hideLoadingIndicator();
    } catch (error) {
        console.error('Error generating PNG:', error);
        hideLoadingIndicator();
        alert('Failed to generate PNG: ' + error.message);
    }
}

/**
 * Show a loading indicator
 */
function showLoadingIndicator(message) {
    // Create loading indicator if it doesn't exist
    let loadingEl = document.getElementById('export-loading');

```

```

    if (!loadingEl) {
        loadingEl = document.createElement('div');
        loadingEl.id = 'export-loading';
        loadingEl.className = 'export-loading';

        loadingEl.innerHTML = `
            <div class="export-loading-content">
                <div class="export-spinner"></div>
                <div id="export-loading-message">${message || 'Loading...'}
            </div>
        `;

        document.body.appendChild(loadingEl);
    } else {
        // Update message if it exists
        const messageEl = document.getElementById('export-loading-message');
        if (messageEl) {
            messageEl.textContent = message || 'Loading...';
        }

        // Show the existing element
        loadingEl.style.display = 'flex';
    }
}

/**
 * Hide the loading indicator
 */
function hideLoadingIndicator() {
    const loadingEl = document.getElementById('export-loading');
    if (loadingEl) {
        loadingEl.style.display = 'none';
    }
}

/**
 * Load html2canvas library dynamically if needed
 */
function loadHtml2Canvas() {
    if (typeof html2canvas === 'undefined') {
        return new Promise((resolve, reject) => {
            const script = document.createElement('script');
            script.src =
'https://html2canvas.hertzen.com/dist/html2canvas.min.js';
            script.onload = resolve;
            script.onerror = reject;
            document.head.appendChild(script);
        });
    }

    return Promise.resolve();
}

```

```
// Initialize html2canvas if not present
loadHtml2Canvas().catch(error => {
  console.warn('Failed to load html2canvas library:', error);
});
})();
```

[6] operator-framework.js

- **Bytes:** 149
- **Type:** text

```
/**
 * operator-framework.js
 *
 * Base framework that provides the shared namespace for all Operator modules
 */
const OperatorFramework = {};
```

[7] operator-language.js

- **Bytes:** 78175
- **Type:** text

```
/**
 * Operator Language System
 * A Turing-complete programming language with graphical interface for the
 * Operator System
 */

class OperatorLanguage {
  constructor(canvasId, consoleId) {
    // Language state
    this.variables = new Map(); // Variable store
    this.functions = new Map(); // Function store
    this.callStack = []; // Function call stack
    this.programCounter = 0; // Current instruction pointer
    this.program = []; // Current program
    this.running = false; // Is program running
    this.waitingForInput = false; // Is waiting for input
    this.breakpoints = new Set(); // Set of instruction numbers

    // Runtime configuration
    this.maxIterations = 100000; // Safety against infinite loops
    this.executionDelay = 0; // Delay between statements (ms)
    this.executionMode = 'normal'; // 'normal', 'step', 'fast'

    // Graphics context
    this.canvas = document.getElementById(canvasId);
    this.ctx = this.canvas.getContext('2d');
```

```

    this.graphicsStack = []; // For saved states

    // Console output
    this.consoleElement = document.getElementById(consoleId);
    this.consoleHistory = [];
    this.maxConsoleHistory = 100;

    // Initialize standard library
    this.initializeStandardLibrary();
}

/**
 * Initialize the standard library of functions
 */
initializeStandardLibrary() {
    // Math operations
    this.registerNativeFunction('add', (a, b) => a + b);
    this.registerNativeFunction('sub', (a, b) => a - b);
    this.registerNativeFunction('mul', (a, b) => a * b);
    this.registerNativeFunction('div', (a, b) => a / b);
    this.registerNativeFunction('mod', (a, b) => a % b);
    this.registerNativeFunction('pow', (a, b) => Math.pow(a, b));
    this.registerNativeFunction('sqrt', (a) => Math.sqrt(a));
    this.registerNativeFunction('sin', (a) => Math.sin(a));
    this.registerNativeFunction('cos', (a) => Math.cos(a));
    this.registerNativeFunction('tan', (a) => Math.tan(a));
    this.registerNativeFunction('floor', (a) => Math.floor(a));
    this.registerNativeFunction('ceil', (a) => Math.ceil(a));
    this.registerNativeFunction('round', (a) => Math.round(a));
    this.registerNativeFunction('abs', (a) => Math.abs(a));
    this.registerNativeFunction('random', (min, max) => min + Math.random() *
(max - min));

    // Comparison operations
    this.registerNativeFunction('eq', (a, b) => a === b);
    this.registerNativeFunction('neq', (a, b) => a !== b);
    this.registerNativeFunction('lt', (a, b) => a < b);
    this.registerNativeFunction('gt', (a, b) => a > b);
    this.registerNativeFunction('lte', (a, b) => a <= b);
    this.registerNativeFunction('gte', (a, b) => a >= b);
    this.registerNativeFunction('and', (a, b) => a && b);
    this.registerNativeFunction('or', (a, b) => a || b);
    this.registerNativeFunction('not', (a) => !a);

    // String operations
    this.registerNativeFunction('concat', (a, b) => String(a) + String(b));
    this.registerNativeFunction('substr', (str, start, length) =>
String(str).substring(start, start + length));
    this.registerNativeFunction('length', (str) => {
        if (Array.isArray(str)) {
            return str.length;
        } else if (typeof str === 'string') {
            return str.length;
        }
    })
}

```



```

        return 0;
    });
    this.registerNativeFunction('toString', (a) => String(a));
    this.registerNativeFunction('tonumber', (a) => Number(a));

    // Array operations
    this.registerNativeFunction('array', (...args) => args);
    this.registerNativeFunction('get', (arr, index) => {
        if (!Array.isArray(arr)) {
            throw new Error(`Expected an array, got ${typeof arr}`);
        }
        return arr[index];
    });
    this.registerNativeFunction('set', (arr, index, value) => {
        if (!Array.isArray(arr)) {
            throw new Error(`Expected an array, got ${typeof arr}`);
        }
        const result = [...arr];
        result[index] = value;
        return result;
    });
    this.registerNativeFunction('push', (arr, value) => {
        if (!Array.isArray(arr)) {
            return [value]; // If arr is not an array, create a new array with
the value
        }
        return [...arr, value];
    });
    this.registerNativeFunction('pop', (arr) => {
        if (!Array.isArray(arr) || arr.length === 0) {
            return [];
        }
        const result = [...arr];
        result.pop();
        return result;
    });
    this.registerNativeFunction('join', (arr, separator) => {
        if (!Array.isArray(arr)) {
            return String(arr);
        }
        return arr.join(separator || '');
    });
    this.registerNativeFunction('split', (str, separator) => {
        return String(str).split(separator || '');
    });

    // I/O operations
    this.registerNativeFunction('print', (...args) => {
        this.writeToConsole(args.join(' '));
        return args[args.length - 1];
    });
    this.registerNativeFunction('clear', () => {
        this.clearConsole();
        return null;
    });

```

```

});

// Graphics operations
this.registerNativeFunction('clearCanvas', (color = '#ffffff') => {
    this.ctx.fillStyle = color;
    this.ctx.fillRect(0, 0, this.canvas.width, this.canvas.height);
    return null;
});
this.registerNativeFunction('setFillColor', (color) => {
    this.ctx.fillStyle = color;
    return color;
});
this.registerNativeFunction('setStrokeColor', (color) => {
    this.ctx.strokeStyle = color;
    return color;
});
this.registerNativeFunction('setLineWidth', (width) => {
    this.ctx.lineWidth = width;
    return width;
});
this.registerNativeFunction('drawRect', (x, y, width, height, fill = true)
=> {
    if (fill) {
        this.ctx.fillRect(x, y, width, height);
    } else {
        this.ctx.strokeRect(x, y, width, height);
    }
    return null;
});
this.registerNativeFunction('drawCircle', (x, y, radius, fill = true) => {
    this.ctx.beginPath();
    this.ctx.arc(x, y, radius, 0, Math.PI * 2);
    if (fill) {
        this.ctx.fill();
    } else {
        this.ctx.stroke();
    }
    return null;
});
this.registerNativeFunction('drawLine', (x1, y1, x2, y2) => {
    this.ctx.beginPath();
    this.ctx.moveTo(x1, y1);
    this.ctx.lineTo(x2, y2);
    this.ctx.stroke();
    return null;
});
this.registerNativeFunction('drawText', (text, x, y, maxWidth) => {
    if (maxWidth) {
        this.ctx.fillText(text, x, y, maxWidth);
    } else {
        this.ctx.fillText(text, x, y);
    }
    return null;
});

```

```

    this.registerNativeFunction('setFont', (font) => {
        this.ctx.font = font;
        return font;
    });
    this.registerNativeFunction('saveGraphicsState', () => {
        this.ctx.save();
        return null;
    });
    this.registerNativeFunction('restoreGraphicsState', () => {
        this.ctx.restore();
        return null;
    });
    this.registerNativeFunction('translate', (x, y) => {
        this.ctx.translate(x, y);
        return null;
    });
    this.registerNativeFunction('rotate', (angle) => {
        this.ctx.rotate(angle);
        return null;
    });
    this.registerNativeFunction('scale', (x, y) => {
        this.ctx.scale(x, y);
        return null;
    });

    // Time operations
    this.registerNativeFunction('delay', (ms) => {
        return new Promise(resolve => setTimeout(resolve, ms));
    });
    this.registerNativeFunction('now', () => Date.now());

    // System operations
    this.registerNativeFunction('getVariable', (name) =>
this.getVariable(name));
    this.registerNativeFunction('setVariable', (name, value) =>
this.setVariable(name, value));

    // Type checking
    this.registerNativeFunction('isNumber', (value) => typeof value ===
'number');
    this.registerNativeFunction('isString', (value) => typeof value ===
'string');
    this.registerNativeFunction('isArray', (value) => Array.isArray(value));
    this.registerNativeFunction('isBoolean', (value) => typeof value ===
'boolean');
    this.registerNativeFunction('isNull', (value) => value === null);
    this.registerNativeFunction('isUndefined', (value) => value ===
undefined);
}

/**
 * Register a native JavaScript function in the language
 */
registerNativeFunction(name, func) {

```

```

        this.functions.set(name, {
            type: 'native',
            func: func
        });
    }

    /**
     * Register a user-defined function
     */
    defineFunction(name, params, body) {
        this.functions.set(name, {
            type: 'user',
            params: params,
            body: body
        });
    }

    /**
     * Get a variable value
     */
    getVariable(name) {
        if (this.variables.has(name)) {
            return this.variables.get(name);
        }
        return null;
    }

    /**
     * Set a variable value
     */
    setVariable(name, value) {
        this.variables.set(name, value);
        return value;
    }

    /**
     * Write text to the console
     */
    writeToConsole(text) {
        if (this.consoleElement) {
            const line = document.createElement('div');
            line.className = 'console-line';
            line.textContent = text;
            this.consoleElement.appendChild(line);
            this.consoleElement.scrollTop = this.consoleElement.scrollHeight;

            // Limit console history
            this.consoleHistory.push(text);
            if (this.consoleHistory.length > this.maxConsoleHistory) {
                this.consoleHistory.shift();
                if (this.consoleElement.childNodes.length >
this.maxConsoleHistory) {
this.consoleElement.removeChild(this.consoleElement.childNodes[0]);

```

```

        }
    }
}

/**
 * Clear the console
 */
clearConsole() {
    if (this.consoleElement) {
        this.consoleElement.innerHTML = '';
        this.consoleHistory = [];
    }
}

/**
 * Parse a string into a program
 * This is the main entry point for the parser
 */
parseProgram(code) {
    const parser = new OperatorLangParser(code, this);
    return parser.parse();
}

/**
 * Load a program from a string
 */
loadProgram(code) {
    this.program = this.parseProgram(code);
    this.programCounter = 0;
    this.running = false;
    this.callStack = [];
    return this.program;
}

/**
 * Set a breakpoint at a specific line
 */
setBreakpoint(line) {
    this.breakpoints.add(line);
}

/**
 * Clear a breakpoint at a specific line
 */
clearBreakpoint(line) {
    this.breakpoints.delete(line);
}

/**
 * Clear all breakpoints
 */
clearAllBreakpoints() {
    this.breakpoints.clear();
}

```

```

}

/**
 * Start program execution
 */
async run() {
    if (this.running) {
        return;
    }

    this.running = true;
    this.programCounter = 0;
    this.callStack = [];

    return this.executeProgram();
}

/**
 * Stop program execution
 */
stop() {
    this.running = false;
}

/**
 * Execute the current program
 */
async executeProgram() {
    let iterations = 0;

    while (this.running && this.programCounter < this.program.length) {
        // Check for breakpoints
        if (this.breakpoints.has(this.programCounter)) {
            this.writeToConsole(`Breakpoint hit at instruction
${this.programCounter}`);
            this.running = false;
            return;
        }

        // Execute the current instruction
        const instruction = this.program[this.programCounter];
        this.programCounter++;

        try {
            await this.executeInstruction(instruction);
        } catch (e) {
            this.writeToConsole(`Runtime error: ${e.message}`);
            this.running = false;
            return;
        }

        // Safety check against infinite loops
        iterations++;
        if (iterations > this.maxIterations) {

```

```

        this.writeToConsole('Program exceeded maximum iterations -
stopping execution');
        this.running = false;
        return;
    }

    // Add delay between statements if needed
    if (this.executionDelay > 0) {
        await new Promise(resolve => setTimeout(resolve,
this.executionDelay));
    }

    // Step mode - pause after each instruction
    if (this.executionMode === 'step') {
        this.running = false;
        return;
    }
}

if (this.programCounter >= this.program.length) {
    this.writeToConsole('Program execution completed');
    this.running = false;
}
}

/**
 * Execute the next step of the program
 */
async step() {
    if (this.programCounter >= this.program.length) {
        this.writeToConsole('Program execution completed');
        return;
    }

    this.running = true;
    const instruction = this.program[this.programCounter];
    this.programCounter++;

    try {
        await this.executeInstruction(instruction);
    } catch (e) {
        this.writeToConsole(`Runtime error: ${e.message}`);
    }

    this.running = false;
}

/**
 * Execute a single instruction
 */
async executeInstruction(instruction) {
    switch (instruction.type) {
        case 'assign':
            const value = await

```

```

this.evaluateExpression(instruction.expression);
    this.setVariable(instruction.variable, value);
    return value;

    case 'if':
        const condition = await
this.evaluateExpression(instruction.condition);
        if (condition) {
            // Execute the if body
            for (const stmt of instruction.body) {
                await this.executeInstruction(stmt);
                if (!this.running) return;
            }
        } else if (instruction.elseBody) {
            // Execute the else body
            for (const stmt of instruction.elseBody) {
                await this.executeInstruction(stmt);
                if (!this.running) return;
            }
        }
        return null;

    case 'while':
        while (this.running) {
            const condition = await
this.evaluateExpression(instruction.condition);
            if (!condition) break;

            for (const stmt of instruction.body) {
                await this.executeInstruction(stmt);
                if (!this.running) return;
            }
        }
        return null;

    case 'for':
        const start = await this.evaluateExpression(instruction.start);
        const end = await this.evaluateExpression(instruction.end);

        for (let i = start; i < end && this.running; i++) {
            this.setVariable(instruction.variable, i);

            for (const stmt of instruction.body) {
                await this.executeInstruction(stmt);
                if (!this.running) return;
            }
        }
        return null;

    case 'return':
        const returnValue = await
this.evaluateExpression(instruction.expression);
        return returnValue;

```


execution

```
        case 'expression':
            return await this.evaluateExpression(instruction.expression);

        case 'end':
        case 'else':
            // These instruction types should be handled during parsing
            // They should not appear as standalone instructions during
            // execution
            return null;

        default:
            throw new Error(`Unknown instruction type: ${instruction.type}`);
    }
}

/**
 * Evaluate an expression to a value
 */
async evaluateExpression(expression) {
    switch (expression.type) {
        case 'value':
            return expression.value;

        case 'variable':
            return this.getVariable(expression.name);

        case 'call':
            const func = this.functions.get(expression.function);
            if (!func) {
                throw new Error(`Function not found: ${expression.function}`);
            }

            // Evaluate all arguments
            const args = [];
            for (const arg of expression.arguments) {
                args.push(await this.evaluateExpression(arg));
            }

            // Call the function
            if (func.type === 'native') {
                return await func.func(...args);
            } else if (func.type === 'user') {
                // Create new scope for function
                this.callStack.push({
                    variables: new Map(this.variables),
                    programCounter: this.programCounter
                });

                // Set up parameters
                for (let i = 0; i < func.params.length; i++) {
                    this.setVariable(func.params[i], args[i]);
                }

                // Execute function body
            }
        }
    }
}
```

```

        let result = null;
        for (const stmt of func.body) {
            result = await this.executeInstruction(stmt);
            if (!this.running) break;
            if (stmt.type === 'return') break;
        }

        // Restore original scope
        const callFrame = this.callStack.pop();
        this.variables = callFrame.variables;
        this.programCounter = callFrame.programCounter;

        return result;
    }
    break;

case 'binary':
    const left = await this.evaluateExpression(expression.left);
    const right = await this.evaluateExpression(expression.right);

    switch (expression.operator) {
        case '+': return left + right;
        case '-': return left - right;
        case '*': return left * right;
        case '/': return left / right;
        case '%': return left % right;
        case '==': return left === right;
        case '!=': return left !== right;
        case '<': return left < right;
        case '>': return left > right;
        case '<=': return left <= right;
        case '>=': return left >= right;
        case '&&': return left && right;
        case '||': return left || right;
        default:
            throw new Error(`Unknown binary operator:
${expression.operator}`);
    }

case 'unary':
    const value = await
this.evaluateExpression(expression.expression);

    switch (expression.operator) {
        case '!': return !value;
        case '-': return -value;
        default:
            throw new Error(`Unknown unary operator:
${expression.operator}`);
    }

case 'array':
    const elements = [];
    for (const element of expression.elements) {

```

```

        elements.push(await this.evaluateExpression(element));
    }
    return elements;

    case 'object':
        const obj = {};
        for (const key in expression.properties) {
            obj[key] = await
this.evaluateExpression(expression.properties[key]);
        }
        return obj;

    default:
        throw new Error(`Unknown expression type: ${expression.type}`);
    }
}

/**
 * Save the program as JSON
 */
exportToJSON() {
    return JSON.stringify({
        program: this.program,
        variables: Array.from(this.variables.entries()),
        functions: Array.from(this.functions.entries())
            .filter(([_, func]) => func.type === 'user')
            .map([name, func]) => ({
                name,
                params: func.params,
                body: func.body
            })),
        breakpoints: Array.from(this.breakpoints),
        version: '1.0'
    }, null, 2);
}

/**
 * Load a program from JSON
 */
importFromJSON(json) {
    try {
        const data = JSON.parse(json);

        // Check version
        if (!data.version || data.version !== '1.0') {
            throw new Error('Incompatible program version');
        }

        // Load program
        this.program = data.program;

        // Load variables
        this.variables = new Map(data.variables);
    }
}

```

```

        // Load user-defined functions
        for (const func of data.functions) {
            this.defineFunction(func.name, func.params, func.body);
        }

        // Load breakpoints
        this.breakpoints = new Set(data.breakpoints);

        // Reset state
        this.programCounter = 0;
        this.running = false;
        this.callStack = [];

        return true;
    } catch (e) {
        this.writeToConsole(`Error importing program: ${e.message}`);
        return false;
    }
}

/**
 * TokenType enum for the lexer
 */
const TokenType = {
    // Literals
    NUMBER: 'NUMBER',
    STRING: 'STRING',
    IDENTIFIER: 'IDENTIFIER',
    TRUE: 'TRUE',
    FALSE: 'FALSE',
    NULL: 'NULL',

    // Keywords
    IF: 'IF',
    ELSE: 'ELSE',
    WHILE: 'WHILE',
    FOR: 'FOR',
    IN: 'IN',
    RANGE: 'RANGE',
    FUNC: 'FUNC',
    RETURN: 'RETURN',
    LET: 'LET',
    CONTINUE: 'CONTINUE',
    BREAK: 'BREAK',

    // Operators
    PLUS: 'PLUS',
    MINUS: 'MINUS',
    TIMES: 'TIMES',
    DIVIDE: 'DIVIDE',
    MODULO: 'MODULO',
    POWER: 'POWER',

```

```

// Comparison operators
EQUAL: 'EQUAL',
NOT_EQUAL: 'NOT_EQUAL',
LESS: 'LESS',
GREATER: 'GREATER',
LESS_EQUAL: 'LESS_EQUAL',
GREATER_EQUAL: 'GREATER_EQUAL',

// Logical operators
AND: 'AND',
OR: 'OR',
NOT: 'NOT',

// Assignment
ASSIGN: 'ASSIGN',

// Delimiters
LPAREN: 'LPAREN',
RPAREN: 'RPAREN',
LBRACE: 'LBRACE',
RBRACE: 'RBRACE',
LBRACKET: 'LBRACKET',
RBRACKET: 'RBRACKET',
COMMA: 'COMMA',
DOT: 'DOT',
COLON: 'COLON',
SEMICOLON: 'SEMICOLON',

// End of file
EOF: 'EOF'
};

/**
 * Token class for the lexer
 */
class Token {
    constructor(type, value, line, column) {
        this.type = type;
        this.value = value;
        this.line = line;
        this.column = column;
    }

    toString() {
        return `Token(${this.type}, ${this.value})`;
    }
}

/**
 * Lexer class for tokenizing the input code
 */
class Lexer {
    constructor(source) {
        this.source = source;
    }
}

```

```

    this.position = 0;
    this.line = 1;
    this.column = 1;
    this.tokens = [];

    this.keywords = {
        'if': TokenType.IF,
        'else': TokenType.ELSE,
        'while': TokenType.WHILE,
        'for': TokenType.FOR,
        'in': TokenType.IN,
        'range': TokenType.RANGE,
        'func': TokenType.FUNC,
        'return': TokenType.RETURN,
        'let': TokenType.LET,
        'continue': TokenType.CONTINUE,
        'break': TokenType.BREAK,
        'true': TokenType.TRUE,
        'false': TokenType.FALSE,
        'null': TokenType.NULL
    };
}

/**
 * Get the current character
 */
currentChar() {
    if (this.position >= this.source.length) {
        return null;
    }
    return this.source[this.position];
}

/**
 * Look ahead at the next character without advancing the position
 */
peekChar() {
    if (this.position + 1 >= this.source.length) {
        return null;
    }
    return this.source[this.position + 1];
}

/**
 * Advance to the next character
 */
advance() {
    if (this.currentChar() === '\n') {
        this.line++;
        this.column = 1;
    } else {
        this.column++;
    }
    this.position++;
}

```

```

}

/**
 * Skip whitespace
 */
skipWhitespace() {
    while (this.currentChar() && /\s/.test(this.currentChar())) {
        this.advance();
    }
}

/**
 * Skip a comment (everything from # to the end of the line)
 */
skipComment() {
    while (this.currentChar() && this.currentChar() !== '\n') {
        this.advance();
    }
    if (this.currentChar() === '\n') {
        this.advance();
    }
}

/**
 * Tokenize a number
 */
tokenizeNumber() {
    let start = this.position;
    let startColumn = this.column;
    let hasDot = false;

    while (this.currentChar() && /[0-9.]/.test(this.currentChar())) {
        if (this.currentChar() === '.') {
            if (hasDot) {
                break;
            }
            hasDot = true;
        }
        this.advance();
    }

    let value = this.source.substring(start, this.position);
    return new Token(TokenType.NUMBER, parseFloat(value), this.line,
startColumn);
}

/**
 * Tokenize a string
 */
tokenizeString(quoteChar) {
    let start = this.position;
    let startColumn = this.column;
    let value = '';

```

```

    // Skip the opening quote
    this.advance();

    while (this.currentChar() && this.currentChar() !== quoteChar) {
        // Handle escape sequences
        if (this.currentChar() === '\\') {
            this.advance();
            if (this.currentChar() === 'n') {
                value += '\n';
            } else if (this.currentChar() === 't') {
                value += '\t';
            } else if (this.currentChar() === 'r') {
                value += '\r';
            } else if (this.currentChar() === quoteChar) {
                value += quoteChar;
            } else if (this.currentChar() === '\\') {
                value += '\\';
            } else {
                value += this.currentChar();
            }
        } else {
            value += this.currentChar();
        }
        this.advance();
    }

    // Skip the closing quote
    if (this.currentChar() === quoteChar) {
        this.advance();
    }

    return new Token(TokenType.STRING, value, this.line, startColumn);
}

/**
 * Tokenize an identifier or keyword
 */
tokenizeIdentifier() {
    let start = this.position;
    let startColumn = this.column;

    while (this.currentChar() && /[A-Za-z0-9_]/.test(this.currentChar())) {
        this.advance();
    }

    let value = this.source.substring(start, this.position);

    // Check if it's a keyword
    let tokenType = this.keywords[value] || TokenType.IDENTIFIER;

    return new Token(tokenType, value, this.line, startColumn);
}

/**

```



```

    * Tokenize the input code
    */
    tokenize() {
        while (this.position < this.source.length) {
            // Skip whitespace
            if (/s/.test(this.currentChar())) {
                this.skipWhitespace();
                continue;
            }

            // Skip comments
            if (this.currentChar() === '#') {
                this.skipComment();
                continue;
            }

            // Numbers
            if (/0-9/.test(this.currentChar())) {
                this.tokens.push(this.tokenizeNumber());
                continue;
            }

            // Strings
            if (this.currentChar() === '"' || this.currentChar() === "'") {
                this.tokens.push(this.tokenizeString(this.currentChar()));
                continue;
            }

            // Identifiers and keywords
            if (/A-Za-z_/.test(this.currentChar())) {
                this.tokens.push(this.tokenizeIdentifier());
                continue;
            }

            // Operators and punctuation
            switch (this.currentChar()) {
                case '+':
                    this.tokens.push(new Token(TokenType.PLUS, '+', this.line,
this.column));
                    this.advance();
                    break;
                case '-':
                    this.tokens.push(new Token(TokenType.MINUS, '-', this.line,
this.column));
                    this.advance();
                    break;
                case '*':
                    this.tokens.push(new Token(TokenType.TIMES, '*', this.line,
this.column));
                    this.advance();
                    break;
                case '/':
                    this.tokens.push(new Token(TokenType.DIVIDE, '/', this.line,
this.column));

```

```

        this.advance();
        break;
    case '%':
        this.tokens.push(new Token(TokenType.MODULO, '%', this.line,
this.column));
        this.advance();
        break;
    case '=':
        if (this.peekChar() === '=') {
            this.advance();
            this.advance();
            this.tokens.push(new Token(TokenType.EQUAL, '==',
this.line, this.column - 2));
        } else {
            this.advance();
            this.tokens.push(new Token(TokenType.ASSIGN, '=',
this.line, this.column - 1));
        }
        break;
    case '!':
        if (this.peekChar() === '=') {
            this.advance();
            this.advance();
            this.tokens.push(new Token(TokenType.NOT_EQUAL, '!=',
this.line, this.column - 2));
        } else {
            this.advance();
            this.tokens.push(new Token(TokenType.NOT, '!', this.line,
this.column - 1));
        }
        break;
    case '<':
        if (this.peekChar() === '=') {
            this.advance();
            this.advance();
            this.tokens.push(new Token(TokenType.LESS_EQUAL, '<=',
this.line, this.column - 2));
        } else {
            this.advance();
            this.tokens.push(new Token(TokenType.LESS, '<', this.line,
this.column - 1));
        }
        break;
    case '>':
        if (this.peekChar() === '=') {
            this.advance();
            this.advance();
            this.tokens.push(new Token(TokenType.GREATER_EQUAL, '>=',
this.line, this.column - 2));
        } else {
            this.advance();
            this.tokens.push(new Token(TokenType.GREATER, '>',
this.line, this.column - 1));
        }

```

```

        break;
    case '&':
        if (this.peekChar() === '&') {
            this.advance();
            this.advance();
            this.tokens.push(new Token(TokenType.AND, '&&', this.line,
this.column - 2));
        } else {
            throw new Error(`Unexpected character '&' at line
${this.line}, column ${this.column}`);
        }
        break;
    case '|':
        if (this.peekChar() === '|') {
            this.advance();
            this.advance();
            this.tokens.push(new Token(TokenType.OR, '||', this.line,
this.column - 2));
        } else {
            throw new Error(`Unexpected character '|' at line
${this.line}, column ${this.column}`);
        }
        break;
    case '(':
        this.tokens.push(new Token(TokenType.LPAREN, '(', this.line,
this.column));
        this.advance();
        break;
    case ')':
        this.tokens.push(new Token(TokenType.RPAREN, ')', this.line,
this.column));
        this.advance();
        break;
    case '{':
        this.tokens.push(new Token(TokenType.LBRACE, '{', this.line,
this.column));
        this.advance();
        break;
    case '}':
        this.tokens.push(new Token(TokenType.RBRACE, '}', this.line,
this.column));
        this.advance();
        break;
    case '[':
        this.tokens.push(new Token(TokenType.LBRACKET, '[', this.line,
this.column));
        this.advance();
        break;
    case ']':
        this.tokens.push(new Token(TokenType.RBRACKET, ']', this.line,
this.column));
        this.advance();
        break;
    case ',':

```

```

        this.tokens.push(new Token(TokenType.COMMA, ',', this.line,
this.column));
        this.advance();
        break;
    case '.':
        this.tokens.push(new Token(TokenType.DOT, '.', this.line,
this.column));
        this.advance();
        break;
    case ':':
        this.tokens.push(new Token(TokenType.COLON, ':', this.line,
this.column));
        this.advance();
        break;
    case ';':
        this.tokens.push(new Token(TokenType.SEMICOLON, ';',
this.line, this.column));
        this.advance();
        break;
    default:
        throw new Error(`Unexpected character '${this.currentChar()}'
at line ${this.line}, column ${this.column}`);
    }
}

// End of file
this.tokens.push(new Token(TokenType.EOF, null, this.line, this.column));

return this.tokens;
}
}

/**
 * Parser class for parsing tokens into an AST
 */
class Parser {
    constructor(tokens) {
        this.tokens = tokens;
        this.position = 0;
    }

    /**
     * Get the current token
     */
    currentToken() {
        if (this.position >= this.tokens.length) {
            return this.tokens[this.tokens.length - 1]; // Return EOF token
        }
        return this.tokens[this.position];
    }

    /**
     * Look ahead at the next token without advancing the position
     */

```

```

peekToken() {
    if (this.position + 1 >= this.tokens.length) {
        return this.tokens[this.tokens.length - 1]; // Return EOF token
    }
    return this.tokens[this.position + 1];
}

/**
 * Advance to the next token
 */
advance() {
    this.position++;
}

/**
 * Check if the current token is of the given type
 */
check(type) {
    return this.currentToken().type === type;
}

/**
 * Consume a token of the given type and advance
 */
consume(type, errorMessage) {
    if (this.check(type)) {
        const token = this.currentToken();
        this.advance();
        return token;
    }

    throw new Error(`${errorMessage} at line ${this.currentToken().line},
column ${this.currentToken().column}`);
}

/**
 * Parse the program
 */
parse() {
    const statements = [];

    while (!this.check(TokenType.EOF)) {
        statements.push(this.parseStatement());
    }

    return statements;
}

/**
 * Parse a statement
 */
parseStatement() {
    // Variable declaration with 'let'
    if (this.check(TokenType.LET)) {

```

```

        return this.parseVariableDeclaration();
    }

    // If statement
    if (this.check(TokenType.IF)) {
        return this.parseIfStatement();
    }

    // While loop
    if (this.check(TokenType.WHILE)) {
        return this.parseWhileLoop();
    }

    // For loop
    if (this.check(TokenType.FOR)) {
        return this.parseForLoop();
    }

    // Function declaration
    if (this.check(TokenType.FUNC)) {
        return this.parseFunctionDeclaration();
    }

    // Return statement
    if (this.check(TokenType.RETURN)) {
        return this.parseReturnStatement();
    }

    // Assignment (variable = expression)
    if (this.check(TokenType.IDENTIFIER) && this.peekToken().type ===
TokenType.ASSIGN) {
        return this.parseAssignment();
    }

    // Expression statement
    return this.parseExpressionStatement();
}

/**
 * Parse a variable declaration (let x = expr)
 */
parseVariableDeclaration() {
    this.consume(TokenType.LET, "Expected 'let'");
    const name = this.consume(TokenType.IDENTIFIER, "Expected variable
name").value;
    this.consume(TokenType.ASSIGN, "Expected '=' after variable name");
    const expression = this.parseExpression();

    return {
        type: 'assign',
        variable: name,
        expression: expression
    };
}

```

```

/**
 * Parse an assignment (x = expr)
 */
parseAssignment() {
    const name = this.consume(TokenType.IDENTIFIER, "Expected variable
name").value;
    this.consume(TokenType.ASSIGN, "Expected '=' after variable name");
    const expression = this.parseExpression();

    return {
        type: 'assign',
        variable: name,
        expression: expression
    };
}

/**
 * Parse an if statement (if expr { ... } else { ... })
 */
parseIfStatement() {
    this.consume(TokenType.IF, "Expected 'if'");
    const condition = this.parseExpression();

    this.consume(TokenType.LBRACE, "Expected '{' after if condition");
    const body = this.parseBlock();

    let elseBody = null;

    // Check for else block
    if (this.check(TokenType.ELSE)) {
        this.advance();
        this.consume(TokenType.LBRACE, "Expected '{' after 'else'");
        elseBody = this.parseBlock();
    }

    return {
        type: 'if',
        condition: condition,
        body: body,
        elseBody: elseBody
    };
}

/**
 * Parse a while loop (while expr { ... })
 */
parseWhileLoop() {
    this.consume(TokenType.WHILE, "Expected 'while'");
    const condition = this.parseExpression();

    this.consume(TokenType.LBRACE, "Expected '{' after while condition");
    const body = this.parseBlock();

```

```

        return {
            type: 'while',
            condition: condition,
            body: body
        };
    }

/**
 * Parse a for loop (for i in range(start, end) { ... })
 */
parseForLoop() {
    this.consume(TokenType.FOR, "Expected 'for'");
    const variable = this.consume(TokenType.IDENTIFIER, "Expected variable
name").value;

    this.consume(TokenType.IN, "Expected 'in'");
    this.consume(TokenType.RANGE, "Expected 'range'");
    this.consume(TokenType.LPAREN, "Expected '(' after 'range'");

    const start = this.parseExpression();
    this.consume(TokenType.COMMA, "Expected ',' between range values");
    const end = this.parseExpression();

    this.consume(TokenType.RPAREN, "Expected ')' after range values");
    this.consume(TokenType.LBRACE, "Expected '{' after for loop header");

    const body = this.parseBlock();

    return {
        type: 'for',
        variable: variable,
        start: start,
        end: end,
        body: body
    };
}

/**
 * Parse a function declaration (func name(params) { ... })
 */
parseFunctionDeclaration() {
    this.consume(TokenType.FUNC, "Expected 'func'");
    const name = this.consume(TokenType.IDENTIFIER, "Expected function
name").value;

    this.consume(TokenType.LPAREN, "Expected '(' after function name");
    const params = this.parseParameters();
    this.consume(TokenType.RPAREN, "Expected ')' after parameters");

    this.consume(TokenType.LBRACE, "Expected '{' after function parameters");
    const body = this.parseBlock();

    return {
        type: 'function',

```



```

        name: name,
        params: params,
        body: body
    };
}

/**
 * Parse function parameters
 */
parseParameters() {
    const params = [];

    // Empty parameter list
    if (this.check(TokenType.RPAREN)) {
        return params;
    }

    // Parse first parameter
    params.push(this.consume(TokenType.IDENTIFIER, "Expected parameter
name").value);

    // Parse remaining parameters
    while (this.check(TokenType.COMMA)) {
        this.advance();
        params.push(this.consume(TokenType.IDENTIFIER, "Expected parameter
name").value);
    }

    return params;
}

/**
 * Parse a block of statements
 */
parseBlock() {
    const statements = [];

    while (!this.check(TokenType.RBRACE) && !this.check(TokenType.EOF)) {
        statements.push(this.parseStatement());
    }

    this.consume(TokenType.RBRACE, "Expected '}'");

    return statements;
}

/**
 * Parse a return statement
 */
parseReturnStatement() {
    this.consume(TokenType.RETURN, "Expected 'return'");
    const expression = this.parseExpression();

    return {

```

```

        type: 'return',
        expression: expression
    };
}

/**
 * Parse an expression statement
 */
parseExpressionStatement() {
    const expression = this.parseExpression();

    return {
        type: 'expression',
        expression: expression
    };
}

/**
 * Parse an expression
 */
parseExpression() {
    return this.parseLogicalOr();
}

/**
 * Parse logical OR expressions
 */
parseLogicalOr() {
    let expr = this.parseLogicalAnd();

    while (this.check(TokenType.OR)) {
        const operator = this.currentToken().value;
        this.advance();
        const right = this.parseLogicalAnd();

        expr = {
            type: 'binary',
            operator: operator,
            left: expr,
            right: right
        };
    }

    return expr;
}

/**
 * Parse logical AND expressions
 */
parseLogicalAnd() {
    let expr = this.parseEquality();

    while (this.check(TokenType.AND)) {
        const operator = this.currentToken().value;

```

```

        this.advance();
        const right = this.parseEquality();

        expr = {
            type: 'binary',
            operator: operator,
            left: expr,
            right: right
        };
    };
}

return expr;
}

/**
 * Parse equality expressions (==, !=)
 */
parseEquality() {
    let expr = this.parseComparison();

    while (this.check(TokenType.EQUAL) || this.check(TokenType.NOT_EQUAL)) {
        const operator = this.currentToken().value;
        this.advance();
        const right = this.parseComparison();

        expr = {
            type: 'binary',
            operator: operator,
            left: expr,
            right: right
        };
    }

    return expr;
}

/**
 * Parse comparison expressions (<, >, <=, >=)
 */
parseComparison() {
    let expr = this.parseAddition();

    while (
        this.check(TokenType.LESS) ||
        this.check(TokenType.GREATER) ||
        this.check(TokenType.LESS_EQUAL) ||
        this.check(TokenType.GREATER_EQUAL)
    ) {
        const operator = this.currentToken().value;
        this.advance();
        const right = this.parseAddition();

        expr = {
            type: 'binary',

```

```

        operator: operator,
        left: expr,
        right: right
    };
}

return expr;
}

/**
 * Parse addition and subtraction
 */
parseAddition() {
    let expr = this.parseMultiplication();

    while (this.check(TokenType.PLUS) || this.check(TokenType.MINUS)) {
        const operator = this.currentToken().value;
        this.advance();
        const right = this.parseMultiplication();

        expr = {
            type: 'binary',
            operator: operator,
            left: expr,
            right: right
        };
    }

    return expr;
}

/**
 * Parse multiplication, division, and modulo
 */
parseMultiplication() {
    let expr = this.parseUnary();

    while (
        this.check(TokenType.TIMES) ||
        this.check(TokenType.DIVIDE) ||
        this.check(TokenType.MODULO)
    ) {
        const operator = this.currentToken().value;
        this.advance();
        const right = this.parseUnary();

        expr = {
            type: 'binary',
            operator: operator,
            left: expr,
            right: right
        };
    }
}

```

```

        return expr;
    }

    /**
     * Parse unary expressions (!, -)
     */
    parseUnary() {
        if (this.check(TokenType.NOT) || this.check(TokenType.MINUS)) {
            const operator = this.currentToken().value;
            this.advance();
            const right = this.parseUnary();

            return {
                type: 'unary',
                operator: operator,
                expression: right
            };
        }

        return this.parseCallOrPrimary();
    }

    /**
     * Parse function calls or primary expressions
     */
    parseCallOrPrimary() {
        // Parse the primary expression first
        let expr = this.parsePrimary();

        // If followed by '(', it's a function call
        while (this.check(TokenType.LPAREN)) {
            expr = this.parseCall(expr);
        }

        return expr;
    }

    /**
     * Parse function call
     */
    parseCall(callee) {
        // Function name must be an identifier
        if (callee.type !== 'variable') {
            throw new Error("Expected function name");
        }

        this.consume(TokenType.LPAREN, "Expected '(' after function name");
        const args = this.parseArguments();
        this.consume(TokenType.RPAREN, "Expected ')' after arguments");

        return {
            type: 'call',
            function: callee.name,
            arguments: args
        };
    }

```

```

    };
}

/**
 * Parse function call arguments
 */
parseArguments() {
    const args = [];

    // Empty argument list
    if (this.check(TokenType.RPAREN)) {
        return args;
    }

    // Parse first argument
    args.push(this.parseExpression());

    // Parse remaining arguments
    while (this.check(TokenType.COMMA)) {
        this.advance();
        args.push(this.parseExpression());
    }

    return args;
}

/**
 * Parse primary expressions (literals, identifiers, parenthesized
expressions)
 */
parsePrimary() {
    // Number literal
    if (this.check(TokenType.NUMBER)) {
        const value = this.currentToken().value;
        this.advance();
        return {
            type: 'value',
            value: value
        };
    }

    // String literal
    if (this.check(TokenType.STRING)) {
        const value = this.currentToken().value;
        this.advance();
        return {
            type: 'value',
            value: value
        };
    }

    // Boolean literals
    if (this.check(TokenType.TRUE)) {
        this.advance();
    }

```

```

        return {
            type: 'value',
            value: true
        };
    }

    if (this.check(TokenType.FALSE)) {
        this.advance();
        return {
            type: 'value',
            value: false
        };
    }

    // Null literal
    if (this.check(TokenType.NULL)) {
        this.advance();
        return {
            type: 'value',
            value: null
        };
    }

    // Identifier
    if (this.check(TokenType.IDENTIFIER)) {
        const name = this.currentToken().value;
        this.advance();
        return {
            type: 'variable',
            name: name
        };
    }

    // Array literal
    if (this.check(TokenType.LBRACKET)) {
        return this.parseArrayLiteral();
    }

    // Parenthesized expression
    if (this.check(TokenType.LPAREN)) {
        this.advance();
        const expr = this.parseExpression();
        this.consume(TokenType.RPAREN, "Expected ')' after expression");
        return expr;
    }

    throw new Error(`Unexpected token ${this.currentToken().type} at line
    ${this.currentToken().line}, column ${this.currentToken().column}`);
}

/**
 * Parse array literal
 */
parseArrayLiteral() {

```

```

        this.consume(TokenType.LBRACKET, "Expected '['");
        const elements = [];

        // Empty array
        if (this.check(TokenType.RBRACKET)) {
            this.advance();
            return {
                type: 'array',
                elements: elements
            };
        }

        // Parse first element
        elements.push(this.parseExpression());

        // Parse remaining elements
        while (this.check(TokenType.COMMA)) {
            this.advance();
            elements.push(this.parseExpression());
        }

        this.consume(TokenType.RBRACKET, "Expected ']'");

        return {
            type: 'array',
            elements: elements
        };
    }
}

/**
 * Main parser class for OperatorLang
 */
class OperatorLangParser {
    constructor(code, context) {
        this.code = code;
        this.context = context;
    }

    /**
     * Parse the code into a program
     */
    parse() {
        try {
            // Tokenize the input
            const lexer = new Lexer(this.code);
            const tokens = lexer.tokenize();

            // Parse the tokens into an AST
            const parser = new Parser(tokens);
            const program = parser.parse();

            // Process function declarations
            this.processFunctionDeclarations(program);

```



```

        // Return the program without function declarations
        return program.filter(statement => statement.type !== 'function');
    } catch (e) {
        this.context.writeToConsole(`Parse error: ${e.message}`);
        throw e;
    }
}

/**
 * Process function declarations
 */
processFunctionDeclarations(program) {
    for (const statement of program) {
        if (statement.type === 'function') {
            this.context.defineFunction(statement.name, statement.params,
statement.body);
        }
    }
}

// Example programs
const EXAMPLE_PROGRAMS = {
    helloWorld: `# Hello world program
print("Hello, world!")
`,

    drawingDemo: `# Graphics demo program
clearCanvas("#f0f0f0")

# Draw a colorful pattern
let size = 20
let colors = array("#ff0000", "#00ff00", "#0000ff", "#ffff00", "#ff00ff",
"#00ffff")

for i in range(0, 10) {
    for j in range(0, 10) {
        let x = i * 30 + 50
        let y = j * 30 + 50
        let colorIndex = (i + j) % 6
        setFillColor(get(colors, colorIndex))
        drawRect(x, y, size, size)
    }
}

# Draw a rainbow arc
setLineWidth(5)
let cx = 200
let cy = 200
let radius = 100
let colors2 = array("#ff0000", "#ff7f00", "#ffff00", "#00ff00", "#0000ff",
"#4b0082", "#9400d3")

```

```

for i in range(0, 7) {
    let r = radius - i * 10
    setStrokeColor(get(colors2, i))
    drawCircle(cx, cy, r, false)
}

# Draw some text
setFillColor("#000000")
setFont("20px Arial")
drawText("Operator Graphics Demo", 120, 350)
`,

    fibonacci: `# Calculate Fibonacci sequence
# Iterative implementation to avoid recursion issues

print("Fibonacci Sequence:")

# Initialize first two Fibonacci numbers
let a = 0
let b = 1

# Calculate and display the first 10 Fibonacci numbers
for i in range(0, 10) {
    # Use separate if statements instead of else if
    if i == 0 {
        print("fibonacci(0) = 0")
    }

    if i == 1 {
        print("fibonacci(1) = 1")
    }

    if i > 1 {
        # Calculate the next Fibonacci number
        let c = a + b

        # Update a and b for the next iteration
        a = b
        b = c

        # Print the result
        print("fibonacci(" + i + ") = " + c)
    }
}

print("Fibonacci sequence completed")
`,

    gameOfLife: `# Conway's Game of Life - Simplified Version
# This implementation uses a flat 1D array instead of nested arrays

# Grid dimensions
let width = 20
let height = 15

```

```
let cellSize = 15

# Debug output
func debug(message) {
    print("DEBUG: " + message)
}

# Convert 2D coordinates to 1D index
func coordToIndex(x, y) {
    return y * width + x
}

# Initialize a random grid
func createGrid() {
    debug("Creating grid...")

    # Create a flat array for the grid
    let grid = array()

    # Fill with random cells
    for i in range(0, width * height) {
        let state = 0
        if random(0, 1) < 0.3 {
            state = 1
        }
        grid = push(grid, state)
    }

    debug("Grid created with " + length(grid) + " cells")
    return grid
}

# Get cell state
func getCell(grid, x, y) {
    # Check bounds
    if x < 0 {
        return 0
    }

    if x >= width {
        return 0
    }

    if y < 0 {
        return 0
    }

    if y >= height {
        return 0
    }

    # Get index
    let index = coordToIndex(x, y)
```

```

# Check if grid and index are valid
if grid == null {
    return 0
}

# Get the cell value
let cell = get(grid, index)

# Return 0 if cell is not defined
if cell == null {
    return 0
}

return cell
}

# Count neighbors
func countNeighbors(grid, x, y) {
    let count = 0

    # Check all 8 surrounding cells
    for dy in range(-1, 2) {
        for dx in range(-1, 2) {
            # Skip the cell itself (when dx=0 and dy=0)
            # We'll handle this with an if statement instead of continue
            if dx != 0 || dy != 0 {
                # Add to count if neighbor is alive
                let nx = x + dx
                let ny = y + dy

                if getCell(grid, nx, ny) == 1 {
                    count = count + 1
                }
            }
        }
    }

    return count
}

# Compute next generation
func nextGeneration(grid) {
    debug("Computing next generation...")

    # Create new grid
    let newGrid = array()

    # Initialize with all dead cells
    for i in range(0, width * height) {
        newGrid = push(newGrid, 0)
    }

    # Apply rules to each cell
    for y in range(0, height) {

```

```

    for x in range(0, width) {
        # Get current state
        let state = getCell(grid, x, y)

        # Count neighbors
        let neighbors = countNeighbors(grid, x, y)

        # Index in the flat array
        let index = coordToIndex(x, y)

        # Apply Conway's rules
        let newState = 0

        # Rule 1: Live cell with 2-3 neighbors survives
        if state == 1 {
            if neighbors == 2 {
                newState = 1
            }

            if neighbors == 3 {
                newState = 1
            }
        }

        # Rule 2: Dead cell with 3 neighbors becomes alive
        if state == 0 {
            if neighbors == 3 {
                newState = 1
            }
        }

        # Update the new grid
        newGrid = set(newGrid, index, newState)
    }
}

debug("Next generation computed")
return newGrid
}

# Draw the grid
func drawGrid(grid) {
    debug("Drawing grid...")

    # Clear the canvas
    clearCanvas("#f0f0f0")

    # Draw each cell
    for y in range(0, height) {
        for x in range(0, width) {
            # Get cell state
            let state = getCell(grid, x, y)

            # Set color based on state

```

```

        if state == 1 {
            setFillColor("#000000") # Black for alive
        } else {
            setFillColor("#ffffff") # White for dead
        }

        # Draw the cell
        drawRect(x * cellSize + 1, y * cellSize + 1,
                cellSize - 2, cellSize - 2)
    }
}

# Draw grid lines
setStrokeColor("#cccccc")
setLineWidth(1)

# Vertical lines
for x in range(0, width + 1) {
    drawLine(x * cellSize, 0, x * cellSize, height * cellSize)
}

# Horizontal lines
for y in range(0, height + 1) {
    drawLine(0, y * cellSize, width * cellSize, y * cellSize)
}

debug("Grid drawn")
}

# Start simulation
print("Conway's Game of Life")
print("Initializing...")

# Create and draw initial grid
let grid = createGrid()
drawGrid(grid)

print("Simulation running...")
print("Press 'Run' again to stop")

# Animation loop
while true {
    # Update grid
    grid = nextGeneration(grid)

    # Draw updated grid
    drawGrid(grid)

    # Delay for animation
    delay(200)
}
\
};

```

```

// Set up the OperatorLanguage UI with the Operator System
document.addEventListener('DOMContentLoaded', function() {
  // Add the programming language section to the main UI
  const mainElement = document.querySelector('main');

  if (!mainElement) {
    console.error('Main element not found in the document');
    return;
  }

  // Create programming language section
  const programmingSection = document.createElement('section');
  programmingSection.id = 'programming-section';
  programmingSection.className = 'operation-section';
  programmingSection.innerHTML = `
    <h2>Programmable Interface</h2>
    <div class="operation-container">
      <div class="operation-controls">
        <h3>OperatorLang Code</h3>
        <textarea id="code-editor" spellcheck="false"></textarea>
        <div class="editor-toolbar">
          <button id="run-program" class="button">Run</button>
          <button id="stop-program" class="button button-
secondary">Stop</button>
          <button id="step-program" class="button">Step</button>
          <button id="clear-program" class="button button-
secondary">Clear</button>
          <select id="example-programs">
            <option value="">Load Example...</option>
            <option value="helloWorld">Hello World</option>
            <option value="drawingDemo">Drawing Demo</option>
            <option value="fibonacci">Fibonacci Sequence</option>
            <option value="gameOfLife">Game of Life</option>
          </select>
        </div>
        <div class="language-console-container">
          <h3>Console Output</h3>
          <div id="language-console" class="language-console"></div>
        </div>
        <div class="operation-result">
          <h3>Graphics Output</h3>
          <canvas id="language-canvas" width="400" height="400"></canvas>
          <div class="program-controls">
            <button id="save-program" class="button">Save Program</button>
            <button id="load-program" class="button">Load Program</button>
            <button id="download-canvas" class="button">Download
Image</button>
          </div>
        </div>
      </div>
    </div>
  `;

  // Add programming section to the main UI

```

```

mainElement.appendChild(programmingSection);

// Add it to navigation
const navSection = document.querySelector('#main-nav');
if (navSection) {
  const programmingNav = document.createElement('div');
  programmingNav.className = 'nav-section';
  programmingNav.innerHTML = `
    <h3>Programming</h3>
    <button data-section="programming-section">OperatorLang</button>
  `;
  navSection.appendChild(programmingNav);
}

// Add the styles
const styleElement = document.createElement('style');
styleElement.textContent = `
  #code-editor {
    width: 100%;
    height: 300px;
    font-family: 'Fira Code', 'Consolas', monospace;
    font-size: 14px;
    padding: 10px;
    border: 1px solid var(--secondary-color);
    resize: vertical;
    tab-size: 4;
    white-space: pre;
    overflow-wrap: normal;
    overflow-x: auto;
  }

  .editor-toolbar {
    display: flex;
    gap: 10px;
    margin-top: 10px;
    flex-wrap: wrap;
  }

  .language-console {
    background-color: var(--terminal-bg);
    color: var(--terminal-text);
    height: 150px;
    overflow-y: auto;
    padding: 10px;
    font-family: 'Consolas', 'Courier New', monospace;
    margin-top: 10px;
    border: 1px solid var(--secondary-color);
  }

  .language-console-container {
    margin-top: 15px;
  }

  #language-canvas {

```



```

        border: 1px solid var(--secondary-color);
        background-color: white;
        width: 100%;
        height: 400px;
        max-width: 400px;
    }

    .program-controls {
        display: flex;
        gap: 10px;
        margin-top: 15px;
        flex-wrap: wrap;
    }

    .console-line {
        margin-bottom: 5px;
        word-wrap: break-word;
    }
`;
document.head.appendChild(styleElement);

// Initialize the language
const lang = new OperatorLanguage('language-canvas', 'language-console');
window.operatorLang = lang; // Make it globally accessible

// Set up event listeners
const codeEditor = document.getElementById('code-editor');
const runButton = document.getElementById('run-program');
const stopButton = document.getElementById('stop-program');
const stepButton = document.getElementById('step-program');
const clearButton = document.getElementById('clear-program');
const exampleSelect = document.getElementById('example-programs');
const saveButton = document.getElementById('save-program');
const loadButton = document.getElementById('load-program');
const downloadButton = document.getElementById('download-canvas');

if (!codeEditor || !runButton || !stopButton || !stepButton || !clearButton ||
    !exampleSelect || !saveButton || !loadButton || !downloadButton) {
    console.error('One or more UI elements not found');
    return;
}

// Run program
runButton.addEventListener('click', function() {
    if (lang.running) {
        lang.stop();
        runButton.textContent = 'Run';
    } else {
        const code = codeEditor.value;
        try {
            lang.loadProgram(code);
            lang.run();
            runButton.textContent = 'Pause';
        } catch (error) {

```

```

        lang.writeToConsole(`Error loading program: ${error.message}`);
    }
}
});

// Stop program
stopButton.addEventListener('click', function() {
    lang.stop();
    runButton.textContent = 'Run';
});

// Step program
stepButton.addEventListener('click', function() {
    const code = codeEditor.value;
    if (lang.program.length === 0) {
        try {
            lang.loadProgram(code);
        } catch (error) {
            lang.writeToConsole(`Error loading program: ${error.message}`);
            return;
        }
    }
    lang.step();
});

// Clear program
clearButton.addEventListener('click', function() {
    if (confirm('Clear the code editor?')) {
        codeEditor.value = '';
        lang.clearConsole();
    }
});

// Load example
exampleSelect.addEventListener('change', function() {
    const example = exampleSelect.value;
    if (example && EXAMPLE_PROGRAMS[example]) {
        if (codeEditor.value.trim() !== '' &&
            !confirm('Replace current code with example?')) {
            exampleSelect.value = '';
            return;
        }
        codeEditor.value = EXAMPLE_PROGRAMS[example];
        lang.clearConsole();
        exampleSelect.value = '';
    }
});

// Save program
saveButton.addEventListener('click', function() {
    try {
        // Create the program data
        const programData = {
            code: codeEditor.value,

```

```

        timestamp: new Date().toISOString(),
        version: '1.0'
    };

    // Save to localStorage
    localStorage.setItem('operatorLangProgram',
JSON.stringify(programData));

    // Create download file
    const blob = new Blob([JSON.stringify(programData, null, 2)], {type:
'application/json'});
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = 'operator_program.json';
    a.click();
    URL.revokeObjectURL(url);

    lang.writeToConsole('Program saved successfully');
} catch (e) {
    lang.writeToConsole(`Error saving program: ${e.message}`);
}
});

// Load program
loadButton.addEventListener('click', function() {
    // Create file input for loading JSON
    const fileInput = document.createElement('input');
    fileInput.type = 'file';
    fileInput.accept = '.json';
    fileInput.style.display = 'none';
    document.body.appendChild(fileInput);

    fileInput.addEventListener('change', function(e) {
        const file = e.target.files[0];
        if (!file) return;

        const reader = new FileReader();
        reader.onload = function(e) {
            try {
                const data = JSON.parse(e.target.result);

                // Validate the data
                if (!data.code || !data.version) {
                    throw new Error('Invalid program file format');
                }

                if (data.version !== '1.0') {
                    throw new Error(`Unsupported program version:
${data.version}`);
                }

                // Set the code
                codeEditor.value = data.code;

```

```

        // Save to localStorage
        localStorage.setItem('operatorLangProgram',
JSON.stringify(data));

        lang.writeToConsole(`Program loaded from file (${new
Date(data.timestamp).toLocaleString()})`);
    } catch (e) {
        lang.writeToConsole(`Error loading program: ${e.message}`);
    }
};
reader.onerror = function() {
    lang.writeToConsole('Error reading file');
};
reader.readAsText(file);

document.body.removeChild(fileInput);
});

// Check if we have a saved program in localStorage
const savedProgram = localStorage.getItem('operatorLangProgram');
if (savedProgram) {
    try {
        const data = JSON.parse(savedProgram);

        // Validate the data
        if (!data.code || !data.version) {
            throw new Error('Invalid saved program format');
        }

        if (data.version !== '1.0') {
            throw new Error(`Unsupported program version:
${data.version}`);
        }

        // Prompt to load the saved program
        if (confirm(`Load saved program from ${new
Date(data.timestamp).toLocaleString()}?`)) {
            codeEditor.value = data.code;
            lang.writeToConsole(`Program loaded from local storage (${new
Date(data.timestamp).toLocaleString()})`);
            return;
        }
    } catch (e) {
        lang.writeToConsole(`Error loading saved program: ${e.message}`);
    }
}

// If no saved program or user declined, show file dialog
fileInput.click();
});

// Download canvas as image
downloadButton.addEventListener('click', function() {

```

```

const canvas = document.getElementById('language-canvas');
const url = canvas.toDataURL('image/png');
const a = document.createElement('a');
a.href = url;
a.download = 'operator_canvas.png';
a.click();
});

// Load any saved program on startup
const savedProgram = localStorage.getItem('operatorLangProgram');
if (savedProgram) {
    try {
        const data = JSON.parse(savedProgram);
        if (data.code && data.version === '1.0') {
            codeEditor.value = data.code;
        }
    } catch (e) {
        console.error('Error loading saved program:', e);
    }
} else {
    // Load a default program if no saved program exists
    codeEditor.value = EXAMPLE_PROGRAMS.helloWorld;
}

// Handle key shortcuts in the editor
codeEditor.addEventListener('keydown', function(e) {
    if (e.key === 'Tab') {
        e.preventDefault();

        // Insert a tab at the cursor position
        const start = this.selectionStart;
        const end = this.selectionEnd;

        // If multiple lines are selected, indent all of them
        if (start !== end) {
            const lines = this.value.split('\n');
            let startLine = this.value.substr(0, start).split('\n').length - 1;

            let endLine = this.value.substr(0, end).split('\n').length - 1;

            let newStart = start;
            let newEnd = end;

            if (e.shiftKey) {
                // Unindent
                for (let i = startLine; i <= endLine; i++) {
                    if (lines[i].startsWith('\t')) {
                        lines[i] = lines[i].substring(1);
                        if (i === startLine) {
                            newStart--;
                        }
                    }
                    newEnd--;
                }
            }
        }
    }
});

```

```

        } else {
            // Indent
            for (let i = startLine; i <= endLine; i++) {
                lines[i] = '\t' + lines[i];
                if (i === startLine) {
                    newStart++;
                }
                newEnd++;
            }
        }

        this.value = lines.join('\n');
        this.selectionStart = newStart;
        this.selectionEnd = newEnd;
    } else {
        // Just insert a tab at cursor position
        this.value = this.value.substring(0, start) + '\t' +
this.value.substring(end);
        this.selectionStart = this.selectionEnd = start + 1;
    }
    }
});
});

```

[8] operator-map.js

- **Bytes:** 36234
- **Type:** text

```

/**
 * operator-map.js
 *
 * This module implements the Map System for the Operator framework,
 * providing charset management and string ID calculation functionality.
 * It integrates with the operator-dropdown.js system.
 *
 * @module OperatorMap
 * @version 1.0.0
 * @depends OperatorFramework.Dropdown
 */

// Access the shared namespace for Operator framework

/**
 * Map System for charset operations and string ID calculations
 * @namespace OperatorMap
 */
OperatorFramework.Map = (function() {
    'use strict';

    // Private module variables

```

```

let _initialized = false;
let _section = null;
let _charsetConfig = [];
let _uniqueCharset = [];
let _defaultOptions = {
  containerId: 'main', // ID of the container to add the map section to
  sectionId: 'map-system-section' // ID for the created section element
};
let _options = {};

/**
 * Initialize the Map System
 * @param {Object} options - Configuration options
 * @param {string} [options.containerId='main'] - ID of the container to add
the map section to
 * @param {string} [options.sectionId='map-system-section'] - ID for the
created section element
 * @returns {boolean} Success status
 */
function initialize(options = {}) {
  if (_initialized) {
    console.warn('OperatorMap is already initialized');
    return false;
  }

  // Merge options with defaults
  _options = { ..._defaultOptions, ...options };

  // Initialize the character set configuration
  _initCharsetConfig();

  // Generate the unique charset based on the configuration
  _uniqueCharset = generateCharsetFromConfig(_charsetConfig);

  console.log(`OperatorMap initialized with ${_uniqueCharset.length}
characters`);
  _initialized = true;

  // Register with dropdown system if available
  if (OperatorFramework.Dropdown) {
    OperatorFramework.Dropdown.addSystem({
      id: 'map',
      name: 'Charset App',
      description: 'Calculate String IDs and perform charset
operations',
      loadFunction: loadMapSystem
    });
    console.log('Map System registered with OperatorFramework.Dropdown');
  } else {
    console.warn('OperatorFramework.Dropdown not available, Map System not
registered');
  }

  return true;
}

```

```

}

/**
 * Load the Map System UI and functionality
 * @returns {boolean} Success status
 */
function loadMapSystem() {
    if (!_initialized) {
        console.error('OperatorMap not initialized, call initialize() first');
        return false;
    }

    // Check if already loaded
    if (_section && document.getElementById(_options.sectionId)) {
        showMapSection();
        return true;
    }

    // Get container
    //const container = document.getElementById(_options.containerId);
    // Use:
    const container = document.querySelector('main');
    if (!container) {
        console.error(`Container with ID "${_options.containerId}" not found`);
        return false;
    }

    // Create Map System section
    _section = document.createElement('section');
    _section.id = _options.sectionId;
    _section.className = 'operation-section';

    // Populate with Map System content
    _section.innerHTML = _getMapSystemHTML();

    // Add to container
    container.appendChild(_section);

    // Add styles
    _addMapSystemStyles();

    // Initialize event listeners
    _initMapSystemEventListeners();

    // Show the section
    showMapSection();

    console.log('Map System loaded successfully');
    return true;
}

/**
 * Show the Map System section and hide other sections

```



```

*/
function showMapSection() {
    if (!_section) return;

    // Hide all sections
    const allSections = document.querySelectorAll('.operation-section');
    allSections.forEach(section => {
        section.classList.remove('active');
    });

    // Show Map section
    _section.classList.add('active');
}

/**
 * Generate the Map System HTML content
 * @private
 * @returns {string} HTML content
 */
function _getMapSystemHTML() {
    return `
        <h2>Charset App (Spatial Systems Engineering)</h2>
        <div class="operation-container">
            <div class="operation-controls">
                <div id="main-menu">
                    <h3>Choose an option:</h3>
                    <button id="show-generate-combinations">Generate
Combinations</button>
                    <button id="show-calculate-string-id">Calculate String
ID</button>
                    <button id="show-decode-id">Decode ID to String</button>
                    <button id="show-find-optimal-variable">Find Optimal
Variable and Save to File</button>
                </div>

                <div id="color-bar" style="width: 100%; height: 20px;
background-color: gold; margin-top: 10px;"></div>

                <div id="option-container" style="display: none;">
                    <!-- Mode 1: Generate Combinations -->
                    <div id="generate-combinations" style="display: none;">
                        <h3>Generate Combinations</h3>
                        <label for="combination-size">Enter the size of
combinations (n):</label>
                        <input type="number" id="combination-size">
                        <br>
                        <label for="output-file-generate">Enter the name of
the output file:</label>
                        <input type="text" id="output-file-generate">
                        <br>
                        <label for="use-multithreading">Use Multithreading:
</label>
                        <select id="use-multithreading">
                            <option value="yes">Yes</option>

```

```

        <option value="no">No</option>
    </select>
    <br>
    <button id="generate-combinations-
btn">Generate</button>
</div>

<!-- Mode 2: Calculate String ID -->
<div id="calculate-string-id" style="display: none;">
    <h3>Calculate String ID</h3>
    <label for="custom-string">Enter your custom string:
</label>

    <textarea id="custom-string" rows="10" cols="50"
wrap="off"></textarea>

    <br>
    <button id="calculate-string-id-
btn">Calculate</button>

    <p id="string-id-result"></p>
    <div id="character-word-count">Characters: 0, Words:
0</div>

</div>

<!-- Mode 3: Decode ID to String -->
<div id="decode-id" style="display: none;">
    <h3>Decode ID to String</h3>
    <label for="string-id">Enter the ID to decode:</label>
    <input type="number" id="string-id">
    <br>
    <button id="decode-id-btn">Decode</button>
    <p id="decoded-string-result"></p>
</div>

<!-- Mode 4: Find Optimal Variable -->
<div id="find-optimal-variable" style="display: none;">
    <h3>Find Optimal Variable and Save to File</h3>
    <label for="user-number">Enter the user number:
</label>

    <input type="number" id="user-number">
    <br>
    <label for="output-file-optimal">Enter the name of the
output file:</label>

    <input type="text" id="output-file-optimal">
    <br>
    <button id="find-optimal-variable-btn">Find and
Save</button>

</div>
</div>
</div>

<div class="operation-result">
    <h3>Map System Result</h3>
    <div id="map-result-display" class="result-container">
        <p>Select an operation from the left panel to start.</p>
    </div>

```

```

        </div>
    </div>
    `;
}

/**
 * Add CSS styles for the Map System
 * @private
 */
function _addMapSystemStyles() {
    const styleId = 'operator-map-styles';

    // Don't add styles if they already exist
    if (document.getElementById(styleId)) return;

    const styleElement = document.createElement('style');
    styleElement.id = styleId;
    styleElement.textContent = `
        /* Basic Reset */
        #map-system-section * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        /* Body Styling - adapted for the section */
        #map-system-section {
            font-family: "Fira Code", "Consolas", monospace;
            color: #333333; /* Dark Gray */
        }

        /* Heading Styles */
        #map-system-section h3 {
            color: #4B5320; /* Army Green */
            margin-bottom: 15px;
            text-shadow: 1px 1px 3px rgba(0, 0, 0, 0.3);
            font-weight: 600;
        }

        /* Button Styles */
        #map-system-section button {
            background-color: #4B5320; /* Army Green */
            color: #FFFFFF;
            padding: 10px 20px;
            font-size: 16px;
            border: 2px solid #BDB76B; /* Dark Khaki */
            cursor: pointer;
            transition: background-color 0.3s ease, color 0.3s ease;
            font-family: "Fira Code", "Consolas", monospace;
            margin-top: 10px;
        }

        #map-system-section button:hover {
            background-color: #BDB76B; /* Dark Khaki */

```

```

        color: #333333; /* Dark Gray */
    }

    /* Input and Textarea Styles */
    #map-system-section input[type="text"],
    #map-system-section input[type="number"],
    #map-system-section textarea,
    #map-system-section select {
        width: calc(100% - 20px);
        padding: 10px;
        font-size: 16px;
        background-color: #F5F5DC; /* Beige */
        color: #333333; /* Dark Gray */
        border: 2px solid #BDB76B; /* Dark Khaki */
        font-family: "Fira Code", "Consolas", monospace;
        margin-bottom: 10px;
    }

    #map-system-section textarea {
        resize: none; /* Disables resizing of the textarea */
    }

    /* Results Styling */
    #string-id-result, #decoded-string-result {
        background-color: #FFFFFF; /* White */
        padding: 10px;
        border: 2px solid #BDB76B; /* Dark Khaki */
        font-family: "Fira Code", "Consolas", monospace;
        box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
        color: #333333; /* Dark Gray */
        word-wrap: break-word;
        white-space: pre-wrap; /* Preserves spaces and line breaks */
        margin-top: 10px;
    }

    /* Color bar styling */
    #color-bar {
        width: 100%; /* Full width */
        height: 20px; /* Fixed height */
        background-color: #BDB76B; /* Dark Khaki */
        margin-top: 10px; /* Space above the bar */
        border: 2px solid #4B5320; /* Army Green border */
        box-shadow: 0 2px 5px rgba(0, 0, 0, 0.3); /* Subtle shadow */
        transition: background-color 0.3s ease; /* Smooth transition for
color change */
    }
    `;

    document.head.appendChild(styleElement);
}

/**
 * Initialize the charset configuration
 * @private

```

```

*/
function _initCharsetConfig() {
    _charsetConfig = [
        { name: 'Basic Latin', range: [0x0020, 0x007F] },
        { name: 'Latin-1 Supplement', range: [0x0080, 0x00FF] },
        { name: 'Latin Extended-A', range: [0x0100, 0x017F] },
        { name: 'Latin Extended-B', range: [0x0180, 0x024F] },
        { name: 'Greek and Coptic', range: [0x0370, 0x03FF] },
        { name: 'Cyrillic', range: [0x0400, 0x04FF] },
        { name: 'Arabic', range: [0x0600, 0x06FF] },
        { name: 'Hebrew', range: [0x0590, 0x05FF] },
        { name: 'Devanagari', range: [0x0900, 0x097F] },
        { name: 'Mathematical Operators', range: [0x2200, 0x22FF] },
        { name: 'Supplemental Mathematical Operators', range: [0x2A00, 0x2AFF] },
    ],

    { name: 'Miscellaneous Technical', range: [0x2300, 0x23FF] },
    { name: 'Miscellaneous Symbols and Arrows', range: [0x2190, 0x21FF] },
    { name: 'CJK Unified Ideographs', range: [0x4E00, 0x9FFF] },
    { name: 'Hangul Syllables', range: [0xAC00, 0xD7AF] },
    { name: 'Hiragana', range: [0x3040, 0x309F] },
    { name: 'Katakana', range: [0x30A0, 0x30FF] },
    { name: 'Bopomofo', range: [0x3100, 0x312F] },
    { name: 'Currency Symbols', range: [0x20A0, 0x20CF] },
    { name: 'Additional Punctuation', range: [0x2000, 0x206F] }

    ];
}

/**
 * Generate a charset from configuration
 * @param {Array} config - Array of charset range configurations
 * @returns {Array} Array of characters in the charset
 */
function generateCharsetFromConfig(config) {
    const charset = new Set();
    config.forEach(block => {
        for (let i = block.range[0]; i <= block.range[1]; i++) {
            try {
                charset.add(String.fromCharCode(i));
            } catch (e) {
                console.error(`Failed to add character code ${i}:
${e.message}`);
            }
        }
    });
    // Explicitly add newline and tab characters
    charset.add('\n');
    charset.add('\t');
    return Array.from(charset);
}

/**
 * Initialize event listeners for the Map System
 * @private
 */

```

```

function _initMapSystemEventListeners() {
    // Option buttons
    document.getElementById('show-generate-combinations').addEventListener('click', () => showOption('generate-combinations'));
    document.getElementById('show-calculate-string-id').addEventListener('click', () => showOption('calculate-string-id'));
    document.getElementById('show-decode-id').addEventListener('click', () => showOption('decode-id'));
    document.getElementById('show-find-optimal-variable').addEventListener('click', () => showOption('find-optimal-variable'));

    // Action buttons
    document.getElementById('generate-combinations-btn').addEventListener('click', generateCombinations);
    document.getElementById('calculate-string-id-btn').addEventListener('click', calculateStringID);
    document.getElementById('decode-id-btn').addEventListener('click', decodeID);
    document.getElementById('find-optimal-variable-btn').addEventListener('click', findOptimalVariable);

    // Text area for character count
    const customStringTextarea = document.getElementById('custom-string');
    if (customStringTextarea) {
        customStringTextarea.addEventListener('input', function(event) {
            const inputString = event.target.value;
            displayCharacterAndWordCount(inputString);
        });
    }
}

/**
 * Show an option panel and hide others
 * @param {string} optionId - ID of the option to show
 */
function showOption(optionId) {
    const options = document.querySelectorAll('#option-container > div');
    options.forEach(opt => opt.style.display = 'none');
    document.getElementById(optionId).style.display = 'block';
    document.getElementById('option-container').style.display = 'block';
}

/**
 * Generate combinations
 */
function generateCombinations() {
    try {
        const n = BigInt(document.getElementById('combination-size').value);
        const outputFile = document.getElementById('output-file-generate').value || 'combinations.txt';

        if (isNaN(Number(n)) || n <= 0n) {
            throw new Error("Invalid combination size. Please enter a positive

```

```

integer.");
    }

    // For large n values, show a warning
    if (n > 4n) {
        if (!confirm(`This will generate up to ${_uniqueCharset.length **
Number(n)} combinations. This could be a large file. Continue?`)) {
            return;
        }
    }

    const resultDisplay = document.getElementById('map-result-display');
    resultDisplay.innerHTML = '<p>Generating combinations... This may take
a while for large values.</p>';

    // Use setTimeout to allow UI to update before starting computation
    setTimeout(() => {
        try {
            let combinationCount = 0;
            let combinations = '';
            const k = BigInt(_uniqueCharset.length);
            const nbrComb = k ** n;

            // For large combinations, limit how many we generate
            const maxToGenerate = 1000000n; // 1 million max
            const actualGenerate = nbrComb > maxToGenerate ? maxToGenerate
: nbrComb;

            // Generate combinations
            for (let i = 0n; i < actualGenerate; i++) {
                let id = i;
                let combination = '';
                for (let j = 0n; j < n; j++) {
                    combination = _uniqueCharset[Number(id % k)] +
combination;

                    id = id / k;
                }
                combinations += combination + '\n';
                combinationCount++;
            }

            // Download the file
            downloadFile(outputFile, combinations);

            // Update the result display
            resultDisplay.innerHTML = `
                <p>Successfully generated
${combinationCount.toLocaleString()} combinations.</p>
                <p>Total possible combinations: ${nbrComb.toString()}.</p>
                <p>File saved as: ${outputFile}</p>
            `;
        } catch (error) {
            resultDisplay.innerHTML = `<p class="error">Error during
generation: ${error.message}</p>`;

```

```

        }
    }, 100));
} catch (error) {
    alert(`Error: ${error.message}`);
}
}

/**
 * Calculate a string ID
 */
function calculateStringID() {
    try {
        const inputString = document.getElementById('custom-string').value;
        if (inputString.trim() === '') {
            throw new Error("Input string cannot be empty.");
        }

        let id = 0n;
        const resultDisplay = document.getElementById('map-result-display');
        resultDisplay.innerHTML = '<p>Calculating string ID...</p>';

        // Use setTimeout to allow UI to update before starting computation
        setTimeout(() => {
            try {
                // Check all characters are in charset
                const invalidChars = [];
                for (const char of inputString) {
                    const charIndex = _uniqueCharset.indexOf(char);
                    if (charIndex === -1) {
                        invalidChars.push(char);
                    }
                }

                if (invalidChars.length > 0) {
                    throw new Error(`Invalid characters found:
${invalidChars.join(' ')}`);
                }

                // Calculate ID
                for (const char of inputString) {
                    const charIndex = _uniqueCharset.indexOf(char);
                    id = id * BigInt(_uniqueCharset.length) +
BigInt(charIndex);
                }

                const result = id.toString() + "\t\n\n" + inputString;
                document.getElementById('string-id-result').innerText = "The
ID for the string is: " + id.toString();

                // Display character and word count
                displayCharacterAndWordCount(inputString);

                // Decode ID to color indexes and update color bar
                const colorIndexes = decodeIDtoColorIndexes(id.toString());

```



```

        updateColorBar(colorIndexes);

        // Update result display
        resultDisplay.innerHTML = `
            <p>Calculation completed successfully.</p>
            <p>String ID: ${id.toString()}</p>
            <p>Character count: ${inputString.length}</p>
            <p>Color indexes: ${colorIndexes.length}</p>
            <p><button id="download-id-result">Download
Result</button></p>
        `;

        // Add event listener for download button
        document.getElementById('download-id-
result').addEventListener('click', () => {
            const outputFile = `${colorIndexes.length}.txt`;
            downloadFile(outputFile, result);
        });
    } catch (error) {
        resultDisplay.innerHTML = `<p class="error">Error calculating
ID: ${error.message}</p>`;
    }
    }, 100);
} catch (error) {
    alert(`Error: ${error.message}`);
}
}

/**
 * Decode an ID to a string
 */
function decodeID() {
    try {
        let id = BigInt(document.getElementById('string-id').value);
        if (id < 0n) {
            throw new Error("ID cannot be negative.");
        }

        const resultDisplay = document.getElementById('map-result-display');
        resultDisplay.innerHTML = '<p>Decoding ID to string...</p>';

        // Use setTimeout to allow UI to update before starting computation
        setTimeout(() => {
            try {
                const decodedString = [];
                const originalId = id;

                // If ID is too large, show warning
                if (id > 10n ** 30n) {
                    resultDisplay.innerHTML += '<p class="warning">Warning:
Very large ID. Decoding may be incomplete.</p>';
                }

                // Decode ID to string

```

```

        while (id > 0n) {
            decodedString.push(_uniqueCharset[Number(id %
BigInt(_uniqueCharset.length))]);
            id = id / BigInt(_uniqueCharset.length);
        }

        if (decodedString.length === 0) {
            throw new Error("Decoded string is empty.");
        }

        const decodedResult = decodedString.reverse().join('');
        document.getElementById('decoded-string-result').innerText =
"The decoded string is: " + decodedResult;

        // Update result display
        resultDisplay.innerHTML = `
            <p>Decoding completed successfully.</p>
            <p>Original ID: ${originalId.toString()}</p>
            <p>Decoded string length: ${decodedResult.length}</p>
            <p>Decoded string:</p>
            <pre>${decodedResult}</pre>
        `;
    } catch (error) {
        resultDisplay.innerHTML = `<p class="error">Error decoding ID:
${error.message}</p>`;
    }
}, 100);
} catch (error) {
    alert(`Error: ${error.message}`);
}
}

/**
 * Find optimal variable
 */
function findOptimalVariable() {
    try {
        const userNumber = BigInt(document.getElementById('user-
number').value);
        const outputFile = document.getElementById('output-file-
optimal').value || 'optimal_variables.txt';

        if (userNumber <= 0n) {
            throw new Error("User number must be positive.");
        }

        const resultDisplay = document.getElementById('map-result-display');
        resultDisplay.innerHTML = '<p>Finding optimal variables...</p>';

        // Use setTimeout to allow UI to update before starting computation
        setTimeout(() => {
            try {
                const charsetLength = BigInt(_uniqueCharset.length);
                const extendedCharsetLength = BigInt(_uniqueCharset.length);

```

```

        const generator = optimalVariableGenerator(userNumber,
charsetLength, extendedCharsetLength);

        const results = [];
        for (let i = 0; i < 100; i++) {
            const optVar = generator.next().value;
            const decodedString = decodeIDFromNumber(optVar,
_uniqueCharset);

            results.push({
                variable: optVar,
                string: decodedString,
                length: decodedString.length
            });
        }

        // Format the results for download
        const resultString = results.map(r =>
`${r.variable}\t${r.string}`).join('\n');

        // Download the file
        downloadFile(outputFile, resultString);

        // Display the results
        let resultsHTML = '<p>Results (showing first 10):</p><table
class="results-table">';
        resultsHTML += '<tr><th>Variable</th><th>String</th>
<th>Length</th></tr>';

        results.slice(0, 10).forEach(r => {
            resultsHTML += `<tr><td>${r.variable}</td><td>${r.string}
</td><td>${r.length}</td></tr>`;
        });

        resultsHTML += '</table>';

        // Update result display
        resultDisplay.innerHTML = `
            <p>Successfully found 100 optimal variables.</p>
            <p>File saved as: ${outputFile}</p>
            ${resultsHTML}
        `;
    } catch (error) {
        resultDisplay.innerHTML = `<p class="error">Error finding
optimal variables: ${error.message}</p>`;
    }
    }, 100);
} catch (error) {
    alert(`Error: ${error.message}`);
}
}

/**
 * Generate an optimal variable
 * @param {BigInt} userNumber - The user number

```

```

* @param {BigInt} charsetLength - The charset length
* @param {BigInt} extendedCharsetLength - The extended charset length
* @returns {Object} Iterator for generating optimal variables
*/
function optimalVariableGenerator(userNumber, charsetLength,
extendedCharsetLength) {
    let k = 1n;
    return {
        next: function() {
            while (true) {
                const x = k * userNumber;
                if (x % extendedCharsetLength < charsetLength) {
                    const result = { value: x, done: false };
                    k += 1n;
                    return result;
                }
                k += 1n;
            }
        }
    };
}

/**
 * Check if a number is a perfect square
 * @param {number} n - Number to check
 * @returns {boolean} Whether the number is a perfect square
 */
function isPerfectSquare(n) {
    if (n <= 0) return false; // Grids must have a positive number of tiles
    const sqrt = Math.sqrt(n);
    return sqrt === Math.floor(sqrt);
}

/**
 * Validate color indexes
 * @param {Array<number>} indexes - Array of color indexes
 * @returns {boolean} Whether the indexes are valid
 */
function areValidColorIndexes(indexes) {
    const maxColorIndex = Math.pow(16, 6); // Maximum valid color index is
16777216
    return indexes.every(index => index >= 1 && index <= maxColorIndex);
}

/**
 * Update the color bar based on color indexes
 * @param {Array<number>} indexes - Array of color indexes
 */
function updateColorBar(indexes) {
    const colorBar = document.getElementById('color-bar');
    if (!colorBar) return;

    if (isPerfectSquare(indexes.length) && areValidColorIndexes(indexes)) {
        colorBar.style.backgroundColor = 'green'; // Valid grid
    }
}

```

```

    } else {
        colorBar.style.backgroundColor = 'red'; // Invalid grid
    }
}

/**
 * Decode an ID to color indexes
 * @param {string} idString - The ID string
 * @returns {Array<number>} Array of color indexes
 */
function decodeIDtoColorIndexes(idString) {
    const indexes = [];
    const segmentLength = 7; // Each color index is represented by a 7-digit
segment

    // Split the integer string into 7-digit segments
    for (let i = 0; i < idString.length; i += segmentLength) {
        const segment = idString.slice(i, i + segmentLength);
        const index = parseInt(segment, 10);

        if (!isNaN(index)) {
            indexes.push(index);
        }
    }
    return indexes;
}

/**
 * Display character and word count
 * @param {string} inputText - The input text
 */
function displayCharacterAndWordCount(inputText) {
    const characterCount = inputText.length;
    const wordCount = inputText.trim() === '' ? 0 :
inputText.trim().split(/\s+/).length;

    const charWordCountElement = document.getElementById('character-word-
count');
    if (charWordCountElement) {
        charWordCountElement.innerHTML = `Characters: ${characterCount},
Words: ${wordCount}`;
    }
}

/**
 * Decode an ID from a number
 * @param {BigInt} id - The ID to decode
 * @param {Array<string>} charset - The charset to use
 * @returns {string} The decoded string
 */
function decodeIDFromNumber(id, charset) {
    const decodedString = [];
    while (id > 0n) {
        decodedString.push(charset[Number(id % BigInt(charset.length))]);
    }
}

```

```

        id = id / BigInt(charset.length);
    }
    return decodedString.reverse().join('');
}

/**
 * Download a file
 * @param {string} filename - The filename
 * @param {string} content - The file content
 */
function downloadFile(filename, content) {
    const blob = new Blob([content], { type: 'text/plain' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = filename;
    document.body.appendChild(a);
    a.click();
    document.body.removeChild(a);
    URL.revokeObjectURL(url);
}

/**
 * Get the charset configuration
 * @returns {Array} The charset configuration
 */
function getCharsetConfig() {
    return [..._charsetConfig];
}

/**
 * Get the unique charset
 * @returns {Array<string>} The unique charset
 */
function getUniqueCharset() {
    return [..._uniqueCharset];
}

/**
 * Calculate a string ID (functional version for API access)
 * @param {string} inputString - The input string
 * @returns {BigInt} The calculated ID
 */
function calculateID(inputString) {
    if (!inputString || typeof inputString !== 'string') {
        throw new Error("Input must be a non-empty string");
    }

    let id = 0n;
    for (const char of inputString) {
        const charIndex = _uniqueCharset.indexOf(char);
        if (charIndex === -1) {
            throw new Error(`Character "${char}" is not in the charset`);
        }
    }

```

```

        id = id * BigInt(_uniqueCharset.length) + BigInt(charIndex);
    }

    return id;
}

/**
 * Decode an ID to a string (functional version for API access)
 * @param {BigInt|string} id - The ID to decode
 * @returns {string} The decoded string
 */
function decodeIDtoString(id) {
    try {
        id = BigInt(id);
    } catch (e) {
        throw new Error("Invalid ID format");
    }

    if (id < 0n) {
        throw new Error("ID cannot be negative");
    }

    const decodedString = [];
    while (id > 0n) {
        decodedString.push(_uniqueCharset[Number(id %
BigInt(_uniqueCharset.length))]);
        id = id / BigInt(_uniqueCharset.length);
    }

    return decodedString.reverse().join('');
}

// Public API
return {
    initialize,
    loadMapSystem,
    showMapSection,
    calculateID,
    decodeIDtoString,
    getCharsetConfig,
    getUniqueCharset,
    isPerfectSquare,
    areValidColorIndexes,
    updateColorBar,
    decodeIDtoColorIndexes,
    downloadFile
};
})();

// Automatically initialize when the DOM is ready
document.addEventListener('DOMContentLoaded', function() {
    // Look for the main container
    const mainElement = document.querySelector('main');
    if (mainElement) {

```

```

    OperatorFramework.Map.initialize({
        containerId: mainElement.id || 'main'
    });
    console.log('Map System initialized');
} else {
    console.warn('Main element not found, Map System not automatically
initialized');
}
});

```

[9] operator-persistence.js

- **Bytes:** 75169
- **Type:** text

```

/**
 * Operator Persistence System
 * A comprehensive state management and persistence system for the Operator OS
 *
 * This module handles saving and loading the state of all components:
 * - Data Structures (arrays, objects, sets, quadtrees)
 * - Algorithms (recursion, iteration, search, binary operations)
 * - System Operations (file system, memory management, process management)
 * - CRUD Database
 * - Advanced Concepts (serialization, compression, encoding, transactions)
 * - OperatorLang programming environment
 */

(function() {
    'use strict';

    // Configuration
    const CONFIG = {
        metadataKey: 'operatorSessionMetadata',
        chunkPrefix: 'operatorSessionChunk_',
        chunkSize: 1024 * 1024, // 1MB chunks
        version: '1.1',
        encryptionKey: 'OPERATOR_SYSTEM_ENCRYPTION_KEY',
        autoSaveKey: 'operatorAutoSave'
    };

    // State Registry - tracks all persistable components and their save/load
    // handlers
    const stateRegistry = new Map();

    // Initialize when document is ready
    document.addEventListener('DOMContentLoaded', function() {
        // Wait for all components to initialize
        setTimeout(initPersistenceSystem, 500);
    });

```



```

/**
 * Initialize the persistence system
 */
function initPersistenceSystem() {
    console.log("Initializing Operator Persistence System...");

    // Register all state handlers
    registerStateHandlers();

    // Create or enhance the persistence controls
    setupPersistenceControls();

    // Check for auto-recovery if configured
    checkAutoRecovery();

    // Register window unload handler to offer auto-save
    setupUnloadHandler();

    // Add this to the initPersistenceSystem function
    // Update workspace info every 30 seconds
    setInterval(updateWorkspaceInfo, 30000);
}

/**
 * Register state handlers for all components
 */
function registerStateHandlers() {
    // Data Structures
    registerComponent('arrays', {
        selector: '#array-operations',
        save: saveArrayState,
        load: loadArrayState
    });

    registerComponent('objects', {
        selector: '#object-operations',
        save: saveObjectState,
        load: loadObjectState
    });

    registerComponent('sets', {
        selector: '#set-operations',
        save: saveSetState,
        load: loadSetState
    });

    registerComponent('quadtree', {
        selector: '#tree-operations',
        save: saveQuadtreeState,
        load: loadQuadtreeState
    });

    // CRUD Database
    registerComponent('database', {

```

```

        selector: '#create-operations',
        save: saveDatabaseState,
        load: loadDatabaseState,
        priority: 10 // Higher priority components are processed first during
load
    });

// System Operations
registerComponent('fileSystem', {
    selector: '#file-operations',
    save: saveFileSystemState,
    load: loadFileSystemState,
    priority: 20
});

registerComponent('memoryManagement', {
    selector: '#memory-operations',
    save: saveMemoryState,
    load: loadMemoryState
});

registerComponent('processManagement', {
    selector: '#process-operations',
    save: saveProcessState,
    load: loadProcessState
});

// Algorithm States
registerComponent('recursion', {
    selector: '#recursion-operations',
    save: saveRecursionState,
    load: loadRecursionState
});

registerComponent('iteration', {
    selector: '#iteration-operations',
    save: saveIterationState,
    load: loadIterationState
});

registerComponent('search', {
    selector: '#search-operations',
    save: saveSearchState,
    load: loadSearchState
});

registerComponent('binaryOps', {
    selector: '#binary-operations',
    save: saveBinaryOperationsState,
    load: loadBinaryOperationsState
});

// Advanced Components
registerComponent('serialization', {

```

```

        selector: '#serialization-operations',
        save: saveSerializationState,
        load: loadSerializationState
    });

    registerComponent('compression', {
        selector: '#compression-operations',
        save: saveCompressionState,
        load: loadCompressionState
    });

    registerComponent('encoding', {
        selector: '#encoding-operations',
        save: saveEncodingState,
        load: loadEncodingState
    });

    registerComponent('transactions', {
        selector: '#transactions-operations',
        save: saveTransactionState,
        load: loadTransactionState
    });

    // Programming Language
    registerComponent('operatorLang', {
        selector: '#programming-section',
        save: saveOperatorLangState,
        load: loadOperatorLangState,
        priority: 30
    });
}

/**
 * Register a component with the state registry
 */
function registerComponent(id, config) {
    // Check if component exists in DOM
    const element = document.querySelector(config.selector);
    if (!element) {
        console.log(`Component ${id} not found in DOM, skipping
registration`);
        return;
    }

    // Set default priority (0) if not specified
    if (config.priority === undefined) {
        config.priority = 0;
    }

    // Register component
    stateRegistry.set(id, {
        id: id,
        element: element,
        save: config.save,

```

```

        load: config.load,
        priority: config.priority,
        active: true
    });

    console.log(`Registered component: ${id}`);
}

function setupPersistenceControls() {
    // Check for existing controls in the main navigation
    const mainNav = document.getElementById('main-nav');
    if (mainNav) {
        // Create session section in nav if it doesn't exist
        // This line has the error:
        // let sessionSection = mainNav.querySelector('.nav-section:has(h3:contains("Session"))');

        // Find Session section without using :has or :contains
        let sessionSection = null;
        const navSections = mainNav.querySelectorAll('.nav-section');
        for (const section of navSections) {
            const h3 = section.querySelector('h3');
            if (h3 && h3.textContent === 'Session') {
                sessionSection = section;
                break;
            }
        }

        if (!sessionSection) {
            sessionSection = document.createElement('div');
            sessionSection.className = 'nav-section';
            sessionSection.innerHTML = `
                <h3>Session</h3>
                <div class="persistence-controls">
                    <button id="save-session" title="Save current
workspace">Save Workspace</button>
                    <button id="load-session" title="Load saved
workspace">Load Workspace</button>
                    <button id="auto-save-toggle" class="button-secondary"
title="Toggle automatic saving">
                        Auto Save: <span id="auto-save-status">Off</span>
                    </button>
                </div>
            `;
            mainNav.appendChild(sessionSection);
        }

        // Set up event listeners
        const saveButton = document.getElementById('save-session');
        const loadButton = document.getElementById('load-session');
        const autoSaveToggle = document.getElementById('auto-save-toggle');

        if (saveButton) {
            saveButton.removeEventListener('click', saveAllStates);

```

```

        saveButton.addEventListener('click', saveAllStates);
    }

    if (loadButton) {
        loadButton.removeEventListener('click', loadWorkspaceFromFile);
        loadButton.addEventListener('click', loadWorkspaceFromFile);
    }

    if (autoSaveToggle) {
        const autoSaveStatus = document.getElementById('auto-save-
status');

        const autoSaveEnabled = localStorage.getItem(CONFIG.autoSaveKey)
=== 'true';

        // Set initial status
        if (autoSaveStatus) {
            autoSaveStatus.textContent = autoSaveEnabled ? 'On' : 'Off';
        }

        // Toggle auto-save on click
        autoSaveToggle.addEventListener('click', function() {
            const currentStatus = localStorage.getItem(CONFIG.autoSaveKey)
=== 'true';

            const newStatus = !currentStatus;

            localStorage.setItem(CONFIG.autoSaveKey,
newStatus.toString());

            if (autoSaveStatus) {
                autoSaveStatus.textContent = newStatus ? 'On' : 'Off';
            }

            showNotification(`Auto save ${newStatus ? 'enabled' :
'disabled'}`, 'info');
        });
    }

    console.log("Set up persistence controls in main navigation");
} else {
    // Create floating controls if main nav not found
    createFloatingPersistenceControls();
}

// Add this to operator-persistence.js inside the setupPersistenceControls
function

function createSessionManagementSection() {
    // Check if the section already exists
    if (document.getElementById('session-management')) {
        return;
    }

    // Create a new section
    const section = document.createElement('section');

```

```

section.id = 'session-management';
section.className = 'operation-section';

// Add the section content
section.innerHTML = `
    <h2>Session Management</h2>
    <div class="operation-container">
        <div class="operation-controls">
            <h3>Workspace Management</h3>
            <div class="control-group">
                <button id="save-workspace-btn" class="button">Save
Workspace</button>
                <button id="load-workspace-btn" class="button">Load
Workspace</button>
            </div>
            <div class="control-group mt-20">
                <h4>Backup Options</h4>
                <button id="export-workspace-btn"
class="button">Export Workspace</button>
                <button id="import-workspace-btn"
class="button">Import Workspace</button>
            </div>
            <div class="control-group mt-20">
                <h4>Auto-Save</h4>
                <div class="toggle-container">
                    <label class="toggle">
                        <input type="checkbox" id="auto-save-toggle">
                        <span class="toggle-slider"></span>
                    </label>
                    <span id="auto-save-label">Auto-save is off</span>
                </div>
                <p class="hint-text">When enabled, your workspace will
be automatically saved when you leave the page.</p>
            </div>
        </div>
        <div class="operation-result">
            <h3>Workspace Status</h3>
            <div id="workspace-info" class="workspace-info">
                <div class="info-item">
                    <span class="info-label">Last Saved:</span>
                    <span id="last-saved-time">Never</span>
                </div>
                <div class="info-item">
                    <span class="info-label">Components:</span>
                    <span id="component-count">0</span>
                </div>
                <div class="info-item">
                    <span class="info-label">Storage Used:</span>
                    <span id="storage-used">0 KB</span>
                </div>
            </div>
            <div class="mt-20">
                <h4>Recent Actions</h4>
                <div id="session-history" class="session-history">

```

```

        <div class="empty-history">No recent actions</div>
    </div>
</div>
<div class="mt-20">
    <button id="clear-storage-btn" class="button button-
secondary">Clear Stored Data</button>
</div>
</div>
</div>
`;

// Add the section to the main content
const main = document.querySelector('main');
if (main) {
    main.appendChild(section);

    // Add to navigation
    const navSection = document.querySelector('#main-nav');
    if (navSection) {
        const sessionNav = document.createElement('div');
        sessionNav.className = 'nav-section';
        sessionNav.innerHTML = `
            <h3>Session</h3>
            <button data-section="session-management">Workspace
Management</button>
        `;
        navSection.appendChild(sessionNav);
    }

    // Set up event listeners
    setupSessionManagementListeners();
    updateWorkspaceInfo();
}

}

// Call this function from setupPersistenceControls
createSessionManagementSection();

}

/**
 * Create floating persistence controls if they don't exist in main nav
 */
function createFloatingPersistenceControls() {
    // Check if floating controls already exist
    if (document.getElementById('floating-persistence-controls')) {
        return;
    }

    // Create container
    const controlsContainer = document.createElement('div');
    controlsContainer.id = 'floating-persistence-controls';
    controlsContainer.className = 'persistence-controls-container';
    controlsContainer.style.cssText = `

```

```

        position: fixed;
        bottom: 20px;
        right: 20px;
        background-color: white;
        border: 1px solid var(--secondary-color);
        border-radius: 5px;
        padding: 10px;
        box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
        z-index: 1000;
        display: flex;
        flex-direction: column;
        gap: 10px;
    `;

    // Add buttons
    const saveButton = document.createElement('button');
    saveButton.textContent = 'Save Workspace';
    saveButton.className = 'button';
    saveButton.addEventListener('click', saveAllStates);

    const loadButton = document.createElement('button');
    loadButton.textContent = 'Load Workspace';
    loadButton.className = 'button';
    loadButton.addEventListener('click', loadWorkspaceFromFile);

    const autoSaveToggle = document.createElement('button');
    autoSaveToggle.className = 'button button-secondary';

    const autoSaveEnabled = localStorage.getItem(CONFIG.autoSaveKey) ===
    'true';
    autoSaveToggle.textContent = `Auto Save: ${autoSaveEnabled ? 'On' :
    'Off'}`;

    autoSaveToggle.addEventListener('click', function() {
        const currentStatus = localStorage.getItem(CONFIG.autoSaveKey) ===
        'true';
        const newStatus = !currentStatus;

        localStorage.setItem(CONFIG.autoSaveKey, newStatus.toString());
        autoSaveToggle.textContent = `Auto Save: ${newStatus ? 'On' : 'Off'}`;

        showNotification(`Auto save ${newStatus ? 'enabled' : 'disabled'}`,
        'info');
    });

    // Add to container
    controlsContainer.appendChild(saveButton);
    controlsContainer.appendChild(loadButton);
    controlsContainer.appendChild(autoSaveToggle);

    // Add to document
    document.body.appendChild(controlsContainer);
    console.log("Created floating persistence controls");
}

```



```

/**
 * Save all component states
 */
async function saveAllStates() {
  try {
    console.log("Saving all states...");
    showNotification('Saving workspace...', 'info');

    // Collect state from all registered components
    const fullState = {
      version: CONFIG.version,
      timestamp: new Date().toISOString(),
      components: {}
    };

    // Save each component's state
    for (const [id, component] of stateRegistry.entries()) {
      if (!component.active) continue;

      try {
        const componentState = await component.save();
        if (componentState !== undefined) {
          fullState.components[id] = componentState;
        }
      } catch (error) {
        console.error(`Error saving state for ${id}:`, error);
      }
    }

    // Convert to JSON and encrypt
    const jsonData = JSON.stringify(fullState);
    const encryptedData = encryptData(jsonData);

    // Split into chunks for localStorage
    const chunks = chunkString(encryptedData, CONFIG.chunkSize);

    // Store metadata
    localStorage.setItem(CONFIG.metadataKey, JSON.stringify({
      version: CONFIG.version,
      timestamp: new Date().toISOString(),
      chunks: chunks.length,
      size: encryptedData.length
    }));

    // Store chunks
    chunks.forEach((chunk, index) => {
      localStorage.setItem(`${CONFIG.chunkPrefix}${index}`, chunk);
    });

    // Create download link for backup
    offerDownload(fullState);

    showNotification('Workspace saved successfully', 'success');
  }
}

```

```

        // Add to the end of the saveAllStates function, before the return
statement:
        addHistoryItem('Workspace saved');
        updateWorkspaceInfo();
    } catch (error) {
        console.error('Error saving workspace:', error);
        showNotification('Failed to save workspace: ' + error.message,
'error');
    }
}

/**
 * Load workspace from a file selected by the user
 */
function loadWorkspaceFromFile() {
    // Create file input for loading JSON
    const fileInput = document.createElement('input');
    fileInput.type = 'file';
    fileInput.accept = '.json';
    fileInput.style.display = 'none';
    document.body.appendChild(fileInput);

    fileInput.addEventListener('change', function(e) {
        const file = e.target.files[0];
        if (!file) return;

        const reader = new FileReader();
        reader.onload = function(e) {
            try {
                // Parse the file content
                const workspaceData = JSON.parse(e.target.result);

                // Basic validation
                if (!workspaceData.timestamp) {
                    throw new Error('Invalid workspace file format');
                }

                // Confirm before loading
                if (!confirm(`Load workspace from ${new
Date(workspaceData.timestamp).toLocaleString()}? This will replace your current
workspace.`)) {
                    return;
                }

                // Load the workspace
                loadFromObject(workspaceData);
            } catch (error) {
                console.error('Error loading workspace file:', error);
                showNotification('Failed to load workspace: ' + error.message,
'error');
            }
        };

        reader.onerror = function() {

```

```

        showNotification('Error reading file', 'error');
    };

    reader.readAsText(file);

    // Clean up
    document.body.removeChild(fileInput);
});

// Trigger file selection
fileInput.click();
}

/**
 * Load workspace from localStorage
 */
async function loadFromLocalStorage() {
    try {
        // Check if there's a saved state
        const metadataStr = localStorage.getItem(CONFIG.metadataKey);
        if (!metadataStr) {
            showNotification('No saved workspace found in browser storage',
'error');

            return;
        }

        showNotification('Loading workspace...', 'info');

        // Parse metadata
        const metadata = JSON.parse(metadataStr);

        // Retrieve and combine chunks
        let encryptedData = '';
        for (let i = 0; i < metadata.chunks; i++) {
            const chunk = localStorage.getItem(`${CONFIG.chunkPrefix}${i}`);
            if (!chunk) {
                throw new Error(`Missing session data chunk ${i}`);
            }
            encryptedData += chunk;
        }

        // Decrypt data
        const jsonData = decryptData(encryptedData);
        const fullState = JSON.parse(jsonData);

        // Load the workspace
        await loadFromObject(fullState);
    } catch (error) {
        console.error('Error loading from localStorage:', error);
        showNotification('Failed to load workspace: ' + error.message,
'error');
    }
}

```

```

/**
 * Load state from a workspace object
 */
async function loadFromObject(workspaceData) {
  try {
    showNotification('Loading workspace...', 'info');

    // Verify version compatibility
    if (workspaceData.version && workspaceData.version !== CONFIG.version)
    {
      console.warn(`Version mismatch: saved=${workspaceData.version},
current=${CONFIG.version}`);
      // Allow loading but warn
      showNotification('Loading workspace from a different version',
'warning');
    }

    // Handle both formats:
    // 1. New format: { components: { id: state } }
    // 2. Legacy format: { virtualFS, database, etc. }
    const hasComponents = workspaceData.components && typeof
workspaceData.components === 'object';

    if (hasComponents) {
      // Get components sorted by priority
      const sortedComponents = Array.from(stateRegistry.values())
        .sort((a, b) => b.priority - a.priority);

      // Load each component state
      for (const component of sortedComponents) {
        if (!component.active) continue;

        const id = component.id;
        if (workspaceData.components[id]) {
          try {
            await component.load(workspaceData.components[id]);
            console.log(`Loaded state for ${id}`);
          } catch (error) {
            console.error(`Error loading state for ${id}:`,
error);
          }
        }
      }
    }
    } else {
      // Legacy format - map directly to components
      // File System
      if (workspaceData.virtualFS) {
        try {
          window.virtualFS = workspaceData.virtualFS;
          if (typeof window.renderFileTree === 'function') {
            window.renderFileTree();
          }
        } catch (error) {
          console.error('Error loading file system:', error);
        }
      }
    }
  }
}

```

```

    }
  }

  // Database
  if (workspaceData.database && workspaceData.database.records) {
    try {
      if (window.database) {
        window.database.deleteAll();
        workspaceData.database.records.forEach(([id, record])
=> {
          window.database.records.set(id, record);
        });

        if (window.databaseDisplayUpdater) {
          window.databaseDisplayUpdater.updateDatabaseDisplay();
        }
      }
    } catch (error) {
      console.error('Error loading database:', error);
    }
  }

  // Memory
  if (workspaceData.memorySlots && window.memoryManager) {
    try {
      window.memoryManager.memorySlots =
workspaceData.memorySlots;
      if (typeof window.updateMemoryDisplay === 'function') {
        window.updateMemoryDisplay();
      }
    } catch (error) {
      console.error('Error loading memory state:', error);
    }
  }

  // OperatorLang
  if (workspaceData.programming && window.operatorLang) {
    try {
      const codeEditor = document.getElementById('code-editor');
      if (codeEditor && workspaceData.programming.code) {
        codeEditor.value = workspaceData.programming.code;
      }

      if (workspaceData.programming.variables) {
        window.operatorLang.variables = new
Map(workspaceData.programming.variables);
      }
    } catch (error) {
      console.error('Error loading programming state:', error);
    }
  }

  // Update workspace info display
  updateWorkspaceInfo();

```

```

    }

    showNotification('Workspace loaded successfully', 'success');
  } catch (error) {
    console.error('Error loading workspace:', error);
    showNotification('Failed to load workspace: ' + error.message,
'error');
  }
}

/**
 * Offer to download state as a backup file
 */
function offerDownload(state) {
  try {
    // Create a JSON file
    const blob = new Blob([JSON.stringify(state)], {type:
'application/json'});
    const url = URL.createObjectURL(blob);

    // Create a temporary link element
    const a = document.createElement('a');
    a.href = url;
    a.download = `operator_workspace_${new
Date().toISOString().replace(/:/g, '-')}.json`;

    // Ask user if they want to download
    if (confirm('Would you like to download a backup of your workspace?'))
{
      a.click();
    }

    // Clean up
    setTimeout(() => URL.revokeObjectURL(url), 5000);
  } catch (error) {
    console.error('Error creating download:', error);
  }
}

// Modify checkAutoRecovery to update UI:
function checkAutoRecovery() {
  // Check for a saved state
  const metadataStr = localStorage.getItem(CONFIG.metadataKey);
  if (!metadataStr) return;

  try {
    const metadata = JSON.parse(metadataStr);
    const savedDate = new Date(metadata.timestamp);
    const now = new Date();

    // Update the workspace info display
    updateWorkspaceInfo();

    // If the saved state is recent (within the last day)

```

```

    if ((now - savedDate) < 24 * 60 * 60 * 1000) {
      // Show recovery banner
      const banner = document.createElement('div');
      banner.className = 'recovery-banner';
      banner.style.cssText = `
        position: fixed;
        top: 0;
        left: 0;
        right: 0;
        background-color: var(--primary-color);
        color: white;
        padding: 10px 20px;
        text-align: center;
        z-index: 2000;
        display: flex;
        justify-content: center;
        align-items: center;
        gap: 20px;
      `;

      banner.innerHTML = `
        <div>Saved workspace found from ${savedDate.toLocaleString()}.
        Would you like to restore it?</div>
        <div>
          <button id="restore-workspace"
class="button">Restore</button>
          <button id="dismiss-banner" class="button button-
secondary">Dismiss</button>
        </div>
      `;

      document.body.appendChild(banner);

      // Add event listeners
      document.getElementById('restore-
workspace').addEventListener('click', () => {
        loadFromLocalStorage();
        document.body.removeChild(banner);
        addHistoryItem('Workspace restored from auto-recovery');
      });

      document.getElementById('dismiss-
banner').addEventListener('click', () => {
        document.body.removeChild(banner);
      });
    }
  } catch (error) {
    console.error('Error checking for recovery:', error);
  }
}

/**
 * Setup window unload handler
 */

```

```

function setupUnloadHandler() {
    window.addEventListener('beforeunload', function(e) {
        // Check if auto-save is enabled
        const autoSave = localStorage.getItem(CONFIG.autoSaveKey) === 'true';
        if (autoSave) {
            // Attempt to save state (async, might not complete)
            saveAllStates();
            return;
        }

        // Otherwise, show confirmation dialog
        const confirmationMessage = 'You have unsaved changes. Would you like
to save your workspace before leaving?';
        e.returnValue = confirmationMessage;
        return confirmationMessage;
    });
}

/**
 * Show a notification to the user
 */
function showNotification(message, type = 'info') {
    // Check if a notification container exists
    let container = document.getElementById('persistence-notifications');
    if (!container) {
        // Create a container
        container = document.createElement('div');
        container.id = 'persistence-notifications';
        container.style.cssText = `
            position: fixed;
            bottom: 20px;
            left: 20px;
            z-index: 2000;
            display: flex;
            flex-direction: column;
            gap: 10px;
        `;
        document.body.appendChild(container);
    }

    // Create notification
    const notification = document.createElement('div');
    notification.className = `persistence-notification ${type}`;
    notification.style.cssText = `
        background-color: white;
        border-left: 4px solid ${getTypeColor(type)};
        padding: 15px 20px;
        box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
        border-radius: 4px;
        max-width: 300px;
        animation: fadeIn 0.3s ease-out;
        position: relative;
    `;

```



```

notification.innerHTML = `
    <div style="margin-right: 20px;">${message}</div>
    <button class="close-notification" style="
        position: absolute;
        top: 5px;
        right: 5px;
        border: none;
        background: none;
        cursor: pointer;
        font-size: 16px;
        color: #999;
    ">&times;</button>
`;

// Add to container
container.appendChild(notification);

// Add close button event
notification.querySelector('.close-
notification').addEventListener('click', () => {
    container.removeChild(notification);
});

// Auto-hide after a delay
setTimeout(() => {
    notification.style.opacity = '0';
    notification.style.transition = 'opacity 0.3s';
    setTimeout(() => {
        if (container.contains(notification)) {
            container.removeChild(notification);
        }
    }, 300);
}, 5000);
}

/**
 * Get color for notification type
 */
function getTypeColor(type) {
    switch (type) {
        case 'success': return '#4caf50';
        case 'error': return '#f44336';
        case 'warning': return '#ff9800';
        case 'info':
        default: return '#2196f3';
    }
}

/**
 * Encryption functions for secure storage
 */
function encryptData(data) {
    try {
        // Simple XOR encryption for demonstration

```

```

        // In production, use a proper encryption library
        const key = CONFIG.encryptionKey;
        let result = '';

        for (let i = 0; i < data.length; i++) {
            const charCode = data.charCodeAt(i) ^ key.charCodeAt(i %
key.length);
            result += String.fromCharCode(charCode);
        }

        // Convert to base64 for safer storage
        return btoa(result);
    } catch (error) {
        console.error('Error encrypting data:', error);
        return btoa(data); // Fallback to basic encoding
    }
}

function decryptData(encrypted) {
    try {
        // Decode from base64
        const encoded = atob(encrypted);
        const key = CONFIG.encryptionKey;
        let result = '';

        for (let i = 0; i < encoded.length; i++) {
            const charCode = encoded.charCodeAt(i) ^ key.charCodeAt(i %
key.length);
            result += String.fromCharCode(charCode);
        }

        return result;
    } catch (error) {
        console.error('Error decrypting data:', error);
        return atob(encrypted); // Fallback to basic decoding
    }
}

/**
 * Split a string into chunks
 */
function chunkString(str, size) {
    const chunks = [];
    for (let i = 0; i < str.length; i += size) {
        chunks.push(str.substring(i, i + size));
    }
    return chunks;
}

/**
 * Component State Handlers
 * These functions handle saving and loading state for specific components
 */

```

```
// ----- Data Structures -----
```

```
async function saveArrayState() {
  const container = document.getElementById('array-operations');
  if (!container) return null;

  const inputElement = container.querySelector('#array-input');
  const resultContainer = container.querySelector('#array-result');

  return {
    input: inputElement ? inputElement.value : '',
    // Also capture the current visualization state if possible
    displayHTML: resultContainer ? resultContainer.innerHTML : ''
  };
}

async function loadArrayState(state) {
  const container = document.getElementById('array-operations');
  if (!container || !state) return;

  const inputElement = container.querySelector('#array-input');
  if (inputElement && state.input) {
    inputElement.value = state.input;
  }

  // Attempt to restore visualization
  const resultContainer = container.querySelector('#array-result');
  if (resultContainer && state.displayHTML) {
    resultContainer.innerHTML = state.displayHTML;
  }
}

async function saveObjectState() {
  const container = document.getElementById('object-operations');
  if (!container) return null;

  // Try to access the current object state
  let currentObject = {};
  try {
    currentObject = window.currentObject || {};
  } catch (e) {
    console.warn("Couldn't access current object state");
  }

  return {
    objectState: currentObject,
    displayHTML: container.querySelector('#object-result')?.innerHTML
  };
}

async function loadObjectState(state) {
  const container = document.getElementById('object-operations');
  if (!container || !state) return;
}
```

```

// Try to restore the object state
try {
    window.currentObject = state.objectState || {};
} catch (e) {
    console.warn("Couldn't restore object state");
}

// Restore display
if (state.displayHTML) {
    const resultContainer = container.querySelector('#object-result');
    if (resultContainer) {
        resultContainer.innerHTML = state.displayHTML;
    }
}
}

async function saveSetState() {
    const container = document.getElementById('set-operations');
    if (!container) return null;

    const setAInput = container.querySelector('#set-a-input');
    const setBInput = container.querySelector('#set-b-input');

    return {
        setA: setAInput ? setAInput.value : '',
        setB: setBInput ? setBInput.value : '',
        displayHTML: container.querySelector('#set-result')?.innerHTML
    };
}

async function loadSetState(state) {
    const container = document.getElementById('set-operations');
    if (!container || !state) return;

    if (state.setA) {
        const input = container.querySelector('#set-a-input');
        if (input) input.value = state.setA;
    }

    if (state.setB) {
        const input = container.querySelector('#set-b-input');
        if (input) input.value = state.setB;
    }

    if (state.displayHTML) {
        const result = container.querySelector('#set-result');
        if (result) result.innerHTML = state.displayHTML;
    }
}

async function saveQuadtreeState() {
    const container = document.getElementById('tree-operations');
    if (!container) return null;

```

```

// Get quadtree parameters
const maxDepth = document.getElementById('max-depth')?.value || 3;
const quadtreeSize = document.getElementById('quadtree-size')?.value ||
400;

// Get the HTML structure of all cells
const quadtreeContainer = document.getElementById('quadtree-container');
const cellsHTML = quadtreeContainer?.innerHTML || '';

// Try to find customized cells
const customizedCells = [];
quadtreeContainer?.querySelectorAll('.quadtree-cell').forEach(cell => {
  if (cell.classList.contains('customized') ||
    cell.style.backgroundColor ||
    cell.querySelector('.quadtree-cell-custom-text') ||
    cell.querySelector('.quadtree-cell-image')) {

    // Extract the customization details
    const customization = {
      x: parseFloat(cell.style.left) || 0,
      y: parseFloat(cell.style.top) || 0,
      width: parseFloat(cell.style.width) || 0,
      height: parseFloat(cell.style.height) || 0,
      backgroundColor: cell.style.backgroundColor,
      opacity: cell.style.opacity,
      customized: cell.classList.contains('customized')
    };

    // Extract text if present
    const textElement = cell.querySelector('.quadtree-cell-custom-
text');
    if (textElement) {
      customization.text = {
        content: textElement.textContent,
        color: textElement.style.color,
        fontSize: textElement.style.fontSize
      };
    }

    // Extract image if present
    const imageElement = cell.querySelector('.quadtree-cell-image');
    if (imageElement) {
      customization.image = {
        src: imageElement.src
      };
    }

    customizedCells.push(customization);
  }
});

return {
  maxDepth,
  quadtreeSize,

```

```

        cellsHTML,
        customizedCells
    };
}

async function loadQuadtreeState(state) {
    const container = document.getElementById('tree-operations');
    if (!container || !state) return;

    // Set quadtree parameters
    if (state.maxDepth) {
        const input = document.getElementById('max-depth');
        if (input) input.value = state.maxDepth;
    }

    if (state.quadtreeSize) {
        const input = document.getElementById('quadtree-size');
        if (input) input.value = state.quadtreeSize;
    }

    // Recreate the quadtree
    const createBtn = document.getElementById('create-quadtree');
    if (createBtn) {
        createBtn.click();

        // Wait for the quadtree to be created
        setTimeout(() => {
            // Try to restore customized cells
            const quadtreeContainer = document.getElementById('quadtree-
container');
            if (!quadtreeContainer || !state.customizedCells) return;

            state.customizedCells.forEach(customization => {
                // Find a cell at the same position
                const cells = quadtreeContainer.querySelectorAll('.quadtree-
cell');

                let matchingCell = null;

                for (const cell of cells) {
                    const x = parseFloat(cell.style.left) || 0;
                    const y = parseFloat(cell.style.top) || 0;
                    const width = parseFloat(cell.style.width) || 0;
                    const height = parseFloat(cell.style.height) || 0;

                    if (Math.abs(x - customization.x) < 1 &&
                        Math.abs(y - customization.y) < 1 &&
                        Math.abs(width - customization.width) < 1 &&
                        Math.abs(height - customization.height) < 1) {
                        matchingCell = cell;
                        break;
                    }
                }

                if (matchingCell) {

```

```

        // Apply customizations
        if (customization.backgroundColor) {
            matchingCell.style.backgroundColor =
customization.backgroundColor;
        }

        if (customization.opacity) {
            matchingCell.style.opacity = customization.opacity;
        }

        if (customization.customized) {
            matchingCell.classList.add('customized');
        }

        // Add text if present
        if (customization.text) {
            const textElement = document.createElement('div');
            textElement.className = 'quadtree-cell-custom-text';
            textElement.textContent = customization.text.content;
            textElement.style.color = customization.text.color;
            textElement.style.fontSize =
customization.text.fontSize;
            matchingCell.appendChild(textElement);
        }

        // Add image if present
        if (customization.image && customization.image.src) {
            const imageElement = document.createElement('img');
            imageElement.className = 'quadtree-cell-image';
            imageElement.src = customization.image.src;
            matchingCell.appendChild(imageElement);
        }
    }
    });
}, 500);
}
}

// ----- CRUD Database -----

async function saveDatabaseState() {
    if (!window.database) return null;

    try {
        // Capture database records
        const records =
Array.from(window.database.records.entries()).map(([id, record]) => {
            // Clone the record to avoid circular references
            return {
                id: id,
                name: record.name,
                data: record.data,
                createdAt: record.createdAt ? record.createdAt.toISOString() :
null,

```

```

        updatedAt: record.updatedAt ? record.updatedAt.toISOString() :
null
        });
    });

    return { records };
} catch (e) {
    console.error('Error saving database state:', e);
    return null;
}
}

async function loadDatabaseState(state) {
    if (!window.database || !state || !state.records) return;

    try {
        // Clear database
        window.database.deleteAll();

        // Restore records
        state.records.forEach(record => {
            window.database.create({
                id: record.id,
                name: record.name,
                data: record.data,
                createdAt: record.createdAt ? new Date(record.createdAt) : new
Date(),
                updatedAt: record.updatedAt ? new Date(record.updatedAt) :
null
            });
        });

        // Update display if possible
        if (window.databaseDisplayUpdater &&
window.databaseDisplayUpdater.updateDatabaseDisplay) {
            window.databaseDisplayUpdater.updateDatabaseDisplay();
        }
    } catch (e) {
        console.error('Error loading database state:', e);
    }
}

// ----- System Operations -----

async function saveFileSystemState() {
    try {
        // Get the virtual file system or create an empty one
        const virtualFS = window.virtualFS || {
            root: { type: 'directory', name: 'root', children: {} }
        };

        return virtualFS;
    } catch (e) {
        console.error('Error saving file system state:', e);
    }
}

```



```

        return null;
    }
}

async function loadFileSystemState(state) {
    if (!state) return;

    try {
        // Restore virtual file system
        window.virtualFS = state;

        // Update file tree display if possible
        if (typeof window.renderFileTree === 'function') {
            window.renderFileTree();
        }
    } catch (e) {
        console.error('Error loading file system state:', e);
    }
}

async function saveMemoryState() {
    try {
        if (!window.memoryManager) return null;

        return {
            memorySlots: window.memoryManager.memorySlots
        };
    } catch (e) {
        console.error('Error saving memory state:', e);
        return null;
    }
}

async function loadMemoryState(state) {
    if (!window.memoryManager || !state || !state.memorySlots) return;

    try {
        // Restore memory slots
        window.memoryManager.memorySlots = state.memorySlots;

        // Update memory display if possible
        if (typeof window.updateMemoryDisplay === 'function') {
            window.updateMemoryDisplay();
        }
    } catch (e) {
        console.error('Error loading memory state:', e);
    }
}

async function saveProcessState() {
    try {
        // Process manager is mostly runtime state, not much to save
        return {
            // Could save process templates/configurations if needed

```

```

    };
  } catch (e) {
    console.error('Error saving process state:', e);
    return null;
  }
}

async function loadProcessState(state) {
  // Mainly for future expansion
  if (!state) return;

  try {
    // Restore any saved process configurations
  } catch (e) {
    console.error('Error loading process state:', e);
  }
}

// ----- Algorithm States -----

async function saveRecursionState() {
  const container = document.getElementById('recursion-operations');
  if (!container) return null;

  return {
    n: document.getElementById('recursion-n')?.value,
    result: document.getElementById('recursion-result')?.innerHTML
  };
}

async function loadRecursionState(state) {
  if (!state) return;

  try {
    if (state.n) {
      const input = document.getElementById('recursion-n');
      if (input) input.value = state.n;
    }

    if (state.result) {
      const result = document.getElementById('recursion-result');
      if (result) result.innerHTML = state.result;
    }
  } catch (e) {
    console.error('Error loading recursion state:', e);
  }
}

async function saveIterationState() {
  const container = document.getElementById('iteration-operations');
  if (!container) return null;

  return {
    n: document.getElementById('iteration-n')?.value,

```

```

        charset: document.getElementById('charset-input')?.value,
        result: document.getElementById('iteration-result')?.innerHTML
    };
}

async function loadIterationState(state) {
    if (!state) return;

    try {
        if (state.n) {
            const input = document.getElementById('iteration-n');
            if (input) input.value = state.n;
        }

        if (state.charset) {
            const input = document.getElementById('charset-input');
            if (input) input.value = state.charset;
        }

        if (state.result) {
            const result = document.getElementById('iteration-result');
            if (result) result.innerHTML = state.result;
        }
    } catch (e) {
        console.error('Error loading iteration state:', e);
    }
}

async function saveSearchState() {
    const container = document.getElementById('search-operations');
    if (!container) return null;

    return {
        data: document.getElementById('search-data')?.value,
        query: document.getElementById('search-query')?.value,
        result: document.getElementById('search-result')?.innerHTML
    };
}

async function loadSearchState(state) {
    if (!state) return;

    try {
        if (state.data) {
            const input = document.getElementById('search-data');
            if (input) input.value = state.data;
        }

        if (state.query) {
            const input = document.getElementById('search-query');
            if (input) input.value = state.query;
        }

        if (state.result) {

```

```

        const result = document.getElementById('search-result');
        if (result) result.innerHTML = state.result;
    }
} catch (e) {
    console.error('Error loading search state:', e);
}
}

async function saveBinaryOperationsState() {
    const container = document.getElementById('binary-operations');
    if (!container) return null;

    return {
        valueA: document.getElementById('binary-input-a')?.value,
        valueB: document.getElementById('binary-input-b')?.value,
        result: document.getElementById('binary-result')?.innerHTML
    };
}

async function loadBinaryOperationsState(state) {
    if (!state) return;

    try {
        if (state.valueA) {
            const input = document.getElementById('binary-input-a');
            if (input) input.value = state.valueA;
        }

        if (state.valueB) {
            const input = document.getElementById('binary-input-b');
            if (input) input.value = state.valueB;
        }

        if (state.result) {
            const result = document.getElementById('binary-result');
            if (result) result.innerHTML = state.result;
        }
    } catch (e) {
        console.error('Error loading binary operations state:', e);
    }
}

// ----- Advanced Components -----

async function saveSerializationState() {
    const container = document.getElementById('serialization-operations');
    if (!container) return null;

    return {
        input: document.getElementById('serialize-input')?.value,
        format: document.getElementById('serialization-format')?.value,
        result: document.getElementById('serialization-result')?.innerHTML
    };
}

```

```

async function loadSerializationState(state) {
  if (!state) return;

  try {
    if (state.input) {
      const input = document.getElementById('serialize-input');
      if (input) input.value = state.input;
    }

    if (state.format) {
      const select = document.getElementById('serialization-format');
      if (select) select.value = state.format;
    }

    if (state.result) {
      const result = document.getElementById('serialization-result');
      if (result) result.innerHTML = state.result;
    }
  } catch (e) {
    console.error('Error loading serialization state:', e);
  }
}

async function saveCompressionState() {
  const container = document.getElementById('compression-operations');
  if (!container) return null;

  return {
    input: document.getElementById('compression-input')?.value,
    method: document.getElementById('compression-method')?.value,
    result: document.getElementById('compression-result')?.innerHTML,
    stats: document.getElementById('compression-stats')?.innerHTML
  };
}

async function loadCompressionState(state) {
  if (!state) return;

  try {
    if (state.input) {
      const input = document.getElementById('compression-input');
      if (input) input.value = state.input;
    }

    if (state.method) {
      const select = document.getElementById('compression-method');
      if (select) select.value = state.method;
    }

    if (state.result) {
      const result = document.getElementById('compression-result');
      if (result) result.innerHTML = state.result;
    }
  }
}

```

```

        if (state.stats) {
            const stats = document.getElementById('compression-stats');
            if (stats) stats.innerHTML = state.stats;
        }
    } catch (e) {
        console.error('Error loading compression state:', e);
    }
}

async function saveEncodingState() {
    const container = document.getElementById('encoding-operations');
    if (!container) return null;

    return {
        input: document.getElementById('encoding-input')?.value,
        method: document.getElementById('encoding-method')?.value,
        result: document.getElementById('encoding-result')?.innerHTML
    };
}

async function loadEncodingState(state) {
    if (!state) return;

    try {
        if (state.input) {
            const input = document.getElementById('encoding-input');
            if (input) input.value = state.input;
        }

        if (state.method) {
            const select = document.getElementById('encoding-method');
            if (select) select.value = state.method;
        }

        if (state.result) {
            const result = document.getElementById('encoding-result');
            if (result) result.innerHTML = state.result;
        }
    } catch (e) {
        console.error('Error loading encoding state:', e);
    }
}

async function saveTransactionState() {
    const container = document.getElementById('transactions-operations');
    if (!container) return null;

    try {
        // Capture canvas state as image
        const canvas = document.getElementById('drawing-canvas');
        let canvasDataURL = null;

        if (canvas) {

```

```

        canvasDataURL = canvas.toDataURL('image/png');
    }

    return {
        canvasState: canvasDataURL,
        history: document.getElementById('transaction-history')?.innerHTML
    };
} catch (e) {
    console.error('Error saving transaction state:', e);
    return null;
}
}

async function loadTransactionState(state) {
    if (!state) return;

    try {
        // Restore canvas state if possible
        if (state.canvasState) {
            const canvas = document.getElementById('drawing-canvas');
            if (canvas) {
                const ctx = canvas.getContext('2d');
                if (ctx) {
                    const img = new Image();
                    img.onload = function() {
                        ctx.clearRect(0, 0, canvas.width, canvas.height);
                        ctx.drawImage(img, 0, 0);
                    };
                    img.src = state.canvasState;
                }
            }
        }

        // Restore history display
        if (state.history) {
            const history = document.getElementById('transaction-history');
            if (history) history.innerHTML = state.history;
        }
    } catch (e) {
        console.error('Error loading transaction state:', e);
    }
}

// ----- Programming Language -----

async function saveOperatorLangState() {
    const container = document.getElementById('programming-section');
    if (!container) return null;

    try {
        // Get editor content
        const editorContent = document.getElementById('code-editor')?.value;

        // Get console output

```

```

const consoleOutput = Array.from(
  document.getElementById('language-console')?.childNodes || []
).map(node => node.textContent).join('\n');

// Get canvas state
const canvas = document.getElementById('language-canvas');
let canvasDataURL = null;

if (canvas) {
  canvasDataURL = canvas.toDataURL('image/png');
}

// Get variables from operatorLang if possible
let variables = [];
if (window.operatorLang && window.operatorLang.variables) {
  variables = Array.from(window.operatorLang.variables.entries());
}

return {
  code: editorContent,
  console: consoleOutput,
  canvasState: canvasDataURL,
  variables: variables
};
} catch (e) {
  console.error('Error saving OperatorLang state:', e);
  return null;
}
}

async function loadOperatorLangState(state) {
  if (!state) return;

  try {
    // Restore editor content
    if (state.code) {
      const editor = document.getElementById('code-editor');
      if (editor) editor.value = state.code;
    }

    // Restore console output
    if (state.console) {
      const consoleEl = document.getElementById('language-console');
      if (consoleEl) {
        consoleEl.innerHTML = '';
        state.console.split('\n').forEach(line => {
          const lineElement = document.createElement('div');
          lineElement.className = 'console-line';
          lineElement.textContent = line;
          consoleEl.appendChild(lineElement);
        });
      }
    }
  }
}

```



```

// Restore canvas state
if (state.canvasState) {
  const canvas = document.getElementById('language-canvas');
  if (canvas) {
    const ctx = canvas.getContext('2d');
    if (ctx) {
      const img = new Image();
      img.onload = function() {
        ctx.clearRect(0, 0, canvas.width, canvas.height);
        ctx.drawImage(img, 0, 0);
      };
      img.src = state.canvasState;
    }
  }
}

// Restore variables if possible
if (state.variables && window.operatorLang) {
  try {
    window.operatorLang.variables = new Map(state.variables);
  } catch (e) {
    console.warn('Could not restore OperatorLang variables:', e);
  }
} catch (e) {
  console.error('Error loading OperatorLang state:', e);
}

// Add this function to operator-persistence.js
function setupSessionManagementListeners() {
  // Save workspace button
  const saveWorkspaceBtn = document.getElementById('save-workspace-btn');
  if (saveWorkspaceBtn) {
    saveWorkspaceBtn.addEventListener('click', saveAllStates);
  }

  // Load workspace button
  const loadWorkspaceBtn = document.getElementById('load-workspace-btn');
  if (loadWorkspaceBtn) {
    loadWorkspaceBtn.addEventListener('click', loadFromLocalStorage);
  }

  // Export workspace button
  const exportWorkspaceBtn = document.getElementById('export-workspace-
btn');
  if (exportWorkspaceBtn) {
    exportWorkspaceBtn.addEventListener('click', exportWorkspace);
  }

  // Import workspace button
  const importWorkspaceBtn = document.getElementById('import-workspace-
btn');
  if (importWorkspaceBtn) {

```

```

importWorkspaceBtn.addEventListener('click', importWorkspace);
}

// Auto-save toggle
const autoSaveToggle = document.getElementById('auto-save-toggle');
const autoSaveLabel = document.getElementById('auto-save-label');
if (autoSaveToggle && autoSaveLabel) {
  // Set initial state
  const autoSaveEnabled = localStorage.getItem(CONFIG.autoSaveKey) ===
'true';
  autoSaveToggle.checked = autoSaveEnabled;
  autoSaveLabel.textContent = `Auto-save is ${autoSaveEnabled ? 'on' :
'off'}`;

  // Add change listener
  autoSaveToggle.addEventListener('change', function() {
    const newState = this.checked;
    localStorage.setItem(CONFIG.autoSaveKey, newState.toString());
    autoSaveLabel.textContent = `Auto-save is ${newState ? 'on' :
'off'}`;
    showNotification(`Auto-save ${newState ? 'enabled' : 'disabled'}`,
'info');
  });
}

// Clear storage button
const clearStorageBtn = document.getElementById('clear-storage-btn');
if (clearStorageBtn) {
  clearStorageBtn.addEventListener('click', function() {
    if (confirm('Are you sure you want to clear all saved workspace
data? This cannot be undone.')) {
      clearAllStoredData();
      updateWorkspaceInfo();
      showNotification('All stored workspace data has been cleared',
'info');
    }
  });
}

// Add this function to operator-persistence.js
async function collectAllStates() {
  const fullState = {
    version: CONFIG.version,
    timestamp: new Date().toISOString(),
    components: {}
  };
  // Save each component's state
  for (const [id, component] of stateRegistry.entries()) {
    if (!component.active) continue;

    try {
      const componentState = await component.save();

```

```

        if (componentState !== undefined) {
            fullState.components[id] = componentState;
        }
    } catch (error) {
        console.error(`Error saving state for ${id}:`, error);
    }
}

return fullState;
}

// Add these functions to operator-persistence.js
function exportWorkspace() {
    safeExecute(async () => {
        // Collect all state
        const fullState = await collectAllStates();

        // Create a JSON file
        const blob = new Blob([JSON.stringify(fullState, null, 2)], {type:
'application/json'});
        const url = URL.createObjectURL(blob);

        // Create download link
        const a = document.createElement('a');
        a.href = url;
        a.download = `operator_workspace_${new
Date().toISOString().replace(/:/g, '-')}.json`;
        a.click();

        // Clean up
        setTimeout(() => URL.revokeObjectURL(url), 5000);

        // Add to history
        addHistoryItem('Workspace exported to file');

        showNotification('Workspace exported successfully', 'success');
        return 'Workspace exported successfully';
    }, error => {
        showNotification('Failed to export workspace: ' + error.message,
'error');
    });
}

function importWorkspace() {
    // Create file input
    const fileInput = document.createElement('input');
    fileInput.type = 'file';
    fileInput.accept = '.json';
    fileInput.style.display = 'none';
    document.body.appendChild(fileInput);

    fileInput.addEventListener('change', function(e) {
        const file = e.target.files[0];
        if (!file) return;
    });
}

```

```

const reader = new FileReader();
reader.onload = function(e) {
    safeExecute(async () => {
        // Parse the file
        const data = JSON.parse(e.target.result);

        // Validate
        if (!data.timestamp) {
            throw new Error('Invalid workspace file format');
        }

        // Confirm
        if (!confirm(`Load workspace from ${new
Date(data.timestamp).toLocaleString())? This will replace your current
workspace.`)) {
            return;
        }

        // Load the data
        await loadFromObject(data);

        // Add to history
        addHistoryItem('Workspace imported from file');

        showNotification('Workspace imported successfully',
'success');

        updateWorkspaceInfo();

        return 'Workspace imported successfully';
    }, error => {
        showNotification('Failed to import workspace: ' +
error.message, 'error');
    });
};

reader.readAsText(file);

// Clean up
document.body.removeChild(fileInput);
});

fileInput.click();
}

function clearAllStoredData() {
    // Clear localStorage
    const metadataKey = localStorage.getItem(CONFIG.metadataKey);
    if (metadataKey) {
        try {
            const metadata = JSON.parse(metadataKey);
            // Remove chunks
            for (let i = 0; i < metadata.chunks; i++) {
                localStorage.removeItem(`${CONFIG.chunkPrefix}${i}`);
            }
        } catch (e) {
            console.error('Error clearing stored data:', e);
        }
    }
}

```

```

    }
  } catch (e) {
    console.error('Error parsing metadata:', e);
  }

  // Remove metadata
  localStorage.removeItem(CONFIG.metadataKey);
}

// Add to history
addHistoryItem('Cleared all stored workspace data');
}

function addHistoryItem(action) {
  const historyContainer = document.getElementById('session-history');
  if (!historyContainer) return;

  // Remove empty history message if present
  const emptyHistory = historyContainer.querySelector('.empty-history');
  if (emptyHistory) {
    historyContainer.removeChild(emptyHistory);
  }

  // Create history item
  const historyItem = document.createElement('div');
  historyItem.className = 'history-item';

  const now = new Date();
  historyItem.innerHTML = `
    <div class="history-time">${now.toLocaleTimeString()}</div>
    <div class="history-action">${action}</div>
  `;

  // Add to container (at the top)
  if (historyContainer.firstChild) {
    historyContainer.insertBefore(historyItem,
historyContainer.firstChild);
  } else {
    historyContainer.appendChild(historyItem);
  }

  // Limit history items
  const maxItems = 10;
  const items = historyContainer.querySelectorAll('.history-item');
  if (items.length > maxItems) {
    for (let i = maxItems; i < items.length; i++) {
      historyContainer.removeChild(items[i]);
    }
  }
}

function updateWorkspaceInfo() {
  const lastSavedTime = document.getElementById('last-saved-time');
  const componentCount = document.getElementById('component-count');

```

```

const storageUsed = document.getElementById('storage-used');

if (!lastSavedTime || !componentCount || !storageUsed) return;

// Get last saved time
const metadataStr = localStorage.getItem(CONFIG.metadataKey);
if (metadataStr) {
    try {
        const metadata = JSON.parse(metadataStr);
        lastSavedTime.textContent = new
Date(metadata.timestamp).toLocaleString();
        componentCount.textContent = stateRegistry.size.toString();

        // Calculate storage used
        let size = metadata.size || 0;
        let sizeStr;

        if (size < 1024) {
            sizeStr = `${size} bytes`;
        } else if (size < 1024 * 1024) {
            sizeStr = `${(size / 1024).toFixed(2)} KB`;
        } else {
            sizeStr = `${(size / (1024 * 1024)).toFixed(2)} MB`;
        }

        storageUsed.textContent = sizeStr;
    } catch (e) {
        console.error('Error parsing metadata:', e);
        lastSavedTime.textContent = 'Error reading data';
    }
} else {
    lastSavedTime.textContent = 'Never';
    componentCount.textContent = stateRegistry.size.toString();
    storageUsed.textContent = '0 bytes';
}
}

// Add this utility function
function safeExecute(func, errorCallback) {
    try {
        return func();
    } catch (error) {
        console.error('Operation error:', error);
        if (errorCallback) {
            errorCallback(error);
        }
        return null;
    }
}

// Add this to initPersistenceSystem function
function addSessionManagementStyles() {
    const styleElement = document.createElement('style');
    styleElement.textContent = `

```

```
/* Session Management Styles */
.workspace-info {
    background-color: var(--background-color);
    border: 1px solid var(--secondary-color);
    border-radius: 4px;
    padding: 15px;
}

.info-item {
    display: flex;
    justify-content: space-between;
    margin-bottom: 10px;
    padding-bottom: 10px;
    border-bottom: 1px solid rgba(0,0,0,0.05);
}

.info-item:last-child {
    margin-bottom: 0;
    padding-bottom: 0;
    border-bottom: none;
}

.info-label {
    font-weight: bold;
    color: var(--primary-color);
}

.session-history {
    max-height: 200px;
    overflow-y: auto;
    background-color: var(--background-color);
    border: 1px solid var(--secondary-color);
    border-radius: 4px;
    padding: 10px;
}

.history-item {
    padding: 8px;
    border-bottom: 1px solid rgba(0,0,0,0.05);
}

.history-item:last-child {
    border-bottom: none;
}

.history-time {
    font-size: 0.8em;
    color: var(--secondary-color);
}

.history-action {
    font-weight: bold;
}
```

```
.empty-history {
  text-align: center;
  color: #888;
  padding: 20px 0;
}

.toggle-container {
  display: flex;
  align-items: center;
  gap: 10px;
  margin-bottom: 10px;
}

.toggle {
  position: relative;
  display: inline-block;
  width: 50px;
  height: 24px;
}

.toggle input {
  opacity: 0;
  width: 0;
  height: 0;
}

.toggle-slider {
  position: absolute;
  cursor: pointer;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background-color: #ccc;
  transition: .4s;
  border-radius: 24px;
}

.toggle-slider:before {
  position: absolute;
  content: "";
  height: 16px;
  width: 16px;
  left: 4px;
  bottom: 4px;
  background-color: white;
  transition: .4s;
  border-radius: 50%;
}

input:checked + .toggle-slider {
  background-color: var(--primary-color);
}
```



```

        input:checked + .toggle-slider:before {
            transform: translateX(26px);
        }

        .hint-text {
            font-size: 0.8em;
            color: #666;
            margin-top: 5px;
        }

        .mt-20 {
            margin-top: 20px;
        }
    `;
    document.head.appendChild(styleElement);
}

// Call this function
addSessionManagementStyles();

// Update the export object at the bottom of operator-persistence.js
window.OperatorPersistence = {
    saveAllStates,
    loadAllStates: loadFromLocalStorage,
    loadFromFile: loadWorkspaceFromFile,
    loadFromObject,
    exportWorkspace,
    importWorkspace,
    registerComponent,
    showNotification,
    updateWorkspaceInfo,
    addHistoryItem
};

})();

```

[10] operator.css

- **Bytes:** 14260
- **Type:** text

```

/* Base Styles and CSS Variables */
:root {
    --primary-color: #4B5320; /* Army Green */
    --secondary-color: #BDB76B; /* Dark Khaki */
    --background-color: #F5F5DC; /* Beige */
    --text-color: #333333; /* Dark Gray */
    --accent-color: #8B4513; /* Saddle Brown */
    --success-color: #4CAF50; /* Green */

```

```

--error-color: #F44336;          /* Red */
--warning-color: #FFC107;        /* Amber */
--info-color: #2196F3;          /* Blue */
--font-main: "Fira Code", "Consolas", monospace;
--border-width: 2px;
--shadow-standard: 0 2px 5px rgba(0, 0, 0, 0.2);
--transition-standard: all 0.3s ease;
}

/* Global Reset */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

/* Body Styling */
body {
  font-family: var(--font-main);
  background-color: var(--background-color);
  color: var(--text-color);
  line-height: 1.6;
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  overflow-x: hidden;
}

/* Header Styling */
header {
  background-color: var(--primary-color);
  color: white;
  padding: 20px;
  text-align: center;
  box-shadow: var(--shadow-standard);
}

header h1 {
  font-size: 2.5rem;
  margin-bottom: 10px;
  text-transform: uppercase;
  letter-spacing: 1px;
}

header p {
  font-size: 1.1rem;
  opacity: 0.9;
}

/* Navigation and Main Layout */
nav#main-nav {
  background-color: var(--secondary-color);
  padding: 15px;
  display: flex;

```

```

    flex-wrap: wrap;
    justify-content: space-around;
    position: sticky;
    top: 0;
    z-index: 100;
    box-shadow: var(--shadow-standard);
}

.nav-section {
    padding: 8px;
    margin: 5px;
    min-width: 200px;
}

.nav-section h3 {
    color: var(--text-color);
    margin-bottom: 10px;
    font-size: 1.1rem;
    border-bottom: 1px solid var(--text-color);
    padding-bottom: 5px;
}

/* Buttons */
button {
    background-color: var(--primary-color);
    color: white;
    border: var(--border-width) solid var(--secondary-color);
    padding: 8px 15px;
    margin: 5px;
    cursor: pointer;
    font-family: var(--font-main);
    font-size: 0.9rem;
    transition: var(--transition-standard);
}

button:hover {
    background-color: var(--secondary-color);
    color: var(--text-color);
    transform: translateY(-2px);
}

button:active {
    transform: translateY(1px);
}

button:disabled {
    background-color: #cccccc;
    color: #666666;
    cursor: not-allowed;
    transform: none;
    border-color: #999999;
}

/* Main Content Area */

```

```
main {
  flex: 1;
  padding: 20px;
  max-width: 1200px;
  margin: 0 auto;
  width: 100%;
}

/* Operation Sections */
.operation-section {
  background-color: white;
  margin-bottom: 30px;
  border: var(--border-width) solid var(--secondary-color);
  box-shadow: var(--shadow-standard);
  display: none; /* Hidden by default, shown when selected */
}

.operation-section.active {
  display: block;
}

.operation-section h2 {
  background-color: var(--secondary-color);
  color: var(--text-color);
  padding: 15px;
  margin: 0;
  font-size: 1.5rem;
}

.operation-container {
  display: flex;
  flex-wrap: wrap;
  padding: 20px;
  gap: 20px;
}

.operation-controls {
  flex: 1;
  min-width: 300px;
}

.operation-result {
  flex: 1;
  min-width: 300px;
}

.operation-controls h3,
.operation-result h3 {
  color: var(--primary-color);
  margin-bottom: 15px;
  padding-bottom: 5px;
  border-bottom: 1px solid var(--secondary-color);
}
```

```
/* Form Styling */
.form-group {
  margin-bottom: 15px;
}

.form-group label {
  display: block;
  margin-bottom: 5px;
  color: var(--text-color);
  font-weight: 500;
}

.form-group input,
.form-group select,
.form-group textarea {
  width: 100%;
  padding: 10px;
  border: 1px solid var(--secondary-color);
  font-family: var(--font-main);
  background-color: #fcfcf7;
  color: var(--text-color);
}

.form-group input:focus,
.form-group select:focus,
.form-group textarea:focus {
  outline: none;
  border-color: var(--primary-color);
  box-shadow: 0 0 5px rgba(75, 83, 32, 0.3);
}

.form-group textarea {
  min-height: 100px;
  resize: vertical;
}

.control-row {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  margin-top: 15px;
}

/* Result Containers */
.result-container {
  background-color: #fcfcf7;
  border: 1px solid var(--secondary-color);
  padding: 15px;
  min-height: 150px;
  max-height: 400px;
  overflow-y: auto;
  overflow-x: hidden;
  white-space: pre-wrap;
  word-break: break-word;
}
```

```
    font-family: var(--font-main);
}

/* Binary and Tree Visualizations */
.binary-display {
    display: flex;
    flex-direction: column;
    gap: 5px;
    font-family: var(--font-main);
    font-size: 0.9rem;
}

.binary-row {
    display: flex;
    align-items: center;
    gap: 5px;
}

.bit {
    width: 30px;
    height: 30px;
    display: flex;
    align-items: center;
    justify-content: center;
    border: 1px solid var(--secondary-color);
}

.bit.one {
    background-color: var(--primary-color);
    color: white;
}

.bit.zero {
    background-color: white;
    color: var(--text-color);
}

/* Quadtree Container */
#quadtree-container {
    position: relative;
    width: 100%;
    min-height: 400px;
    border: 2px solid var(--secondary-color);
    background-color: white;
    margin-top: 15px;
}

.quadtree-cell {
    position: absolute;
    border: 1px solid var(--secondary-color);
    background-color: var(--background-color);
    display: flex;
    justify-content: center;
    align-items: center;
}
```

```
    font-family: var(--font-main);
    color: var(--text-color);
    text-align: center;
    cursor: pointer;
    transition: transform 0.2s ease;
}

.quadtree-cell:hover {
    transform: scale(1.05);
    box-shadow: var(--shadow-standard);
}

.cell-content {
    width: 100%;
    height: 100%;
    display: flex;
    justify-content: center;
    align-items: center;
    overflow: hidden;
}

/* Process Visualization */
.process {
    background-color: var(--background-color);
    border: 1px solid var(--secondary-color);
    padding: 10px;
    margin-bottom: 10px;
    position: relative;
}

.process-name {
    font-weight: bold;
    margin-bottom: 5px;
}

.process-progress {
    background-color: #e0e0e0;
    height: 10px;
    width: 100%;
    overflow: hidden;
}

.process-bar {
    background-color: var(--primary-color);
    height: 100%;
    width: 0%;
    transition: width 0.3s linear;
}

.process-status {
    margin-top: 5px;
    font-size: 0.8rem;
}
```

```
.process-close {
  position: absolute;
  top: 5px;
  right: 5px;
  cursor: pointer;
  color: var(--error-color);
  font-size: 1.2rem;
}

/* Database Visualization */
.database-record {
  background-color: var(--background-color);
  border: 1px solid var(--secondary-color);
  padding: 10px;
  margin-bottom: 10px;
}

.record-header {
  display: flex;
  justify-content: space-between;
  margin-bottom: 5px;
}

.record-id {
  font-weight: bold;
}

.record-controls {
  display: flex;
  gap: 5px;
}

.record-control {
  cursor: pointer;
  color: var(--primary-color);
  font-size: 0.8rem;
}

.record-content {
  padding: 5px;
  background-color: white;
}

/* File System Visualization */
#file-tree {
  list-style-type: none;
}

.file-item {
  padding: 5px;
  cursor: pointer;
}

.file-item:hover {
```



```
        background-color: var(--secondary-color);
    }

    .file-icon {
        margin-right: 5px;
    }

    .file-name {
        font-weight: bold;
    }

    .directory-item {
        font-weight: bold;
        cursor: pointer;
    }

    .directory-children {
        padding-left: 20px;
        list-style-type: none;
    }

    /* Memory Display */
    .memory-file {
        margin-bottom: 15px;
    }

    .memory-file-title {
        font-weight: bold;
        padding-bottom: 5px;
        border-bottom: 1px solid var(--secondary-color);
    }

    .memory-slot {
        padding: 8px;
        border-bottom: 1px solid #f0f0e0;
    }

    .memory-slot:hover {
        background-color: #f0f0e0;
    }

    .memory-slot-title {
        font-weight: bold;
        margin-bottom: 5px;
    }

    .memory-slot-value {
        padding: 5px;
        background-color: #f8f8f8;
        border-radius: 2px;
        font-size: 0.9em;
    }

    /* Progress Bar */
```

```
.progress-bar {
  background-color: #e0e0e0;
  height: 20px;
  width: 100%;
  margin: 10px 0;
  overflow: hidden;
}

.progress-fill {
  background-color: var(--primary-color);
  height: 100%;
  width: 0%;
  transition: width 0.3s linear;
}

/* Canvas Styling */
#drawing-canvas {
  border: 2px solid var(--secondary-color);
  cursor: crosshair;
  background-color: white;
  width: 100%;
  max-width: 400px;
  height: 300px;
}

/* Transaction History */
#transaction-history {
  margin-top: 10px;
  font-size: 0.9rem;
  max-height: 150px;
  overflow-y: auto;
}

.transaction-item {
  padding: 3px 5px;
  margin: 2px 0;
  background-color: #f0f0e0;
}

/* Modal Styling */
.modal {
  display: none;
  position: fixed;
  z-index: 1000;
  left: 0;
  top: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.7);
  overflow: auto;
}

.modal-content {
  background-color: white;
```

```
margin: 5% auto;
padding: 20px;
border: 1px solid var(--secondary-color);
width: 80%;
max-width: 600px;
box-shadow: var(--shadow-standard);
position: relative;
}

.close-btn {
color: var(--secondary-color);
position: absolute;
top: 10px;
right: 15px;
font-size: 28px;
font-weight: bold;
cursor: pointer;
}

.close-btn:hover {
color: var(--primary-color);
}

/* Status Messages */
.status-message {
padding: 10px;
margin: 10px 0;
border-radius: 4px;
}

.status-success {
background-color: rgba(76, 175, 80, 0.2);
border: 1px solid var(--success-color);
color: var(--success-color);
}

.status-error {
background-color: rgba(244, 67, 54, 0.2);
border: 1px solid var(--error-color);
color: var(--error-color);
}

.status-warning {
background-color: rgba(255, 193, 7, 0.2);
border: 1px solid var(--warning-color);
color: var(--warning-color);
}

.status-info {
background-color: rgba(33, 150, 243, 0.2);
border: 1px solid var(--info-color);
color: var(--info-color);
}
```

```
/* Compression Stats Display */
#compression-stats {
  margin-top: 10px;
  padding: 10px;
  background-color: #f0f0e0;
  border: 1px solid var(--secondary-color);
}

/* Footer Styling */
footer {
  background-color: var(--primary-color);
  color: white;
  text-align: center;
  padding: 15px;
  margin-top: 30px;
  font-size: 0.9rem;
}

/* Responsive Design */
@media (max-width: 992px) {
  .operation-container {
    flex-direction: column;
  }

  .operation-controls,
  .operation-result {
    width: 100%;
  }

  .nav-section {
    width: 100%;
  }

  .modal-content {
    width: 95%;
  }
}

@media (max-width: 768px) {
  header h1 {
    font-size: 2rem;
  }

  .control-row {
    flex-direction: column;
  }

  .control-row button {
    width: 100%;
  }
}

@media (max-width: 480px) {
  body {
```

```
        font-size: 14px;
    }

    header h1 {
        font-size: 1.8rem;
    }

    .operation-section h2 {
        font-size: 1.2rem;
    }
}

/* Animations */
@keyframes fadeIn {
    from {
        opacity: 0;
        transform: translateY(20px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

.fade-in {
    animation: fadeIn 0.5s ease forwards;
}

@keyframes pulse {
    0% {
        transform: scale(1);
    }
    50% {
        transform: scale(1.05);
    }
    100% {
        transform: scale(1);
    }
}

.pulse {
    animation: pulse 1s infinite;
}

/* Utility Classes */
.hidden {
    display: none !important;
}

.text-center {
    text-align: center;
}

.text-right {
```

```
    text-align: right;
}

.mt-10 {
    margin-top: 10px;
}

.mb-10 {
    margin-bottom: 10px;
}

.mt-20 {
    margin-top: 20px;
}

.mb-20 {
    margin-bottom: 20px;
}

.flex-row {
    display: flex;
}

.justify-between {
    justify-content: space-between;
}

.align-center {
    align-items: center;
}

/* Session persistence styling */
.persistence-controls {
    display: flex;
    flex-direction: column;
    gap: 5px;
}

.persistence-controls button {
    width: 100%;
    padding: 8px;
    font-size: 0.9rem;
}

.notification {
    position: fixed;
    top: 20px;
    right: 20px;
    z-index: 9999;
    background-color: white;
    border: 2px solid var(--secondary-color);
    border-left: 5px solid var(--primary-color);
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.2);
    padding: 15px;
}
```

```

    max-width: 350px;
    font-size: 0.9rem;
    animation: slideIn 0.3s ease-out;
}

.success-notification {
    border-left: 5px solid var(--success-color);
}

.notification-content p {
    margin: 0 0 10px 0;
}

.notification-actions {
    display: flex;
    gap: 10px;
    justify-content: flex-end;
}

.notification-actions button {
    padding: 5px 10px;
    font-size: 0.8rem;
    background-color: var(--primary-color);
    color: white;
    border: none;
    cursor: pointer;
}

.notification-actions button:last-child {
    background-color: var(--secondary-color);
}

.fade-out {
    opacity: 0;
    transition: opacity 0.3s;
}

@keyframes slideIn {
    from {
        transform: translateX(100px);
        opacity: 0;
    }
    to {
        transform: translateX(0);
        opacity: 1;
    }
}

```

[11] operator.html

- **Bytes:** 29881
- **Type:** text

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Operator System - Integrated CS Principles</title>
  <link rel="stylesheet" href="operator.css">
  <!-- Math.js library for expression evaluation -->
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/mathjs/11.8.0/math.min.js"></script>
  <!-- JSZip library for file compression -->
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/jszip/3.10.1/jszip.min.js"></script>
</head>
<body>
  <header>
    <h1>Operator System</h1>
    <p>Integrated Computer Science, Operating System, and CRUD Database
Principles</p>
  </header>

  <nav id="main-nav">
    <div class="nav-section">
      <h3>Data Structures</h3>
      <button data-section="array-operations">Arrays</button>
      <button data-section="object-operations">Objects</button>
      <button data-section="set-operations">Sets</button>
      <button data-section="tree-operations">Trees (Quadtree)</button>
    </div>
    <div class="nav-section">
      <h3>Algorithms</h3>
      <button data-section="recursion-operations">Recursion</button>
      <button data-section="iteration-operations">Iteration</button>
      <button data-section="search-operations">Search</button>
      <button data-section="binary-operations">Binary Operations</button>
    </div>
    <div class="nav-section">
      <h3>System Operations</h3>
      <button data-section="file-operations">File System</button>
      <button data-section="memory-operations">Memory Management</button>
      <button data-section="process-operations">Process Management</button>
    </div>
    <div class="nav-section">
      <h3>CRUD Operations</h3>
      <button data-section="create-operations">Create</button>
      <button data-section="read-operations">Read</button>
      <button data-section="update-operations">Update</button>
      <button data-section="delete-operations">Delete</button>
    </div>
    <div class="nav-section">
      <h3>Advanced Concepts</h3>
      <button data-section="serialization-operations">Serialization</button>
    </div>
  </nav>

```



```

        <button data-section="compression-operations">Compression</button>
        <button data-section="encoding-operations">Binary Encoding</button>
        <button data-section="transactions-operations">Transactions</button>
    </div>
</nav>

<main id="main">
    <!-- Section for Array Operations -->
    <section id="array-operations" class="operation-section">
        <h2>Array Operations</h2>
        <div class="operation-container">
            <div class="operation-controls">
                <h3>Create Array</h3>
                <textarea id="array-input" placeholder="Enter comma-separated
values (e.g. 1,2,3,4,5)"></textarea>
                <div class="control-row">
                    <button id="create-array">Create Array</button>
                    <button id="sort-array">Sort</button>
                    <button id="filter-array">Filter > 5</button>
                    <button id="map-array">Map (x2)</button>
                    <button id="reduce-array">Sum</button>
                </div>
            </div>
            <div class="operation-result">
                <h3>Result</h3>
                <div id="array-result" class="result-container"></div>
            </div>
        </div>
    </section>

    <!-- Section for Object Operations -->
    <section id="object-operations" class="operation-section">
        <h2>Object Operations</h2>
        <div class="operation-container">
            <div class="operation-controls">
                <h3>Create Object</h3>
                <div class="form-group">
                    <label for="object-key">Key:</label>
                    <input type="text" id="object-key" placeholder="Enter
property name">
                </div>
                <div class="form-group">
                    <label for="object-value">Value:</label>
                    <input type="text" id="object-value" placeholder="Enter
property value">
                </div>
                <div class="control-row">
                    <button id="add-property">Add Property</button>
                    <button id="remove-property">Remove Property</button>
                    <button id="clear-object">Clear Object</button>
                </div>
            </div>
            <div class="operation-result">
                <h3>Object Contents</h3>

```

```

        <div id="object-result" class="result-container"></div>
    </div>
</div>
</section>

<!-- Section for Set Operations -->
<section id="set-operations" class="operation-section">
    <h2>Set Operations</h2>
    <div class="operation-container">
        <div class="operation-controls">
            <h3>Create Sets</h3>
            <div class="form-group">
                <label for="set-a-input">Set A:</label>
                <input type="text" id="set-a-input" placeholder="Enter
comma-separated values">
            </div>
            <div class="form-group">
                <label for="set-b-input">Set B:</label>
                <input type="text" id="set-b-input" placeholder="Enter
comma-separated values">
            </div>
            <div class="control-row">
                <button id="set-union">Union</button>
                <button id="set-intersection">Intersection</button>
                <button id="set-difference">Difference (A-B)</button>
            </div>
        </div>
        <div class="operation-result">
            <h3>Result</h3>
            <div id="set-result" class="result-container"></div>
        </div>
    </div>
</section>

<!-- Section for Tree (Quadtree) Operations -->
<section id="tree-operations" class="operation-section">
    <h2>Quadtree Operations</h2>
    <div class="operation-container">
        <div class="operation-controls">
            <h3>Quadtree Settings</h3>
            <div class="form-group">
                <label for="max-depth">Max Depth:</label>
                <input type="number" id="max-depth" min="1" max="6"
value="3">
            </div>
            <div class="form-group">
                <label for="quadtree-size">Size (px):</label>
                <input type="number" id="quadtree-size" min="200"
max="800" value="400">
            </div>
            <div class="control-row">
                <button id="create-quadtree">Create Quadtree</button>
                <button id="reset-quadtree">Reset</button>
                <button id="export-quadtree">Export PNG</button>
            </div>
        </div>
    </div>
</section>

```

```

        </div>
    </div>
    <div class="operation-result">
        <div id="quadtree-container"></div>
    </div>
</div>
</section>

<!-- Section for Recursion Operations -->
<section id="recursion-operations" class="operation-section">
    <h2>Recursion Operations</h2>
    <div class="operation-container">
        <div class="operation-controls">
            <h3>Recursive Functions</h3>
            <div class="form-group">
                <label for="recursion-n">Value (n):</label>
                <input type="number" id="recursion-n" min="1" max="40"
value="5">

            </div>
            <div class="control-row">
                <button id="calc-factorial">Factorial</button>
                <button id="calc-fibonacci">Fibonacci</button>
                <button id="generate-tree">Generate Tree</button>
            </div>
        </div>
        <div class="operation-result">
            <h3>Result</h3>
            <div id="recursion-result" class="result-container"></div>
        </div>
    </div>
</section>

<!-- Section for Iteration Operations -->
<section id="iteration-operations" class="operation-section">
    <h2>Iteration Operations</h2>
    <div class="operation-container">
        <div class="operation-controls">
            <h3>Generate Data</h3>
            <div class="form-group">
                <label for="iteration-n">Size (n):</label>
                <input type="number" id="iteration-n" min="1" max="10"
value="3">

            </div>
            <div class="form-group">
                <label for="charset-input">Character Set:</label>
                <input type="text" id="charset-input" value="abcdefghijklmnopqrstuvwxyz">
            </div>
            <div class="control-row">
                <button id="generate-combinations">Generate
Combinations</button>
                <button id="stop-generation">Stop</button>
            </div>
        </div>
        <div class="operation-result">

```

```

        <h3>Generated Data</h3>
        <div id="iteration-progress" class="progress-bar">
            <div class="progress-fill"></div>
        </div>
        <div id="iteration-result" class="result-container">
            <p>Generated combinations will appear here</p>
        </div>
    </div>
</div>
</section>

<!-- Section for Search Operations -->
<section id="search-operations" class="operation-section">
    <h2>Search Operations</h2>
    <div class="operation-container">
        <div class="operation-controls">
            <h3>Search Data</h3>
            <div class="form-group">
                <label for="search-data">Dataset:</label>
                <textarea id="search-data" placeholder="Enter comma-
separated values"></textarea>
            </div>
            <div class="form-group">
                <label for="search-query">Search For:</label>
                <input type="text" id="search-query" placeholder="Enter
search term">
            </div>
            <div class="control-row">
                <button id="linear-search">Linear Search</button>
                <button id="binary-search">Binary Search</button>
                <button id="fuzzy-search">Fuzzy Search</button>
            </div>
        </div>
        <div class="operation-result">
            <h3>Search Results</h3>
            <div id="search-result" class="result-container"></div>
        </div>
    </div>
</section>

<!-- Section for Binary Operations -->
<section id="binary-operations" class="operation-section">
    <h2>Binary Operations</h2>
    <div class="operation-container">
        <div class="operation-controls">
            <h3>Binary Manipulation</h3>
            <div class="form-group">
                <label for="binary-input-a">Value A:</label>
                <input type="number" id="binary-input-a" min="0" max="255"
value="42">
            </div>
            <div class="form-group">
                <label for="binary-input-b">Value B:</label>
                <input type="number" id="binary-input-b" min="0" max="255"

```

```

value="27">
    </div>
    <div class="control-row">
        <button id="binary-and">AND</button>
        <button id="binary-or">OR</button>
        <button id="binary-xor">XOR</button>
        <button id="binary-not">NOT (A)</button>
        <button id="binary-shift">Shift Left</button>
    </div>
</div>
<div class="operation-result">
    <h3>Binary Result</h3>
    <div id="binary-result" class="result-container"></div>
</div>
</div>
</section>

<!-- Section for File System Operations -->
<section id="file-operations" class="operation-section">
    <h2>File System Operations</h2>
    <div class="operation-container">
        <div class="operation-controls">
            <h3>File Management</h3>
            <div class="form-group">
                <label for="file-content">File Content:</label>
                <textarea id="file-content" placeholder="Enter file
content to save"></textarea>
            </div>
            <div class="form-group">
                <label for="file-name">File Name:</label>
                <input type="text" id="file-name" placeholder="Enter file
name" value="data.txt">
            </div>
            <div class="control-row">
                <button id="save-file">Save to File</button>
                <button id="load-file">Load File</button>
                <input type="file" id="file-upload" style="display:none">
            </div>
            <div class="control-row">
                <button id="create-directory">Create Directory</button>
                <button id="create-zip">Create ZIP</button>
            </div>
        </div>
        <div class="operation-result">
            <h3>File System</h3>
            <div id="file-system" class="result-container">
                <ul id="file-tree"></ul>
            </div>
        </div>
    </div>
</section>

<!-- Section for Memory Operations -->
<section id="memory-operations" class="operation-section">

```

```

<h2>Memory Management</h2>
<div class="operation-container">
  <div class="operation-controls">
    <h3>Memory Slots</h3>
    <div class="form-group">
      <label for="memory-file">File Number:</label>
      <input type="number" id="memory-file" min="1" value="1">
    </div>
    <div class="form-group">
      <label for="memory-slot">Slot Number:</label>
      <input type="number" id="memory-slot" min="1" value="1">
    </div>
    <div class="form-group">
      <label for="memory-content">Content:</label>
      <textarea id="memory-content" placeholder="Enter slot
content"></textarea>
    </div>
    <div class="control-row">
      <button id="assign-slot">Assign Slot</button>
      <button id="read-slot">Read Slot</button>
      <button id="clear-all-slots">Clear All</button>
    </div>
  </div>
  <div class="operation-result">
    <h3>Memory Contents</h3>
    <div id="memory-display" class="result-container"></div>
  </div>
</div>
</section>

<!-- Section for Process Operations -->
<section id="process-operations" class="operation-section">
  <h2>Process Management</h2>
  <div class="operation-container">
    <div class="operation-controls">
      <h3>Processes</h3>
      <div class="form-group">
        <label for="process-name">Process Name:</label>
        <input type="text" id="process-name" placeholder="Enter
process name">
      </div>
      <div class="form-group">
        <label for="process-duration">Duration (ms):</label>
        <input type="number" id="process-duration" min="100"
max="10000" value="2000">
      </div>
      <div class="control-row">
        <button id="start-process">Start Process</button>
        <button id="stop-all-processes">Stop All</button>
      </div>
    </div>
    <div class="operation-result">
      <h3>Process Status</h3>
      <div id="process-list" class="result-container"></div>
    </div>
  </div>
</section>

```

```

        </div>
    </div>
</section>

<!-- Section for Create Operations -->
<section id="create-operations" class="operation-section">
    <h2>CRUD - Create</h2>
    <div class="operation-container">
        <div class="operation-controls">
            <h3>Create Record</h3>
            <div class="form-group">
                <label for="record-id">ID:</label>
                <input type="text" id="record-id" placeholder="Enter
record ID">
            </div>
            <div class="form-group">
                <label for="record-name">Name:</label>
                <input type="text" id="record-name" placeholder="Enter
record name">
            </div>
            <div class="form-group">
                <label for="record-data">Data:</label>
                <textarea id="record-data" placeholder="Enter record
data"></textarea>
            </div>
            <div class="control-row">
                <button id="create-record">Create Record</button>
            </div>
        </div>
        <div class="operation-result">
            <h3>Database Contents</h3>
            <div id="database-display" class="result-container"></div>
        </div>
    </div>
</section>

<!-- Section for Read Operations -->
<section id="read-operations" class="operation-section">
    <h2>CRUD - Read</h2>
    <div class="operation-container">
        <div class="operation-controls">
            <h3>Read Record</h3>
            <div class="form-group">
                <label for="read-id">Record ID:</label>
                <input type="text" id="read-id" placeholder="Enter record
ID to read">
            </div>
            <div class="control-row">
                <button id="read-record">Read Record</button>
                <button id="read-all-records">Read All Records</button>
                <button id="query-records">Query Records</button>
            </div>
        </div>
        <div class="operation-result">

```

```

        <h3>Read Results</h3>
        <div id="read-result" class="result-container"></div>
    </div>
</div>
</section>

<!-- Section for Update Operations -->
<section id="update-operations" class="operation-section">
    <h2>CRUD - Update</h2>
    <div class="operation-container">
        <div class="operation-controls">
            <h3>Update Record</h3>
            <div class="form-group">
                <label for="update-id">Record ID:</label>
                <input type="text" id="update-id" placeholder="Enter
record ID to update">
            </div>
            <div class="form-group">
                <label for="update-field">Field:</label>
                <select id="update-field">
                    <option value="name">Name</option>
                    <option value="data">Data</option>
                </select>
            </div>
            <div class="form-group">
                <label for="update-value">New Value:</label>
                <textarea id="update-value" placeholder="Enter new value">
</textarea>
            </div>
            <div class="control-row">
                <button id="update-record">Update Record</button>
            </div>
        </div>
        <div class="operation-result">
            <h3>Update Result</h3>
            <div id="update-result" class="result-container"></div>
        </div>
    </div>
</section>

<!-- Section for Delete Operations -->
<section id="delete-operations" class="operation-section">
    <h2>CRUD - Delete</h2>
    <div class="operation-container">
        <div class="operation-controls">
            <h3>Delete Record</h3>
            <div class="form-group">
                <label for="delete-id">Record ID:</label>
                <input type="text" id="delete-id" placeholder="Enter
record ID to delete">
            </div>
            <div class="control-row">
                <button id="delete-record">Delete Record</button>
                <button id="delete-all-records">Delete All

```



```

Records</button>
        </div>
    </div>
    <div class="operation-result">
        <h3>Delete Result</h3>
        <div id="delete-result" class="result-container"></div>
    </div>
</div>
</section>

<!-- Section for Serialization Operations -->
<section id="serialization-operations" class="operation-section">
    <h2>Serialization & Deserialization</h2>
    <div class="operation-container">
        <div class="operation-controls">
            <h3>Serialize Data</h3>
            <div class="form-group">
                <label for="serialize-input">Input Object:</label>
                <textarea id="serialize-input" placeholder="Enter JSON
object"></textarea>
            </div>
            <div class="form-group">
                <label for="serialization-format">Format:</label>
                <select id="serialization-format">
                    <option value="json">JSON</option>
                    <option value="xml">XML</option>
                    <option value="csv">CSV</option>
                </select>
            </div>
            <div class="control-row">
                <button id="serialize-data">Serialize</button>
                <button id="deserialize-data">Deserialize</button>
            </div>
        </div>
        <div class="operation-result">
            <h3>Serialized Data</h3>
            <div id="serialization-result" class="result-container"></div>
        </div>
    </div>
</section>

<!-- Section for Compression Operations -->
<section id="compression-operations" class="operation-section">
    <h2>Data Compression</h2>
    <div class="operation-container">
        <div class="operation-controls">
            <h3>Compress Data</h3>
            <div class="form-group">
                <label for="compression-input">Input Text:</label>
                <textarea id="compression-input" placeholder="Enter text
to compress"></textarea>
            </div>
            <div class="form-group">
                <label for="compression-method">Method:</label>

```

```

        <select id="compression-method">
            <option value="rle">Run-Length Encoding</option>
            <option value="huffman">Huffman Coding</option>
            <option value="lz">LZ Compression</option>
        </select>
    </div>
    <div class="control-row">
        <button id="compress-data">Compress</button>
        <button id="decompress-data">Decompress</button>
    </div>
</div>
<div class="operation-result">
    <h3>Compression Result</h3>
    <div id="compression-result" class="result-container"></div>
    <div id="compression-stats"></div>
</div>
</div>
</section>

<!-- Section for Binary Encoding Operations -->
<section id="encoding-operations" class="operation-section">
    <h2>Binary Encoding/Decoding</h2>
    <div class="operation-container">
        <div class="operation-controls">
            <h3>Encode/Decode</h3>
            <div class="form-group">
                <label for="encoding-input">Input:</label>
                <textarea id="encoding-input" placeholder="Enter text to
encode"></textarea>
            </div>
            <div class="form-group">
                <label for="encoding-method">Method:</label>
                <select id="encoding-method">
                    <option value="base64">Base64</option>
                    <option value="hex">Hexadecimal</option>
                    <option value="binary">Binary</option>
                    <option value="custom">Custom ID</option>
                </select>
            </div>
            <div class="control-row">
                <button id="encode-data">Encode</button>
                <button id="decode-data">Decode</button>
            </div>
        </div>
        <div class="operation-result">
            <h3>Encoding Result</h3>
            <div id="encoding-result" class="result-container"></div>
        </div>
    </div>
</section>

<!-- Section for Transaction Operations -->
<section id="transactions-operations" class="operation-section">
    <h2>Transactions & History</h2>

```

```

        <div class="operation-container">
            <div class="operation-controls">
                <h3>Canvas Operations</h3>
                <div class="form-group">
                    <label for="drawing-color">Color:</label>
                    <input type="color" id="drawing-color" value="#000000">
                </div>
                <div class="form-group">
                    <label for="drawing-size">Size:</label>
                    <input type="range" id="drawing-size" min="1" max="50"
value="5">
                </div>
                <div class="control-row">
                    <button id="transaction-undo">Undo</button>
                    <button id="transaction-redo">Redo</button>
                    <button id="transaction-clear">Clear</button>
                </div>
            </div>
            <div class="operation-result">
                <h3>Drawing Canvas</h3>
                <canvas id="drawing-canvas" width="400" height="300"></canvas>
                <div id="transaction-history"></div>
            </div>
        </div>
    </section>
</main>

<footer>
    <p>Operator System - Comprehensive Computer Science Principles
Implementation</p>
    <p>&copy; 2025 - An educational tool demonstrating software engineering
concepts</p>
</footer>

<!-- Error modal for handling and displaying errors -->
<div id="error-modal" class="modal">
    <div class="modal-content">
        <span class="close-btn">&times;</span>
        <h2>Error</h2>
        <p id="error-message"></p>
    </div>
</div>

<!-- Result modal for displaying large results -->
<div id="result-modal" class="modal">
    <div class="modal-content">
        <span class="close-btn">&times;</span>
        <h2>Result</h2>
        <div id="modal-result"></div>
    </div>
</div>
<script src="operator-framework.js"></script>
<!-- Add this before operator.js in operator.html -->
<script src="operator-language.js"></script>

```

```

<script src="operator.js"></script>
<!-- Add this right after operator.js and operator-language.js -->
<script src="operator-extensions.js"></script>
<script src="operator-dropdown.js"></script>
<script src="operator-map.js"></script>
<script src="operator-alphabet.js"></script>
<script src="operator-build.js"></script>
</body>
</html>

```

[12] operator.js

- **Bytes:** 153742
- **Type:** text

```

/**
 * operator.js
 * Operator System - Integrated Computer Science Principles
 *
 * This JavaScript file implements various computer science principles:
 * - Data Structures (Arrays, Objects, Sets, Trees)
 * - Algorithms (Recursion, Iteration, Search, Binary Operations)
 * - System Operations (File System, Memory Management, Process Management)
 * - CRUD Database Operations
 * - Advanced Concepts (Serialization, Compression, Encoding, Transactions)
 */

// Immediately Invoked Function Expression (IIFE) to avoid global scope pollution
(function() {
    'use strict';

    // ===== UTILITY FUNCTIONS =====

    /**
     * Utility function to show an error modal
     */
    function showError(message) {
        const modal = document.getElementById('error-modal');
        const messageElement = document.getElementById('error-message');
        messageElement.textContent = message;
        modal.style.display = 'block';
    }

    /**
     * Utility function to show a result in the modal
     */
    function showResultModal(content) {
        const modal = document.getElementById('result-modal');
        const resultElement = document.getElementById('modal-result');
        resultElement.innerHTML = '';
    }

```

```

        if (typeof content === 'string') {
            resultElement.textContent = content;
        } else {
            resultElement.appendChild(content);
        }

        modal.style.display = 'block';
    }

    /**
     * Creates a status message element
     */
    function createStatusMessage(message, type) {
        const statusElement = document.createElement('div');
        statusElement.className = `status-message status-${type}`;
        statusElement.textContent = message;
        return statusElement;
    }

    /**
     * Global error handler with try-catch pattern
     */
    function safeExecute(func, errorCallback) {
        try {
            return func();
        } catch (error) {
            console.error('Operation error:', error);
            if (errorCallback) {
                errorCallback(error);
            } else {
                showError(error.message || 'An unexpected error occurred');
            }
            return null;
        }
    }

    // ===== SECTION NAVIGATION =====

    /**
     * Set up section navigation
     */
    function setupNavigation() {
        // Get all section buttons
        const sectionButtons = document.querySelectorAll('#main-nav button');

        // Add click event to each button
        sectionButtons.forEach(button => {
            button.addEventListener('click', () => {
                const sectionId = button.getAttribute('data-section');
                showSection(sectionId);
            });
        });

        // Show the first section by default
    }

```

```

    if (sectionButtons.length > 0) {
        const firstSectionId = sectionButtons[0].getAttribute('data-section');
        showSection(firstSectionId);
    }
}

/**
 * Show a specific section and hide all others
 */
function showSection(sectionId) {
    // Hide all sections
    const allSections = document.querySelectorAll('.operation-section');
    allSections.forEach(section => {
        section.classList.remove('active');
    });

    // Show the selected section
    const selectedSection = document.getElementById(sectionId);
    if (selectedSection) {
        selectedSection.classList.add('active');
        selectedSection.classList.add('fade-in');
        setTimeout(() => {
            selectedSection.classList.remove('fade-in');
        }, 500);
    }
}

// ===== 1. DATA STRUCTURES =====

// ===== 1.1 Array Operations =====
function setupArrayOperations() {
    const arrayInput = document.getElementById('array-input');
    const createArrayBtn = document.getElementById('create-array');
    const sortArrayBtn = document.getElementById('sort-array');
    const filterArrayBtn = document.getElementById('filter-array');
    const mapArrayBtn = document.getElementById('map-array');
    const reduceArrayBtn = document.getElementById('reduce-array');
    const resultContainer = document.getElementById('array-result');

    // Current array state
    let currentArray = [];

    // Create array from input
    createArrayBtn.addEventListener('click', () => {
        safeExecute(() => {
            const input = arrayInput.value;
            currentArray = input.split(',').map(item => {
                const trimmed = item.trim();
                // Try to convert to number if possible
                const num = Number(trimmed);
                return isNaN(num) ? trimmed : num;
            });

            displayArray(currentArray);
        });
    });
}

```

```

        return `Created array with ${currentArray.length} elements`;
    }, error => {
        resultContainer.textContent = `Error creating array:
${error.message}`;
    });
});

// Sort the array
sortArrayBtn.addEventListener('click', () => {
    safeExecute(() => {
        if (currentArray.length === 0) {
            throw new Error('Array is empty. Create an array first.');
```

sorting

```

        }

        // Check if array has all numbers or all strings for proper
        const allNumbers = currentArray.every(item => typeof item ===
'number');
        const allStrings = currentArray.every(item => typeof item ===
'string');

        if (allNumbers) {
            currentArray.sort((a, b) => a - b);
        } else if (allStrings) {
            currentArray.sort();
        } else {
            // Mixed type sorting - convert all to strings
            currentArray.sort((a, b) =>
String(a).localeCompare(String(b)));
        }

        displayArray(currentArray);
        return 'Array sorted successfully';
    }, error => {
        resultContainer.textContent = `Error sorting array:
${error.message}`;
    });
});

// Filter array elements > 5
filterArrayBtn.addEventListener('click', () => {
    safeExecute(() => {
        if (currentArray.length === 0) {
            throw new Error('Array is empty. Create an array first.');
```

```

        }

        const filtered = currentArray.filter(item => {
            // Only filter numerical values
            return typeof item === 'number' && item > 5;
        });

        displayArray(filtered);
        return `Filtered array to ${filtered.length} elements > 5`;
    }, error => {
```

```

        resultContainer.textContent = `Error filtering array:
${error.message}`;
    });
});

// Map array (multiply by 2)
mapArrayBtn.addEventListener('click', () => {
    safeExecute(() => {
        if (currentArray.length === 0) {
            throw new Error('Array is empty. Create an array first.');
        }

        const mapped = currentArray.map(item => {
            // Only multiply numbers, leave strings as is
            return typeof item === 'number' ? item * 2 : item;
        });

        displayArray(mapped);
        return 'Mapped array (numbers multiplied by 2)';
    }, error => {
        resultContainer.textContent = `Error mapping array:
${error.message}`;
    });
});

// Reduce array (sum)
reduceArrayBtn.addEventListener('click', () => {
    safeExecute(() => {
        if (currentArray.length === 0) {
            throw new Error('Array is empty. Create an array first.');
        }

        // Extract numbers from the array
        const numbers = currentArray.filter(item => typeof item ===
'number');

        if (numbers.length === 0) {
            throw new Error('No numerical values found in the array');
        }

        const sum = numbers.reduce((acc, curr) => acc + curr, 0);

        resultContainer.innerHTML = '';
        const resultElement = document.createElement('div');
        resultElement.textContent = `Sum of numerical values: ${sum}`;
        resultContainer.appendChild(resultElement);

        return `Reduced array to sum: ${sum}`;
    }, error => {
        resultContainer.textContent = `Error reducing array:
${error.message}`;
    });
});

```



```

// Helper function to display the array
function displayArray(array) {
    resultContainer.innerHTML = '';

    const arrayElement = document.createElement('div');
    arrayElement.className = 'array-display';

    array.forEach((item, index) => {
        const itemElement = document.createElement('div');
        itemElement.className = 'array-item';
        itemElement.innerHTML = `<span class="array-index">[${index}]
</span>: <span class="array-value">${item}</span>`;
        arrayElement.appendChild(itemElement);
    });

    resultContainer.appendChild(arrayElement);
}

// ===== 1.2 Object Operations =====
function setupObjectOperations() {
    const objectKey = document.getElementById('object-key');
    const objectValue = document.getElementById('object-value');
    const addPropertyBtn = document.getElementById('add-property');
    const removePropertyBtn = document.getElementById('remove-property');
    const clearObjectBtn = document.getElementById('clear-object');
    const resultContainer = document.getElementById('object-result');

    // Current object state
    let currentObject = {};

    // Add a property to the object
    addPropertyBtn.addEventListener('click', () => {
        safeExecute(() => {
            const key = objectKey.value.trim();
            const value = objectValue.value.trim();

            if (!key) {
                throw new Error('Property key cannot be empty');
            }

            // Try to parse the value if it looks like a number or boolean
            let parsedValue = value;

            if (value.toLowerCase() === 'true') {
                parsedValue = true;
            } else if (value.toLowerCase() === 'false') {
                parsedValue = false;
            } else if (!isNaN(Number(value))) {
                parsedValue = Number(value);
            }

            // Add or update the property
            currentObject[key] = parsedValue;
        });
    });
}

```

```

        // Clear inputs
        objectKey.value = '';
        objectValue.value = '';

        // Display updated object
        displayObject(currentObject);
        return `Property '${key}' added to object`;
    }, error => {
        resultContainer.textContent = `Error adding property:
${error.message}`;
    });
});

// Remove a property from the object
removePropertyBtn.addEventListener('click', () => {
    safeExecute(() => {
        const key = objectKey.value.trim();

        if (!key) {
            throw new Error('Property key cannot be empty');
        }

        if (!(key in currentObject)) {
            throw new Error(`Property '${key}' does not exist in the
object`);
        }

        // Remove the property
        delete currentObject[key];

        // Clear input
        objectKey.value = '';

        // Display updated object
        displayObject(currentObject);
        return `Property '${key}' removed from object`;
    }, error => {
        resultContainer.textContent = `Error removing property:
${error.message}`;
    });
});

// Clear the object
clearObjectBtn.addEventListener('click', () => {
    currentObject = {};
    displayObject(currentObject);
});

// Helper function to display the object
function displayObject(obj) {
    resultContainer.innerHTML = '';

    if (Object.keys(obj).length === 0) {

```

```

        resultContainer.textContent = 'Empty object: {}';
        return;
    }

    const objectElement = document.createElement('div');
    objectElement.className = 'object-display';

    // Create JSON representation with syntax highlighting
    const jsonPre = document.createElement('pre');
    jsonPre.className = 'json-display';

    // Format JSON with indentation
    const formattedJson = JSON.stringify(obj, null, 2);
    jsonPre.textContent = formattedJson;

    objectElement.appendChild(jsonPre);
    resultContainer.appendChild(objectElement);
}

// Initialize with empty object
displayObject(currentObject);
}

// ===== 1.3 Set Operations =====
function setupSetOperations() {
    const setAInput = document.getElementById('set-a-input');
    const setBInput = document.getElementById('set-b-input');
    const unionBtn = document.getElementById('set-union');
    const intersectionBtn = document.getElementById('set-intersection');
    const differenceBtn = document.getElementById('set-difference');
    const resultContainer = document.getElementById('set-result');

    // Helper function to parse input into a Set
    function parseSetInput(input) {
        const values = input.split(',').map(item => item.trim());
        return new Set(values);
    }

    // Helper function to display a Set
    function displaySet(set, label) {
        const setElement = document.createElement('div');
        setElement.className = 'set-display';

        const setLabel = document.createElement('div');
        setLabel.className = 'set-label';
        setLabel.textContent = label;

        const setContent = document.createElement('div');
        setContent.className = 'set-content';
        setContent.textContent = Array.from(set).join(', ');

        setElement.appendChild(setLabel);
        setElement.appendChild(setContent);
    }

```

```

        return setElement;
    }

    // Union operation
    unionBtn.addEventListener('click', () => {
        safeExecute(() => {
            const setA = parseSetInput(setAInput.value);
            const setB = parseSetInput(setBInput.value);

            // Perform union operation
            const union = new Set([...setA, ...setB]);

            // Display the result
            resultContainer.innerHTML = '';
            resultContainer.appendChild(displaySet(setA, 'Set A:'));
            resultContainer.appendChild(displaySet(setB, 'Set B:'));
            resultContainer.appendChild(displaySet(union, 'Union (A  $\cup$  B):'));

            return `Union operation complete: ${union.size} elements`;
        }, error => {
            resultContainer.textContent = `Error performing union:
            ${error.message}`;
        });
    });

    // Intersection operation
    intersectionBtn.addEventListener('click', () => {
        safeExecute(() => {
            const setA = parseSetInput(setAInput.value);
            const setB = parseSetInput(setBInput.value);

            // Perform intersection operation
            const intersection = new Set(
                [...setA].filter(item => setB.has(item))
            );

            // Display the result
            resultContainer.innerHTML = '';
            resultContainer.appendChild(displaySet(setA, 'Set A:'));
            resultContainer.appendChild(displaySet(setB, 'Set B:'));
            resultContainer.appendChild(displaySet(intersection, 'Intersection
            (A  $\cap$  B):'));

            return `Intersection operation complete: ${intersection.size}
            elements`;
        }, error => {
            resultContainer.textContent = `Error performing intersection:
            ${error.message}`;
        });
    });

    // Difference operation
    differenceBtn.addEventListener('click', () => {
        safeExecute(() => {

```

```

        const setA = parseSetInput(setAInput.value);
        const setB = parseSetInput(setBInput.value);

        // Perform difference operation (A - B)
        const difference = new Set(
            [...setA].filter(item => !setB.has(item))
        );

        // Display the result
        resultContainer.innerHTML = '';
        resultContainer.appendChild(displaySet(setA, 'Set A:'));
        resultContainer.appendChild(displaySet(setB, 'Set B:'));
        resultContainer.appendChild(displaySet(difference, 'Difference (A
- B):'));

        return `Difference operation complete: ${difference.size}
elements`;
    }, error => {
        resultContainer.textContent = `Error performing difference:
${error.message}`;
    });
});
}

// ===== 1.4 Tree (Quadtree) Operations =====
function setupTreeOperations() {
    const maxDepthInput = document.getElementById('max-depth');
    const quadtreeSizeInput = document.getElementById('quadtree-size');
    const createQuadtreeBtn = document.getElementById('create-quadtree');
    const resetQuadtreeBtn = document.getElementById('reset-quadtree');
    const exportQuadtreeBtn = document.getElementById('export-quadtree');
    const quadtreeContainer = document.getElementById('quadtree-container');

    // Quadtree state
    let quadtreeRoot = null;
    let maxDepth = 3;
    let quadtreeSize = 400;

    // QuadtreeNode class
    class QuadtreeNode {
        constructor(x, y, size, depth) {
            this.x = x;
            this.y = y;
            this.size = size;
            this.depth = depth;
            this.children = [];
            this.data = {};
            this.element = this.createElement();
            this.updateVisualState();
        }

        createElement() {
            const cell = document.createElement('div');
            cell.className = 'quadtree-cell';

```

```

    cell.style.left = `${this.x}px`;
    cell.style.top = `${this.y}px`;
    cell.style.width = `${this.size}px`;
    cell.style.height = `${this.size}px`;

    const content = document.createElement('div');
    content.className = 'cell-content';
    content.textContent = `d: ${this.depth}`;
    cell.appendChild(content);

    cell.addEventListener('click', () => {
        this.handleClick();
    });

    quadtreeContainer.appendChild(cell);
    return cell;
}

updateVisualState() {
    if (this.depth < maxDepth) {
        this.element.classList.add('can-subdivide');
        this.element.title = 'Click to subdivide';
    } else {
        this.element.classList.remove('can-subdivide');
        this.element.title = 'Max depth reached';
    }

    // Update color based on depth
    const hue = (this.depth * 30) % 360;
    this.element.style.backgroundColor = `hsl(${hue}, 70%, 90%)`;
    this.element.style.borderColor = `hsl(${hue}, 70%, 60%)`;
}

handleClick() {
    if (this.children.length === 0 && this.depth < maxDepth) {
        this.subdivide();
    }
}

subdivide() {
    if (this.children.length > 0 || this.depth >= maxDepth) return
false;

    const childSize = this.size / 2;

    this.children.push(
        new QuadtreeNode(this.x, this.y, childSize, this.depth + 1),
        new QuadtreeNode(this.x + childSize, this.y, childSize,
this.depth + 1),
        new QuadtreeNode(this.x, this.y + childSize, childSize,
this.depth + 1),
        new QuadtreeNode(this.x + childSize, this.y + childSize,
childSize, this.depth + 1)
    );
}

```

```

    );

    this.element.style.display = 'none';
    return true;
  }
}

// Create the initial quadtree
function initQuadtree() {
  // Clear the container
  quadtreeContainer.innerHTML = '';

  // Get current settings
  maxDepth = parseInt(maxDepthInput.value) || 3;
  quadtreeSize = parseInt(quadtreeSizeInput.value) || 400;

  // Set container size
  quadtreeContainer.style.width = `${quadtreeSize}px`;
  quadtreeContainer.style.height = `${quadtreeSize}px`;

  // Create root node
  quadtreeRoot = new QuadTreeNode(0, 0, quadtreeSize, 0);
}

// Export the quadtree as PNG
function exportQuadtreeAsPNG() {
  const canvas = document.createElement('canvas');
  const context = canvas.getContext('2d');

  canvas.width = quadtreeSize;
  canvas.height = quadtreeSize;

  // Fill background
  context.fillStyle = 'white';
  context.fillRect(0, 0, canvas.width, canvas.height);

  // Recursive function to draw nodes
  function drawNode(node) {
    if (!node) return;

    // Only draw visible nodes
    if (node.element.style.display !== 'none') {
      // Get computed styles
      const styles = window.getComputedStyle(node.element);

      // Draw cell background
      context.fillStyle = styles.backgroundColor;
      context.fillRect(node.x, node.y, node.size, node.size);

      // Draw cell border
      context.strokeStyle = styles.borderColor;
      context.lineWidth = 1;
      context.strokeRect(node.x, node.y, node.size, node.size);
    }
  }
}

```

```

        // Draw text
        context.fillStyle = '#333';
        context.font = '12px sans-serif';
        context.textAlign = 'center';
        context.textBaseline = 'middle';
        context.fillText(`d: ${node.depth}`,
                        node.x + node.size/2,
                        node.y + node.size/2);
    }

    // Draw children
    for (const child of node.children) {
        drawNode(child);
    }
}

// Draw the quadtree
drawNode(quadtreeRoot);

// Create download link
const link = document.createElement('a');
link.download = 'quadtree.png';
link.href = canvas.toDataURL('image/png');
link.click();
}

// Event listeners
createQuadtreeBtn.addEventListener('click', initQuadtree);
resetQuadtreeBtn.addEventListener('click', initQuadtree);
exportQuadtreeBtn.addEventListener('click', exportQuadtreeAsPNG);

// Initialize
initQuadtree();
}

// ===== 2. ALGORITHMS =====

// ===== 2.1 Recursion Operations =====
function setupRecursionOperations() {
    const recursionInput = document.getElementById('recursion-n');
    const factorialBtn = document.getElementById('calc-factorial');
    const fibonacciBtn = document.getElementById('calc-fibonacci');
    const generateTreeBtn = document.getElementById('generate-tree');
    const resultContainer = document.getElementById('recursion-result');

    // Recursive factorial function
    function factorial(n) {
        // Base case
        if (n <= 1) return 1;

        // Recursive case
        return n * factorial(n - 1);
    }
}

```



```

// Recursive Fibonacci function
function fibonacci(n) {
  // Base cases
  if (n <= 0) return 0;
  if (n === 1) return 1;

  // Recursive case
  return fibonacci(n - 1) + fibonacci(n - 2);
}

// Generate recursive tree visualization
function generateTree(n) {
  const container = document.createElement('div');
  container.className = 'tree-container';

  function createTreeNode(value, depth) {
    const node = document.createElement('div');
    node.className = 'tree-node';

    const nodeContent = document.createElement('div');
    nodeContent.className = 'tree-node-content';
    nodeContent.textContent = value;

    node.appendChild(nodeContent);

    // Add children if depth > 0
    if (depth > 0) {
      const children = document.createElement('div');
      children.className = 'tree-children';

      // Left child is depth-1
      children.appendChild(createTreeNode(depth-1, depth-1));

      // Right child is depth/2 (integer division)
      if (depth > 1) {
        children.appendChild(createTreeNode(Math.floor(depth/2),
Math.floor(depth/2)));
      }

      node.appendChild(children);
    }

    return node;
  }

  container.appendChild(createTreeNode(n, n));
  return container;
}

// Calculate factorial button
factorialBtn.addEventListener('click', () => {
  safeExecute(() => {
    const n = parseInt(recursionInput.value);

```

```

    if (isNaN(n) || n < 0) {
        throw new Error('Please enter a non-negative integer');
    }

    if (n > 20) {
        throw new Error('Value too large (max 20)');
    }

    const result = factorial(n);

    resultContainer.innerHTML = '';
    const resultElement = document.createElement('div');
    resultElement.innerHTML = `Factorial of ${n}:</strong>
    ${result}`;

    // Add explanation
    const explanation = document.createElement('div');
    explanation.className = 'recursion-explanation';
    explanation.innerHTML = `

Factorial is calculated recursively
    as:</p>
    <p>factorial(${n}) = ${n} * factorial(${n-
    1})</p>
    <p>factorial(1) = 1 (base case)</p>`;

    resultContainer.appendChild(resultElement);
    resultContainer.appendChild(explanation);

    return `Calculated factorial of ${n}: ${result}`;
  }, error => {
    resultContainer.textContent = `Error calculating factorial:
    ${error.message}`;
  });
});

// Calculate Fibonacci button
fibonacciBtn.addEventListener('click', () => {
  safeExecute(() => {
    const n = parseInt(recursionInput.value);

    if (isNaN(n) || n < 0) {
        throw new Error('Please enter a non-negative integer');
    }

    if (n > 40) {
        throw new Error('Value too large (max 40). Recursive Fibonacci
    is inefficient for large values.');


```

```

        // Add explanation
        const explanation = document.createElement('div');
        explanation.className = 'recursion-explanation';
        explanation.innerHTML = `<p>Fibonacci is calculated recursively
as:</p>
                                <p>fibonacci(${n}) = fibonacci(${n-1}) +
fibonacci(${n-2})</p>
                                <p>fibonacci(0) = 0, fibonacci(1) = 1
(base cases)</p>`;

        resultContainer.appendChild(resultElement);
        resultContainer.appendChild(explanation);

        return `Calculated Fibonacci(${n}): ${result}`;
    }, error => {
        resultContainer.textContent = `Error calculating Fibonacci:
${error.message}`;
    });
});

// Generate Tree button
generateTreeBtn.addEventListener('click', () => {
    safeExecute(() => {
        const n = parseInt(recursionInput.value);

        if (isNaN(n) || n < 0) {
            throw new Error('Please enter a non-negative integer');
        }

        if (n > 5) {
            throw new Error('Value too large (max 5). Larger values create
very large trees.');
```

```

const resultContainer = document.getElementById('iteration-result');
const progressBar = document.getElementById('iteration-progress');
const progressFill = progressBar.querySelector('.progress-fill');

// Flag to control generation
let isGenerating = false;

// Generate all possible combinations of length n from charset
function generateCombinations(charset, n) {
  const combinations = [];
  const charsetArray = Array.from(charset);

  // Total number of combinations
  const total = Math.pow(charsetArray.length, n);

  // Iterator function using batch processing for UI responsiveness
  async function* combinationIterator() {
    const batchSize = 1000;
    let count = 0;

    // For each possible combination
    for (let i = 0; i < total && isGenerating; i++) {
      let combination = '';
      let index = i;

      // Generate combination by extracting characters
      for (let j = 0; j < n; j++) {
        combination = charsetArray[index % charsetArray.length] +
combination;

        index = Math.floor(index / charsetArray.length);
      }

      combinations.push(combination);
      count++;

      // Yield after each batch to allow UI updates
      if (count % batchSize === 0 || i === total - 1) {
        yield {
          progress: (i + 1) / total * 100,
          combinations: combinations.slice() // Copy current
state
        };
        await new Promise(resolve => setTimeout(resolve, 0));
      }
    }
  }

  return {
    iterator: combinationIterator(),
    total
  };
}

// Generate button click handler

```

```

generateBtn.addEventListener('click', async () => {
  safeExecute(async () => {
    const n = parseInt(iterationInput.value);
    const charset = charsetInput.value;

    if (isNaN(n) || n < 1) {
      throw new Error('Please enter a positive integer for n');
    }

    if (charset.length === 0) {
      throw new Error('Charset cannot be empty');
    }

    if (n > 5 && charset.length > 5) {
      throw new Error('Combination too large. Try a smaller n or
charset.');
```

performance)

```

    }

    // Calculate total combinations
    const totalCombinations = Math.pow(charset.length, n);

    // Update UI
    resultContainer.innerHTML = `<p>Generating ${totalCombinations}
combinations...</p>`;
    progressFill.style.width = '0%';
    isGenerating = true;
    generateBtn.disabled = true;
    stopBtn.disabled = false;

    // Generate combinations
    const generator = generateCombinations(charset, n);

    // Process each batch
    let result;
    try {
      for await (result of generator.iterator) {
        if (!isGenerating) break;

        // Update progress bar
        progressFill.style.width = `${result.progress}%`;

        // Display current state (limited to first 100 for
        const displayCombinations = result.combinations.slice(0,
100);

        resultContainer.innerHTML = `<p>Generated
${result.combinations.length} of ${totalCombinations} combinations</p>

```

```

    }

    // Update UI after completion
    isGenerating = false;
    generateBtn.disabled = false;
    stopBtn.disabled = true;

    if (result && result.combinations) {
        return `Generated ${result.combinations.length} combinations
with n=${n} and charset="${charset}"`;
    } else {
        return 'Generation stopped';
    }
}, error => {
    isGenerating = false;
    generateBtn.disabled = false;
    stopBtn.disabled = true;
    resultContainer.innerHTML = `
```

```

        if (arr[i] === query) {
            results.push({
                index: i,
                value: arr[i],
                comparisons
            });
        }
    }

    const endTime = performance.now();

    return {
        found: results.length > 0,
        results,
        time: endTime - startTime
    };
}

// Binary search implementation (requires sorted array)
function binarySearch(arr, query) {
    // Sort array if it contains all numbers or all strings
    const isNumber = arr.every(item => !isNaN(Number(item)));
    const sorted = [...arr];

    // Sort the array
    if (isNumber) {
        sorted.sort((a, b) => Number(a) - Number(b));
    } else {
        sorted.sort();
    }

    const startTime = performance.now();
    let comparisons = 0;

    // Binary search algorithm
    let left = 0;
    let right = sorted.length - 1;
    let foundIndex = -1;

    while (left <= right) {
        const mid = Math.floor((left + right) / 2);
        comparisons++;

        if (sorted[mid] === query) {
            foundIndex = mid;
            break;
        } else if (sorted[mid] < query) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    const endTime = performance.now();

```

```

// Map back to original array if found
let results = [];
if (foundIndex !== -1) {
    const originalIndex = arr.indexOf(sorted[foundIndex]);
    results.push({
        index: originalIndex,
        value: arr[originalIndex],
        comparisons
    });
}

return {
    found: foundIndex !== -1,
    results,
    time: endTime - startTime,
    sortedArray: sorted
};
}

// Fuzzy search implementation
function fuzzySearch(arr, query) {
    const results = [];
    const startTime = performance.now();

    // Search for partial matches
    for (let i = 0; i < arr.length; i++) {
        let comparisons = 1;
        const strValue = String(arr[i]);
        const strQuery = String(query);

        // Check if item contains the query
        if (strValue.includes(strQuery)) {
            results.push({
                index: i,
                value: arr[i],
                comparisons,
                matchType: 'contains'
            });
        }
        // Check if item is "close" to query using Levenshtein distance
        else {
            const distance = levenshteinDistance(strValue, strQuery);
            if (distance <= 2) { // Consider matches with distance <= 2
                results.push({
                    index: i,
                    value: arr[i],
                    comparisons: comparisons + 1,
                    matchType: 'similar',
                    distance
                });
            }
        }
    }
}

```



```

    const endTime = performance.now();

    return {
        found: results.length > 0,
        results,
        time: endTime - startTime
    };
}

// Levenshtein distance for fuzzy matching
function levenshteinDistance(a, b) {
    const matrix = [];

    // Initialize the matrix
    for (let i = 0; i <= b.length; i++) {
        matrix[i] = [i];
    }

    for (let j = 0; j <= a.length; j++) {
        matrix[0][j] = j;
    }

    // Fill the matrix
    for (let i = 1; i <= b.length; i++) {
        for (let j = 1; j <= a.length; j++) {
            if (b.charAt(i - 1) === a.charAt(j - 1)) {
                matrix[i][j] = matrix[i - 1][j - 1];
            } else {
                matrix[i][j] = Math.min(
                    matrix[i - 1][j - 1] + 1, // substitution
                    matrix[i][j - 1] + 1,     // insertion
                    matrix[i - 1][j] + 1       // deletion
                );
            }
        }
    }

    return matrix[b.length][a.length];
}

// Display search results
function displayResults(searchType, result) {
    resultContainer.innerHTML = '';

    const header = document.createElement('div');
    header.innerHTML = `

### ${searchType} Search Results</h3>`; resultContainer.appendChild(header); const timeElement = document.createElement('div'); timeElement.className = 'search-time'; timeElement.textContent = `Search time: ${result.time.toFixed(4)} ms`; resultContainer.appendChild(timeElement); }


```

```

    if (result.sortedArray) {
        const sortedElement = document.createElement('div');
        sortedElement.className = 'sorted-array';
        sortedElement.innerHTML = `<strong>Sorted Array:</strong>
${result.sortedArray.join(', ')}`;
        resultContainer.appendChild(sortedElement);
    }

    if (result.found) {
        const resultsElement = document.createElement('div');
        resultsElement.className = 'search-results';

        result.results.forEach(item => {
            const resultItem = document.createElement('div');
            resultItem.className = 'search-result-item';

            let details = `Found at index ${item.index}: ${item.value}
(${item.comparisons} comparisons)`;
            if (item.matchType) {
                details += ` - ${item.matchType} match`;
                if (item.distance !== undefined) {
                    details += ` (distance: ${item.distance})`;
                }
            }

            resultItem.textContent = details;
            resultsElement.appendChild(resultItem);
        });

        resultContainer.appendChild(resultsElement);
    } else {
        const notFoundElement = document.createElement('div');
        notFoundElement.className = 'not-found';
        notFoundElement.textContent = 'Item not found in the array';
        resultContainer.appendChild(notFoundElement);
    }
}

// Linear search button click handler
linearSearchBtn.addEventListener('click', () => {
    safeExecute(() => {
        const data = parseData();
        const query = searchQuery.value.trim();

        if (data.length === 0) {
            throw new Error('Please enter comma-separated data');
        }

        if (query === '') {
            throw new Error('Please enter a search query');
        }

        const result = linearSearch(data, query);
        displayResults('Linear', result);
    });
});

```

```

        return `Linear search complete: ${result.found ? 'Found' : 'Not
found'}`;
    }, error => {
        resultContainer.innerHTML = `<div class="error">${error.message}
</div>`;
    });
});

// Binary search button click handler
binarySearchBtn.addEventListener('click', () => {
    safeExecute(() => {
        const data = parseData();
        const query = searchQuery.value.trim();

        if (data.length === 0) {
            throw new Error('Please enter comma-separated data');
        }

        if (query === '') {
            throw new Error('Please enter a search query');
        }

        const result = binarySearch(data, query);
        displayResults('Binary', result);

        return `Binary search complete: ${result.found ? 'Found' : 'Not
found'}`;
    }, error => {
        resultContainer.innerHTML = `<div class="error">${error.message}
</div>`;
    });
});

// Fuzzy search button click handler
fuzzySearchBtn.addEventListener('click', () => {
    safeExecute(() => {
        const data = parseData();
        const query = searchQuery.value.trim();

        if (data.length === 0) {
            throw new Error('Please enter comma-separated data');
        }

        if (query === '') {
            throw new Error('Please enter a search query');
        }

        const result = fuzzySearch(data, query);
        displayResults('Fuzzy', result);

        return `Fuzzy search complete: ${result.found ? 'Found' : 'Not
found'}`;
    }, error => {

```

```

        resultContainer.innerHTML = `
```

```

// Display binary operation result
function displayBinaryOperation(valueA, valueB, result, operationSymbol) {
    resultContainer.innerHTML = '';

    // Display inputs
    resultContainer.appendChild(displayBinary('Value A', valueA));
    if (valueB !== undefined) {
        resultContainer.appendChild(displayBinary('Value B', valueB));
    }

    // Display operation
    const operationElement = document.createElement('div');
    operationElement.className = 'binary-operation';
    operationElement.textContent = operationSymbol;
    resultContainer.appendChild(operationElement);

    // Display result
    resultContainer.appendChild(displayBinary('Result', result));
}

// AND operation
andBtn.addEventListener('click', () => {
    safeExecute(() => {
        const valueA = parseInt(binaryInputA.value) || 0;
        const valueB = parseInt(binaryInputB.value) || 0;

        if (valueA < 0 || valueA > 255 || valueB < 0 || valueB > 255) {
            throw new Error('Values must be between 0 and 255');
        }

        const result = valueA & valueB;
        displayBinaryOperation(valueA, valueB, result, 'AND');

        return `Binary AND: ${valueA} & ${valueB} = ${result}`;
    }, error => {
        resultContainer.innerHTML = `<div class="error">${error.message}
</div>`;
    });
});

// OR operation
orBtn.addEventListener('click', () => {
    safeExecute(() => {
        const valueA = parseInt(binaryInputA.value) || 0;
        const valueB = parseInt(binaryInputB.value) || 0;

        if (valueA < 0 || valueA > 255 || valueB < 0 || valueB > 255) {
            throw new Error('Values must be between 0 and 255');
        }

        const result = valueA | valueB;
        displayBinaryOperation(valueA, valueB, result, 'OR');
    });
});

```

```

        return `Binary OR: ${valueA} | ${valueB} = ${result}`;
    }, error => {
        resultContainer.innerHTML = `<div class="error">${error.message}
</div>`;
    });
});

// XOR operation
xorBtn.addEventListener('click', () => {
    safeExecute(() => {
        const valueA = parseInt(binaryInputA.value) || 0;
        const valueB = parseInt(binaryInputB.value) || 0;

        if (valueA < 0 || valueA > 255 || valueB < 0 || valueB > 255) {
            throw new Error('Values must be between 0 and 255');
        }

        const result = valueA ^ valueB;
        displayBinaryOperation(valueA, valueB, result, 'XOR');

        return `Binary XOR: ${valueA} ^ ${valueB} = ${result}`;
    }, error => {
        resultContainer.innerHTML = `<div class="error">${error.message}
</div>`;
    });
});

// NOT operation
notBtn.addEventListener('click', () => {
    safeExecute(() => {
        const valueA = parseInt(binaryInputA.value) || 0;

        if (valueA < 0 || valueA > 255) {
            throw new Error('Value must be between 0 and 255');
        }

        const result = ~valueA & 0xFF; // Mask to 8 bits
        displayBinaryOperation(valueA, undefined, result, 'NOT');

        return `Binary NOT: ~${valueA} = ${result}`;
    }, error => {
        resultContainer.innerHTML = `<div class="error">${error.message}
</div>`;
    });
});

// Shift Left operation
shiftBtn.addEventListener('click', () => {
    safeExecute(() => {
        const valueA = parseInt(binaryInputA.value) || 0;
        const shiftAmount = parseInt(binaryInputB.value) || 0;

        if (valueA < 0 || valueA > 255) {
            throw new Error('Value must be between 0 and 255');

```

```

    }

    if (shiftAmount < 0 || shiftAmount > 7) {
        throw new Error('Shift amount must be between 0 and 7');
    }

    const result = (valueA << shiftAmount) & 0xFF; // Mask to 8 bits
    displayBinaryOperation(valueA, shiftAmount, result, `SHIFT LEFT
(${shiftAmount} bits)`);

    return `Binary Shift Left: ${valueA} << ${shiftAmount} =
${result}`;
}, error => {
    resultContainer.innerHTML = `<div class="error">${error.message}
</div>`;
});
});
}

// ===== 3. SYSTEM OPERATIONS =====

// ===== 3.1 File System Operations =====
function setupFileOperations() {
    const fileContent = document.getElementById('file-content');
    const fileName = document.getElementById('file-name');
    const saveFileBtn = document.getElementById('save-file');
    const loadFileBtn = document.getElementById('load-file');
    const fileUpload = document.getElementById('file-upload');
    const createDirBtn = document.getElementById('create-directory');
    const createZipBtn = document.getElementById('create-zip');
    const fileTree = document.getElementById('file-tree');

    console.log("Setting up file operations...");

    // Initialize virtual file system if it doesn't exist
    if (!window.virtualFS) {
        console.log("Creating new virtual file system");
        window.virtualFS = {
            root: {
                type: 'directory',
                name: 'root',
                children: {}
            }
        };
    } else {
        console.log("Using existing virtual file system", window.virtualFS);
    }

    // Load file button click handler
    loadFileBtn.addEventListener('click', () => {
        fileUpload.click();
    });

    // File upload change handler

```

```

fileUpload.addEventListener('change', (event) => {
  safeExecute(() => {
    const file = event.target.files[0];

    if (!file) return;

    const reader = new FileReader();

    reader.onload = (e) => {
      const content = e.target.result;
      fileContent.value = content;
      fileName.value = file.name;

      // Add to virtual filesystem
      addFileToVFS(file.name, content);
      renderFileTree();
    };

    reader.onerror = () => {
      throw new Error('Error reading file');
    };

    reader.readAsText(file);
  }, error => {
    showError(`File loading error: ${error.message}`);
  });
});

// Save file button click handler
saveFileBtn.addEventListener('click', () => {
  safeExecute(() => {
    const content = fileContent.value;
    const name = fileName.value.trim();

    if (!name) {
      throw new Error('File name cannot be empty');
    }

    // Save file to browser (download)
    const blob = new Blob([content], { type: 'text/plain' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = name;
    a.click();

    // Clean up
    setTimeout(() => URL.revokeObjectURL(url), 1000);

    // Add to virtual filesystem
    addFileToVFS(name, content);
    renderFileTree();

    return `File "${name}" saved successfully`;
  });
});

```



```

    }, error => {
        showError(`File saving error: ${error.message}`);
    });
});

// Create directory button click handler
createDirBtn.addEventListener('click', () => {
    safeExecute(() => {
        const dirName = prompt('Enter directory name:');

        if (!dirName) return;

        if (dirName.trim() === '') {
            throw new Error('Directory name cannot be empty');
        }

        addDirectoryToVFS(dirName);
        renderFileTree();

        return `Directory "${dirName}" created`;
    }, error => {
        showError(`Directory creation error: ${error.message}`);
    });
});

// Create ZIP button click handler (using JSZip)
createZipBtn.addEventListener('click', () => {
    safeExecute(async () => {
        const zip = new JSZip();

        // Add all files from virtual filesystem
        addFilesToZip(zip, virtualFS.root);

        // Generate ZIP blob
        const blob = await zip.generateAsync({ type: 'blob' });

        // Download ZIP file
        const url = URL.createObjectURL(blob);
        const a = document.createElement('a');
        a.href = url;
        a.download = 'files.zip';
        a.click();

        // Clean up
        setTimeout(() => URL.revokeObjectURL(url), 1000);

        return 'ZIP file created and downloaded';
    }, error => {
        showError(`ZIP creation error: ${error.message}`);
    });
});

// Helper function to add files to ZIP
function addFilesToZip(zip, directory, path = '') {

```

```

    for (const name in directory.children) {
        const item = directory.children[name];
        const itemPath = path ? `${path}/${name}` : name;

        if (item.type === 'file') {
            zip.file(itemPath, item.content);
        } else if (item.type === 'directory') {
            const folder = zip.folder(itemPath);
            addFilesToZip(zip, item, itemPath);
        }
    }
}

// Add file to virtual filesystem
function addFileToVFS(filePath, content) {
    const parts = filePath.split('/');
    const fileName = parts.pop();
    let currentDir = virtualFS.root;

    // Create directories in path if needed
    for (const part of parts) {
        if (!part) continue;

        if (!currentDir.children[part]) {
            currentDir.children[part] = {
                type: 'directory',
                name: part,
                children: {}
            };
        }

        currentDir = currentDir.children[part];
    }

    // Add file
    currentDir.children[fileName] = {
        type: 'file',
        name: fileName,
        content: content
    };
}

// Add directory to virtual filesystem
function addDirectoryToVFS(dirPath) {
    const parts = dirPath.split('/');
    let currentDir = virtualFS.root;

    // Create directories in path
    for (const part of parts) {
        if (!part) continue;

        if (!currentDir.children[part]) {
            currentDir.children[part] = {
                type: 'directory',

```

```

        name: part,
        children: {}
    };
}

currentDir = currentDir.children[part];
}
}

// Render file tree
function renderFileTree() {
    fileTree.innerHTML = '';

    function createTreeItem(item, parent) {
        const li = document.createElement('li');

        if (item.type === 'directory') {
            const dirElement = document.createElement('div');
            dirElement.className = 'directory-item';
            dirElement.innerHTML = `<span class="file-icon">📁</span>
${item.name}`;

            li.appendChild(dirElement);

            const childList = document.createElement('ul');
            childList.className = 'directory-children';

            // Add children
            const children = Object.values(item.children)
                .sort((a, b) => {
                    // Sort directories first, then files
                    if (a.type !== b.type) {
                        return a.type === 'directory' ? -1 : 1;
                    }
                    return a.name.localeCompare(b.name);
                });

            for (const child of children) {
                createTreeItem(child, childList);
            }

            li.appendChild(childList);

            // Toggle expand/collapse
            dirElement.addEventListener('click', () => {
                childList.style.display = childList.style.display ===
'none' ? 'block' : 'none';
            });
        } else {
            li.className = 'file-item';
            li.innerHTML = `<span class="file-icon">📄</span> <span
class="file-name">${item.name}</span>`;

            // Click to load file
            li.addEventListener('click', () => {

```

```

        fileContent.value = item.content;
        fileName.value = item.name;
    });
}

parent.appendChild(li);
}

// Create the root node
createTreeItem(virtualFS.root, fileTree);
}

// Initialize with empty file tree
renderFileTree();
}

// ===== 3.2 Memory Operations =====
function setupMemoryOperations() {
    const memoryFile = document.getElementById('memory-file');
    const memorySlot = document.getElementById('memory-slot');
    const memoryContent = document.getElementById('memory-content');
    const assignSlotBtn = document.getElementById('assign-slot');
    const readSlotBtn = document.getElementById('read-slot');
    const clearAllSlotsBtn = document.getElementById('clear-all-slots');
    const memoryDisplay = document.getElementById('memory-display');

    // Memory manager class
    class MemorySlotManager {
        constructor() {
            this.memorySlots = {};
        }

        assignSlot(fileNumber, slotNumber, value) {
            if (!fileNumber || !slotNumber || !value) {
                return "Error: Invalid input parameters.";
            }

            if (this.valueExists(value)) {
                return "Error: The value already exists in another memory slot.";
            }

            if (!this.memorySlots[fileNumber]) {
                this.memorySlots[fileNumber] = {};
            }

            this.memorySlots[fileNumber][slotNumber] = value;
            return `Assigned value to slot ${slotNumber} in file ${fileNumber}.`;
        }

        readSlot(fileNumber, slotNumber) {
            if (this.memorySlots[fileNumber] && this.memorySlots[fileNumber][slotNumber] !== undefined) {

```

```

        return this.memorySlots[fileNumber][slotNumber];
    }
    return `Slot ${slotNumber} not found in file ${fileNumber}.`;
}

valueExists(value) {
    const valueLower = value.toLowerCase();

    for (const file in this.memorySlots) {
        for (const slot in this.memorySlots[file]) {
            if (this.memorySlots[file][slot].toLowerCase() ===
valueLower) {
                return true;
            }
        }
    }

    return false;
}

clearAll() {
    this.memorySlots = {};
    return 'All memory slots cleared';
}
}

// Initialize memory slot manager
const memoryManager = new MemorySlotManager();

// Update memory display
function updateMemoryDisplay() {
    memoryDisplay.innerHTML = '';

    const memorySlots = memoryManager.memorySlots;

    if (Object.keys(memorySlots).length === 0) {
        memoryDisplay.innerHTML = `
                <div style="text-align: center; color: #888; margin-top:
20px;">
                    No memory slots available.
                    <br><br>
                    Use the controls to add memory slots.
                </div>
            `;
        return;
    }

    // Sort file numbers numerically if possible
    const sortedFiles = Object.keys(memorySlots).sort((a, b) => {
        if (/^\d+$/.test(a) && /^\d+$/.test(b)) {
            return parseInt(a) - parseInt(b);
        }
        return a.localeCompare(b);
    });

```

```

for (const fileNumber of sortedFiles) {
  const slots = memorySlots[fileNumber];
  const slotCount = Object.keys(slots).length;

  const fileElement = document.createElement('div');
  fileElement.className = 'memory-file';

  const fileTitle = document.createElement('div');
  fileTitle.className = 'memory-file-title';
  fileTitle.textContent = `FILE ${fileNumber} (${slotCount} slots)`;
  fileElement.appendChild(fileTitle);

  // Sort slot numbers numerically if possible
  const sortedSlots = Object.keys(slots).sort((a, b) => {
    if (/^\d+$/.test(a) && /^\d+$/.test(b)) {
      return parseInt(a) - parseInt(b);
    }
    return a.localeCompare(b);
  });

  for (const slotNumber of sortedSlots) {
    const value = slots[slotNumber];

    // Truncate long values for display
    let displayValue = value;
    if (displayValue.length > 100) {
      displayValue = displayValue.substring(0, 97) + '...';
    }

    const slotElement = document.createElement('div');
    slotElement.className = 'memory-slot';

    const slotTitle = document.createElement('div');
    slotTitle.className = 'memory-slot-title';
    slotTitle.textContent = `Slot ${slotNumber}`;

    const slotValue = document.createElement('div');
    slotValue.className = 'memory-slot-value';
    slotValue.textContent = displayValue;

    slotElement.appendChild(slotTitle);
    slotElement.appendChild(slotValue);
    fileElement.appendChild(slotElement);
  }

  memoryDisplay.appendChild(fileElement);
}

// Assign slot button click handler
assignSlotBtn.addEventListener('click', () => {
  safeExecute(() => {
    const fileNum = memoryFile.value.trim();

```

```

        const slotNum = memorySlot.value.trim();
        const content = memoryContent.value;

        if (!fileNum) {
            throw new Error('File number cannot be empty');
        }

        if (!slotNum) {
            throw new Error('Slot number cannot be empty');
        }

        if (!content) {
            throw new Error('Content cannot be empty');
        }

        const result = memoryManager.assignSlot(fileNum, slotNum,
content);

        if (result.startsWith('Error')) {
            throw new Error(result.substring(7));
        }

        updateMemoryDisplay();

        return result;
    }, error => {
        showError(`Memory error: ${error.message}`);
    });
});

// Read slot button click handler
readSlotBtn.addEventListener('click', () => {
    safeExecute(() => {
        const fileNum = memoryFile.value.trim();
        const slotNum = memorySlot.value.trim();

        if (!fileNum) {
            throw new Error('File number cannot be empty');
        }

        if (!slotNum) {
            throw new Error('Slot number cannot be empty');
        }

        const result = memoryManager.readSlot(fileNum, slotNum);

        if (result.startsWith('Slot')) {
            throw new Error(result);
        }

        memoryContent.value = result;

        return `Read value from slot ${slotNum} in file ${fileNum}`;
    }, error => {

```

```

        showError(`Memory error: ${error.message}`);
    });
});

// Clear all slots button click handler
clearAllSlotsBtn.addEventListener('click', () => {
    if (confirm('Are you sure you want to clear all memory slots?')) {
        memoryManager.clearAll();
        updateMemoryDisplay();
        memoryContent.value = '';
    }
});

// Initialize memory display
updateMemoryDisplay();
}

// ===== 3.3 Process Operations =====
function setupProcessOperations() {
    const processName = document.getElementById('process-name');
    const processDuration = document.getElementById('process-duration');
    const startProcessBtn = document.getElementById('start-process');
    const stopAllProcessesBtn = document.getElementById('stop-all-processes');
    const processList = document.getElementById('process-list');

    // Process manager class
    class ProcessManager {
        constructor() {
            this.processes = new Map();
            this.nextProcessId = 1;
        }

        startProcess(name, duration, progressCallback, completeCallback) {
            const processId = this.nextProcessId++;
            const startTime = performance.now();
            const endTime = startTime + duration;

            // Create process object
            const process = {
                id: processId,
                name,
                duration,
                startTime,
                endTime,
                progress: 0,
                status: 'running',
                intervalId: null
            };

            // Start progress updates
            process.intervalId = setInterval(() => {
                const now = performance.now();
                const elapsed = now - startTime;

```



```

        if (elapsed >= duration) {
            // Process complete
            process.progress = 100;
            process.status = 'completed';
            clearInterval(process.intervalId);

            // Remove process after a delay
            setTimeout(() => {
                this.processes.delete(processId);
                if (completeCallback) {
                    completeCallback(process);
                }
            }, 1000);
        } else {
            // Update progress
            process.progress = (elapsed / duration) * 100;
        }

        if (progressCallback) {
            progressCallback(process);
        }
    }, 50);

    // Add to process list
    this.processes.set(processId, process);

    return processId;
}

stopProcess(processId) {
    const process = this.processes.get(processId);

    if (process) {
        clearInterval(process.intervalId);
        process.status = 'stopped';
        this.processes.delete(processId);
        return true;
    }

    return false;
}

stopAllProcesses() {
    for (const [processId, process] of this.processes.entries()) {
        clearInterval(process.intervalId);
        process.status = 'stopped';
    }

    this.processes.clear();
    return true;
}
}

```

// Create process manager

```

const processManager = new ProcessManager();

// Create process element
function createProcessElement(process) {
    const processElement = document.createElement('div');
    processElement.className = 'process';
    processElement.id = `process-${process.id}`;

    const processHeader = document.createElement('div');
    processHeader.className = 'process-header';

    const processNameElement = document.createElement('div');
    processNameElement.className = 'process-name';
    processNameElement.textContent = process.name;

    const processClose = document.createElement('span');
    processClose.className = 'process-close';
    processClose.textContent = 'x';
    processClose.title = 'Stop process';
    processClose.addEventListener('click', () => {
        processManager.stopProcess(process.id);
        processElement.remove();
    });

    processHeader.appendChild(processNameElement);
    processHeader.appendChild(processClose);

    const processProgress = document.createElement('div');
    processProgress.className = 'process-progress';

    const processBar = document.createElement('div');
    processBar.className = 'process-bar';
    processBar.style.width = `${process.progress}%`;

    processProgress.appendChild(processBar);

    const processStatus = document.createElement('div');
    processStatus.className = 'process-status';
    processStatus.textContent = `Status: ${process.status} - Progress:
    ${Math.round(process.progress)}%`;

    processElement.appendChild(processHeader);
    processElement.appendChild(processProgress);
    processElement.appendChild(processStatus);

    return processElement;
}

// Update process element
function updateProcessElement(process) {
    const processElement =
document.getElementById(`process-${process.id}`);

    if (processElement) {

```

```

        const processBar = processElement.querySelector('.process-bar');
        const processStatus = processElement.querySelector('.process-
status');

        processBar.style.width = `${process.progress}%`;
        processStatus.textContent = `Status: ${process.status} - Progress:
${Math.round(process.progress)}%`;

        if (process.status === 'completed') {
            processElement.classList.add('completed');
        }
    }
}

// Start process button click handler
startProcessBtn.addEventListener('click', () => {
    safeExecute(() => {
        const name = processName.value.trim();
        const duration = parseInt(processDuration.value);

        if (!name) {
            throw new Error('Process name cannot be empty');
        }

        if (isNaN(duration) || duration < 100 || duration > 10000) {
            throw new Error('Duration must be between 100 and 10000 ms');
        }

        // Start the process
        const processId = processManager.startProcess(
            name,
            duration,
            updateProcessElement,
            process => {
                // Process completed callback
                const processElement =
document.getElementById(`process-${process.id}`);
                if (processElement) {
                    setTimeout(() => {
                        processElement.classList.add('fade-out');
                        setTimeout(() => {
                            processElement.remove();
                        }, 500);
                    }, 1000);
                }
            }
        );

        // Create and add process element
        const process = processManager.processes.get(processId);
        const processElement = createProcessElement(process);
        processList.appendChild(processElement);

        // Clear input

```

```

        processName.value = '';

        return `Process "${name}" started`;
    }, error => {
        showError(`Process error: ${error.message}`);
    });
});

// Stop all processes button click handler
stopAllProcessesBtn.addEventListener('click', () => {
    processManager.stopAllProcesses();
    processList.innerHTML = '';
});
}

// ===== 4. CRUD OPERATIONS =====

// Simple in-memory database
const database = {
    records: new Map(),

    create(record) {
        if (!record.id) {
            throw new Error('Record must have an ID');
        }

        if (this.records.has(record.id)) {
            throw new Error(`Record with ID "${record.id}" already exists`);
        }

        this.records.set(record.id, { ...record, createdAt: new Date() });
        return true;
    },

    read(id) {
        if (!this.records.has(id)) {
            throw new Error(`Record with ID "${id}" not found`);
        }

        return { ...this.records.get(id) };
    },

    readAll() {
        return Array.from(this.records.values()).map(record => ({ ...record
    }));
    },

    update(id, updates) {
        if (!this.records.has(id)) {
            throw new Error(`Record with ID "${id}" not found`);
        }

        const record = this.records.get(id);
        const updatedRecord = { ...record, ...updates, updatedAt: new Date()

```

```

};

    this.records.set(id, updatedRecord);

    return updatedRecord;
},

delete(id) {
    if (!this.records.has(id)) {
        throw new Error(`Record with ID "${id}" not found`);
    }

    return this.records.delete(id);
},

deleteAll() {
    this.records.clear();
    return true;
},

query(criteria) {
    return Array.from(this.records.values())
        .filter(record => {
            for (const key in criteria) {
                if (record[key] !== criteria[key]) {
                    return false;
                }
            }
            return true;
        })
        .map(record => ({ ...record }));
}

};

// ===== 4.1 Create Operations =====
function setupCreateOperations() {
    const recordId = document.getElementById('record-id');
    const recordName = document.getElementById('record-name');
    const recordData = document.getElementById('record-data');
    const createRecordBtn = document.getElementById('create-record');
    const databaseDisplay = document.getElementById('database-display');

    // Create record button click handler
    createRecordBtn.addEventListener('click', () => {
        safeExecute(() => {
            const id = recordId.value.trim();
            const name = recordName.value.trim();
            const data = recordData.value.trim();

            if (!id) {
                throw new Error('Record ID cannot be empty');
            }

            if (!name) {
                throw new Error('Record name cannot be empty');
            }
        });
    });
}

```

```

    }

    // Create the record
    database.create({
        id,
        name,
        data
    });

    // Clear inputs
    recordId.value = '';
    recordName.value = '';
    recordData.value = '';

    // Update the display
    updateDatabaseDisplay();

    return `Record "${id}" created successfully`;
}, error => {
    showError(`Create error: ${error.message}`);
});
});

// Update database display
function updateDatabaseDisplay() {
    databaseDisplay.innerHTML = '';

    const records = database.readAll();

    if (records.length === 0) {
        databaseDisplay.innerHTML = `
            <div style="text-align: center; color: #888; margin-top:
20px;">
                No records in the database.
                <br><br>
                Use the form to create records.
            </div>
        `;
        return;
    }

    // Sort records by ID
    records.sort((a, b) => a.id.localeCompare(b.id));

    for (const record of records) {
        const recordElement = document.createElement('div');
        recordElement.className = 'database-record';

        const recordHeader = document.createElement('div');
        recordHeader.className = 'record-header';

        const recordIdElement = document.createElement('div');
        recordIdElement.className = 'record-id';
        recordIdElement.textContent = record.id;
    }
}

```

```

    const recordControls = document.createElement('div');
    recordControls.className = 'record-controls';

    recordHeader.appendChild(recordIdElement);
    recordHeader.appendChild(recordControls);

    const recordContent = document.createElement('div');
    recordContent.className = 'record-content';
    recordContent.innerHTML = `
        <div><strong>Name:</strong> ${record.name}</div>
        <div><strong>Data:</strong> ${record.data || '<empty>'}</div>
        <div><strong>Created:</strong>
    ${record.createdAt.toLocaleString()}</div>
        ${record.updatedAt ? `<div><strong>Updated:</strong>
    ${record.updatedAt.toLocaleString()}</div>` : ''}
    `;

    recordElement.appendChild(recordHeader);
    recordElement.appendChild(recordContent);

    databaseDisplay.appendChild(recordElement);
}
}

// Initialize the database display
updateDatabaseDisplay();

// Export for other functions
return { updateDatabaseDisplay };
}

// ===== 4.2 Read Operations =====
function setupReadOperations(displayUpdater) {
    const readId = document.getElementById('read-id');
    const readRecordBtn = document.getElementById('read-record');
    const readAllRecordsBtn = document.getElementById('read-all-records');
    const queryRecordsBtn = document.getElementById('query-records');
    const readResult = document.getElementById('read-result');

    // Read record button click handler
    readRecordBtn.addEventListener('click', () => {
        safeExecute(() => {
            const id = readId.value.trim();

            if (!id) {
                throw new Error('Record ID cannot be empty');
            }

            const record = database.read(id);

            // Display the record
            readResult.innerHTML = '';

```

```

const recordElement = document.createElement('div');
recordElement.className = 'database-record';

const recordHeader = document.createElement('div');
recordHeader.className = 'record-header';

const recordIdElement = document.createElement('div');
recordIdElement.className = 'record-id';
recordIdElement.textContent = record.id;

recordHeader.appendChild(recordIdElement);

const recordContent = document.createElement('div');
recordContent.className = 'record-content';
recordContent.innerHTML = `
    <div><strong>Name:</strong> ${record.name}</div>
    <div><strong>Data:</strong> ${record.data || '<empty>'}</div>
    <div><strong>Created:</strong>
    ${record.createdAt.toLocaleString()}</div>
    ${record.updatedAt ? `<div><strong>Updated:</strong>
    ${record.updatedAt.toLocaleString()}</div>` : ''}
`;

recordElement.appendChild(recordHeader);
recordElement.appendChild(recordContent);

readResult.appendChild(recordElement);

return `Record "${id}" read successfully`;
}, error => {
    readResult.innerHTML = `<div class="error">${error.message}
</div>`;
});
});

// Read all records button click handler
readAllRecordsBtn.addEventListener('click', () => {
    safeExecute(() => {
        const records = database.readAll();

        if (records.length === 0) {
            readResult.innerHTML = `
                <div style="text-align: center; color: #888; margin-top:
                20px;">
                    No records in the database.
                </div>
            `;
            return `No records found in the database`;
        }

        // Display all records
        readResult.innerHTML = '';

        // Sort records by ID

```



```

records.sort((a, b) => a.id.localeCompare(b.id));

for (const record of records) {
  const recordElement = document.createElement('div');
  recordElement.className = 'database-record';

  const recordHeader = document.createElement('div');
  recordHeader.className = 'record-header';

  const recordIdElement = document.createElement('div');
  recordIdElement.className = 'record-id';
  recordIdElement.textContent = record.id;

  recordHeader.appendChild(recordIdElement);

  const recordContent = document.createElement('div');
  recordContent.className = 'record-content';
  recordContent.innerHTML = `
    <div><strong>Name:</strong> ${record.name}</div>
    <div><strong>Data:</strong> ${record.data || '<empty>'}
  </div>
    <div><strong>Created:</strong>
    ${record.createdAt.toLocaleString()}</div>
    ${record.updatedAt ? `<div><strong>Updated:</strong>
    ${record.updatedAt.toLocaleString()}</div>` : ''}
  `;

  recordElement.appendChild(recordHeader);
  recordElement.appendChild(recordContent);

  readResult.appendChild(recordElement);
}

return `${records.length} records read successfully`;
}, error => {
  readResult.innerHTML = `<div class="error">${error.message}
</div>`;
});
});

// Query records button click handler
queryRecordsBtn.addEventListener('click', () => {
  safeExecute(() => {
    // Create a query dialog
    const queryValue = prompt('Enter search term to query records by
name:');

    if (queryValue === null) return; // User cancelled

    const query = queryValue.trim();

    if (!query) {
      throw new Error('Search term cannot be empty');
    }
  });
});

```

```

// Simple case-insensitive name search
const records = database.readAll().filter(record =>
  record.name.toLowerCase().includes(query.toLowerCase())
);

if (records.length === 0) {
  readResult.innerHTML = `
    <div style="text-align: center; color: #888; margin-top:
20px;">
      No records found matching the query "${query}".
    </div>
  `;
  return `No records found matching the query "${query}"`;
}

// Display all matching records
readResult.innerHTML = '';

// Sort records by ID
records.sort((a, b) => a.id.localeCompare(b.id));

for (const record of records) {
  const recordElement = document.createElement('div');
  recordElement.className = 'database-record';

  const recordHeader = document.createElement('div');
  recordHeader.className = 'record-header';

  const recordIdElement = document.createElement('div');
  recordIdElement.className = 'record-id';
  recordIdElement.textContent = record.id;

  recordHeader.appendChild(recordIdElement);

  const recordContent = document.createElement('div');
  recordContent.className = 'record-content';
  recordContent.innerHTML = `
    <div><strong>Name:</strong> ${record.name}</div>
    <div><strong>Data:</strong> ${record.data || '<empty>'}
</div>
    <div><strong>Created:</strong>
${record.createdAt.toLocaleString()}</div>
    ${record.updatedAt ? `<div><strong>Updated:</strong>
${record.updatedAt.toLocaleString()}</div>` : ''}
  `;

  recordElement.appendChild(recordHeader);
  recordElement.appendChild(recordContent);

  readResult.appendChild(recordElement);
}

return `${records.length} records found matching the query

```

```

`${query}`";
    }, error => {
        readResult.innerHTML = `
```

```

        const recordContent = document.createElement('div');
        recordContent.className = 'record-content';
        recordContent.innerHTML = `
            <div><strong>Name:</strong> ${updatedRecord.name}</div>
            <div><strong>Data:</strong> ${updatedRecord.data || '<empty>'}
        </div>
            <div><strong>Created:</strong>
        ${updatedRecord.createdAt.toLocaleString()}</div>
            <div><strong>Updated:</strong>
        ${updatedRecord.updatedAt.toLocaleString()}</div>
        `;

        recordElement.appendChild(recordHeader);
        recordElement.appendChild(recordContent);

        updateResult.appendChild(recordElement);

        // Clear inputs
        updateId.value = '';
        updateValue.value = '';

        // Update the database display
        if (displayUpdater) displayUpdater.updateDatabaseDisplay();

        return `Record "${id}" updated successfully`;
    }, error => {
        updateResult.innerHTML = `<div class="error">${error.message}
    </div>`;
    });
});
}

// ===== 4.4 Delete Operations =====
function setupDeleteOperations(displayUpdater) {
    const deleteId = document.getElementById('delete-id');
    const deleteRecordBtn = document.getElementById('delete-record');
    const deleteAllRecordsBtn = document.getElementById('delete-all-records');
    const deleteResult = document.getElementById('delete-result');

    // Delete record button click handler
    deleteRecordBtn.addEventListener('click', () => {
        safeExecute(() => {
            const id = deleteId.value.trim();

            if (!id) {
                throw new Error('Record ID cannot be empty');
            }

            // Delete the record
            const result = database.delete(id);

            if (!result) {
                throw new Error(`Failed to delete record "${id}"`);
            }
        });
    });
}

```

```

        // Clear input
        deleteId.value = '';

        // Update the database display
        if (displayUpdater) displayUpdater.updateDatabaseDisplay();

        deleteResult.innerHTML = `
```

```

let serializedData = '';
let originalObject = null;

// Serialize button click handler
serializeBtn.addEventListener('click', () => {
  safeExecute(() => {
    const input = serializeInput.value.trim();
    const format = serializationFormat.value;

    if (!input) {
      throw new Error('Input cannot be empty');
    }

    // Parse the input as JSON to get an object
    try {
      originalObject = JSON.parse(input);
    } catch (e) {
      throw new Error('Invalid JSON input. Please check your
syntax.');
```

```

    }

    // Serialize the object according to the selected format
    serializedData = serializeObject(originalObject, format);

    // Display the serialized data
    resultContainer.innerHTML = '';

    const resultHeader = document.createElement('div');
    resultHeader.className = 'serialization-header';
    resultHeader.innerHTML = `<strong>Serialized Data (${format}):
</strong>`;

    resultContainer.appendChild(resultHeader);

    const resultContent = document.createElement('pre');
    resultContent.className = 'serialization-content';
    resultContent.textContent = serializedData;
    resultContainer.appendChild(resultContent);

    return `Data serialized to ${format} format`;
  }, error => {
    resultContainer.innerHTML = `<div class="error">${error.message}
</div>`;
  });
});

// Deserialize button click handler
deserializeBtn.addEventListener('click', () => {
  safeExecute(() => {
    const format = serializationFormat.value;

    if (!serializedData) {
      throw new Error('No serialized data to deserialize');
    }
  });
});

```

```

// Deserialize the data according to the selected format
const deserializedObject = deserializeData(serializedData,
format);

// Display the deserialized object
resultContainer.innerHTML = '';

const resultHeader = document.createElement('div');
resultHeader.className = 'serialization-header';
resultHeader.innerHTML = `<strong>Deserialized Object:</strong>`;
resultContainer.appendChild(resultHeader);

const resultContent = document.createElement('pre');
resultContent.className = 'serialization-content';
resultContent.textContent = JSON.stringify(deserializedObject,
null, 2);
resultContainer.appendChild(resultContent);

// Display comparison
const comparisonHeader = document.createElement('div');
comparisonHeader.className = 'serialization-header mt-20';
comparisonHeader.innerHTML = `<strong>Comparison with Original:
</strong>`;
resultContainer.appendChild(comparisonHeader);

const isEqual = compareObjects(originalObject,
deserializedObject);

const comparisonResult = document.createElement('div');
comparisonResult.className = isEqual ? 'status-success' : 'status-
error';
comparisonResult.textContent = isEqual
    ? 'Objects are identical. Serialization and deserialization
completed successfully.'
    : 'Objects are different. Some data may have been lost in the
process.';
resultContainer.appendChild(comparisonResult);

return `Data deserialized from ${format} format`;
}, error => {
    resultContainer.innerHTML = `<div class="error">${error.message}
</div>`;
});
});

// Serialize an object to the specified format
function serializeObject(obj, format) {
    switch (format) {
        case 'json':
            return JSON.stringify(obj, null, 2);

        case 'xml':
            return objectToXML(obj);
    }
}

```

```

        case 'csv':
            return objectToCSV(obj);

        default:
            throw new Error(`Unsupported format: ${format}`);
    }
}

// Deserialize data from the specified format
function deserializeData(data, format) {
    switch (format) {
        case 'json':
            return JSON.parse(data);

        case 'xml':
            return xmlToObject(data);

        case 'csv':
            return csvToObject(data);

        default:
            throw new Error(`Unsupported format: ${format}`);
    }
}

// Convert object to XML
function objectToXML(obj) {
    function addIndent(level) {
        return ' '.repeat(level * 2);
    }

    function objectToXMLRecursive(obj, name, level = 0) {
        let xml = `${addIndent(level)}<${name}>`;

        if (obj === null) {
            return `${xml}null</${name}>`;
        }

        if (typeof obj !== 'object') {
            return `${xml}${String(obj)}</${name}>`;
        }

        xml += '\n';

        if (Array.isArray(obj)) {
            for (let i = 0; i < obj.length; i++) {
                xml += objectToXMLRecursive(obj[i], 'item', level + 1) +
                    '\n';
            }
        } else {
            for (const key in obj) {
                if (obj.hasOwnProperty(key)) {
                    xml += objectToXMLRecursive(obj[key], key, level + 1)
                        + '\n';
                }
            }
        }
    }
}

```



```

    }
  }
}

xml += `${addIndent(level)}</${name}>`;
return xml;
}

return objectToXMLRecursive(obj, 'root');
}

// Convert XML to object (simplified)
function xmlToObject(xml) {
  // This is a very simplified XML parser
  // In a real-world scenario, you would use a proper XML parser

  // Extract content within root tags
  const rootMatch = xml.match(/<root>([\s\S]*)<\/root>/);
  if (!rootMatch) {
    throw new Error('Invalid XML: root tag not found');
  }

  const rootContent = rootMatch[1];

  // Parse key-value pairs
  const result = {};
  const keyPattern = /<(\w+)>([\s\S]*?)<\/\1>/g;
  let match;

  while ((match = keyPattern.exec(rootContent)) !== null) {
    const key = match[1];
    const value = match[2].trim();

    // Try to parse nested objects
    if (value.includes('<')) {
      // Check if it's an array of items
      if (value.includes('<item>')) {
        result[key] = [];
        const itemPattern = /<item>([\s\S]*?)<\/item>/g;
        let itemMatch;

        while ((itemMatch = itemPattern.exec(value)) !== null) {
          const itemValue = itemMatch[1].trim();

          if (itemValue.includes('<')) {
            // Try to parse as object
            try {
              const dummyXml = `<root>${itemValue}<\/root>`;
              result[key].push(xmlToObject(dummyXml));
            } catch (e) {
              result[key].push(itemValue);
            }
          } else {
            // Try to parse as number or boolean

```

```

        result[key].push(parseValue(itemValue));
    }
}
} else {
    // Try to parse as object
    try {
        const dummyXml = `${value}</root>`;
        result[key] = xmlToObject(dummyXml);
    } catch (e) {
        result[key] = value;
    }
}
} else {
    // Parse as primitive value
    result[key] = parseValue(value);
}
}

return result;
}

// Convert object to CSV (for flat objects or arrays of flat objects)
function objectToCSV(obj) {
    if (Array.isArray(obj)) {
        // Array of objects - create CSV with headers
        if (obj.length === 0) {
            return '';
        }

        // Get headers from the first object
        const headers = Object.keys(obj[0]);
        let csv = headers.join(',') + '\n';

        // Add rows
        for (const item of obj) {
            const row = headers.map(header => {
                const value = item[header];
                return escapeCSV(value);
            }).join(',');

            csv += row + '\n';
        }

        return csv;
    } else {
        // Single object - create key-value CSV
        let csv = 'key,value\n';

        for (const key in obj) {
            if (obj.hasOwnProperty(key)) {
                const value = obj[key];
                csv += `${escapeCSV(key)},${escapeCSV(value)}\n`;
            }
        }
    }
}

```

```

        return csv;
    }
}

// Convert CSV to object
function csvToObject(csv) {
    const lines = csv.split('\n').filter(line => line.trim() !== '');
    if (lines.length < 2) {
        return {};
    }

    const headers = lines[0].split(',');

    // Check if it's key-value format
    if (headers.length === 2 && headers[0] === 'key' && headers[1] ===
'value') {
        const result = {};

        for (let i = 1; i < lines.length; i++) {
            const [key, value] = parseCSVLine(lines[i]);
            result[key] = parseValue(value);
        }

        return result;
    } else {
        // Array of objects format
        const result = [];

        for (let i = 1; i < lines.length; i++) {
            const values = parseCSVLine(lines[i]);
            const item = {};

            for (let j = 0; j < headers.length; j++) {
                item[headers[j]] = parseValue(values[j] || '');
            }

            result.push(item);
        }

        return result;
    }
}

// Parse a CSV line, handling quoted values
function parseCSVLine(line) {
    const values = [];
    let current = '';
    let inQuotes = false;

    for (let i = 0; i < line.length; i++) {
        const char = line[i];

        if (char === '"' {

```

```

        if (i + 1 < line.length && line[i + 1] === '"') {
            // Escaped quote
            current += '"';
            i++;
        } else {
            // Toggle quote mode
            inQuotes = !inQuotes;
        }
    } else if (char === ',' && !inQuotes) {
        // End of value
        values.push(current);
        current = '';
    } else {
        current += char;
    }
}

// Add the last value
values.push(current);

return values;
}

// Escape a value for CSV
function escapeCSV(value) {
    if (value == null) {
        return '';
    }

    const strValue = String(value);

    if (strValue.includes(',') || strValue.includes('"') ||
    strValue.includes('\n')) {
        // Escape quotes by doubling them and wrap in quotes
        return `"${strValue.replace(/"/g, '""')}"`;
    }

    return strValue;
}

// Parse a value to the appropriate type
function parseValue(value) {
    if (value === 'null') {
        return null;
    }

    if (value === 'true') {
        return true;
    }

    if (value === 'false') {
        return false;
    }
}

```

```

        if (!isNaN(Number(value))) {
            return Number(value);
        }

        return value;
    }

    // Compare two objects for equality
    function compareObjects(a, b) {
        if (a === b) {
            return true;
        }

        if (typeof a !== 'object' || typeof b !== 'object' || a == null || b
== null) {
            return false;
        }

        const keysA = Object.keys(a);
        const keysB = Object.keys(b);

        if (keysA.length !== keysB.length) {
            return false;
        }

        for (const key of keysA) {
            if (!keysB.includes(key)) {
                return false;
            }

            if (!compareObjects(a[key], b[key])) {
                return false;
            }
        }

        return true;
    }
}

// ===== 5.2 Compression Operations =====
function setupCompressionOperations() {
    const compressionInput = document.getElementById('compression-input');
    const compressionMethod = document.getElementById('compression-method');
    const compressBtn = document.getElementById('compress-data');
    const decompressBtn = document.getElementById('decompress-data');
    const resultContainer = document.getElementById('compression-result');
    const statsContainer = document.getElementById('compression-stats');

    // Currently compressed data
    let compressedData = '';
    let originalData = '';
    let currentMethod = '';

    // Compress button click handler

```

```

compressBtn.addEventListener('click', () => {
  safeExecute(() => {
    originalData = compressionInput.value;
    currentMethod = compressionMethod.value;

    if (!originalData) {
      throw new Error('Input cannot be empty');
    }

    // Compress the data according to the selected method
    const compressionResult = compressData(originalData,
currentMethod);
    compressedData = compressionResult.data;

    // Display the compressed data
    resultContainer.innerHTML = '';

    const resultHeader = document.createElement('div');
    resultHeader.className = 'compression-header';
    resultHeader.innerHTML = `<strong>Compressed Data
(${currentMethod}):</strong>`;
    resultContainer.appendChild(resultHeader);

    const resultContent = document.createElement('pre');
    resultContent.className = 'compression-content';
    resultContent.textContent = compressedData;
    resultContainer.appendChild(resultContent);

    // Display compression stats
    statsContainer.innerHTML = '';

    const originalSize = new Blob([originalData]).size;
    const compressedSize = new Blob([compressedData]).size;
    const compressionRatio = originalSize / compressedSize;
    const spaceSavings = (1 - (compressedSize / originalSize)) * 100;

    statsContainer.innerHTML = `
      <div><strong>Original Size:</strong> ${originalSize}
bytes</div>
      <div><strong>Compressed Size:</strong> ${compressedSize}
bytes</div>
      <div><strong>Compression Ratio:</strong>
${compressionRatio.toFixed(2)}:1</div>
      <div><strong>Space Savings:</strong>
${spaceSavings.toFixed(2)}%</div>
    `;

    return `Data compressed using ${currentMethod}`;
  }, error => {
    resultContainer.innerHTML = `<div class="error">${error.message}
</div>`;
    statsContainer.innerHTML = '';
  });
});

```

```

// Decompress button click handler
decompressBtn.addEventListener('click', () => {
  safeExecute(() => {
    if (!compressedData) {
      throw new Error('No compressed data to decompress');
    }

    // Decompress the data
    const decompressedData = decompressData(compressedData,
currentMethod);

    // Display the decompressed data
    resultContainer.innerHTML = '';

    const resultHeader = document.createElement('div');
    resultHeader.className = 'compression-header';
    resultHeader.innerHTML = `<strong>Decompressed Data:</strong>`;
    resultContainer.appendChild(resultHeader);

    const resultContent = document.createElement('pre');
    resultContent.className = 'compression-content';
    resultContent.textContent = decompressedData;
    resultContainer.appendChild(resultContent);

    // Update stats
    statsContainer.innerHTML += `
      <div class="mt-10"><strong>Verification:</strong> ${
        decompressedData === originalData
          ? '<span class="status-success">Data matches
original</span>'
          : '<span class="status-error">Data does not match
original</span>'
      }</div>
    `;

    return `Data decompressed using ${currentMethod}`;
  }, error => {
    resultContainer.innerHTML = `<div class="error">${error.message}
</div>`;
  });
});

// Compress data using the specified method
function compressData(data, method) {
  switch (method) {
    case 'rle':
      return { data: runLengthEncode(data) };

    case 'huffman':
      return { data: huffmanEncode(data) };

    case 'lz':
      return { data: lzCompress(data) };
  }
}

```

```

        default:
            throw new Error(`Unsupported compression method: ${method}`);
    }
}

// Decompress data using the specified method
function decompressData(data, method) {
    switch (method) {
        case 'rle':
            return runLengthDecode(data);

        case 'huffman':
            return huffmanDecode(data);

        case 'lz':
            return lzDecompress(data);

        default:
            throw new Error(`Unsupported compression method: ${method}`);
    }
}

// Run-Length Encoding
function runLengthEncode(data) {
    let encoded = '';
    let count = 1;

    for (let i = 0; i < data.length; i++) {
        // If the current character is the same as the next, increment
        if (data[i] === data[i + 1]) {
            count++;
        } else {
            // Otherwise, add the count and character to the result
            if (count >= 4) {
                // Only use RLE for runs of 4 or more
                encoded += `${count}${data[i]}`;
            } else {
                // Otherwise, just use the raw characters
                encoded += data[i].repeat(count);
            }
            count = 1;
        }
    }

    return encoded;
}

// Run-Length Decoding
function runLengthDecode(data) {
    let decoded = '';
    let countStr = '';

```



```

    for (let i = 0; i < data.length; i++) {
        const char = data[i];

        if (/\\d/.test(char)) {
            // If it's a digit, add it to the count
            countStr += char;
        } else {
            // Otherwise, it's a character
            let count = countStr ? parseInt(countStr) : 1;
            decoded += char.repeat(count);
            countStr = '';
        }
    }

    return decoded;
}

// Huffman Encoding (simplified version)
function huffmanEncode(data) {
    // Count frequency of each character
    const frequencies = {};
    for (const char of data) {
        frequencies[char] = (frequencies[char] || 0) + 1;
    }

    // Build Huffman tree (simplified as a dictionary)
    // In a real Huffman implementation, we would build a tree and
    traverse it
    const codes = {};
    let index = 0;

    for (const char in frequencies) {
        // Use binary representation of index as the code (simpler than a
    real Huffman tree)
        codes[char] = index.toString(2).padStart(4, '0');
        index++;

        if (index >= 16) {
            break; // Limit to 16 distinct characters for simplicity
        }
    }

    // Encode the data
    let encoded = '';

    // First, add the dictionary
    for (const char in codes) {
        const charCode = char.charCodeAt(0).toString(16).padStart(4, '0');
        encoded += `${charCode}:${codes[char]}`;
    }

    encoded += '|'; // Dictionary/data separator

    // Then encode the data

```

```

        for (const char of data) {
            encoded += codes[char] || '';
        }

        return encoded;
    }

    // Huffman Decoding (simplified version)
    function huffmanDecode(data) {
        // Split into dictionary and encoded data
        const parts = data.split('|');
        if (parts.length !== 2) {
            throw new Error('Invalid Huffman encoded data');
        }

        const dictionaryStr = parts[0];
        const encodedData = parts[1];

        // Parse the dictionary
        const codeToChar = {};
        const dictionaryEntries = dictionaryStr.split(';');

        for (const entry of dictionaryEntries) {
            if (!entry) continue;

            const [charHex, code] = entry.split(':');
            if (!charHex || !code) continue;

            const char = String.fromCharCode(parseInt(charHex, 16));
            codeToChar[code] = char;
        }

        // Decode the data
        let decoded = '';
        let currentCode = '';

        for (const bit of encodedData) {
            currentCode += bit;

            if (codeToChar[currentCode]) {
                decoded += codeToChar[currentCode];
                currentCode = '';
            }
        }

        return decoded;
    }

    // LZ Compression (simplified version of LZ77)
    function lzCompress(data) {
        let compressed = '';
        let pos = 0;

        while (pos < data.length) {

```

```

        // Look for longest match in the window
        let bestLength = 0;
        let bestOffset = 0;
        const maxOffset = Math.min(255, pos); // Limit to 255 for
simplicity        const maxLength = Math.min(255, data.length - pos); // Limit to
255 for simplicity

        for (let offset = 1; offset <= maxOffset; offset++) {
            let length = 0;
            while (length < maxLength && data[pos - offset + (length %
offset)] === data[pos + length]) {
                length++;
            }

            if (length > bestLength) {
                bestLength = length;
                bestOffset = offset;
            }
        }

        if (bestLength >= 3) { // Only compress sequences of 3 or more
            // Add a reference as (offset, length)
            compressed += `(${bestOffset},${bestLength})`;
            pos += bestLength;
        } else {
            // Add the character as is
            if (/[\(\)]/.test(data[pos])) {
                compressed += '\\' + data[pos]; // Escape special
characters
            } else {
                compressed += data[pos];
            }
            pos++;
        }
    }

    return compressed;
}

// LZ Decompression
function lzDecompress(data) {
    let decompressed = '';
    let pos = 0;

    while (pos < data.length) {
        if (data[pos] === '\\') {
            // Escaped character
            decompressed += data[pos + 1];
            pos += 2;
        } else if (data[pos] === '(') {
            // Reference (offset, length)
            const match = data.substring(pos).match(/^((\d+),(\d+)\))/);
            if (!match) {

```

```

        throw new Error('Invalid LZ compressed data');
    }

    const offset = parseInt(match[1]);
    const length = parseInt(match[2]);

    for (let i = 0; i < length; i++) {
        decompressed += decompressed[decompressed.length - offset
+ (i % offset)];
    }

    pos += match[0].length;
} else {
    // Literal character
    decompressed += data[pos];
    pos++;
}
}

return decompressed;
}
}

```

// ===== 5.3 Binary Encoding Operations =====

```

function setupEncodingOperations() {
    const encodingInput = document.getElementById('encoding-input');
    const encodingMethod = document.getElementById('encoding-method');
    const encodeBtn = document.getElementById('encode-data');
    const decodeBtn = document.getElementById('decode-data');
    const resultContainer = document.getElementById('encoding-result');

    // Currently encoded data
    let encodedData = '';
    let originalData = '';
    let currentMethod = '';

    // Encode button click handler
    encodeBtn.addEventListener('click', () => {
        safeExecute(() => {
            originalData = encodingInput.value;
            currentMethod = encodingMethod.value;

            if (!originalData) {
                throw new Error('Input cannot be empty');
            }

            // Encode the data according to the selected method
            encodedData = encodeData(originalData, currentMethod);

            // Display the encoded data
            resultContainer.innerHTML = '';

            const resultHeader = document.createElement('div');
            resultHeader.className = 'encoding-header';

```

```

        resultHeader.innerHTML = `

```

```

        case 'hex':
            return [...data].map(c =>
c.charCodeAt(0).toString(16).padStart(2, '0')).join('');

        case 'binary':
            return [...data].map(c =>
c.charCodeAt(0).toString(2).padStart(8, '0')).join(' ');

        case 'custom':
            return generateCustomId(data);

        default:
            throw new Error(`Unsupported encoding method: ${method}`);
    }
}

// Decode data using the specified method
function decodeData(data, method) {
    switch (method) {
        case 'base64':
            return decodeURIComponent(escape(atob(data)));

        case 'hex':
            if (!/^[0-9A-Fa-f]+$/.test(data) || data.length % 2 !== 0) {
                throw new Error('Invalid hexadecimal string');
            }
            let result = '';
            for (let i = 0; i < data.length; i += 2) {
                result += String.fromCharCode(parseInt(data.substr(i, 2),
16));
            }
            return result;

        case 'binary':
            return data.split(' ')
                .filter(binary => binary.trim() !== '')
                .map(binary => String.fromCharCode(parseInt(binary, 2)))
                .join('');

        case 'custom':
            return decodeCustomId(data);

        default:
            throw new Error(`Unsupported encoding method: ${method}`);
    }
}

// Generate a custom string ID
function generateCustomId(data) {
    const charset =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
    const charsetLength = charset.length;

```

```

        let id = '';

        // Convert string to a large integer
        for (let i = 0; i < data.length; i++) {
            const charCode = data.charCodeAt(i);
            id = id * charsetLength + (charCode % charsetLength);
        }

        // Add a simple checksum
        const checksum = data.split('').reduce((sum, char) => sum +
char.charCodeAt(0), 0) % 97;

        return id.toString() + '-' + checksum;
    }

    // Decode a custom string ID (simplified - in practice this would require
the original charset)
    function decodeCustomId(id) {
        // This is a placeholder since we can't fully decode without
additional information
        // In practice, the encoding would need to be reversible

        // Extract the checksum
        const parts = id.split('-');
        if (parts.length !== 2) {
            throw new Error('Invalid custom ID format');
        }

        return `[Original data (ID: ${parts[0]}, Checksum: ${parts[1]})]`;
    }
}

// ===== 5.4 Transaction Operations =====
function setupTransactionOperations() {
    const drawingColor = document.getElementById('drawing-color');
    const drawingSize = document.getElementById('drawing-size');
    const undoBtn = document.getElementById('transaction-undo');
    const redoBtn = document.getElementById('transaction-redo');
    const clearBtn = document.getElementById('transaction-clear');
    const canvas = document.getElementById('drawing-canvas');
    const historyContainer = document.getElementById('transaction-history');
    const context = canvas.getContext('2d');

    // Transaction management
    let actionHistory = [];
    let redoStack = [];
    let isDrawing = false;
    let lastX = 0;
    let lastY = 0;

    // Set up canvas and initial state
    function initCanvas() {
        // Set canvas background
        context.fillStyle = 'white';

```

```

        context.fillRect(0, 0, canvas.width, canvas.height);

        // Add event listeners
        canvas.addEventListener('mousedown', startDrawing);
        canvas.addEventListener('mousemove', draw);
        canvas.addEventListener('mouseup', stopDrawing);
        canvas.addEventListener('mouseout', stopDrawing);

        // Touch support
        canvas.addEventListener('touchstart', handleTouchStart);
        canvas.addEventListener('touchmove', handleTouchMove);
        canvas.addEventListener('touchend', stopDrawing);

        updateButtons();
    }

    // Start drawing
    function startDrawing(e) {
        isDrawing = true;
        const pos = getPosition(e);
        lastX = pos.x;
        lastY = pos.y;

        // Start a new transaction
        saveInitialState();
    }

    // Draw on canvas
    function draw(e) {
        if (!isDrawing) return;

        const pos = getPosition(e);
        const currentX = pos.x;
        const currentY = pos.y;

        context.beginPath();
        context.moveTo(lastX, lastY);
        context.lineTo(currentX, currentY);
        context.strokeStyle = drawingColor.value;
        context.lineWidth = parseInt(drawingSize.value);
        context.lineCap = 'round';
        context.stroke();

        lastX = currentX;
        lastY = currentY;
    }

    // Stop drawing
    function stopDrawing() {
        if (isDrawing) {
            isDrawing = false;
            saveFinalState();
        }
    }
}

```



```

// Handle touch events
function handleTouchStart(e) {
    e.preventDefault();
    if (e.touches.length === 1) {
        const touch = e.touches[0];
        const mouseEvent = new MouseEvent('mousedown', {
            clientX: touch.clientX,
            clientY: touch.clientY
        });
        canvas.dispatchEvent(mouseEvent);
    }
}

function handleTouchMove(e) {
    e.preventDefault();
    if (e.touches.length === 1) {
        const touch = e.touches[0];
        const mouseEvent = new MouseEvent('mousemove', {
            clientX: touch.clientX,
            clientY: touch.clientY
        });
        canvas.dispatchEvent(mouseEvent);
    }
}

// Get mouse position relative to canvas
function getPosition(e) {
    const rect = canvas.getBoundingClientRect();
    return {
        x: e.clientX - rect.left,
        y: e.clientY - rect.top
    };
}

// Transaction handling
function saveInitialState() {
    // Capture the current canvas state
    const imageData = context.getImageData(0, 0, canvas.width,
canvas.height);
    actionHistory.push({
        state: imageData,
        operation: 'Drawing started',
        color: drawingColor.value,
        size: drawingSize.value,
        timestamp: new Date().toISOString()
    });

    // Clear redo stack when a new action is performed
    redoStack = [];

    updateHistory();
    updateButtons();
}

```

```

function saveFinalState() {
    // Capture the final canvas state after drawing
    const imageData = context.getImageData(0, 0, canvas.width,
canvas.height);
    actionHistory.push({
        state: imageData,
        operation: 'Drawing completed',
        color: drawingColor.value,
        size: drawingSize.value,
        timestamp: new Date().toISOString()
    });

    updateHistory();
    updateButtons();
}

// Undo the last action
function undo() {
    if (actionHistory.length <= 1) return;

    // Get the previous state
    const currentState = actionHistory.pop();
    redoStack.push(currentState);

    const previousState = actionHistory[actionHistory.length - 1];

    // Restore the previous state
    context.putImageData(previousState.state, 0, 0);

    updateHistory();
    updateButtons();
}

// Redo the last undone action
function redo() {
    if (redoStack.length === 0) return;

    // Get the next state
    const nextState = redoStack.pop();
    actionHistory.push(nextState);

    // Restore the next state
    context.putImageData(nextState.state, 0, 0);

    updateHistory();
    updateButtons();
}

// Clear the canvas
function clearCanvas() {
    // Save current state before clearing
    const imageData = context.getImageData(0, 0, canvas.width,
canvas.height);

```

```

        actionHistory.push({
            state: imageData,
            operation: 'Before clear',
            timestamp: new Date().toISOString()
        });

        // Clear the canvas
        context.fillStyle = 'white';
        context.fillRect(0, 0, canvas.width, canvas.height);

        // Save the cleared state
        const clearedImageData = context.getImageData(0, 0, canvas.width,
canvas.height);
        actionHistory.push({
            state: clearedImageData,
            operation: 'Canvas cleared',
            timestamp: new Date().toISOString()
        });

        // Clear redo stack
        redoStack = [];

        updateHistory();
        updateButtons();
    }

    // Update transaction history display
    function updateHistory() {
        historyContainer.innerHTML = '';

        // Display last 5 transactions
        const start = Math.max(0, actionHistory.length - 5);
        for (let i = start; i < actionHistory.length; i++) {
            const transaction = actionHistory[i];
            const item = document.createElement('div');
            item.className = 'transaction-item';

            const time = new Date(transaction.timestamp).toLocaleTimeString();
            item.textContent = `${time}: ${transaction.operation}`;

            if (transaction.color) {
                const colorSwatch = document.createElement('span');
                colorSwatch.className = 'color-swatch';
                colorSwatch.style.backgroundColor = transaction.color;
                colorSwatch.style.display = 'inline-block';
                colorSwatch.style.width = '10px';
                colorSwatch.style.height = '10px';
                colorSwatch.style.marginLeft = '5px';

                item.appendChild(colorSwatch);
            }

            historyContainer.appendChild(item);
        }
    }
}

```

```

    }

    // Update button states
    function updateButtons() {
        undoBtn.disabled = actionHistory.length <= 1;
        redoBtn.disabled = redoStack.length === 0;
    }

    // Initialize
    initCanvas();

    // Add event listeners
    undoBtn.addEventListener('click', undo);
    redoBtn.addEventListener('click', redo);
    clearBtn.addEventListener('click', clearCanvas);
}

// ===== INITIALIZATION =====

/**
 * Initialize the application
 */
function init() {
    // Setup error modal close button
    const errorModal = document.getElementById('error-modal');
    const resultModal = document.getElementById('result-modal');
    const closeButton = document.querySelector('.close-btn');

    closeButton.addEventListener('click', function() {
        errorModal.style.display = 'none';
        resultModal.style.display = 'none';
    });

    // Add to window event listeners in the init function
    window.addEventListener('beforeunload', function() {
        // Explicitly clear all sensitive data structures
        virtualFS.root.children = {};
        database.deleteAll();
        memoryManager.clearAll();
        // Other data structures as needed
    });

    // Close modals when clicking outside
    window.addEventListener('click', function(event) {
        if (event.target === errorModal) {
            errorModal.style.display = 'none';
        }
        if (event.target === resultModal) {
            resultModal.style.display = 'none';
        }
    });
}

```

```

    // Setup navigation
    setupNavigation();

    // Setup all operations
    setupArrayOperations();
    setupObjectOperations();
    setupSetOperations();
    setupTreeOperations();
    setupRecursionOperations();
    setupIterationOperations();
    setupSearchOperations();
    setupBinaryOperations();
    setupFileOperations();
    setupMemoryOperations();
    setupProcessOperations();

    // CRUD operations - Create returns an object with updateDatabaseDisplay
function
    // which is passed to the other CRUD operations
    const databaseDisplayUpdater = setupCreateOperations();
    setupReadOperations(databaseDisplayUpdater);
    setupUpdateOperations(databaseDisplayUpdater);
    setupDeleteOperations(databaseDisplayUpdater);

    setupSerializationOperations();
    setupCompressionOperations();
    setupEncodingOperations();
    setupTransactionOperations();

}

// Wait for DOM to be fully loaded before initializing
document.addEventListener('DOMContentLoaded', init);
})();

```